

Прогнозная модель отклика клиента на банковское предложение

Описание проекта

Данный проект создан в рамках портфолио и реализуется в рамках соревнования Kaggle Playground Series - Season 5, Episode 8

- <https://www.kaggle.com/competitions/playground-series-s5e8/>

Цель проекта – построить модель машинного обучения, способную предсказывать, подпишется ли клиент на банковский срочный депозит (переменная y) на основе его демографических характеристик и истории взаимодействия с банком.

Это задача бинарной классификации, где необходимо предсказать вероятность положительного исхода (подписка на депозит) на основе данных о клиенте и истории маркетинговой кампании.

Метрика оценки: ROC AUC между предсказанной вероятностью и фактическим значением целевой переменной.

Прогнозирование отклика клиентов на маркетинговые кампании является ключевой задачей в банковском секторе для оптимизации маркетинговых затрат и повышения эффективности продаж.

Практическая ценность:

- Банки могут использовать модель для таргетинга клиентов в маркетинговых кампаниях
- Оптимизация затрат на рекламу и повышение конверсии
- Анализ факторов, влияющих на решение клиента
- Персонализация предложений для клиентов

Проект демонстрирует навыки:

- загрузки и предобработки данных
- исследовательского анализа (EDA)
- построения и сравнения моделей классификации
- визуализации результатов
- оценки качества предсказаний с использованием ROC AUC

Описание данных

Данные синтетически сгенерированы с помощью глубокого обучения на основе оригинального набора данных **Bank Marketing Dataset Full**(<https://www.kaggle.com/datasets/sushant097/bank-marketing-dataset-full>).

Распределения признаков близки к оригинальным, но не идентичны.

Файлы:

- train.csv – обучающая выборка, содержит бинарную целевую переменную y (0/1)
- test.csv – тестовая выборка, для которой необходимо предсказать вероятность y
- sample_submission.csv – пример файла для отправки предсказаний

Признаки включают:

- Социально-демографические характеристики клиента: возраст, семейное положение(age, job, marital, education)
- Финансовые параметры (balance, housing, loan)
- Информация о маркетинговой кампании (contact, day, month, duration, campaign, pdays, previous, poutcome)
- Целевая переменная: y (подписка на депозит: 0 – нет, 1 – да)

Признаки:

Информация о клиенте:

- **id** — уникальный идентификатор клиента
- **age** - возраст клиента (числовой)
- **job** - тип работы (категориальный: "admin.", "blue-collar", "entrepreneur" и др.)
- **marital** - семейное положение (категориальный: "married", "single", "divorced")
- **education** - уровень образования (категориальный: "primary", "secondary", "tertiary", "unknown")
- **default** - наличие просроченного кредита (категориальный: "yes", "no")
- **balance** - средний годовой баланс в евро (числовой)
- **housing** - наличие ипотечного кредита (категориальный: "yes", "no")
- **loan** - наличие персонального кредита (категориальный: "yes", "no")

Информация о текущей маркетинговой кампании:

- **contact** - тип канала связи (категориальный: "unknown", "telephone", "cellular")
- **day** - день последнего контакта в месяце (числовой, 1-31)
- **month** - месяц последнего контакта (категориальный: "jan", "feb", "mar", ..., "dec")
- **duration** - продолжительность последнего контакта в секундах (числовой)
- **campaign** - количество контактов в течение текущей кампании (числовой)

Информация о предыдущих кампаниях:

- **pdays** - количество дней с последнего контакта в предыдущей кампании (числовой; -1 означает, что клиент не контактировал ранее)
- **previous** - количество контактов до текущей кампании (числовой)
- **poutcome** - результат предыдущей маркетинговой кампании (категориальный: "unknown", "other", "failure", "success")

Целевая переменная: y (только в train.csv) - подписался ли клиент на срочный депозит (бинарная: 0 или 1, где 1 = "yes", 0 = "no")

Особенности данных:

- Синтетические данные содержат как числовые, так и категориальные признаки
- Присутствуют временные характеристики (день, месяц контакта)
- Есть информация о предыдущих маркетинговых кампаниях
- Целевая переменная вероятно несбалансирована (типично для маркетинговых данных)

Можно использовать оригинальный набор данных (Bank Marketing Dataset Full) в рамках проекта — как для исследования различий, так и для проверки, улучшает ли включение оригинальных данных качество модели.

Оригинальный набор данных Bank Marketing Dataset Full (<https://www.kaggle.com/datasets/sushant097/bank-marketing-dataset-full>):

- набор данных для предсказания подписки на срочный банковский депозит
- данный набор данных содержит информацию о клиентах португальского банковского учреждения
- данные были получены в ходе прямой маркетинговой кампании, и каждая запись соответствует одному клиенту

Цели исследования

- Провести исследовательский анализ данных (EDA)
- Изучить распределение целевого признака и его связь с основными характеристиками клиентов
- Выявить ключевые признаки, влияющие на решение клиента о подписке на депозит
- Построить и сравнить различные модели для бинарной классификации (Logistic Regression, Ансамбли (XGBClassifier, LGBMClassifier, CatBoostClassifier))
- Оценить качество моделей с помощью кросс-валидации по метрике ROC AUC и максимизировать эту метрику
- Проверить гипотезы о влиянии признаков (например, duration, pdays, balance) на вероятность подписки
- Проанализировать важность признаков для понимания факторов успеха маркетинговой кампании
- Проанализировать сезонность в маркетинговой кампании (по месяцам)

«Бизнес-задача: повысить эффективность маркетинговых кампаний банка за счёт точного таргетинга клиентов, наиболее склонных к подписке на депозит».

Ход исследования

Шаг 1: Загрузка данных

- Подключение к Kaggle API, скачивание файлов
- Первичный анализ структуры и типов данных
- Проверка согласованности id между train и test

Шаг 2: Предобработка данных

- Уникальные значения категориальных признаков
- Проверка и обработка пропусков
- Проверка дубликатов

Шаг 3: Исследовательский анализ данных (EDA)

- Распределение целевой переменной y (подписка на депозит)
 - проверка баланса классов
- Гистограммы и ящики с усами распределений количественных признаков для двух датасетов
 - проверка и обработка выбросов

- Столбчатые диаграммы категориальных признаков для двух датасетов
- Анализ распределений признаков по классам целевой переменной
- Матрица корреляций(heatmap) для выявления зависимостей
- Попарные графики (pairplot) для признаков
- Поиск мультиколлинеарности и выделение ключевых признаков
- Выявление паттернов: какие характеристики типичны для клиентов, подписавшихся/не подписавшихся на депозит

Шаг 4: Подготовка к обучению моделей

- Создание новых признаков
- Проверка корреляции новых признаков с целевой переменной
- Выбор признаков для моделей
- Разделение данных (признаки и целевая переменная) обучающей выборки на train/valid
- Предобработка данных: масштабирование числовых признаков и кодирование категориальных признаков (One-Hot Encoding, Label Encoding)

Шаг 5: Обучение моделей

- Базовые модели: DummyClassifier
- Линейные модели: Logistic Regression
- Ансамбли: XGBClassifier, LGBMClassifier, CatBoostClassifier
- Подбор гиперпараметров на кросс-валидации с помощью GridSearchCV/RandomizedSearchCV
- Сравнение моделей XGBClassifier, LGBMClassifier, CatBoostClassifier по метрике ROC-AUC на валидации
- Визуализация результатов лучшей модели на валидационной выборке:
 - Предсказание на валидационно выборке
 - Расчёт метрик
 - Матрица ошибок и ROC-кривая
 - Отчёт по классификации: точность, полнота, F1-мера
- Интерпретация важности признаков (Feature Importance)
- Аналитические инсайты
- Рекомендации по улучшению

Шаг 6: Формирование submission

- Предсказание вероятностей для тестовой выборки с помощью лучшей модели
- Сохранение итоговых предсказаний в формате:
 - id,y
 - 750000,0.5
 - 750001,0.7

Шаг 7: Исследование синтетической природы данных:

- Загрузка оригинального датасета Bank Marketing Dataset Full(<https://www.kaggle.com/datasets/sushant097/bank-marketing-dataset-full>)
- Сравнить распределения признаков в синтетическом и с оригинальным датасетом
- Корреляция признаков с целевой переменной в синтетических и оригинальных датасетах
- **Объединение оригинальных данных с синтетическими:**
 - Приведение к единому формату
 - Удаление дубликатов
 - Переобучение моделей на объединенном датасете
 - Сравнение метрик до и после добавления оригинальных данных

Ограничения и вызовы

- Синтетическая природа данных:могут быть артефакты, возможна потеря реальных зависимостей между признаками и откликом
- Дисбаланс классов (обычно подписавшихся меньше)
- Не все категориальные признаки интерпретируемы напрямую (например, “unknown”)
- Категориальные признаки с большим количеством уникальных значений (например, job, education)

Ожидаемые результаты

- Чистый и подготовленный датасет
- Визуализации распределений и корреляций
- Сравнительная таблица моделей по ROC AUC
 - ожидаемая метрика ROC-AUC на валидации — 0.95–0.97, при использовании ансамблей (CatBoost, LightGBM, XGBoost).
- Финальный submission-файл для Kaggle

- Аналитические инсайты:
 - Определение ТОП-5 факторов, влияющих на решение о подписке
 - Выявление профилей клиентов, наиболее склонных к подписке на депозит
 - Рекомендации по оптимизации маркетинговой стратегии

Содержание:

- [Шаг 1: Загрузка данных](#)
- [Шаг 2: Предобработка данных](#)
- [Шаг 3: Исследовательский анализ данных \(EDA\)](#)
- [Шаг 4: Подготовка к обучению моделей](#)
- [Шаг 5: Обучение моделей](#)
- [Шаг 6: Формирование submission](#)
- [Шаг 7: Исследование синтетической природы данных](#)

Выполнение проекта

Технический стек

- Проект написан на Python с использованием следующих библиотек:

```
In [1]: # 📁 Работа с файлами и API
import os, zipfile, requests, re
from IPython.display import display
from kaggle.api.kaggle_api_extended import KaggleApi
from charset_normalizer import from_path

# 📊 Базовые библиотеки анализа данных
import pandas as pd
import numpy as np

# 📈 Визуализация
import matplotlib.pyplot as plt
import seaborn as sns

# 🔍 Корреляционный анализ
import phik
from phik import phik_matrix
from phik.report import plot_correlation_matrix

from scipy import stats
from scipy.stats import mannwhitneyu, ks_2samp, chi2_contingency

import time
# 🤖 Машинное обучение
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score, KFold, RandomizedSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder
from sklearn.metrics import roc_auc_score, classification_report, confusion_matrix, roc_curve, auc

from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.model_selection import GridSearchCV

from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
                             roc_auc_score, roc_curve, precision_recall_curve, f1_score, precision_score, recall_score)

from types import SimpleNamespace
```

```
In [2]: import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
```

```
In [3]: start_all = time.time()
```

Шаг 1: Загрузка данных

```
In [4]: # Работа с файлами и API
import os, zipfile, requests, re
from IPython.display import display
from kaggle.api.kaggle_api_extended import KaggleApi
```



```

from charset_normalizer import from_path

# Базовые библиотеки анализа данных
import pandas as pd
import numpy as np

# Визуализация
import matplotlib.pyplot as plt
import seaborn as sns

# Определение кодировки файла
def detect_encoding(file_path):
    result = from_path(file_path).best()
    if result is None:
        print("⚠ Кодировка не определена, используем utf-8 по умолчанию.")
        return "utf-8"
    print(f"✅ Определена кодировка: {result.encoding}")
    return result.encoding

# Загрузка и первичный анализ CSV-файла
def process_dataframe(file_name, data_dir, sep=",", decimal="."):
    local_path = os.path.join(data_dir, file_name)
    if not os.path.exists(local_path):
        print(f"❌ Файл '{file_name}' не найден в папке {data_dir}.")
        return None
    print(f"📁 Загружаем файл: {file_name}")
    encoding = detect_encoding(local_path)
    print(f"📍 Абсолютный путь к файлу:", os.path.abspath(local_path))
    df = pd.read_csv(local_path, sep=sep, decimal=decimal, encoding=encoding)

    # Первичный анализ датафрейма
    print("\n ♦ Первые 5 строк:")
    display(df.head())
    print("\n ♦ Случайные 5 строк:")
    display(df.sample(5))
    print("\n ♦ Последние 5 строк:")
    display(df.tail())

    print("\n 📊 Информация о датафрейме:")
    df.info()
    print("\n 📏 Размер датафрейма:", df.shape)
    print("\n 📋 Названия столбцов:", df.columns.tolist())

    return df

# Скачивание и обработка конкретного файла из соревнования Kaggle
def fetch_kaggle_file(competition_slug, file_name, data_dir):
    os.makedirs(data_dir, exist_ok=True)
    api = KaggleApi()
    api.authenticate()

    print(f"↓ Скачиваем файл {file_name} из соревнования {competition_slug} ...")
    api.competition_download_file(competition_slug, file_name, path=data_dir)

    file_path = os.path.join(data_dir, file_name)

    # Распаковка, если это архив
    if file_path.endswith(".zip"):
        print("📦 Распаковываем архив ...")
        with zipfile.ZipFile(file_path, 'r') as zip_ref:
            zip_ref.extractall(data_dir)
            extracted_files = zip_ref.namelist()
            if extracted_files:
                file_name = extracted_files[0]
            else:
                print("❌ Архив пуст.")
                return None

    return process_dataframe(file_name, data_dir=data_dir)

# Путь к папке с датасетами
data_dir = r"E:\kaggle_datasets\Kaggle_Playground_S5E8"
competition = "playground-series-s5e8"
files = ["train.csv", "test.csv", "sample_submission.csv"]

# Загрузка и анализ всех файлов
datasets = {name: fetch_kaggle_file(competition, name, data_dir) for name in files}
df_train = datasets["train.csv"]
df_test = datasets["test.csv"]
df_sample = datasets["sample_submission.csv"]

```

↓ Скачиваем файл train.csv из соревнования playground-series-s5e8 ...

train.csv: Skipping, found more recently modified local copy (use --force to force download)

📁 Загружаем файл: train.csv

✅ Определена кодировка: ascii

📍 Абсолютный путь к файлу: E:\kaggle_datasets\Kaggle_Playground_S5E8\train.csv

♦ Первые 5 строк:

	id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	y
0	0	42	technician	married	secondary	no	7	no	no	cellular	25	aug	117	3	-1	0	
1	1	38	blue-collar	married	secondary	no	514	no	no	unknown	18	jun	185	1	-1	0	
2	2	36	blue-collar	married	secondary	no	602	yes	no	unknown	14	may	111	2	-1	0	
3	3	27	student	single	secondary	no	34	yes	no	unknown	28	may	10	2	-1	0	
4	4	26	technician	married	secondary	no	889	yes	no	cellular	3	feb	902	1	-1	0	



◆ Случайные 5 строк:

	id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	y
285282	285282	30	management	single	tertiary	no	0	no	no	cellular	18	aug	106	4	-1	0	
381757	381757	37	technician	single	tertiary	no	980	no	no	cellular	2	aug	155	1	-1	0	
336409	336409	33	blue-collar	married	secondary	no	4	no	no	cellular	5	feb	141	2	-1	0	
130726	130726	33	management	single	tertiary	no	2466	no	no	unknown	18	nov	248	2	-1	0	
697366	697366	32	management	married	tertiary	no	3185	no	no	cellular	26	may	97	1	-1	0	



◆ Последние 5 строк:

	id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	y
749995	749995	29	services	single	secondary	no	1282	no	yes	unknown	4	jul	1006	2	-1	0	
749996	749996	69	retired	divorced	tertiary	no	631	no	no	cellular	19	aug	87	1	-1	0	
749997	749997	50	blue-collar	married	secondary	no	217	yes	no	cellular	17	apr	113	1	-1	0	
749998	749998	32	technician	married	secondary	no	-274	no	no	cellular	26	aug	108	6	-1	0	
749999	749999	42	technician	married	secondary	no	1559	no	no	cellular	4	aug	143	1	1	0	



🖼️ Информация о датафрейме:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 750000 entries, 0 to 749999
Data columns (total 18 columns):
Column Non-Null Count Dtype
--- ---
0 id 750000 non-null int64
1 age 750000 non-null int64
2 job 750000 non-null object
3 marital 750000 non-null object
4 education 750000 non-null object
5 default 750000 non-null object
6 balance 750000 non-null int64
7 housing 750000 non-null object
8 loan 750000 non-null object
9 contact 750000 non-null object
10 day 750000 non-null int64
11 month 750000 non-null object
12 duration 750000 non-null int64
13 campaign 750000 non-null int64
14 pdays 750000 non-null int64
15 previous 750000 non-null int64
16 poutcome 750000 non-null object
17 y 750000 non-null int64
dtypes: int64(9), object(9)
memory usage: 103.0+ MB

📏 Размер датафрейма: (750000, 18)

📄 Названия столбцов: ['id', 'age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous', 'poutcome', 'y']
↓ Скачиваем файл test.csv из соревнования playground-series-s5e8 ...
test.csv: Skipping, found more recently modified local copy (use --force to force download)
📁 Загружаем файл: test.csv
✅ Определена кодировка: ascii
📌 Абсолютный путь к файлу: E:\kaggle_datasets\Kaggle_Playground_S5E8\test.csv

◆ Первые 5 строк:

	id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	pre
0	750000	32	blue-collar	married	secondary	no	1397	yes	no	unknown	21	may	224	1	-1	
1	750001	44	management	married	tertiary	no	23	yes	no	cellular	3	apr	586	2	-1	
2	750002	36	self-employed	married	primary	no	46	yes	yes	cellular	13	may	111	2	-1	
3	750003	58	blue-collar	married	secondary	no	-1380	yes	yes	unknown	29	may	125	1	-1	
4	750004	28	technician	single	secondary	no	1950	yes	no	cellular	22	jul	181	1	-1	



◆ Случайные 5 строк:

	id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	pre
221337	971337	37	management	married	secondary	no	-32	yes	yes	unknown	28	may	114	2	-	
210523	960523	29	management	single	tertiary	no	5359	no	no	cellular	25	may	211	1	18	
225138	975138	32	technician	divorced	tertiary	no	685	yes	no	cellular	18	nov	89	1	-	
107387	857387	56	retired	married	secondary	no	478	no	no	unknown	5	jun	593	2	-	
40851	790851	43	technician	married	secondary	no	-160	no	yes	cellular	8	jul	143	1	-	



◆ Последние 5 строк:

	id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	pre
249995	999995	43	management	married	tertiary	no	0	yes	no	cellular	18	nov	65	2	-	
249996	999996	40	services	married	unknown	no	522	yes	no	cellular	19	nov	531	1	18	
249997	999997	63	retired	married	primary	no	33	no	no	cellular	3	jul	178	1	9	
249998	999998	50	blue-collar	married	primary	no	2629	yes	no	unknown	30	may	163	2	-	
249999	999999	29	student	single	tertiary	no	722	no	no	cellular	6	apr	119	1	-	



 Информация о датафрейме:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250000 entries, 0 to 249999
Data columns (total 17 columns):
#   Column      Non-Null Count  Dtype
---  -
0   id          250000 non-null int64
1   age         250000 non-null int64
2   job         250000 non-null object
3   marital     250000 non-null object
4   education   250000 non-null object
5   default     250000 non-null object
6   balance     250000 non-null int64
7   housing     250000 non-null object
8   loan        250000 non-null object
9   contact     250000 non-null object
10  day         250000 non-null int64
11  month       250000 non-null object
12  duration    250000 non-null int64
13  campaign    250000 non-null int64
14  pdays       250000 non-null int64
15  previous    250000 non-null int64
16  poutcome    250000 non-null object
dtypes: int64(8), object(9)
memory usage: 32.4+ MB
```

 Размер датафрейма: (250000, 17)

 Названия столбцов: ['id', 'age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous', 'poutcome']

↓ Скачиваем файл sample_submission.csv из соревнования playground-series-s5e8 ...

sample_submission.csv: Skipping, found more recently modified local copy (use --force to force download)

 Загружаем файл: sample_submission.csv

 Определена кодировка: ascii

 Абсолютный путь к файлу: E:\kaggle_datasets\Kaggle_Playground_S5E8\sample_submission.csv

◆ Первые 5 строк:

	id	y
0	750000	0.5
1	750001	0.5
2	750002	0.5
3	750003	0.5
4	750004	0.5

Случайные 5 строк:

	id	y
195274	945274	0.5
224828	974828	0.5
248795	998795	0.5
238956	988956	0.5
116107	866107	0.5

Последние 5 строк:

	id	y
249995	999995	0.5
249996	999996	0.5
249997	999997	0.5
249998	999998	0.5
249999	999999	0.5

Информация о датафрейме:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250000 entries, 0 to 249999
Data columns (total 2 columns):
Column Non-Null Count Dtype
--- --- -
0 id 250000 non-null int64
1 y 250000 non-null float64
dtypes: float64(1), int64(1)
memory usage: 3.8 MB

Размер датафрейма: (250000, 2)

Названия столбцов: ['id', 'y']

- df_tarin:

```
In [5]: df_train.head().iloc[:,9]
```

	id	age	job	marital	education	default	balance	housing	loan
0	0	42	technician	married	secondary	no	7	no	no
1	1	38	blue-collar	married	secondary	no	514	no	no
2	2	36	blue-collar	married	secondary	no	602	yes	no
3	3	27	student	single	secondary	no	34	yes	no
4	4	26	technician	married	secondary	no	889	yes	no

```
In [6]: df_train.tail().iloc[:,9]
```

	id	age	job	marital	education	default	balance	housing	loan
749995	749995	29	services	single	secondary	no	1282	no	yes
749996	749996	69	retired	divorced	tertiary	no	631	no	no
749997	749997	50	blue-collar	married	secondary	no	217	yes	no
749998	749998	32	technician	married	secondary	no	-274	no	no
749999	749999	42	technician	married	secondary	no	1559	no	no

```
In [7]: df_train.head().iloc[:,[0] + list(range(9, df_train.shape[1]))]
```

Out[7]:

	id	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	0	cellular	25	aug	117	3	-1	0	unknown	0
1	1	unknown	18	jun	185	1	-1	0	unknown	0
2	2	unknown	14	may	111	2	-1	0	unknown	0
3	3	unknown	28	may	10	2	-1	0	unknown	0
4	4	cellular	3	feb	902	1	-1	0	unknown	1

In [8]: `df_train.tail().iloc[:,[0] + list(range(9, df_train.shape[1]))]`

Out[8]:

	id	contact	day	month	duration	campaign	pdays	previous	poutcome	y
749995	749995	unknown	4	jul	1006	2	-1	0	unknown	1
749996	749996	cellular	19	aug	87	1	-1	0	unknown	0
749997	749997	cellular	17	apr	113	1	-1	0	unknown	0
749998	749998	cellular	26	aug	108	6	-1	0	unknown	0
749999	749999	cellular	4	aug	143	1	1	7	failure	0

In [9]: `df_train['id'].nunique()==df_train.shape[0]`

Out[9]: True

- df_test:

In [10]: `df_test.head().iloc[:, :9]`

Out[10]:

	id	age	job	marital	education	default	balance	housing	loan
0	750000	32	blue-collar	married	secondary	no	1397	yes	no
1	750001	44	management	married	tertiary	no	23	yes	no
2	750002	36	self-employed	married	primary	no	46	yes	yes
3	750003	58	blue-collar	married	secondary	no	-1380	yes	yes
4	750004	28	technician	single	secondary	no	1950	yes	no

In [11]: `df_test.tail().iloc[:, :9]`

Out[11]:

	id	age	job	marital	education	default	balance	housing	loan
249995	999995	43	management	married	tertiary	no	0	yes	no
249996	999996	40	services	married	unknown	no	522	yes	no
249997	999997	63	retired	married	primary	no	33	no	no
249998	999998	50	blue-collar	married	primary	no	2629	yes	no
249999	999999	29	student	single	tertiary	no	722	no	no

In [12]: `df_test.head().iloc[:,[0] + list(range(9, df_test.shape[1]))]`

Out[12]:

	id	contact	day	month	duration	campaign	pdays	previous	poutcome
0	750000	unknown	21	may	224	1	-1	0	unknown
1	750001	cellular	3	apr	586	2	-1	0	unknown
2	750002	cellular	13	may	111	2	-1	0	unknown
3	750003	unknown	29	may	125	1	-1	0	unknown
4	750004	cellular	22	jul	181	1	-1	0	unknown

In [13]: `df_test.tail().iloc[:,[0] + list(range(9, df_test.shape[1]))]`

Out[13]:

	id	contact	day	month	duration	campaign	pdays	previous	poutcome
249995	999995	cellular	18	nov	65	2	-1	0	unknown
249996	999996	cellular	19	nov	531	1	189	1	failure
249997	999997	cellular	3	jul	178	1	92	8	success
249998	999998	unknown	30	may	163	2	-1	0	unknown
249999	999999	cellular	6	apr	119	1	-1	0	unknown

In [14]:

```
df_test['id'].nunique()==df_test.shape[0]
```

Out[14]:

True

In [15]:

```
set(df_train['id']).isdisjoint(df_test['id'])
```

Out[15]:

True

Вернул True, нет пересечений.

In [16]:

```
# 📊 Сбор статистики для train/test
dataframes = {
    'train': df_train,
    'test': df_test
}

stats = []
for name, df in dataframes.items():
    total_rows = df.shape[0]
    unique_ids = df['id'].nunique() if 'id' in df.columns else None
    duplicate_ids = df['id'].duplicated().sum() if 'id' in df.columns else None
    stats.append({
        'DataFrame': name,
        'Rows': total_rows,
        'Unique id': unique_ids,
        'Duplicate id': duplicate_ids
    })

df_stats = pd.DataFrame(stats)

# 📈 Визуализация количества строк
plt.figure(figsize=(6, 4))
sns.barplot(
    data=df_stats,
    x='DataFrame',
    y='Rows',
    hue='DataFrame',      # добавляем hue
    palette='mako'
)
plt.title('Количество строк в train и test')
plt.ylabel('Число строк')
plt.xlabel('Датасет')
plt.tight_layout()
plt.show()

# 📄 Вывод таблицы
print("📌 Статистика по строкам, уникальности и дубликатам id:")
display(df_stats)
```



📌 Статистика по строкам, уникальности и дубликатам id:

	DataFrame	Rows	Unique id	Duplicate id
0	train	750000	750000	0
1	test	250000	250000	0

- df_sample:

```
In [17]: df_sample.head()
```

Out[17]:

	id	y
0	750000	0.5
1	750001	0.5
2	750002	0.5
3	750003	0.5
4	750004	0.5

```
In [18]: df_sample.tail()
```

Out[18]:

	id	y
249995	999995	0.5
249996	999996	0.5
249997	999997	0.5
249998	999998	0.5
249999	999999	0.5

Проверка согласованности id

```
In [19]: print("Train id range:", df_train['id'].min(), "-", df_train['id'].max())
print("Test id range:", df_test['id'].min(), "-", df_test['id'].max())
print(f"Соотношение между тренировочным и тестовым датасетами:", round(df_train.shape[0]/df_test.shape[0],2))
```

Train id range: 0 - 749999
Test id range: 750000 - 999999
Соотношение между тренировочным и тестовым датасетами: 3.0

Диапазон id:

- train: 0–749999
- test: 750000 - 9999995

Пересечений между наборами нет (isdisjoint → True).

Соотношение размеров: train / test = 3.0.

✅ Результат: данные корректно разделены на обучающую и тестовую выборки.

Выводы по шагу 1

Данные успешно загружены, структура корректна и соответствует описанию.

- проверка структуры: в train.csv — 750 000 строк и 18 признаков, в test.csv — 250 000 строк. аналогичная структура, но без целевой переменной y.
- диапазоны по id между df_train и df_test не пересекаются
- есть как числовые, так и категориальные признаки, потребуется кодирование.
- вероятен дисбаланс классов (подписавшихся меньше), что нужно учитывать при валидации.

train.csv (обучающая выборка)

- Размер: 750,000 строк × 18 столбцов
- Структура:
 - 1 идентификатор (id)
 - 8 числовых признаков (age, balance, day, duration, campaign, pdays, previous)
 - 9 категориальных признаков (job, marital, education, default, housing, loan, contact, month, poutcome)
 - 1 целевая переменная (y: 0/1)
 - ✅ Все id уникальны (750,000 уникальных)

test.csv (тестовая выборка)

- Размер: 250,000 строк × 17 столбцов
- Структура: идентична train, но без целевой переменной y
 - ☒ Все id уникальны (250,000 уникальных)

sample_submission.csv

- Размер: 250,000 строк × 2 столбца (id, y)
- Формат: все вероятности заполнены константой 0.5 (шаблон)

Датасет сгенерирован с помощью глубокого обучения на основе оригинального **Bank Marketing Dataset Full** (45,211 записей).

- Синтетический train: 750,000 записей (в **16.6 раз больше** оригинала)
- Оригинальный датасет: 45,211 записей

[* к содержанию](#)

Шаг 2: Предобработка данных

Проверка названий столбцов в обоих датафреймах

```
In [20]: # Преобразуем в множества
set1 = set(df_train.columns)
set2 = set(df_test.columns)

# Найдём различия
only_in_df1 = sorted(set1 - set2)
only_in_df2 = sorted(set2 - set1)

# Выводим результат
print("🔵 Колонки только в df1:", only_in_df1)
print("🟢 Колонки только в df2:", only_in_df2)
```

```
🔵 Колонки только в df1: ['y']
🟢 Колонки только в df2: []
```

Проверка типов данных

```
In [21]: print(df_train.dtypes)
print(df_test.dtypes)
```

```
id          int64
age         int64
job         object
marital     object
education   object
default     object
balance     int64
housing     object
loan        object
contact     object
day         int64
month       object
duration    int64
campaign    int64
pdays      int64
previous    int64
poutcome    object
y           int64
dtype: object
id          int64
age         int64
job         object
marital     object
education   object
default     object
balance     int64
housing     object
loan        object
contact     object
day         int64
month       object
duration    int64
campaign    int64
pdays      int64
previous    int64
poutcome    object
dtype: object
```

```
In [22]: # Получаем типы
types1 = df_train.dtypes
types2 = df_test.dtypes

# Объединяем в таблицу для сравнения
comparison = pd.DataFrame({
    "df_train_type": types1,
    "df_test_type": types2
})

# Добавляем флаг совпадения
comparison["match"] = comparison["df_train_type"] == comparison["df_test_type"]

mismatches = comparison[~comparison["match"]]
mismatches
```

```
Out[22]:
```

	df_train_type	df_test_type	match
y	int64	NaN	False

все числовые признаки → int64 все категориальные признаки → object

целевая переменная y → int64

✅ Типы данных корректные, преобразование не требуется.

Категориальный признак целевой переменной

Создадим новый столбец категориального признака целевой переменной y подписался ли клиент на срочный депозит

```
In [23]: # создаём признак по условию (nun object)
df_train['subscribed_dep'] = df_train['y'].map({1: 'yes', 0: 'no'})

# проверим уникальные значения
print(df_train['subscribed_dep'].unique())

['no' 'yes']
```

Уникальные значения категориальных признаков

```
In [24]: def inspect_object_columns_two(df1, df2, name1="DataFrame1", name2="DataFrame2"):
    obj1 = set(df1.select_dtypes(include="object"))
    obj2 = set(df2.select_dtypes(include="object"))

    for col in sorted(obj1 | obj2):
        print("-" * 60)
        print(f"♦ {col}")
        if col in obj1:
            u1 = set(df1[col].dropna().unique())
            print(f" {name1}: {sorted(u1)}")
        else:
            u1 = None
            print(f" {name1}: ❌ нет колонки")

        if col in obj2:
            u2 = set(df2[col].dropna().unique())
            print(f" {name2}: {sorted(u2)}")
        else:
            u2 = None
            print(f" {name2}: ❌ нет колонки")

    # 🔍 сравнение множеств
    if u1 is not None and u2 is not None:
        if u1 == u2:
            print(" ✅ Совпадают уникальные значения")
        else:
            print(" ⚠️ Различия:")
            print(f" Только в {name1}: {sorted(u1 - u2)}")
            print(f" Только в {name2}: {sorted(u2 - u1)}")
```

```
In [25]: inspect_object_columns_two(df_train, df_test, name1="df_train", name2="df_test")
```

```
-----
♦ contact
df_train: ['cellular', 'telephone', 'unknown']
df_test: ['cellular', 'telephone', 'unknown']
✅ Совпадают уникальные значения
-----

♦ default
df_train: ['no', 'yes']
df_test: ['no', 'yes']
✅ Совпадают уникальные значения
-----

♦ education
df_train: ['primary', 'secondary', 'tertiary', 'unknown']
df_test: ['primary', 'secondary', 'tertiary', 'unknown']
✅ Совпадают уникальные значения
-----

♦ housing
df_train: ['no', 'yes']
df_test: ['no', 'yes']
✅ Совпадают уникальные значения
-----

♦ job
df_train: ['admin.', 'blue-collar', 'entrepreneur', 'housemaid', 'management', 'retired', 'self-employed', 'services', 'student', 'technician', 'unemployed', 'unknown']
df_test: ['admin.', 'blue-collar', 'entrepreneur', 'housemaid', 'management', 'retired', 'self-employed', 'services', 'student', 'technician', 'unemployed', 'unknown']
✅ Совпадают уникальные значения
-----

♦ loan
df_train: ['no', 'yes']
df_test: ['no', 'yes']
✅ Совпадают уникальные значения
-----

♦ marital
df_train: ['divorced', 'married', 'single']
df_test: ['divorced', 'married', 'single']
✅ Совпадают уникальные значения
-----

♦ month
df_train: ['apr', 'aug', 'dec', 'feb', 'jan', 'jul', 'jun', 'mar', 'may', 'nov', 'oct', 'sep']
df_test: ['apr', 'aug', 'dec', 'feb', 'jan', 'jul', 'jun', 'mar', 'may', 'nov', 'oct', 'sep']
✅ Совпадают уникальные значения
-----

♦ poutcome
df_train: ['failure', 'other', 'success', 'unknown']
df_test: ['failure', 'other', 'success', 'unknown']
✅ Совпадают уникальные значения
-----

♦ subscribed_dep
df_train: ['no', 'yes']
df_test: ❌ нет колонки
```

```
In [26]: def cols_with_unknown(df, name="DataFrame"):
        cols = [col for col in df.columns if (df[col] == "unknown").any()]
        print(f"📄 Столбцы с 'unknown' в {name}: {cols}")
        return cols

cols_with_unknown(df_train, "train")
cols_with_unknown(df_test, "test")

📄 Столбцы с 'unknown' в train: ['job', 'education', 'contact', 'poutcome']
📄 Столбцы с 'unknown' в test: ['job', 'education', 'contact', 'poutcome']
```

```
Out[26]: ['job', 'education', 'contact', 'poutcome']
```

Проверка на пропуски

```
In [27]: def analyze_missing_values(df):
        temp = df.copy() # 📄 Создаем копию входного DataFrame
        missing = pd.DataFrame({
            'Кол-во пропусков': temp.isnull().sum(),
            'Доля пропусков': temp.isnull().mean().round(4)
        })
        # 📊 Сортируем столбцы по количеству пропусков (по убыванию)
        missing = missing.sort_values(by='Кол-во пропусков', ascending=False)

        # 🎨 Визуализация пропусков
        return missing.style.background_gradient(cmap='coolwarm')
```

```
In [28]: analyze_missing_values(df_train)
```

Out[28]:

	Кол-во пропусков	Доля пропусков
id	0	0.000000
day	0	0.000000
y	0	0.000000
poutcome	0	0.000000
previous	0	0.000000
pdays	0	0.000000
campaign	0	0.000000
duration	0	0.000000
month	0	0.000000
contact	0	0.000000
age	0	0.000000
loan	0	0.000000
housing	0	0.000000
balance	0	0.000000
default	0	0.000000
education	0	0.000000
marital	0	0.000000
job	0	0.000000
subscribed_dep	0	0.000000

In [29]:

```
analyze_missing_values(df_test)
```

Out[29]:

	Кол-во пропусков	Доля пропусков
id	0	0.000000
contact	0	0.000000
previous	0	0.000000
pdays	0	0.000000
campaign	0	0.000000
duration	0	0.000000
month	0	0.000000
day	0	0.000000
loan	0	0.000000
age	0	0.000000
housing	0	0.000000
balance	0	0.000000
default	0	0.000000
education	0	0.000000
marital	0	0.000000
job	0	0.000000
poutcome	0	0.000000

✅ Пропусков нет (все признаки заполнены). Это подтверждается выводами из Шага 1(корректность синтетически сгенерированного датасета)

Проверка на дубликаты

In [30]:

```
print("Train duplicates:", df_train.duplicated().sum())
print("Test duplicates:", df_test.duplicated().sum())
```

Train duplicates: 0
Test duplicates: 0

Явных дубликатов строк

```
In [31]: # Проверка неявных дубликатов в train
dup_train = df_train.drop(columns=['id']).duplicated().sum()
print(f"Неявные дубликаты в train: {dup_train}")

# Проверка неявных дубликатов в test
dup_test = df_test.drop(columns=['id']).duplicated().sum()
print(f"Неявные дубликаты в test: {dup_test}")

# Если нужно сами строки:
duplicates_train_rows = df_train[df_train.drop(columns=['id']).duplicated()]
display(duplicates_train_rows.head())

duplicates_test_rows = df_test[df_test.drop(columns=['id']).duplicated()]
display(duplicates_test_rows.head())
```

Неявные дубликаты в train: 0
Неявные дубликаты в test: 0

id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome
id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome

неявные дубликаты (одинаковые строки по всем признакам, кроме id) отсутствуют

Выводы по шагу 2

Данные в целом чистые, явных пропусков нет, но есть «псевдопропуски» (unknown).

- В категориальных признаках 'job', 'education', 'contact', 'poutcome) встречается значение "unknown".
 - это не пропуск в техническом смысле, нужно оставить как отдельную категорию

При этом категориальные признаки содержат идентичные наборы значений, что упрощает кодирование

Особенности категориальных признаков:

- Высокая кардинальность (12 значений): job, month
 - job: 12 категорий (включая "unknown")
 - month: 12 месяцев (полный годовой цикл)
- Низкая кардинальность (2-4 значения): default, housing, loan, marital, contact, education, poutcome
 - education: 4 категории

Типы признаков корректны и согласованы между df_train/df_test.

Дубликатов нет

Требуется внимательная работа с признаками pdays, balance, duration.

[* к содержанию](#)

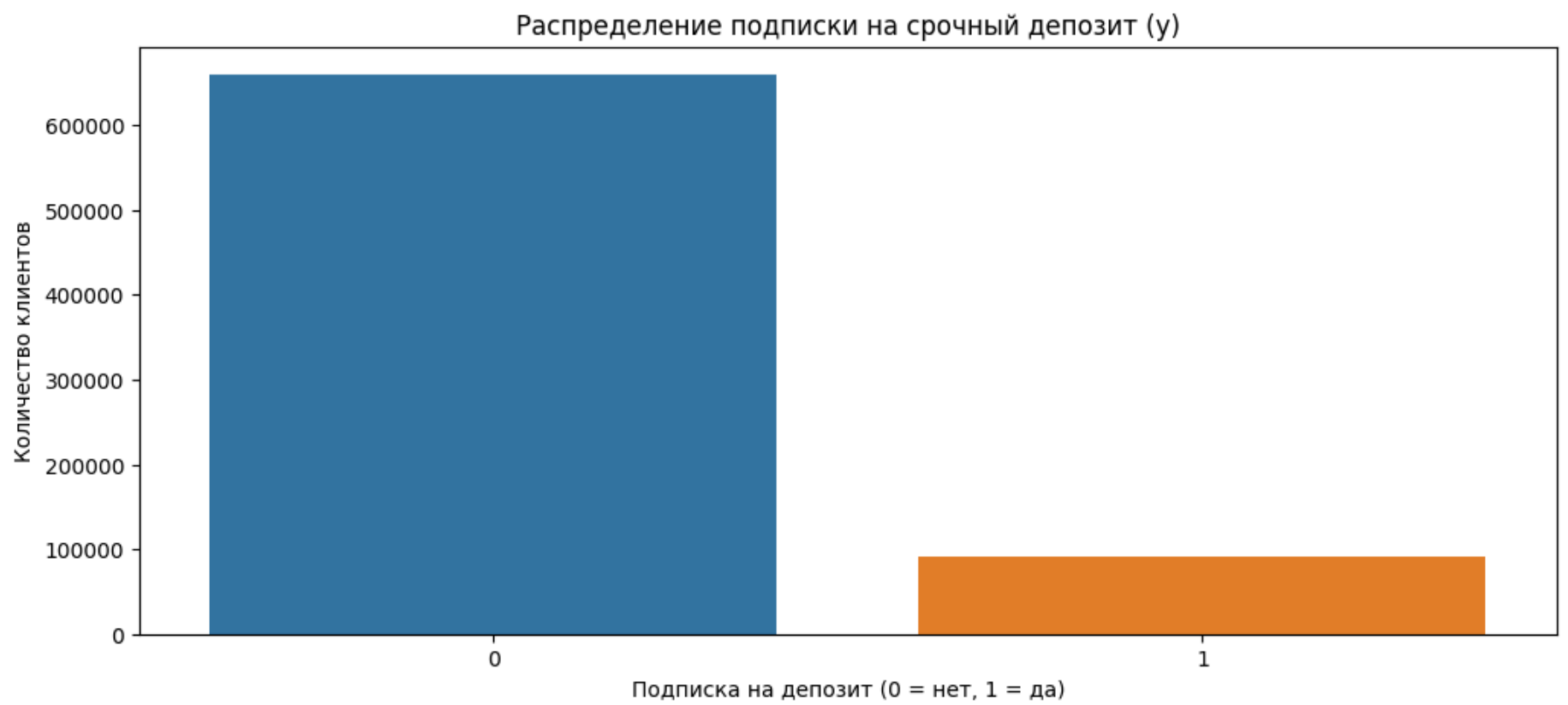
Шаг 3: Исследовательский анализ данных (EDA)

Анализ целевой переменной y

```
In [32]: # Визуализация распределения целевой переменной
plt.figure(figsize=(12, 5))
sns.countplot(data=df_train, x='y')
plt.title('Распределение подписки на срочный депозит (y)')
plt.xlabel('Подписка на депозит (0 = нет, 1 = да)')
plt.ylabel('Количество клиентов')
plt.show()

# Абсолютные значения
subscription_counts = df_train['y'].value_counts()
print("Абсолютные значения подписки на депозит:")
print(subscription_counts)

# Доли (нормализованные значения)
subscription_rate = df_train['y'].value_counts(normalize=True)
print("\nДоли подписки на депозит:")
print(subscription_rate)
```

Абсолютные значения подписки на депозит:

```
0    659512
1     90488
Name: y, dtype: int64
```

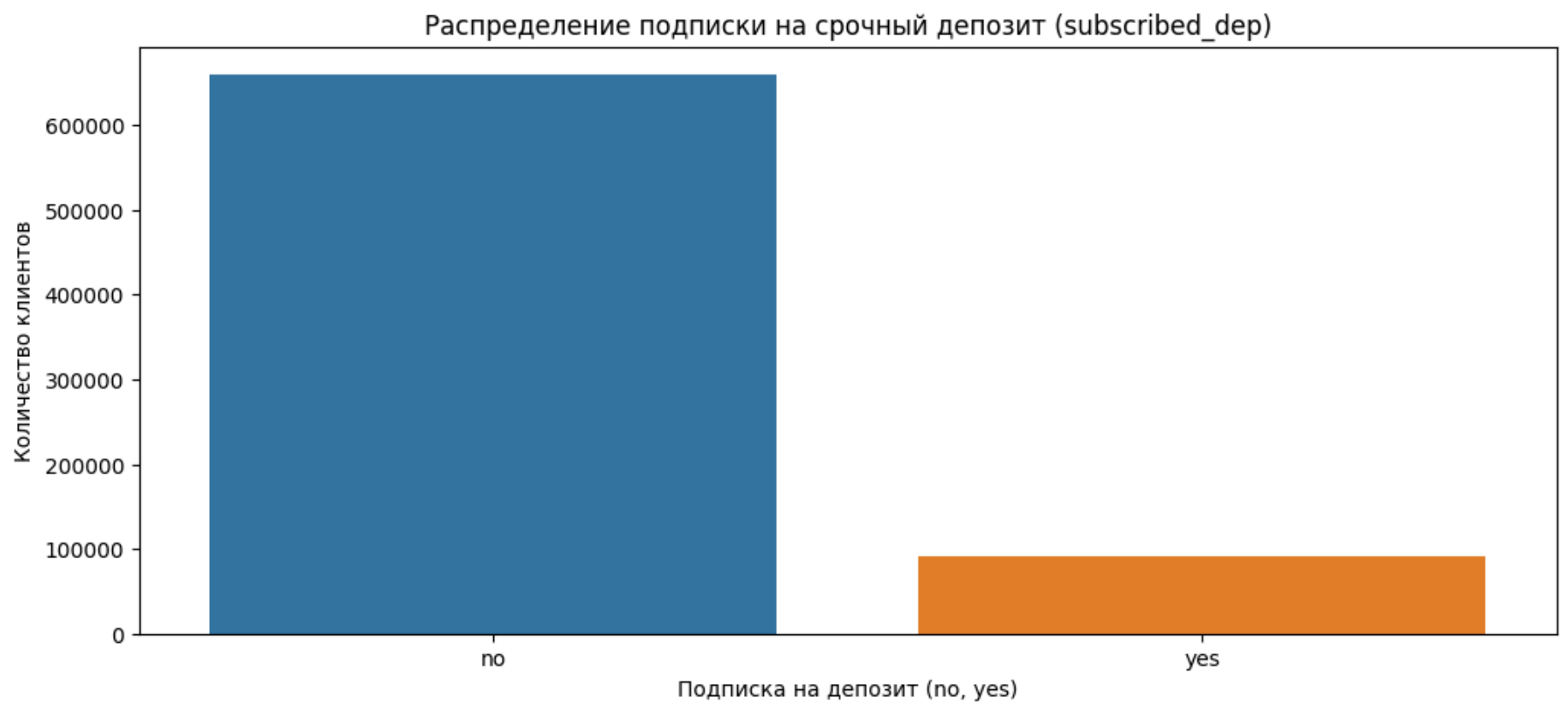
Доли подписки на депозит:

```
0    0.879349
1    0.120651
Name: y, dtype: float64
```

```
In [33]: # Визуализация распределения целевой переменной
plt.figure(figsize=(12, 5))
sns.countplot(data=df_train, x='subscribed_dep')
plt.title('Распределение подписки на срочный депозит (subscribed_dep)')
plt.xlabel('Подписка на депозит (no, yes)')
plt.ylabel('Количество клиентов')
plt.show()

# Абсолютные значения
subscription_counts = df_train['subscribed_dep'].value_counts()
print("Абсолютные значения подписки на депозит:")
print(subscription_counts)

# Доли (нормализованные значения)
subscription_rate = df_train['subscribed_dep'].value_counts(normalize=True)
print("\nДоли подписки на депозит:")
print(subscription_rate)
```



Абсолютные значения подписки на депозит:

```
no    659512
yes    90488
Name: subscribed_dep, dtype: int64
```

Доли подписки на депозит:

```
no    0.879349
yes    0.120651
Name: subscribed_dep, dtype: float64
```


Такой дисбаланс требует использования:

- стратифицированной кросс-валидации (StratifiedKFold),
- метрики ROC-AUC

Признаки для анализа

Для каждого типа признака построим графики

- при это будет сравнивать согласованность распределения данных между датасетами df_train и df_test
- но дальнейший анализ будет с упором на df_train, поскольку на нем будут учиться модели машинного обучения

- Количественные признаки датасета data:

```
In [34]: # Количественные переменные (обычно числовые типы)
numerical = df_train.select_dtypes(include=['number']).columns.tolist()

print("Количественные переменные в датасета df_train:", numerical)
```

Количественные переменные в датасета df_train: ['id', 'age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previous', 'y']

```
In [35]: # Количественные переменные (обычно числовые типы)
numerical = df_test.select_dtypes(include=['number']).columns.tolist()

print("Количественные переменные в датасета df_test:", numerical)
```

Количественные переменные в датасета df_test: ['id', 'age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previous']

- Категориальные признаки датасета data:

```
In [36]: # Категориальные переменные (обычно тип object или category)
categorical = df_train.select_dtypes(include=['object', 'category']).columns.tolist()

print("Категориальные переменные в датасета df_train:", categorical)
```

Категориальные переменные в датасета df_train: ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome', 'subscribed_dep']

```
In [37]: # Категориальные переменные (обычно тип object или category)
categorical = df_test.select_dtypes(include=['object', 'category']).columns.tolist()

print("Категориальные переменные в датасета df_test:", categorical)
```

Категориальные переменные в датасета df_test: ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome']

Распределение количественных признаков для двух датасетов

```
In [38]: df_train.describe().T
```

	count	mean	std	min	25%	50%	75%	max
id	750000.0	374999.500000	216506.495284	0.0	187499.75	374999.5	562499.25	749999.0
age	750000.0	40.926395	10.098829	18.0	33.00	39.0	48.00	95.0
balance	750000.0	1204.067397	2836.096759	-8019.0	0.00	634.0	1390.00	99717.0
day	750000.0	16.117209	8.250832	1.0	9.00	17.0	21.00	31.0
duration	750000.0	256.229144	272.555662	1.0	91.00	133.0	361.00	4918.0
campaign	750000.0	2.577008	2.718514	1.0	1.00	2.0	3.00	63.0
pdays	750000.0	22.412733	77.319998	-1.0	-1.00	-1.0	-1.00	871.0
previous	750000.0	0.298545	1.335926	0.0	0.00	0.0	0.00	200.0
y	750000.0	0.120651	0.325721	0.0	0.00	0.0	0.00	1.0

```
In [39]: df_test.describe().T
```

Out[39]:

	count	mean	std	min	25%	50%	75%	max
id	250000.0	874999.500000	72168.927986	750000.0	812499.75	874999.5	937499.25	999999.0
age	250000.0	40.932332	10.081613	18.0	33.00	39.0	48.00	95.0
balance	250000.0	1197.426352	2741.520699	-8019.0	0.00	631.0	1389.00	98517.0
day	250000.0	16.116068	8.258509	1.0	9.00	17.0	21.00	31.0
duration	250000.0	255.342260	271.404326	3.0	91.00	133.0	353.00	4918.0
campaign	250000.0	2.573548	2.709661	1.0	1.00	2.0	3.00	58.0
pdays	250000.0	22.280028	76.915879	-1.0	-1.00	-1.0	-1.00	871.0
previous	250000.0	0.303728	1.384574	0.0	0.00	0.0	0.00	150.0

In [40]:

```
def compare_box_and_hist(df_train, df_test, feature='rhythm_score', title=None, bins=60):
    # Удаляем пропуски
    train = df_train[feature].dropna()
    test = df_test[feature].dropna()

    # Общая статистика
    print(f"\n📊 Статистика по признаку '{feature}':")
    print("df_train:\n", train.describe())
    print("\ndf_test:\n", test.describe())

    # Заголовок
    if title is None:
        title = f'Сравнение распределения признака "{feature}" между train и test'

    # Сетка графиков
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))
    fig.suptitle(title, fontsize=20, fontweight='bold') # увеличил шрифт заголовка

    # Boxplot
    sns.boxplot(
        data=pd.concat([train.to_frame().assign(dataset='df_train'),
                        test.to_frame().assign(dataset='df_test')]),
        x='dataset', y=feature, ax=axes[0, 0], palette='Set2'
    )
    axes[0, 0].set_title('Boxplot (df_train vs df_test)', fontsize=18)
    axes[0, 0].set_xlabel('Датасет', fontsize=16)
    axes[0, 0].set_ylabel(feature, fontsize=16)
    axes[0, 0].tick_params(axis='x', labelsize=14)
    axes[0, 0].tick_params(axis='y', labelsize=14)

    axes[0, 1].axis('off') # пустая ячейка

    # Гистограммы: частота и плотность
    stats = [('count', 'Частота'), ('density', 'Плотность')]
    colors = [('skyblue', 'salmon'), ('steelblue', 'tomato')]

    for ax, (stat, ylabel), (c1, c2) in zip(axes[1], stats, colors):
        sns.histplot(train, bins=bins, stat=stat, color=c1, label='df_train',
                     ax=ax, alpha=0.6, edgecolor='gray')
        sns.histplot(test, bins=bins, stat=stat, color=c2, label='df_test',
                     ax=ax, alpha=0.6, edgecolor='gray')
        ax.set_title(f'Гистограмма ({ylabel.lower()})', fontsize=18)
        ax.set_xlabel(feature, fontsize=16)
        ax.set_ylabel(ylabel, fontsize=16)
        ax.tick_params(axis='x', labelsize=14)
        ax.tick_params(axis='y', labelsize=14)
        ax.legend(fontsize=12)

    plt.tight_layout(rect=[0, 0, 1, 0.95])
    plt.show()
```

age

- **age** - возраст клиента

In [41]:

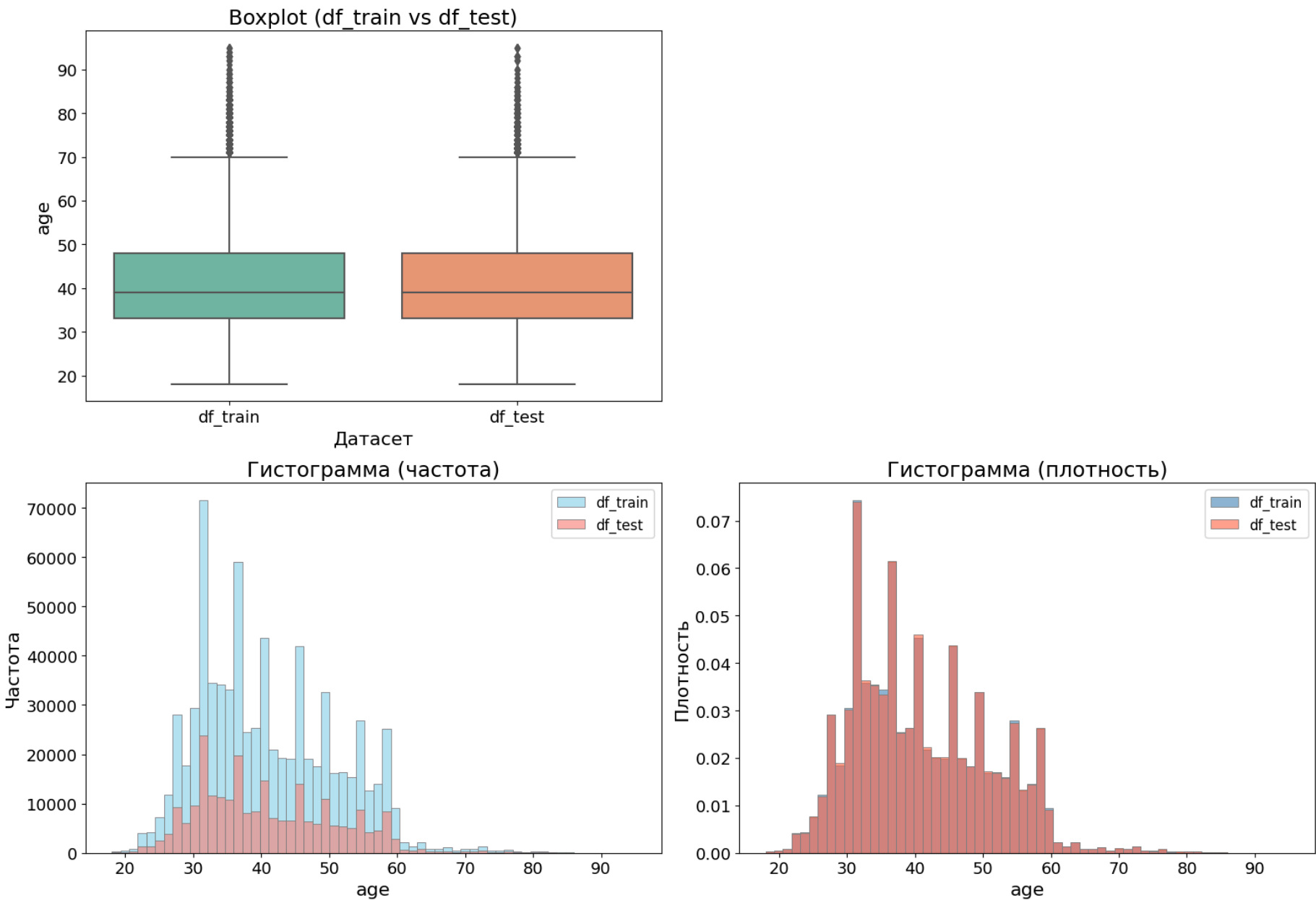
```
compare_box_and_hist(df_train, df_test, feature='age',
                     title="Сравнение распределения возраста клиентов")
```

📊 Статистика по признаку 'age':

df_train:
count 750000.000000
mean 40.926395
std 10.098829
min 18.000000
25% 33.000000
50% 39.000000
75% 48.000000
max 95.000000
Name: age, dtype: float64

df_test:
count 250000.000000
mean 40.932332
std 10.081613
min 18.000000
25% 33.000000
50% 39.000000
75% 48.000000
max 95.000000
Name: age, dtype: float64

Сравнение распределения возраста клиентов



```
In [42]: def compare_distributions(df1, df2, feature='age', label1='df_train', label2='df_test', alpha=0.05):
# Удаляем пропуски
x1 = df1[feature].dropna()
x2 = df2[feature].dropna()

print(f"\n📊 Сравнение распределения признака '{feature}' между {label1} и {label2}")

# U-критерий Манна-Уитни
u_stat, u_pval = mannwhitneyu(x1, x2, alternative='two-sided')
print(f"\n📊 Mann-Whitney U-тест:")
print(f"U-статистика: {u_stat:.2f}, p-value: {u_pval:.4f}")
if u_pval < alpha:
    print("✅ Различия по ранговому распределению статистически значимы.")
else:
    print("⚠️ Статистически значимых различий по ранговому распределению не обнаружено.")

# Критерий Колмогорова-Смирнова
ks_stat, ks_pval = ks_2samp(x1, x2)
print(f"\n📊 Kolmogorov-Smirnov тест:")
print(f"KS-статистика: {ks_stat:.4f}, p-value: {ks_pval:.4f}")
if ks_pval < alpha:
    print("✅ Форма распределений статистически различается.")
else:
    print("⚠️ Статистически значимых различий в форме распределений не обнаружено.")
```

```
# Сравнение средних
mean1 = x1.mean()
mean2 = x2.mean()
print(f"\n📊 Средние значения: {label1} = {mean1:.2f}, {label2} = {mean2:.2f}")
```

In [43]: `compare_distributions(df_train, df_test, feature='age')`

📊 Сравнение распределения признака 'age' между df_train и df_test

📈 Mann-Whitney U-тест:

U-статистика: 93692473861.50, p-value: 0.6452

⚠️ Статистически значимых различий по ранговому распределению не обнаружено.

📈 Kolmogorov-Smirnov тест:

KS-статистика: 0.0020, p-value: 0.4197

⚠️ Статистически значимых различий в форме распределений не обнаружено.

📊 Средние значения: df_train = 40.93, df_test = 40.93

- Бинирование возраста: age_group: молодые (<30), средний возраст (30-50), пожилые (>50)

In [44]:

```
def insert_before_second_last(df, col_name, values):
    second_last_idx = df.shape[1] - 2
    df.insert(second_last_idx, col_name, values)
    return df
```

In [45]:

```
# создаём категориальный признак
age_group = pd.cut(
    df_train['age'],
    bins=[0, 30, 50, 100],
    labels=['young', 'middle', 'senior']
)

# вставляем его перед предпоследней колонкой
df_train = insert_before_second_last(df_train, 'age_group', age_group)
print(df_train['age_group'].value_counts())
```

```
middle    496026
senior    150199
young     103775
Name: age_group, dtype: int64
```

Возраст (age): распределён неравномерно; основная группа клиентов 30–50 лет.

balance

- **balance** - средний годовой баланс в евро

In [46]: `compare_box_and_hist(df_train, df_test, feature='balance', title="Сравнение распределения среднего годового баланса клиентов")`

📊 Статистика по признаку 'balance':

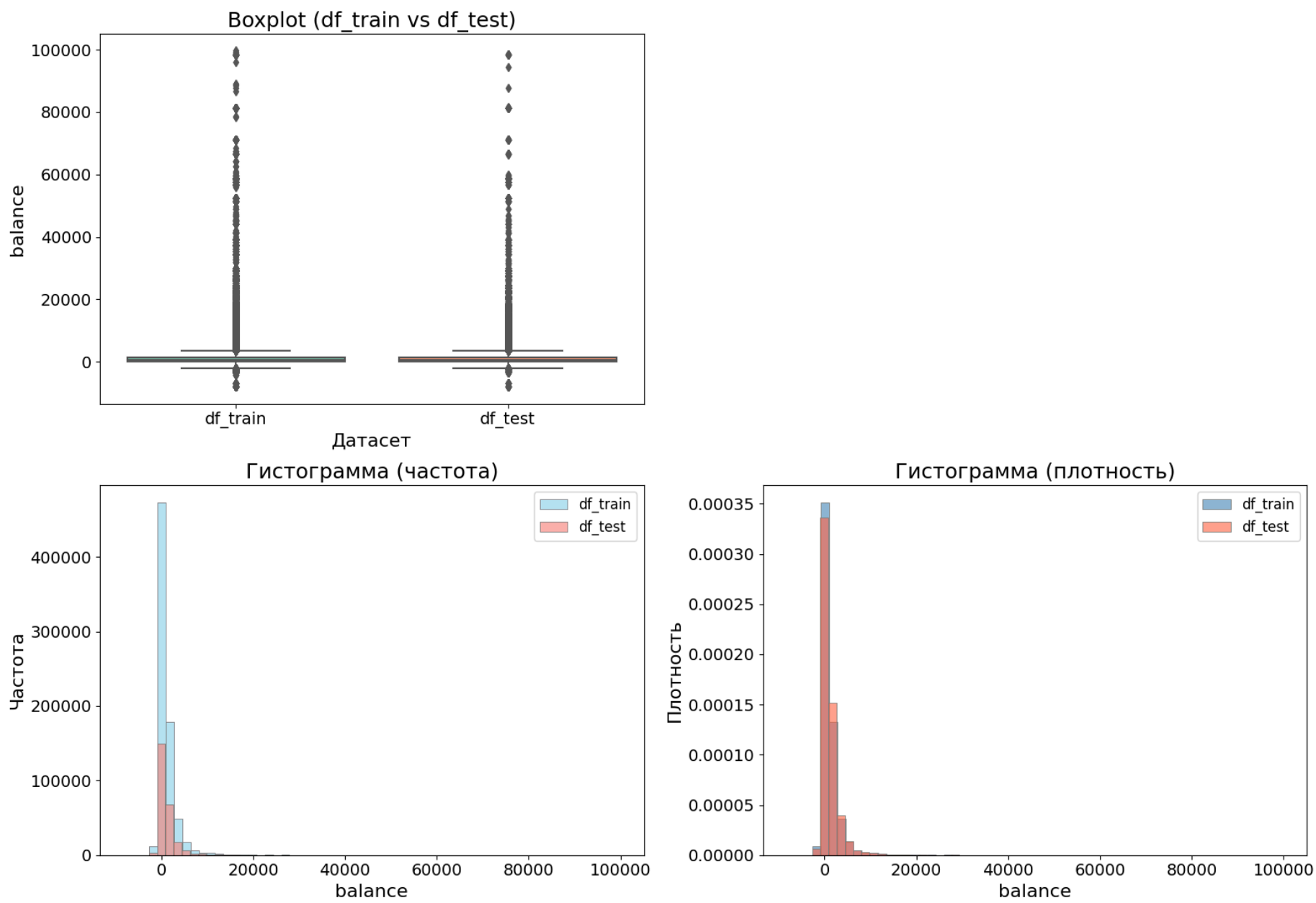
df_train:

```
count    750000.000000
mean      1204.067397
std       2836.096759
min       -8019.000000
25%         0.000000
50%        634.000000
75%       1390.000000
max       99717.000000
Name: balance, dtype: float64
```

df_test:

```
count    250000.000000
mean      1197.426352
std       2741.520699
min       -8019.000000
25%         0.000000
50%        631.000000
75%       1389.000000
max       98517.000000
Name: balance, dtype: float64
```

Сравнение распределения среднего годового баланса клиентов



```
In [47]: # Сортировка по значению balance (от меньшего к большему)
df_train['balance'].value_counts().sort_index()
```

```
Out[47]: -8019      75
          -8016       1
          -7048       1
          -6857        2
          -6848        1
          ..
          98417       37
          98418        2
          98942        1
          99218        1
          99717        1
          Name: balance, Length: 8217, dtype: int64
```

```
In [48]: compare_distributions(df_train, df_test, feature='balance')
```

📊 Сравнение распределения признака 'balance' между df_train и df_test

📊 Mann-Whitney U-тест:
U-статистика: 93789714943.00, p-value: 0.7505
⚠️ Статистически значимых различий по ранговому распределению не обнаружено.

📊 Kolmogorov-Smirnov тест:
KS-статистика: 0.0016, p-value: 0.7273
⚠️ Статистически значимых различий в форме распределений не обнаружено.

📊 Средние значения: df_train = 1204.07, df_test = 1197.43

balance: распределение с длинным правым хвостом, есть отрицательные значения.

day

- **day** - день последнего контакта в месяце (числовой, 1-31)

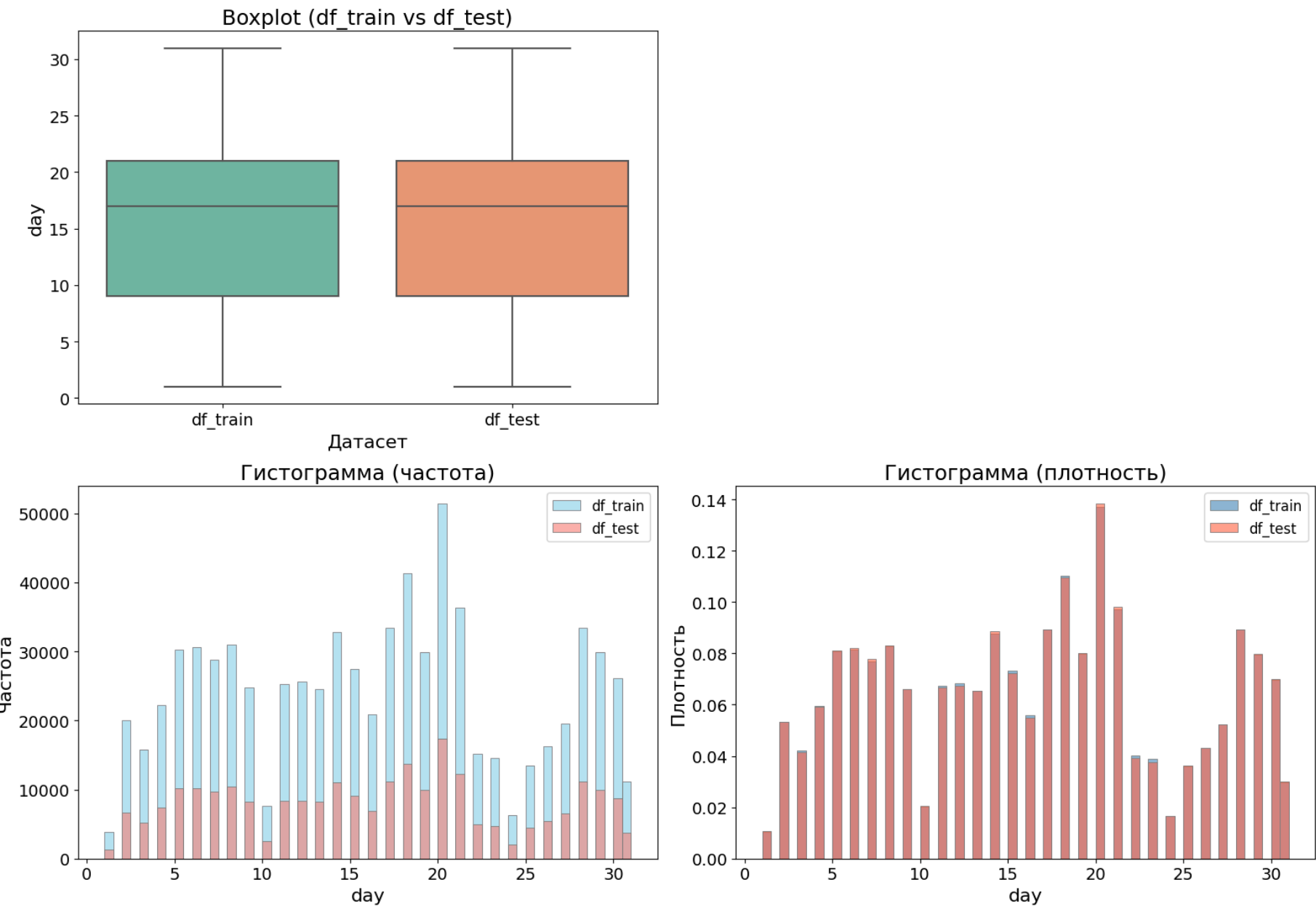
```
In [49]: compare_box_and_hist(df_train, df_test, feature='day',
                               title="Сравнение распределения дня последнего контакта клиентов")
```

📊 Статистика по признаку 'day':

df_train:
count 750000.000000
mean 16.117209
std 8.250832
min 1.000000
25% 9.000000
50% 17.000000
75% 21.000000
max 31.000000
Name: day, dtype: float64

df_test:
count 250000.000000
mean 16.116068
std 8.258509
min 1.000000
25% 9.000000
50% 17.000000
75% 21.000000
max 31.000000
Name: day, dtype: float64

Сравнение распределения дня последнего контакта клиентов



```
In [50]: compare_distributions(df_train, df_test, feature='day')
```

📊 Сравнение распределения признака 'day' между df_train и df_test

📊 Mann-Whitney U-тест:
U-статистика: 93756142318.00, p-value: 0.9608
⚠️ Статистически значимых различий по ранговому распределению не обнаружено.

📊 Kolmogorov-Smirnov тест:
KS-статистика: 0.0010, p-value: 0.9915
⚠️ Статистически значимых различий в форме распределений не обнаружено.

📊 Средние значения: df_train = 16.12, df_test = 16.12

duration

- **duration** - продолжительность последнего контакта в секундах

```
In [51]: compare_box_and_hist(df_train, df_test, feature='duration',  
title="Сравнение распределения продолжительности последнего контакта в секундах клиентов")
```


📊 Статистика по признаку 'duration':

df_train:

count	750000.000000
mean	256.229144
std	272.555662
min	1.000000
25%	91.000000
50%	133.000000
75%	361.000000
max	4918.000000

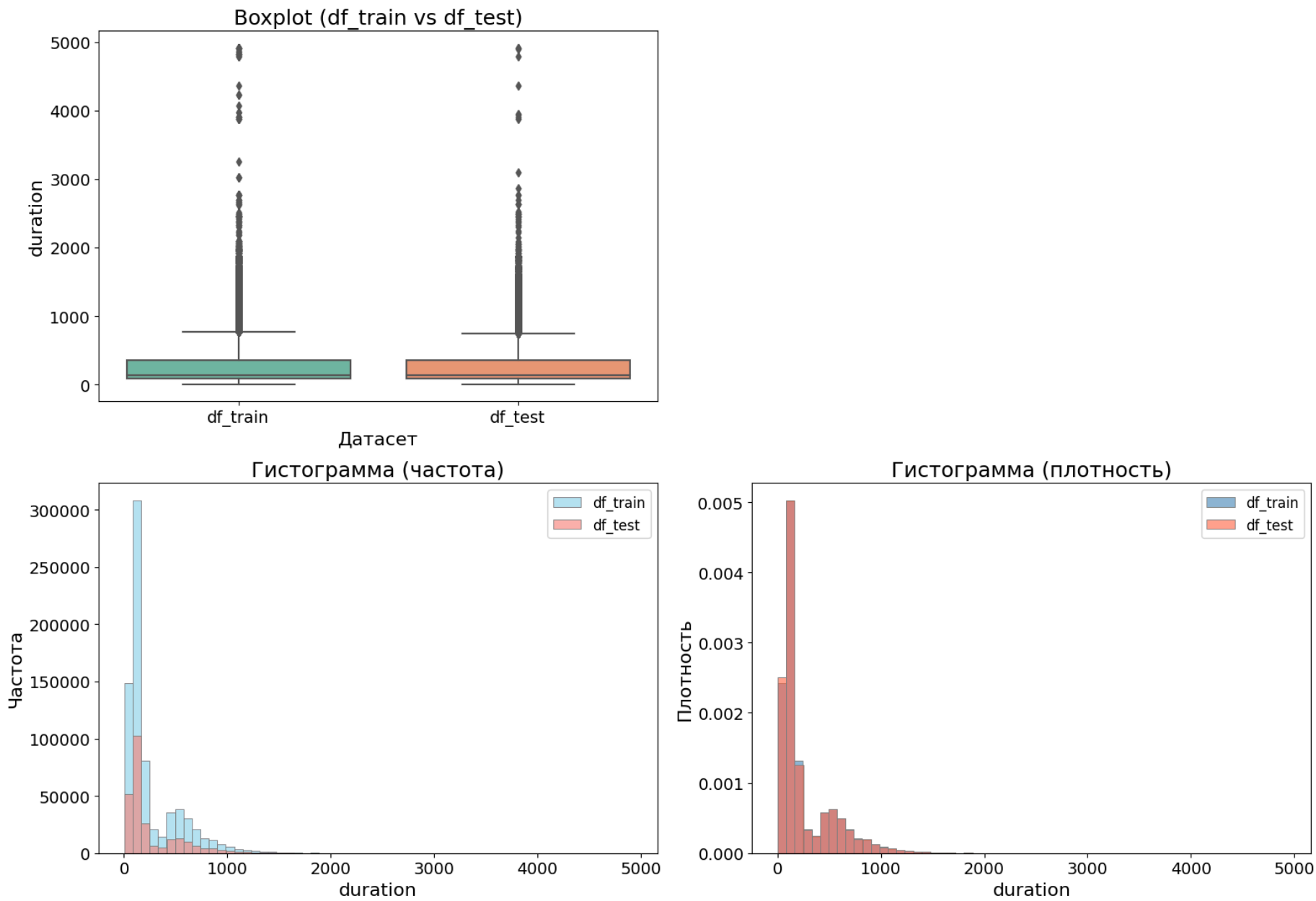
Name: duration, dtype: float64

df_test:

count	250000.000000
mean	255.342260
std	271.404326
min	3.000000
25%	91.000000
50%	133.000000
75%	353.000000
max	4918.000000

Name: duration, dtype: float64

Сравнение распределения продолжительности последнего контакта в секундах клиентов



```
In [52]: compare_distributions(df_train, df_test, feature='duration')
```

📊 Сравнение распределения признака 'duration' между df_train и df_test

📊 Mann-Whitney U-тест:

U-статистика: 93866618835.50, p-value: 0.3508

⚠️ Статистически значимых различий по ранговому распределению не обнаружено.

📊 Kolmogorov-Smirnov тест:

KS-статистика: 0.0019, p-value: 0.5070

⚠️ Статистически значимых различий в форме распределений не обнаружено.

📊 Средние значения: df_train = 256.23, df_test = 255.34

campaign

- **campaign** - количество контактов в течение текущей кампании

```
In [53]: compare_box_and_hist(df_train, df_test, feature='campaign',
title="Сравнение распределения количества контактов в течение текущей кампании клиентов")
```


📊 Статистика по признаку 'campaign':

df_train:

count	750000.000000
mean	2.577008
std	2.718514
min	1.000000
25%	1.000000
50%	2.000000
75%	3.000000
max	63.000000

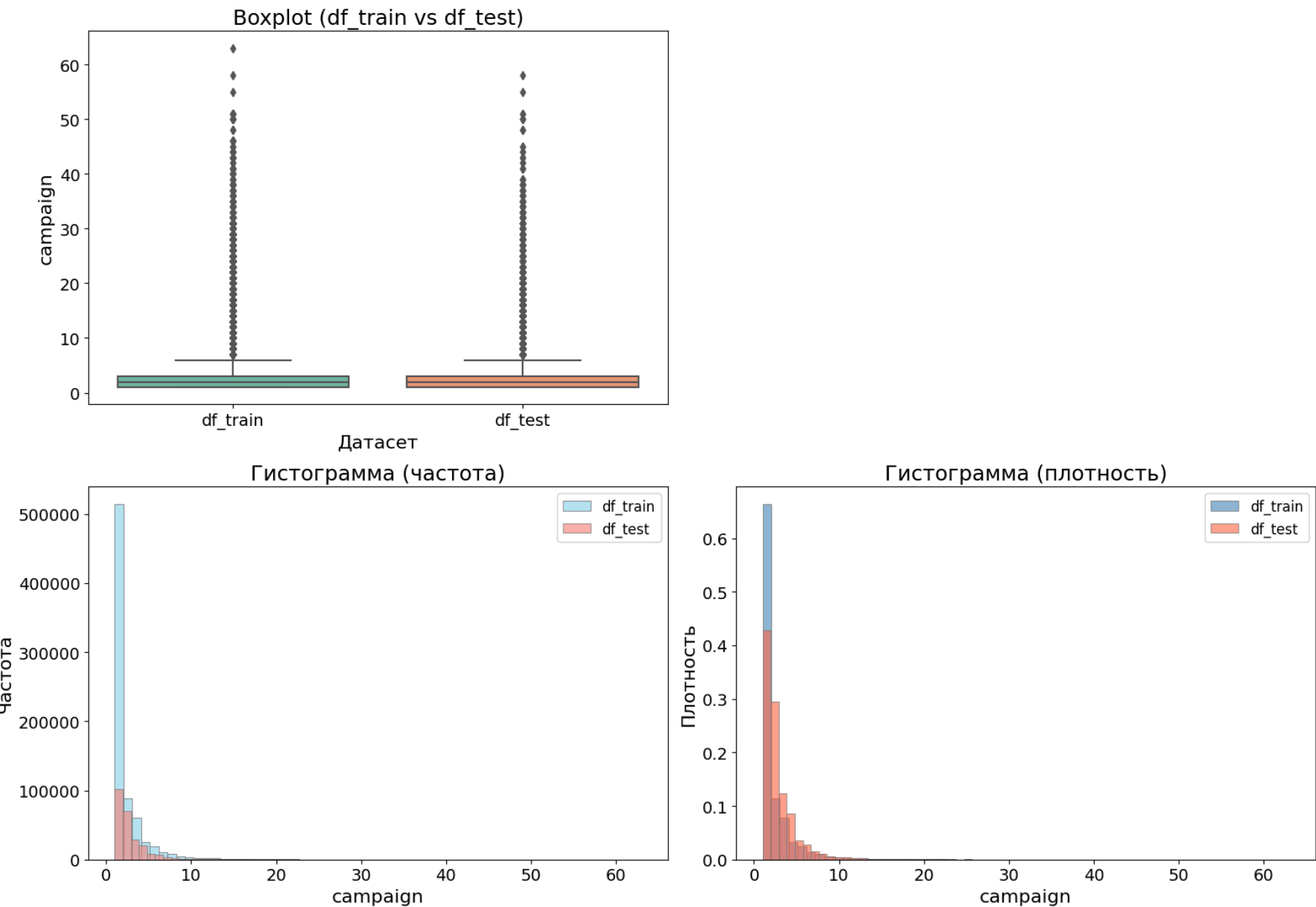
Name: campaign, dtype: float64

df_test:

count	250000.000000
mean	2.573548
std	2.709661
min	1.000000
25%	1.000000
50%	2.000000
75%	3.000000
max	58.000000

Name: campaign, dtype: float64

Сравнение распределения количества контактов в течение текущей кампании клиентов



```
In [54]: compare_distributions(df_train, df_test, feature='campaign')
```

📊 Сравнение распределения признака 'campaign' между df_train и df_test

📊 Mann-Whitney U-тест:

U-статистика: 93850909051.00, p-value: 0.3971

⚠️ Статистически значимых различий по ранговому распределению не обнаружено.

📊 Kolmogorov-Smirnov тест:

KS-статистика: 0.0012, p-value: 0.9637

⚠️ Статистически значимых различий в форме распределений не обнаружено.

📊 Средние значения: df_train = 2.58, df_test = 2.57

pdays

- **pdays** - количество дней с последнего контакта в предыдущей кампании (числовой; -1 означает, что клиент не контактировал ранее)

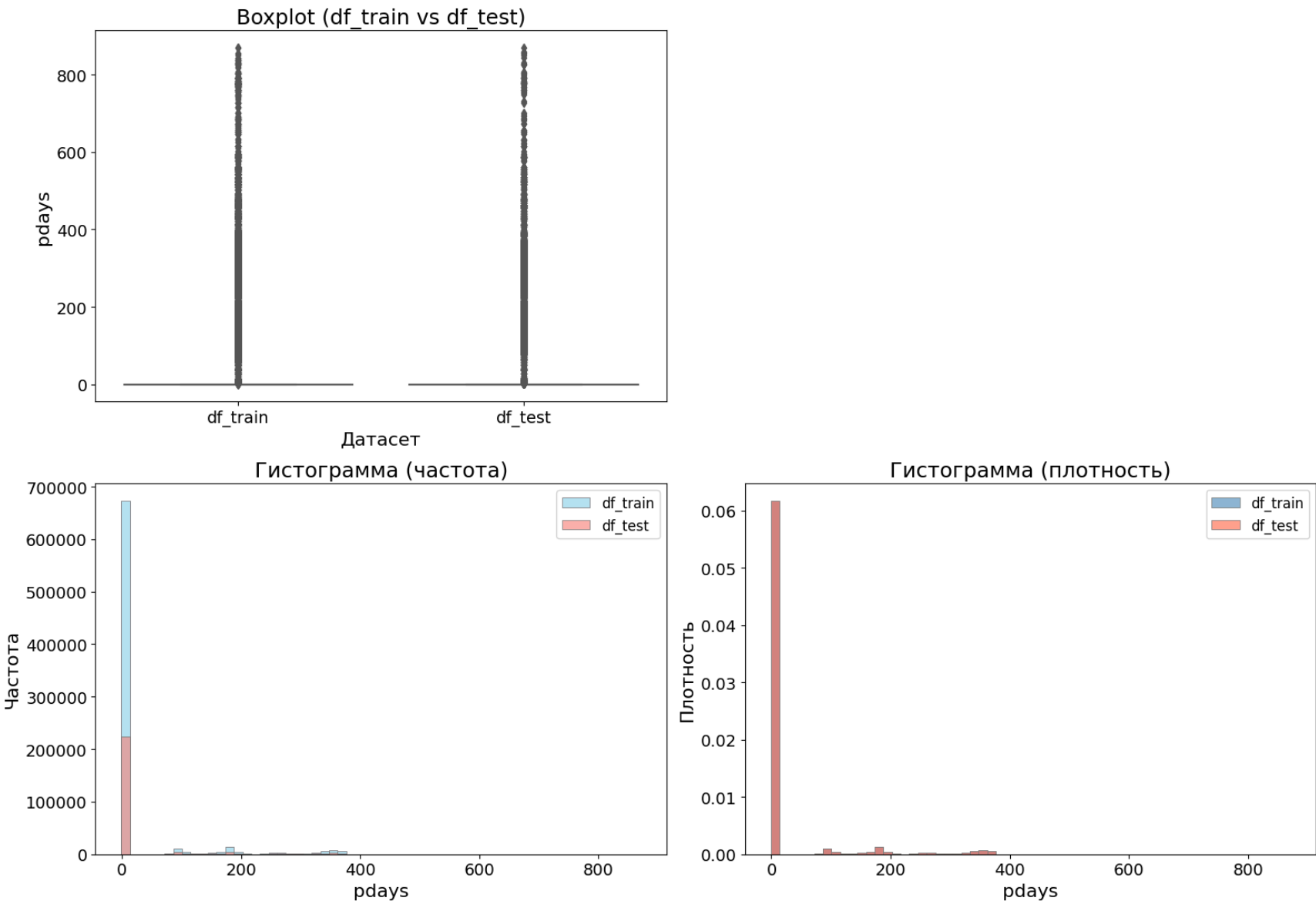
```
In [55]: compare_box_and_hist(df_train, df_test, feature='pdays',
title="Сравнение распределения количества дней с последнего контакта в предыдущей кампании клиентов")
```

📊 Статистика по признаку 'pdays':

df_train:
count 750000.000000
mean 22.412733
std 77.319998
min -1.000000
25% -1.000000
50% -1.000000
75% -1.000000
max 871.000000
Name: pdays, dtype: float64

df_test:
count 250000.000000
mean 22.280028
std 76.915879
min -1.000000
25% -1.000000
50% -1.000000
75% -1.000000
max 871.000000
Name: pdays, dtype: float64

Сравнение распределения количества дней с последнего контакта в предыдущей кампании клиентов



```
In [56]: # Сортировка по значению pdays (от меньшего к большему)
df_train['pdays'].value_counts().sort_index()
```

Out[56]:

-1	672434
0	1
1	45
2	210
3	1
...	
838	4
842	5
850	1
854	2
871	2

Name: pdays, Length: 596, dtype: int64

```
In [57]: compare_distributions(df_train, df_test, feature='pdays')
```

📊 Сравнение распределения признака 'pdays' между df_train и df_test

📊 Mann-Whitney U-тест:

U-статистика: 93747070589.50, p-value: 0.9646

⚠️ Статистически значимых различий по ранговому распределению не обнаружено.

📊 Kolmogorov-Smirnov тест:

KS-статистика: 0.0007, p-value: 1.0000

⚠️ Статистически значимых различий в форме распределений не обнаружено.

📊 Средние значения: df_train = 22.41, df_test = 22.28

- построим также графики для варианта отсутствия -1 в pdays

```
In [58]: df_train_clean = df_train[df_train['pdays'] != -1].copy()
df_test_clean = df_test[df_test['pdays'] != -1].copy()
```

```
In [59]: compare_box_and_hist(df_train_clean, df_test_clean, feature='pdays',
title="Сравнение распределения количества дней с последнего контакта в предыдущей кампании клиентов без учета -1")
```

📊 Статистика по признаку 'pdays':

df_train:

count	77566.000000
mean	225.382049
std	108.892372
min	0.000000
25%	144.000000
50%	187.000000
75%	338.000000
max	871.000000

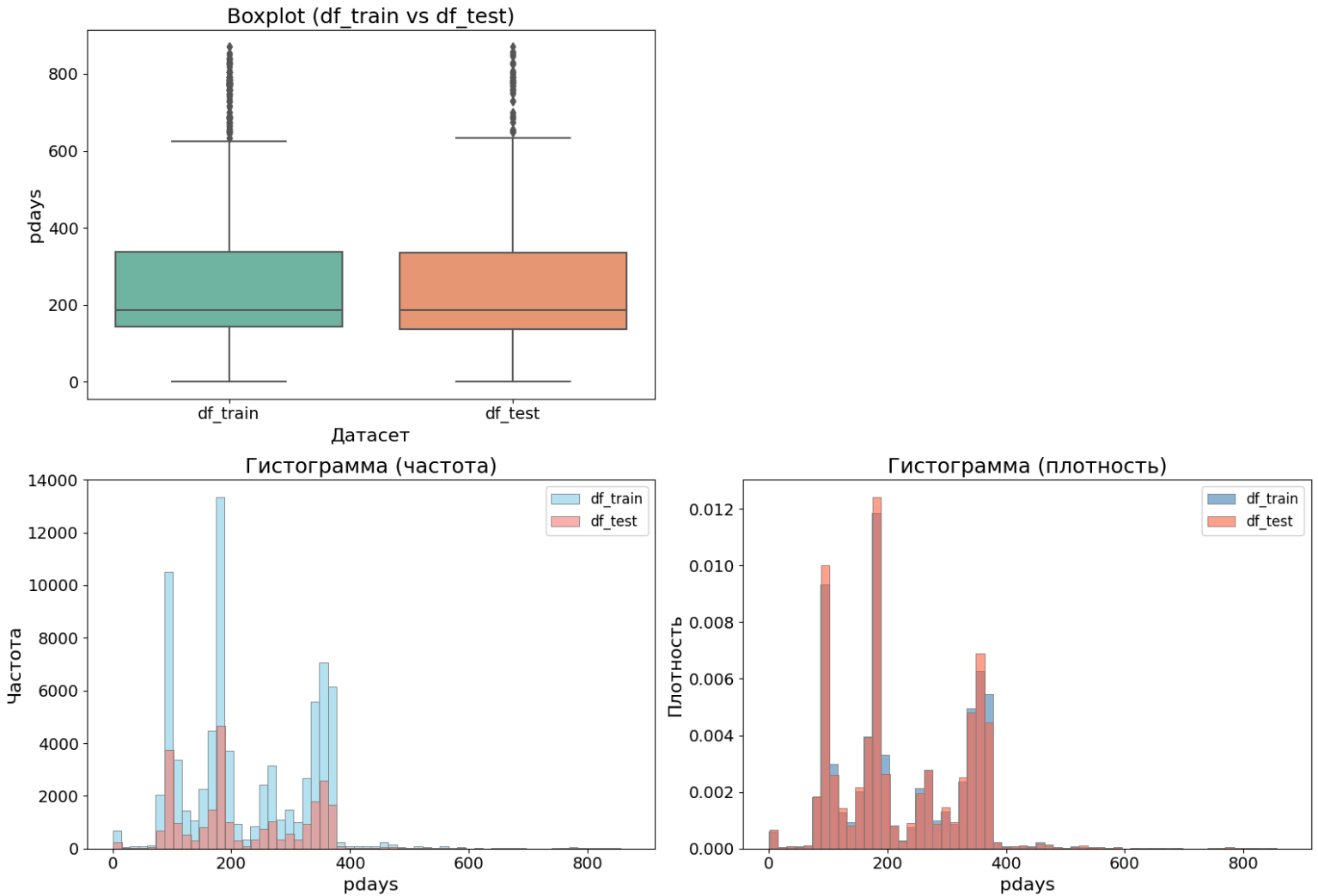
Name: pdays, dtype: float64

df_test:

count	25888.000000
mean	223.814856
std	108.735585
min	1.000000
25%	137.000000
50%	187.000000
75%	336.000000
max	871.000000

Name: pdays, dtype: float64

Сравнение распределения количества дней с последнего контакта в предыдущей кампании клиентов без учета -1



```
In [60]: compare_distributions(df_train_clean, df_test_clean, feature='pdays')
```

 Сравнение распределения признака 'pdays' между df_train и df_test

 Mann-Whitney U-тест:

U-статистика: 1013334893.50, p-value: 0.0251

 Различия по ранговому распределению статистически значимы.

 Kolmogorov-Smirnov тест:

KS-статистика: 0.0071, p-value: 0.2788

 Статистически значимых различий в форме распределений не обнаружено.

 Средние значения: df_train = 225.38, df_test = 223.81

```
In [61]: # Флаг контакта раньше
was_contacted_before = (df_train['pdays'] != -1).map({True: "yes", False: "no"})

# вставляем колонку перед предпоследней
second_last_idx = df_train.shape[1] - 2
df_train.insert(second_last_idx, 'was_contacted_before', was_contacted_before)
print(df_train['was_contacted_before'].value_counts())

no      672434
yes      77566
Name: was_contacted_before, dtype: int64
```

previous

- **previous** - количество контактов до текущей кампании

```
In [62]: compare_box_and_hist(df_train, df_test, feature='previous',
title="Сравнение распределения количества контактов до текущей кампании клиентов")
```

 Статистика по признаку 'previous':

df_train:

count	750000.000000
mean	0.298545
std	1.335926
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	200.000000

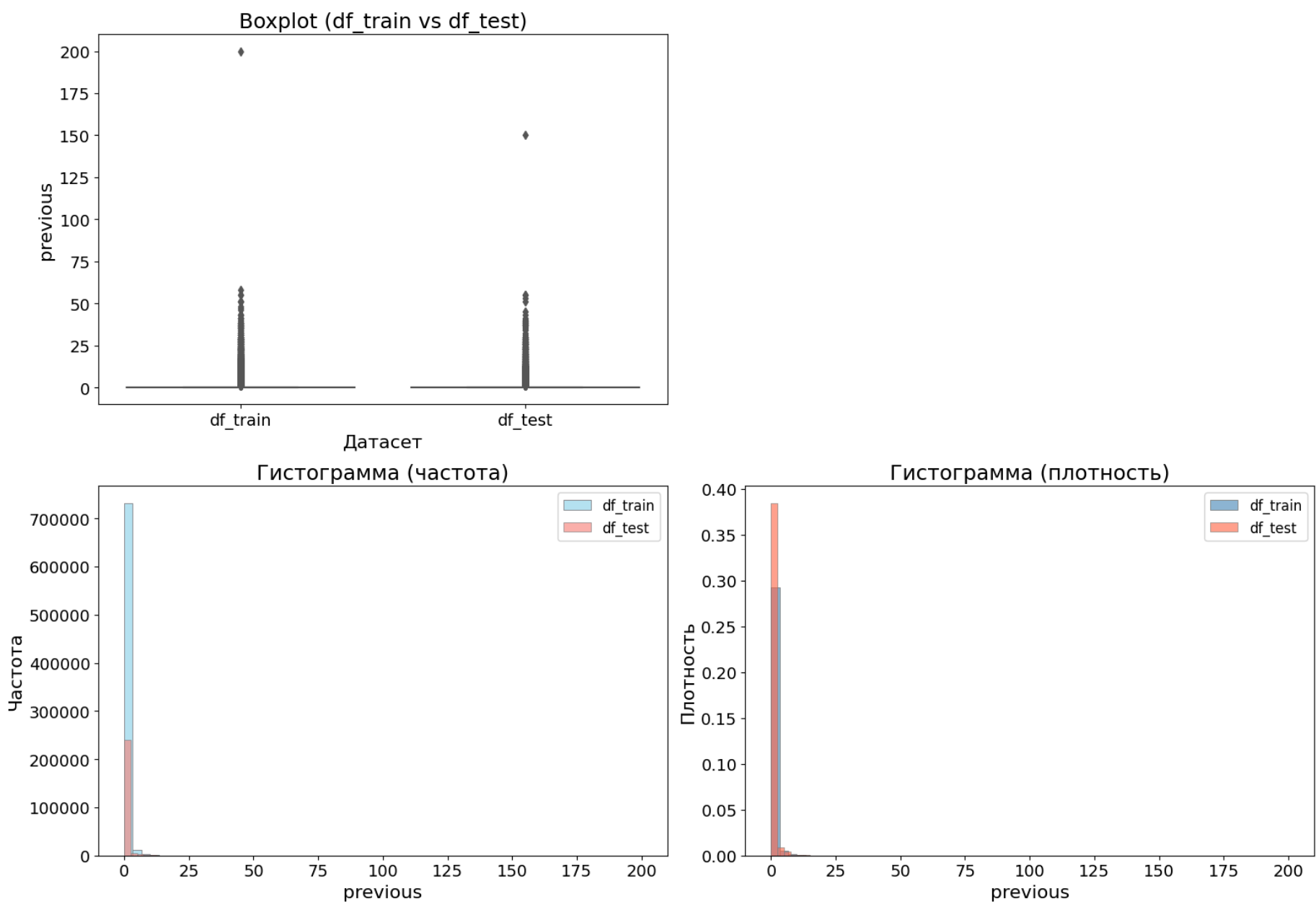
Name: previous, dtype: float64

df_test:

count	250000.000000
mean	0.303728
std	1.384574
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	150.000000

Name: previous, dtype: float64

Сравнение распределения количества контактов до текущей кампании клиентов



```
In [63]: # Сортировка по значению previous (от меньшего к большему)
df_train['previous'].value_counts().sort_index()
```

```
Out[63]: 0      672431
         1      28342
         2     20468
         3     10326
         4      6239
         5      3882
         6      2183
         7      1730
         8      1036
         9       697
        10       501
        11       503
        12       320
        13       327
        14       137
        15       130
        16        81
        17       103
        18        56
        19        77
        20        56
        21        22
        22        43
        23        74
        24        34
        25        20
        26        17
        27        29
        28        23
        29        29
        30        15
        31         9
        32         5
        33         3
        34         3
        35         6
        36         8
        37         7
        38         6
        39         2
        40         1
        41         4
        43         4
        46         1
        47         1
        48         1
        51         4
        55         2
        58         1
        200         1
        Name: previous, dtype: int64
```

```
In [64]: # Удаляем строки, где previous == 200 в train
df_train = df_train[df_train['previous'] != 200].copy()

# Удаляем строки, где previous == 150 в test
df_test = df_test[df_test['previous'] != 150].copy()
```

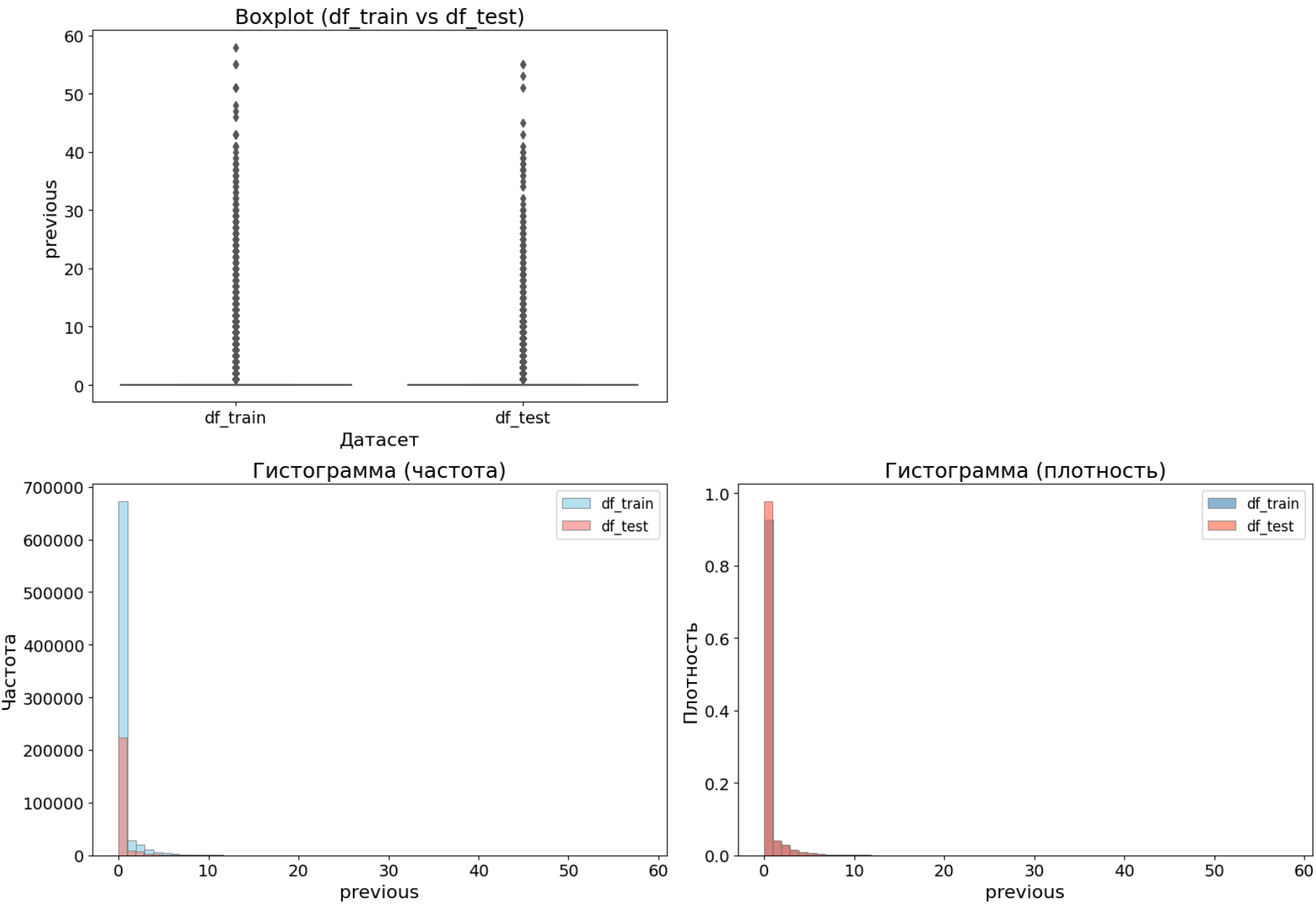
```
In [65]: compare_box_and_hist(df_train, df_test, feature='previous',
                               title="Сравнение распределения количества контактов до текущей кампании клиентов")
```

 Статистика по признаку 'previous':

```
df_train:
count    749999.000000
mean      0.298279
std       1.315875
min       0.000000
25%       0.000000
50%       0.000000
75%       0.000000
max       58.000000
Name: previous, dtype: float64
```

```
df_test:
count    249999.000000
mean      0.303129
std       1.351819
min       0.000000
25%       0.000000
50%       0.000000
75%       0.000000
max       55.000000
Name: previous, dtype: float64
```


Сравнение распределения количества контактов до текущей кампании клиентов



```
In [66]: compare_distributions(df_train, df_test, feature='previous')
```

Сравнение распределения признака 'previous' между df_train и df_test

Mann-Whitney U-тест:
U-статистика: 93732675082.00, p-value: 0.7990
⚠️ Статистически значимых различий по ранговому распределению не обнаружено.

Kolmogorov-Smirnov тест:
KS-статистика: 0.0007, p-value: 1.0000
⚠️ Статистически значимых различий в форме распределений не обнаружено.

Средние значения: df_train = 0.30, df_test = 0.30

```
In [67]: def categorize_previous(x):
    if x == 0:
        return "none"
    elif x == 1:
        return "one"
    elif 2 <= x <= 5:
        return "few"
    elif 6 <= x <= 10:
        return "several"
    else:
        return "many"

# создаём категориальный признак
previous_group = df_train['previous'].apply(categorize_previous)

# вставляем его перед предпоследней колонкой
df_train = insert_before_second_last(df_train, 'previous_group', previous_group)

# Проверим распределение
print(df_train['previous_group'].value_counts())
```

```
none      672431
few       40915
one       28342
several   6147
many      2164
Name: previous_group, dtype: int64
```

Переменные pdays и previous: большая часть клиентов ранее не контактировалась (pdays = -1, previous = 0).

- Создание бинарного флага was_contacted_before улучшит информативность модели.

Анализ выбросов

```
In [68]: def analyze_outliers(df, features):
outliers_info = {}
for feature in features:
    series = df[feature].dropna()
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = series[(series < lower_bound) | (series > upper_bound)]

    outliers_info[feature] = {
        'count': len(outliers),
        'percentage_total': len(outliers) / len(df) * 100,
        'percentage_non_na': len(outliers) / len(series) * 100,
        'min': outliers.min() if not outliers.empty else None,
        'max': outliers.max() if not outliers.empty else None,
        'Q1': Q1,
        'Q3': Q3,
        'lower_bound': lower_bound,
        'upper_bound': upper_bound
    }
return pd.DataFrame(outliers_info).T
```

```
In [69]: key_features = ["age", "balance", "day", "duration", "campaign", "pdays", "previous"]
outliers_summary = analyze_outliers(df_train, key_features)
outliers_summary
```

Out[69]:

	count	percentage_total	percentage_non_na	min	max	Q1	Q3	lower_bound	upper_bound
age	4903.0	0.653734	0.653734	71.0	95.0	33.0	48.0	10.5	70.5
balance	57745.0	7.699344	7.699344	-8019.0	99717.0	0.0	1390.0	-2085.0	3475.0
day	0.0	0.000000	0.000000	NaN	NaN	9.0	21.0	-9.0	39.0
duration	46118.0	6.149075	6.149075	767.0	4918.0	91.0	361.0	-314.0	766.0
campaign	40686.0	5.424807	5.424807	7.0	63.0	1.0	3.0	-2.0	6.0
pdays	77565.0	10.342014	10.342014	0.0	871.0	-1.0	-1.0	-1.0	-1.0
previous	77568.0	10.342414	10.342414	1.0	58.0	0.0	0.0	0.0	0.0

- age: выбросы начинаются с 71 года (верхняя граница 70.5). Это естественная часть распределения.
- balance: выбросы составляют ~7.7% записей, диапазон огромный (от -8019 до 99717).
- day: выбросов нет, всё в пределах границ.
- duration: ~6% выбросов, начиная с 767 секунд (≈12 минут)
 - длительные звонки редки, но не обязательно «ошибки».
- pdays и previous: выбросы >10%.
 - Q1 = Q3 = -1 или 0, а значит распределение очень смещённое (много «-1» как код «не звонили») - категориальная характеристика

Распределение категориальных признаков для двух датасетов

```
In [70]: # Функция для визуализации распределения категориальных признаков в двух выборках
def barh_categorical_features(train_df, test_df, column_name, xlabel=None, ylabel=None, figure_title=None):
    # Подсчет частоты значений
    train_counts = train_df[column_name].value_counts()
    test_counts = test_df[column_name].value_counts()

    # Вычисление процентного распределения
    percent_train = train_counts / train_counts.sum() * 100
    percent_test = test_counts / test_counts.sum() * 100

    # Вывод анализа
    print(f'\nАнализ {column_name}:\n')
    print("Тренировочные данные:")
    print(train_counts, '\n')
    print("Тестовые данные:")
    print(test_counts, '\n')

    print("\nПроцентное распределение:")
    print("Тренировочные данные:")
    print(round(percent_train, 1), '\n')
    print("Тестовые данные:")
    print(round(percent_test, 1), '\n')

    # Создаем график
    fig, axes = plt.subplots(1, 2, figsize=(20, 10), sharey=True)
```



```

# Добавляем общий заголовок, если указан
if figure_title:
    fig.suptitle(figure_title, fontsize=20, fontweight='bold')

# Визуализация тренировочных данных
axes[0].barh(train_counts.index, train_counts.values, color='skyblue', edgecolor='black')
axes[0].set_title('Тренировочные данные (df_train)', fontsize=22)
axes[0].set_xlabel(xlabel if xlabel else 'Количество', fontsize=22)
axes[0].set_ylabel(ylabel if ylabel else column_name, fontsize=22)
axes[0].tick_params(axis='x', labelsizе=16) # размер делений по оси X
axes[0].tick_params(axis='y', labelsizе=16) # размер делений по оси Y

axes[0].grid(axis='x', linestyle='--', alpha=0.6)

# Визуализация тестовых данных
axes[1].barh(test_counts.index, test_counts.values, color='lightcoral', edgecolor='black')
axes[1].set_title('Тестовые данные (df_test)', fontsize=22)
axes[1].set_xlabel(xlabel if xlabel else 'Количество', fontsize=22)
axes[1].set_ylabel(ylabel if ylabel else column_name, fontsize=22)
axes[1].tick_params(axis='x', labelsizе=16) # размер делений по оси X
axes[1].tick_params(axis='y', labelsizе=16) # размер делений по оси Y

axes[1].grid(axis='x', linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()

```

job

- **job** - тип работы (категориальный: "admin.", "blue-collar", "entrepreneur" и др.)

```

In [71]: barh_categorical_features(df_train, df_test,
                                   'job',
                                   figure_title=f"Распределение клиентов банка по профессии")

```

Анализ job:

Тренировочные данные:

management	175541
blue-collar	170498
technician	138106
admin.	81492
services	64209
retired	35185
self-employed	19020
entrepreneur	17718
unemployed	17634
housemaid	15912
student	11767
unknown	2917

Name: job, dtype: int64

Тестовые данные:

management	58636
blue-collar	56970
technician	45936
admin.	27009
services	21312
retired	11611
self-employed	6423
unemployed	6013
entrepreneur	5955
housemaid	5245
student	3867
unknown	1022

Name: job, dtype: int64

Процентное распределение:

Тренировочные данные:

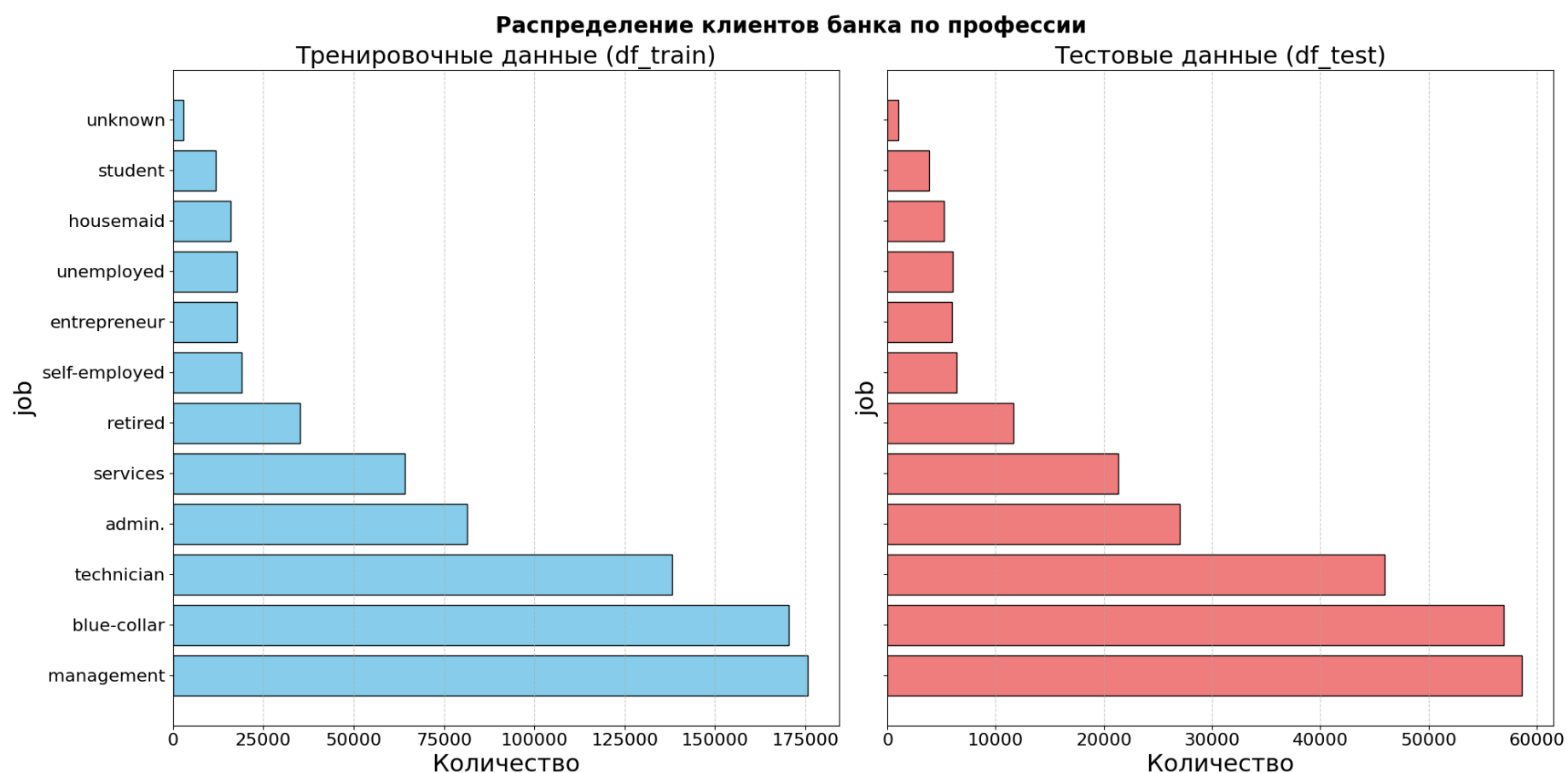
management	23.4
blue-collar	22.7
technician	18.4
admin.	10.9
services	8.6
retired	4.7
self-employed	2.5
entrepreneur	2.4
unemployed	2.4
housemaid	2.1
student	1.6
unknown	0.4

Name: job, dtype: float64

Тестовые данные:

management	23.5
blue-collar	22.8
technician	18.4
admin.	10.8
services	8.5
retired	4.6
self-employed	2.6
unemployed	2.4
entrepreneur	2.4
housemaid	2.1
student	1.5
unknown	0.4

Name: job, dtype: float64



marital

- **marital** - семейное положение

```
In [72]: barh_categorical_features(df_train, df_test,
                                   'marital',
                                   figure_title=f"Распределение клиентов банка по семейному положению")
```

Анализ marital:

Тренировочные данные:

```
married    480759
single     194833
divorced    74407
Name: marital, dtype: int64
```

Тестовые данные:

```
married    160411
single      64717
divorced    24871
Name: marital, dtype: int64
```

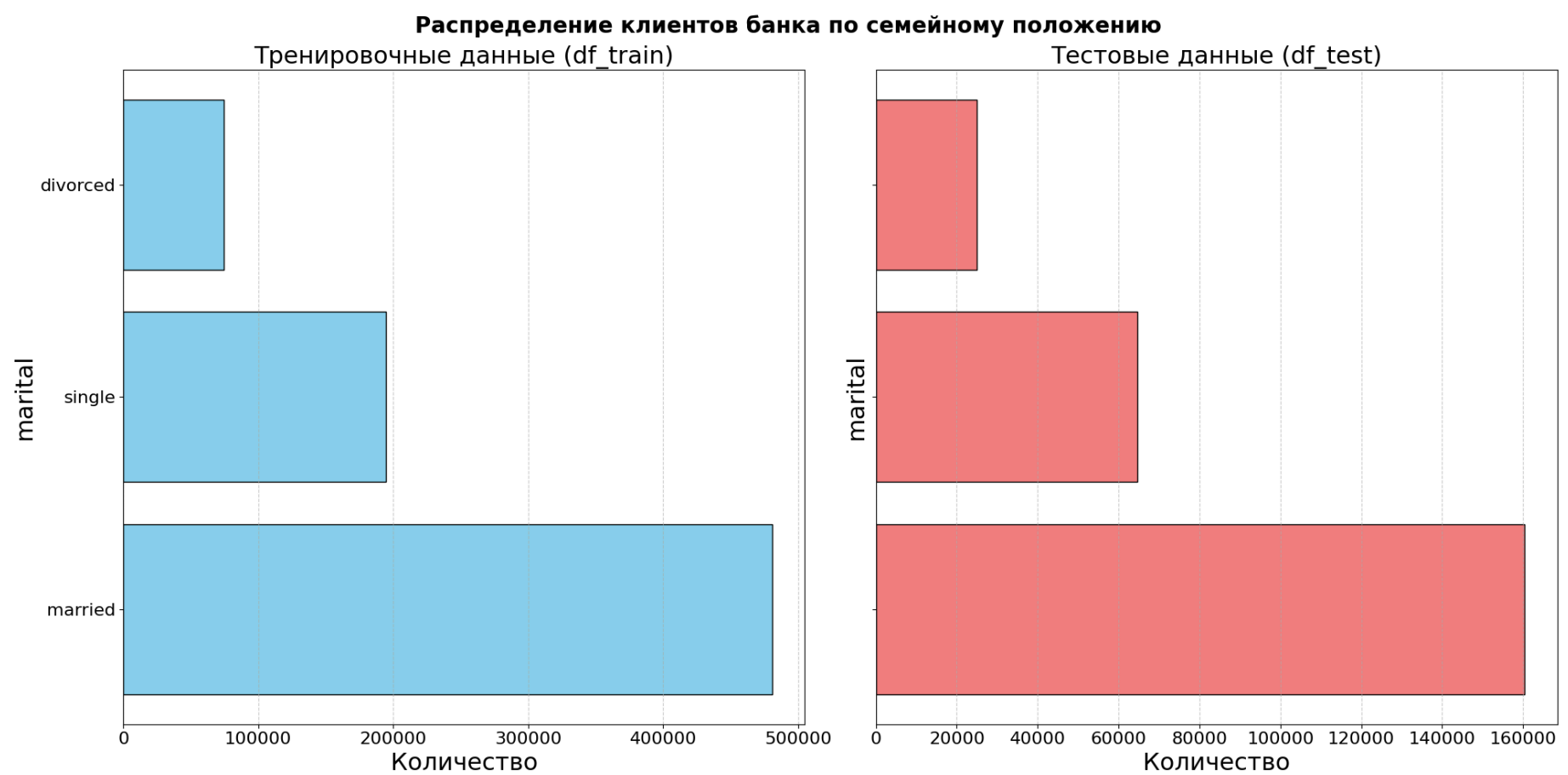
Процентное распределение:

Тренировочные данные:

```
married     64.1
single      26.0
divorced     9.9
Name: marital, dtype: float64
```

Тестовые данные:

```
married     64.2
single      25.9
divorced     9.9
Name: marital, dtype: float64
```



education

- **education** - уровень образования

```
In [73]: barh_categorical_features(df_train, df_test,
                                'education',
                                figure_title=f"Распределение клиентов банка по уровню образования")
```

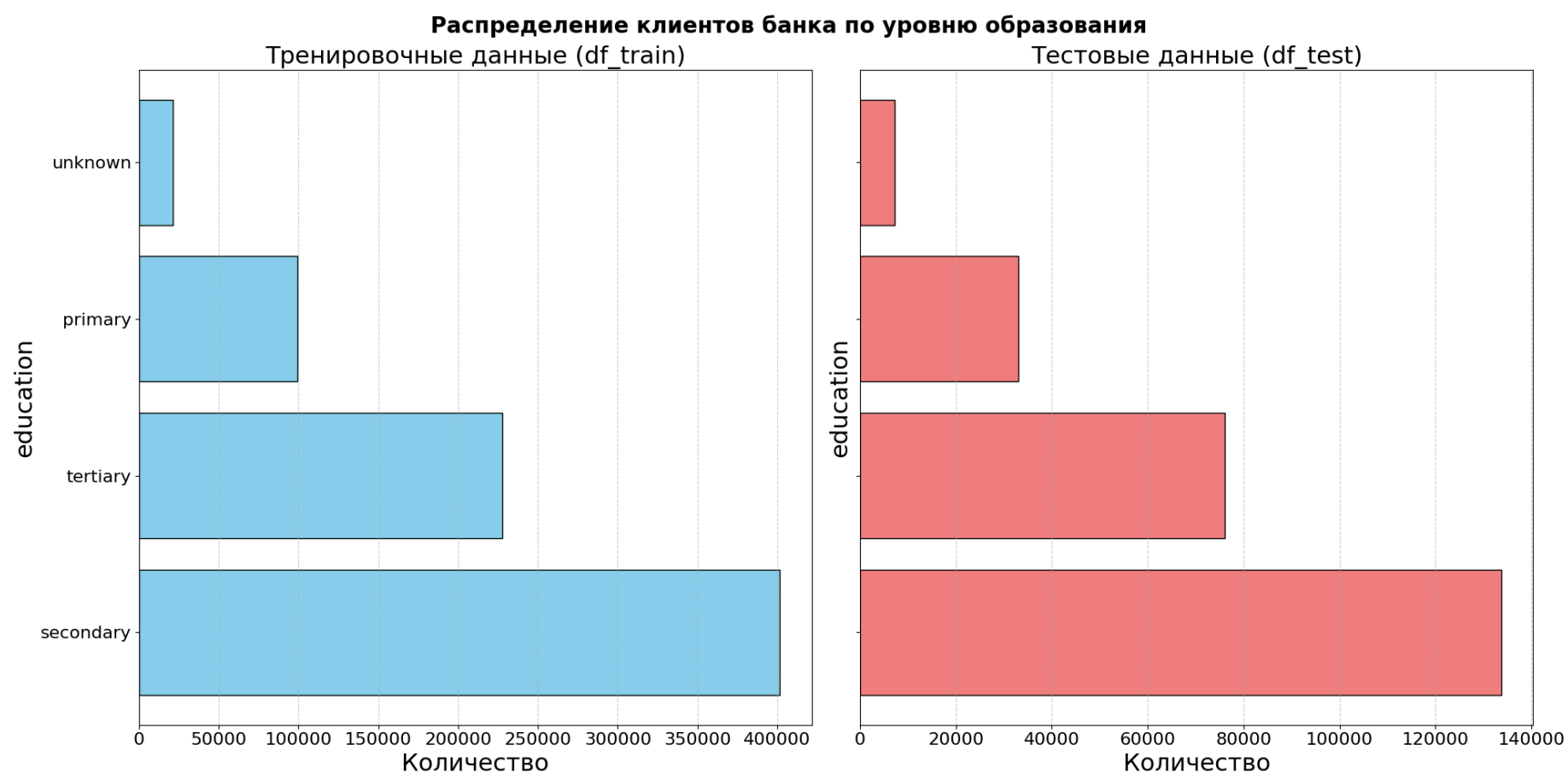
Анализ education:

Тренировочные данные:
secondary 401683
tertiary 227507
primary 99510
unknown 21299
Name: education, dtype: int64

Тестовые данные:
secondary 133723
tertiary 76037
primary 32989
unknown 7250
Name: education, dtype: int64

Процентное распределение:
Тренировочные данные:
secondary 53.6
tertiary 30.3
primary 13.3
unknown 2.8
Name: education, dtype: float64

Тестовые данные:
secondary 53.5
tertiary 30.4
primary 13.2
unknown 2.9
Name: education, dtype: float64



default

- **default** - наличие просроченного кредита

```
In [74]: barh_categorical_features(df_train, df_test,
                                'default',
                                figure_title=f"Распределение клиентов банка по наличию просроченного кредита")
```

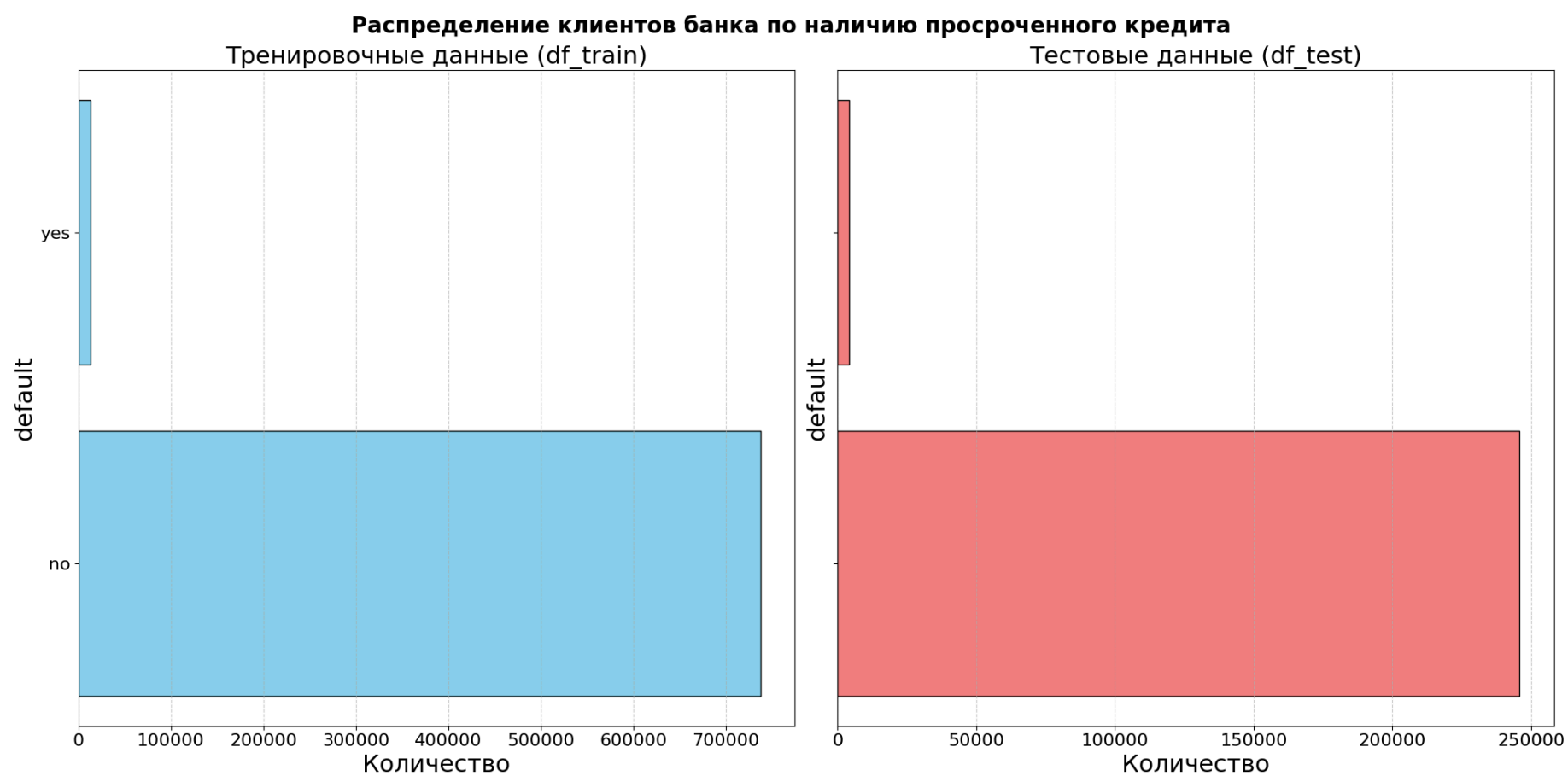
Анализ default:

Тренировочные данные:
no 737150
yes 12849
Name: default, dtype: int64

Тестовые данные:
no 245842
yes 4157
Name: default, dtype: int64

Процентное распределение:
Тренировочные данные:
no 98.3
yes 1.7
Name: default, dtype: float64

Тестовые данные:
no 98.3
yes 1.7
Name: default, dtype: float64



housing

- **housing** - наличие ипотечного кредита

```
In [75]: barh_categorical_features(df_train, df_test,
                                     'housing',
                                     figure_title=f"Распределение клиентов банка по наличию ипотечного кредита")
```

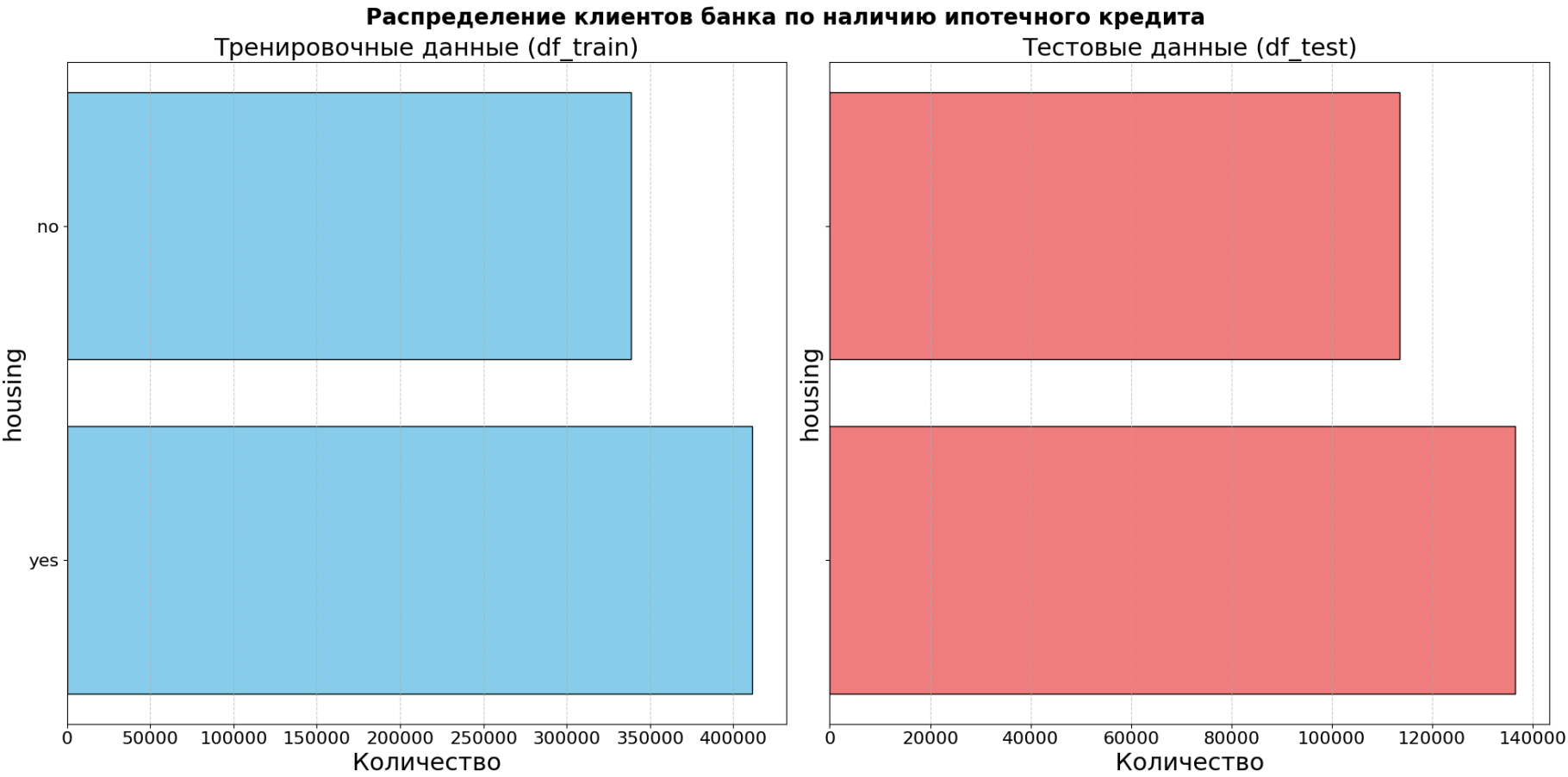
Анализ housing:

Тренировочные данные:
yes 411288
no 338711
Name: housing, dtype: int64

Тестовые данные:
yes 136533
no 113466
Name: housing, dtype: int64

Процентное распределение:
Тренировочные данные:
yes 54.8
no 45.2
Name: housing, dtype: float64

Тестовые данные:
yes 54.6
no 45.4
Name: housing, dtype: float64



loan

- **loan** - наличие персонального кредита

```
In [76]: barh_categorical_features(df_train, df_test,
                                     'loan',
                                     figure_title=f"Распределение клиентов банка по наличию персонального кредита")
```

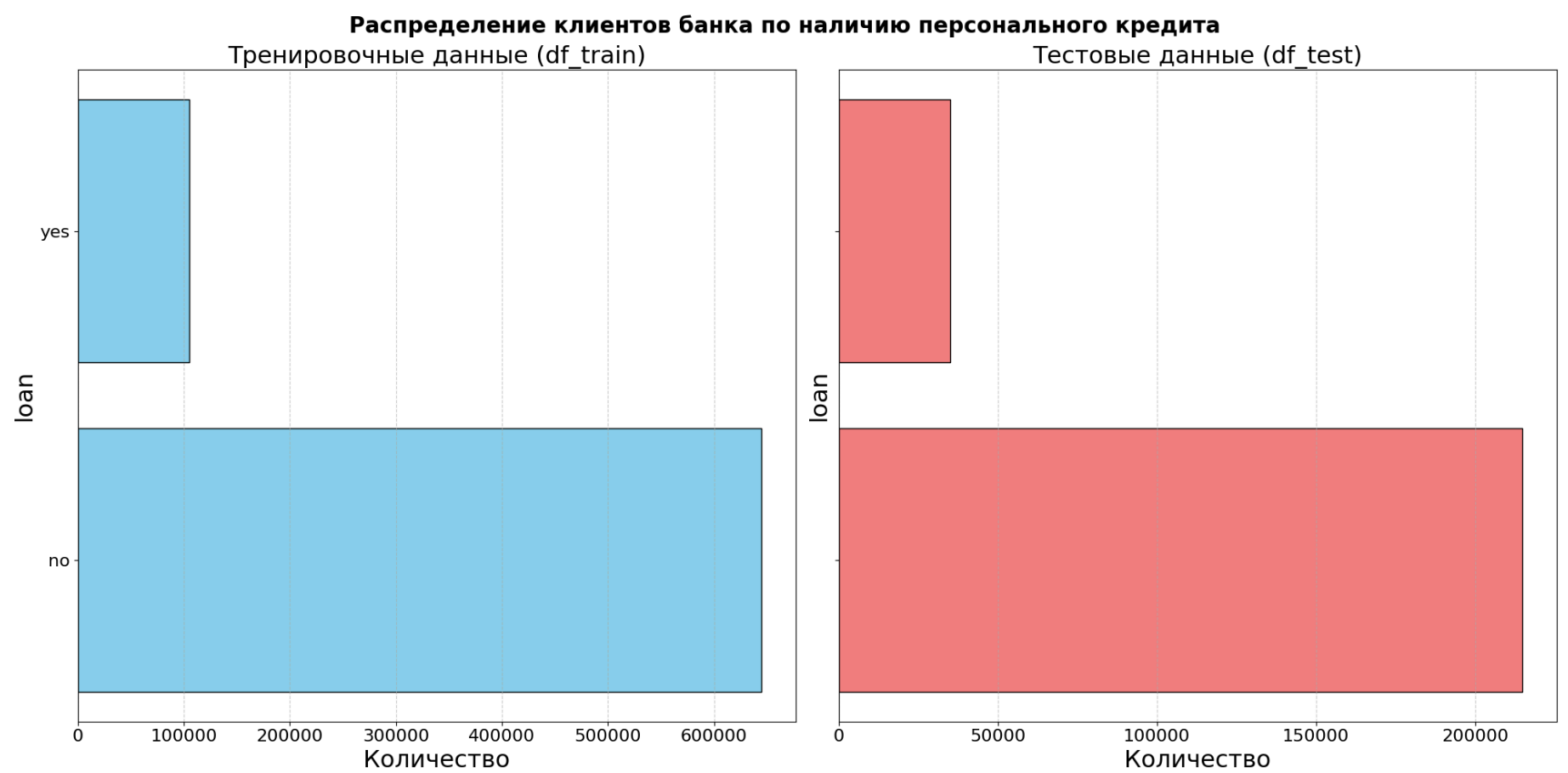
Анализ loan:

Тренировочные данные:
no 645022
yes 104977
Name: loan, dtype: int64

Тестовые данные:
no 214956
yes 35043
Name: loan, dtype: int64

Процентное распределение:
Тренировочные данные:
no 86.0
yes 14.0
Name: loan, dtype: float64

Тестовые данные:
no 86.0
yes 14.0
Name: loan, dtype: float64



contact

- **contact** - тип канала связи

```
In [77]: barh_categorical_features(df_train, df_test,
                                   'contact',
                                   figure_title=f"Распределение клиентов банка по типу канала связи")
```

Анализ contact:

Тренировочные данные:

cellular 486654

unknown 231627

telephone 31718

Name: contact, dtype: int64

Тестовые данные:

cellular 162461

unknown 76896

telephone 10642

Name: contact, dtype: int64

Процентное распределение:

Тренировочные данные:

cellular 64.9

unknown 30.9

telephone 4.2

Name: contact, dtype: float64

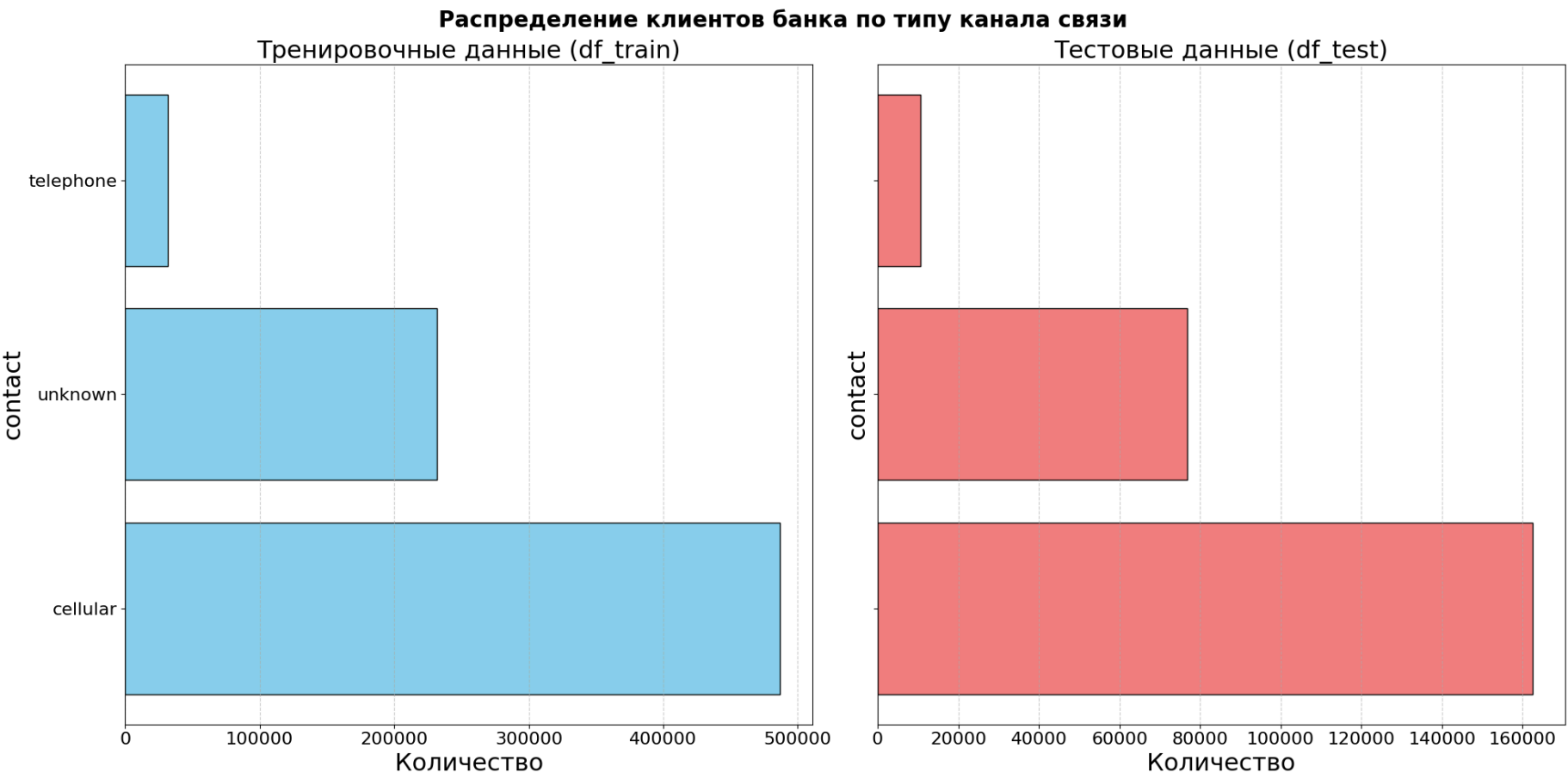
Тестовые данные:

cellular 65.0

unknown 30.8

telephone 4.3

Name: contact, dtype: float64



- **month** - месяц последнего контакта

```
In [78]: barh_categorical_features(df_train, df_test,
                                'month',
                                figure_title=f"Распределение клиентов банка по месяцу последнего контакта")
```

Анализ month:

Тренировочные данные:

may	228411
aug	128859
jul	110647
jun	93670
nov	66062
apr	41319
feb	37611
jan	18937
oct	9204
sep	7408
mar	5802
dec	2069

Name: month, dtype: int64

Тестовые данные:

may	76009
aug	42874
jul	36828
jun	31195
nov	22036
apr	13878
feb	12516
jan	6441
oct	3154
sep	2429
mar	1944
dec	695

Name: month, dtype: int64

Процентное распределение:

Тренировочные данные:

may	30.5
aug	17.2
jul	14.8
jun	12.5
nov	8.8
apr	5.5
feb	5.0
jan	2.5
oct	1.2
sep	1.0
mar	0.8
dec	0.3

Name: month, dtype: float64

Тестовые данные:

may	30.4
aug	17.1
jul	14.7
jun	12.5
nov	8.8
apr	5.6
feb	5.0
jan	2.6
oct	1.3
sep	1.0
mar	0.8
dec	0.3

Name: month, dtype: float64



poutcome

- **poutcome** - результат предыдущей маркетинговой кампании

```
In [79]: barh_categorical_features(df_train, df_test,
                                   'poutcome',
                                   figure_title=f"Распределение клиентов банка по результатам предыдущей маркетинговой кампании")
```

Анализ poutcome:

Тренировочные данные:

```
unknown    672450
failure     45115
success     17691
other       14743
Name: poutcome, dtype: int64
```

Тестовые данные:

```
unknown    224115
failure     14844
success      6002
other        5038
Name: poutcome, dtype: int64
```

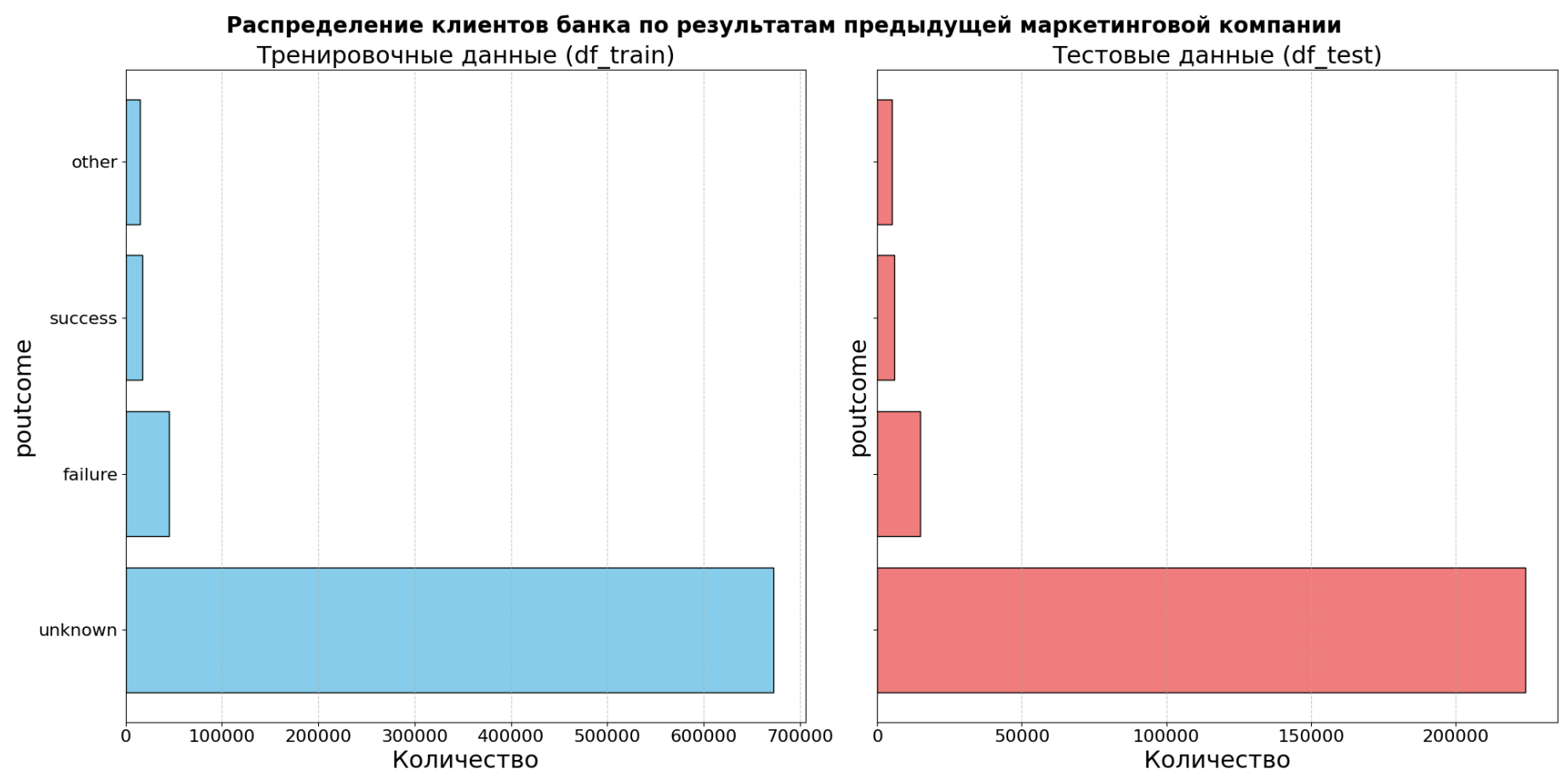
Процентное распределение:

Тренировочные данные:

```
unknown     89.7
failure      6.0
success      2.4
other        2.0
Name: poutcome, dtype: float64
```

Тестовые данные:

```
unknown     89.6
failure      5.9
success      2.4
other        2.0
Name: poutcome, dtype: float64
```



Анализ распределений признаков по классам целевой переменной

Количественные признаки

- **целевая переменная у** для возможности визуализации переведена в категориальную переменную subscribed_dep (подписка на срочный депозит)

```
In [80]: def plot_box_and_hist_by_target(data, feature='age', target='subscribed_dep',
                                         title=None, bins=60):

    # Удаляем пропуски
    df = data[[feature, target]].dropna()

    # Общая статистика по признаку
    print(f"\n📊 Общая статистика по признаку '{feature}':")
    print(df[feature].describe())

    # Статистика по значениям целевой переменной
    print(f"\n📊 Статистика признака '{feature}' по значениям целевой переменной '{target}':")
    for val in sorted(df[target].unique()):
        print(f"\n{target} = {val}:")
        subset_stats = df[df[target] == val][feature].describe()
        print(subset_stats)

    # Заголовок графика по умолчанию
    if title is None:
        title = f'Распределение признака "{feature}" по целевой переменной'

    # Создаём сетку 2 строки × 2 столбца
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))
    fig.suptitle(title, fontsize=20)

    # Ящик с усами – первая строка
    sns.boxplot(data=df, x=target, y=feature, ax=axes[0, 0])
    axes[0, 0].set_title('Ящик с усами', fontsize=16)
    axes[0, 0].set_xlabel(target, fontsize=16)
    axes[0, 0].set_ylabel(feature, fontsize=16)
    axes[0, 0].tick_params(axis='x', labelsize=14)
    axes[0, 0].tick_params(axis='y', labelsize=14)

    # Отключаем вторую ячейку в первой строке
    axes[0, 1].axis('off')

    # Обычная гистограмма – вторая строка, слева
    sns.histplot(data=df, x=feature, hue=target, multiple="stack", bins=bins,
                 palette='pastel', edgecolor='gray', ax=axes[1, 0])
    axes[1, 0].set_title('Гистограмма (частота)', fontsize=18)
    axes[1, 0].set_xlabel(feature, fontsize=16)
    axes[1, 0].set_ylabel('Частота', fontsize=16)
    axes[1, 0].tick_params(axis='x', labelsize=14)
    axes[1, 0].tick_params(axis='y', labelsize=14)

    # Нормированная гистограмма – вторая строка, справа
    sns.histplot(data=df, x=feature, hue=target, multiple="layer", bins=bins,
                 stat="density", common_norm=False, palette='muted', ax=axes[1, 1])
    axes[1, 1].set_title('Нормированная гистограмма (плотность)', fontsize=18)
```

```
axes[1, 1].set_xlabel(feature, fontsize=16)
axes[1, 1].set_ylabel('Плотность', fontsize=16)
axes[1, 1].tick_params(axis='x', labelsizе=12)
axes[1, 1].tick_params(axis='y', labelsizе=12)

plt.tight_layout(rect=[0, 0, 1, 0.96]) # Оставляем место для заголовка
plt.show()
```

```
In [81]: plot_box_and_hist_by_target(df_train, feature='age',
                                     title='Распределение возраста клиентов по целевой переменной')
```

Общая статистика по признаку 'age':

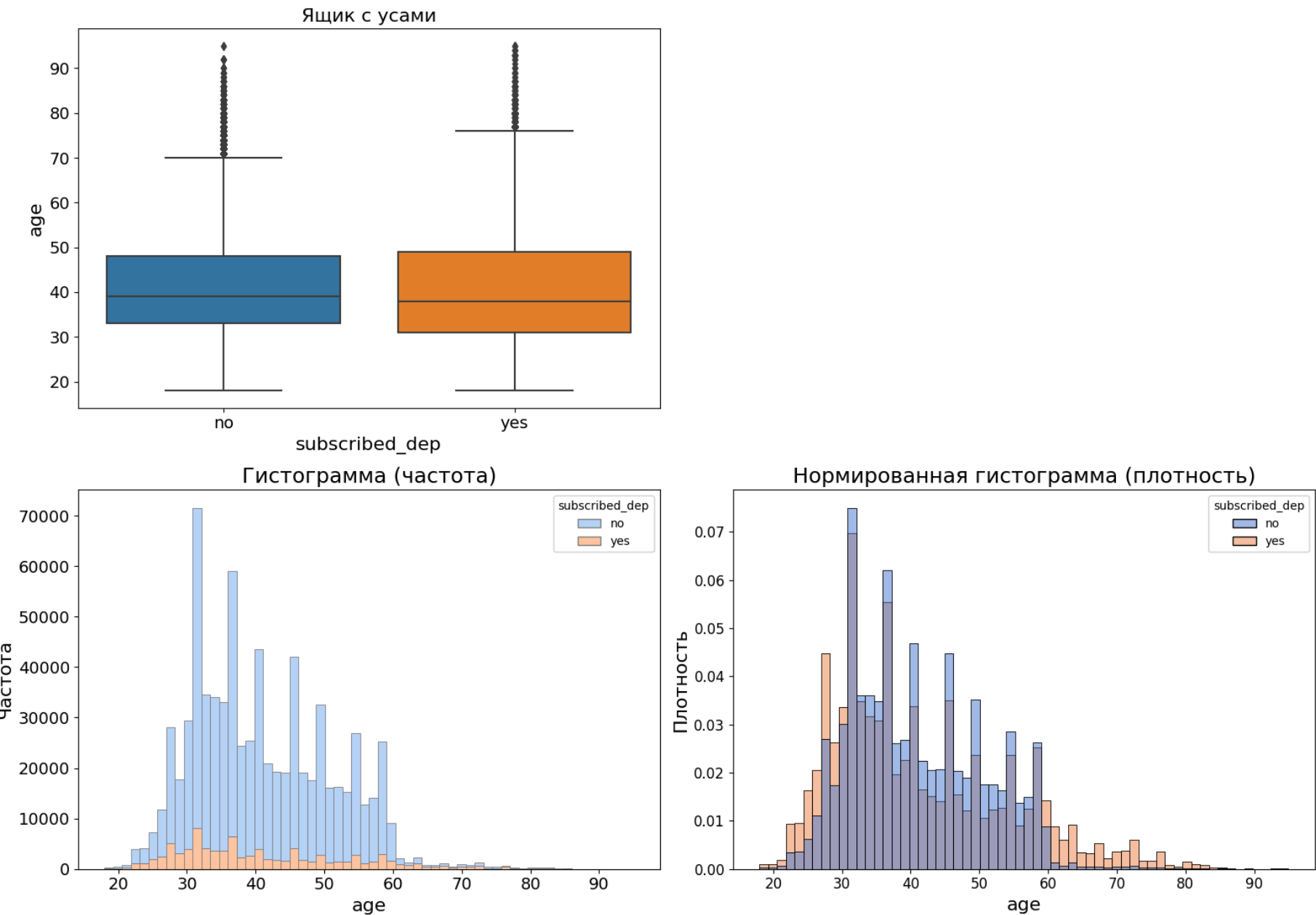
```
count    749999.000000
mean      40.926413
std       10.098823
min       18.000000
25%       33.000000
50%       39.000000
75%       48.000000
max       95.000000
Name: age, dtype: float64
```

Статистика признака 'age' по значениям целевой переменной 'subscribed_dep':

```
subscribed_dep = no:
count    659511.000000
mean      40.890792
std       9.685363
min       18.000000
25%       33.000000
50%       39.000000
75%       48.000000
max       95.000000
Name: age, dtype: float64
```

```
subscribed_dep = yes:
count     90488.000000
mean      41.186036
std      12.709447
min       18.000000
25%       31.000000
50%       38.000000
75%       49.000000
max       95.000000
Name: age, dtype: float64
```

Распределение возраста клиентов по целевой переменной



```
In [82]: group0 = df_train[df_train['subscribed_dep'] == 'no']['age']
group1 = df_train[df_train['subscribed_dep'] == 'yes']['age']

stat, pval = mannwhitneyu(group0, group1, alternative='two-sided')
print(f"U-статистика: {stat:.2f}, p-value: {pval:.4f}")

# Интерпретация результата
alpha = 0.05 # уровень значимости

if pval < alpha:
    print("✅ Различия в распределении возраста между группами статистически значимы.")
    mean0 = group0.mean()
    mean1 = group1.mean()
    if mean1 > mean0:
        print(f"📄 Подписавшиеся клиенты в среднем старше: {mean1:.1f} vs {mean0:.1f}")
    else:
        print(f"📄 Подписавшиеся клиенты в среднем моложе: {mean1:.1f} vs {mean0:.1f}")
else:
    print("⚠️ Статистически значимых различий в распределении возраста между группами не обнаружено.")
```

U-статистика: 31063272209.00, p-value: 0.0000

✅ Различия в распределении возраста между группами статистически значимы.

📄 Подписавшиеся клиенты в среднем старше: 41.2 vs 40.9

```
In [83]: from scipy.stats import mannwhitneyu, ks_2samp

group0 = df_train[df_train['subscribed_dep'] == 'no']['age']
group1 = df_train[df_train['subscribed_dep'] == 'yes']['age']

# Mann-Whitney U-тест
stat_u, pval_u = mannwhitneyu(group0, group1, alternative='two-sided')
print(f"\n📄 Mann-Whitney U-тест: U-статистика = {stat_u:.2f}, p-value = {pval_u:.4f}")

alpha = 0.05
if pval_u < alpha:
    print("✅ Различия по ранговому распределению статистически значимы.")
    mean0 = group0.mean()
    mean1 = group1.mean()
    if mean1 > mean0:
        print(f"📄 Подписавшиеся клиенты в среднем старше: {mean1:.1f} vs {mean0:.1f}")
    else:
        print(f"📄 Подписавшиеся клиенты в среднем моложе: {mean1:.1f} vs {mean0:.1f}")
else:
    print("⚠️ Статистически значимых различий по ранговому распределению не обнаружено.")

# Kolmogorov-Smirnov тест
stat_ks, pval_ks = ks_2samp(group0, group1)
print(f"\n📄 Kolmogorov-Smirnov тест: KS-статистика = {stat_ks:.4f}, p-value = {pval_ks:.4f}")
if pval_ks < alpha:
    print("✅ Форма распределений статистически различается.")
else:
    print("⚠️ Статистически значимых различий в форме распределений не обнаружено.")
```

📄 Mann-Whitney U-тест: U-статистика = 31063272209.00, p-value = 0.0000

✅ Различия по ранговому распределению статистически значимы.

📄 Подписавшиеся клиенты в среднем старше: 41.2 vs 40.9

📄 Kolmogorov-Smirnov тест: KS-статистика = 0.0823, p-value = 0.0000

✅ Форма распределений статистически различается.

- 'Подписавшиеся клиенты немного старше',
- 'Средний возраст: 41.2 года (подписка) vs 40.9 (нет)',

```
In [84]: plot_box_and_hist_by_target(df_train, feature='balance',
                                     title='Распределение средне-годового баланса клиентов по целевой переменной')
```


Общая статистика по признаку 'balance':

count	749999.000000
mean	1204.069003
std	2836.098309
min	-8019.000000
25%	0.000000
50%	634.000000
75%	1390.000000
max	99717.000000

Name: balance, dtype: float64

Статистика признака 'balance' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

count	659511.000000
mean	1075.366309
std	2665.969505
min	-8019.000000
25%	0.000000
50%	584.000000
75%	1288.000000
max	99717.000000

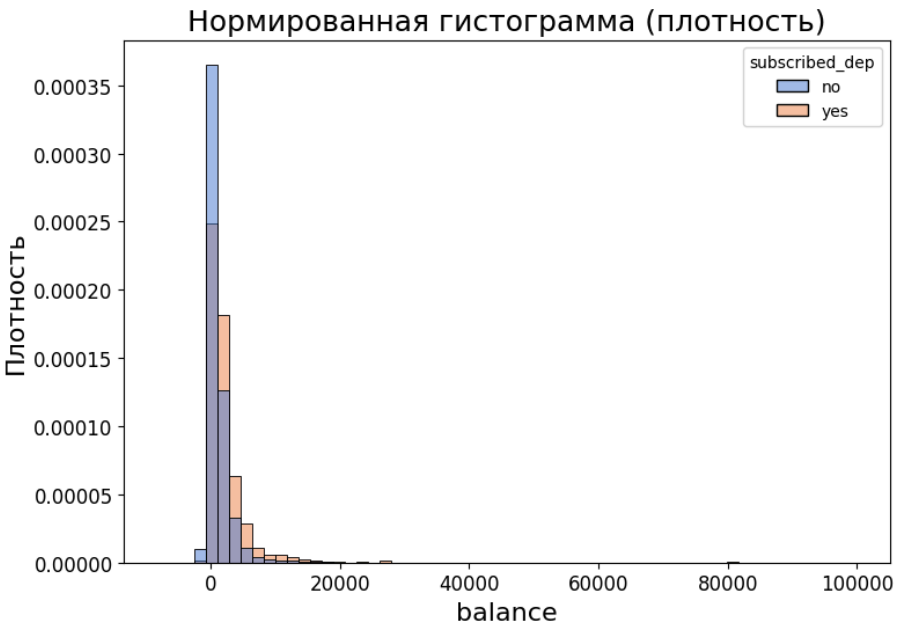
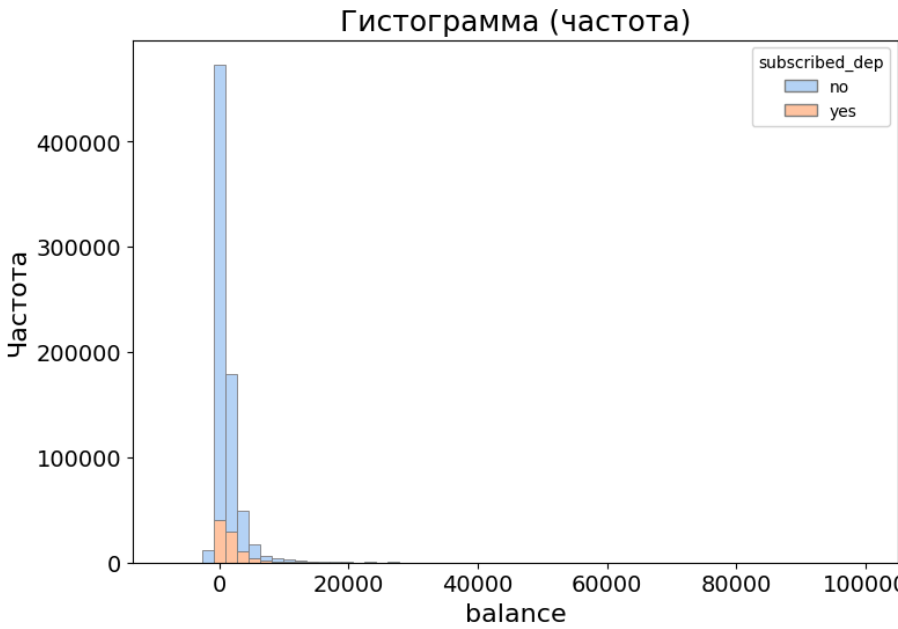
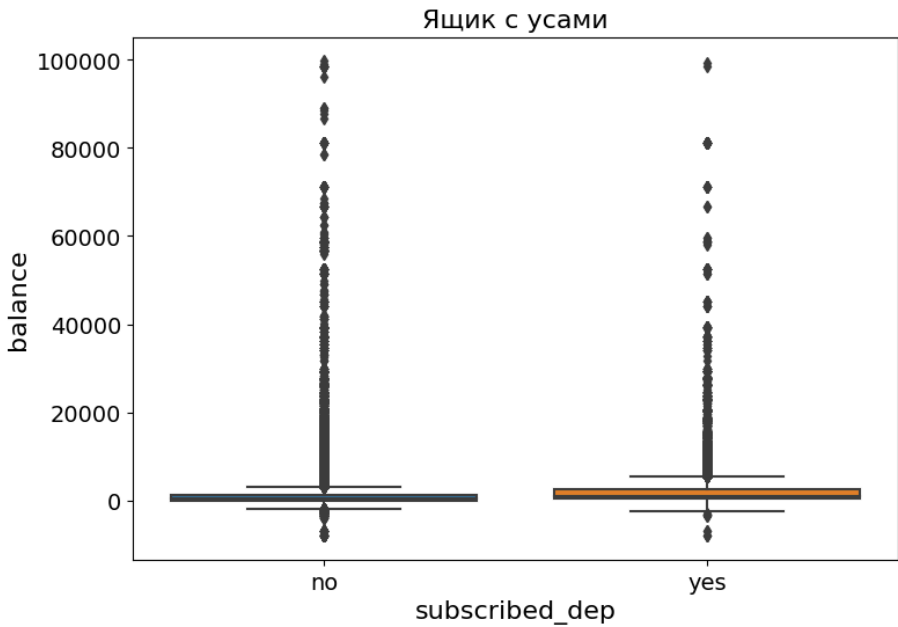
Name: balance, dtype: float64

subscribed_dep = yes:

count	90488.000000
mean	2142.103240
std	3723.617059
min	-8019.000000
25%	565.000000
50%	1134.000000
75%	2552.000000
max	99218.000000

Name: balance, dtype: float64

Распределение средне-годового баланса клиентов по целевой переменной



```
In [85]: # Делим на группы по целевой переменной
group0 = df_train[df_train['subscribed_dep'] == 'no']['balance']
group1 = df_train[df_train['subscribed_dep'] == 'yes']['balance']

# U-критерий Манна-Уитни
stat, pval = mannwhitneyu(group0, group1, alternative='two-sided')
print(f"U-статистика: {stat:.2f}, p-value: {pval:.4f}")

# Интерпретация результата
alpha = 0.05 # уровень значимости

if pval < alpha:
```

```
print("✅ Различия в распределении среднего годового баланса между группами статистически значимы.")
mean0 = group0.mean()
mean1 = group1.mean()
if mean1 > mean0:
    print(f"📈 Подписавшиеся клиенты в среднем имеют более высокий баланс: {mean1:.2f} vs {mean0:.2f} евро")
else:
    print(f"📉 Подписавшиеся клиенты в среднем имеют более низкий баланс: {mean1:.2f} vs {mean0:.2f} евро")
else:
    print("⚠️ Статистически значимых различий в распределении баланса между группами не обнаружено.")
```

U-статистика: 19398615783.00, p-value: 0.0000

✅ Различия в распределении среднего годового баланса между группами статистически значимы.

📈 Подписавшиеся клиенты в среднем имеют более высокий баланс: 2142.10 vs 1075.37 евро

- 'Клиенты с высоким балансом чаще подписываются',
- 'Средний баланс: 2142€ (подписка) vs 1075€ (нет)'

```
In [86]: plot_box_and_hist_by_target(df_train, feature='day',
                                     title='Распределение дня последнего контакта по целевой переменной')
```

📊 Общая статистика по признаку 'day':

count	749999.000000
mean	16.117208
std	8.250838
min	1.000000
25%	9.000000
50%	17.000000
75%	21.000000
max	31.000000

Name: day, dtype: float64

📊 Статистика признака 'day' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

count	659511.000000
mean	16.268872
std	8.201081
min	1.000000
25%	9.000000
50%	17.000000
75%	21.000000
max	31.000000

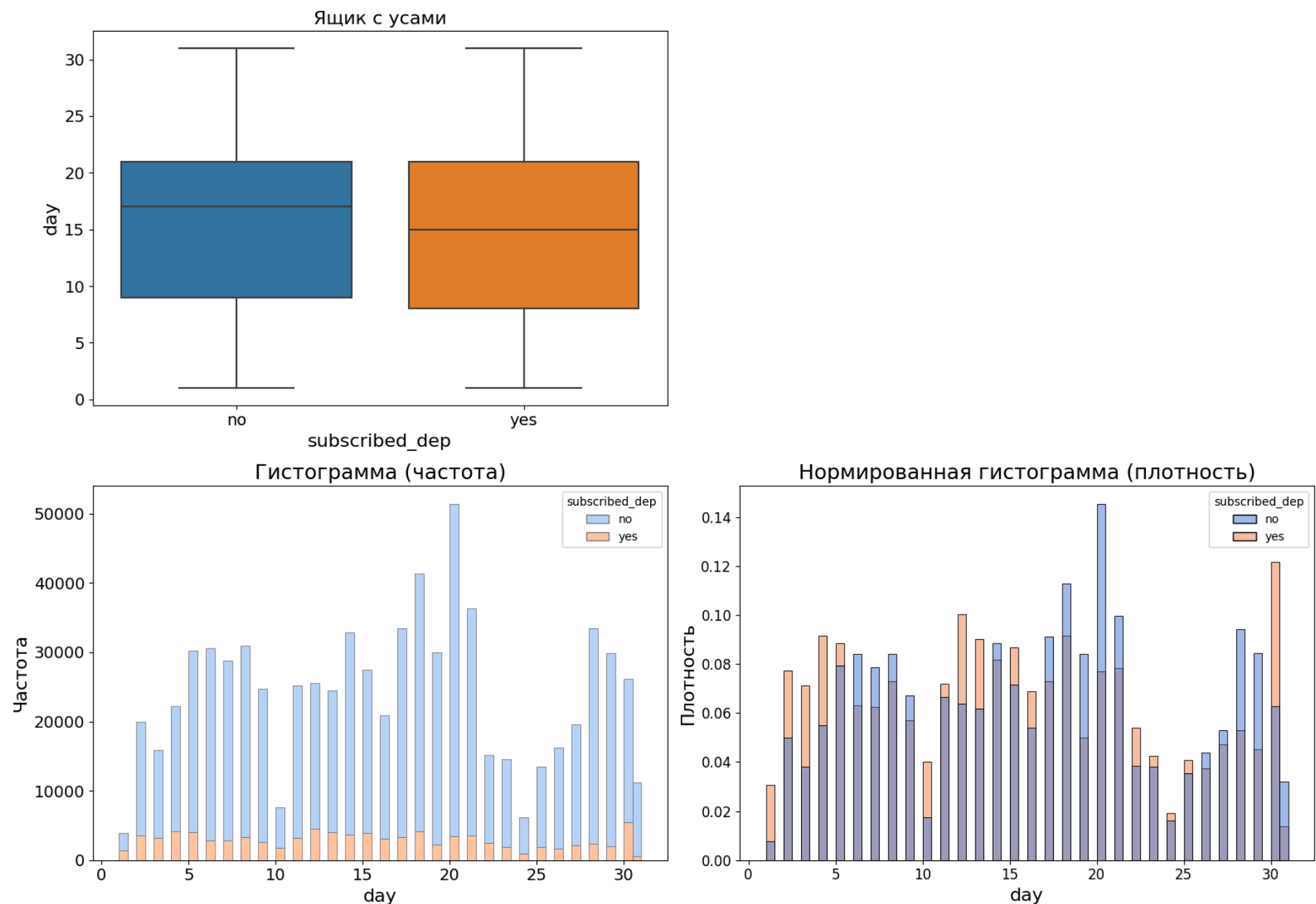
Name: day, dtype: float64

subscribed_dep = yes:

count	90488.000000
mean	15.011825
std	8.523723
min	1.000000
25%	8.000000
50%	15.000000
75%	21.000000
max	31.000000

Name: day, dtype: float64

Распределение дня последнего контакта по целевой переменной



```
In [87]: plot_box_and_hist_by_target(df_train, feature='duration',
                                     title='Распределение продолжительности последнего контакта по целевой переменной')
```

Общая статистика по признаку 'duration':

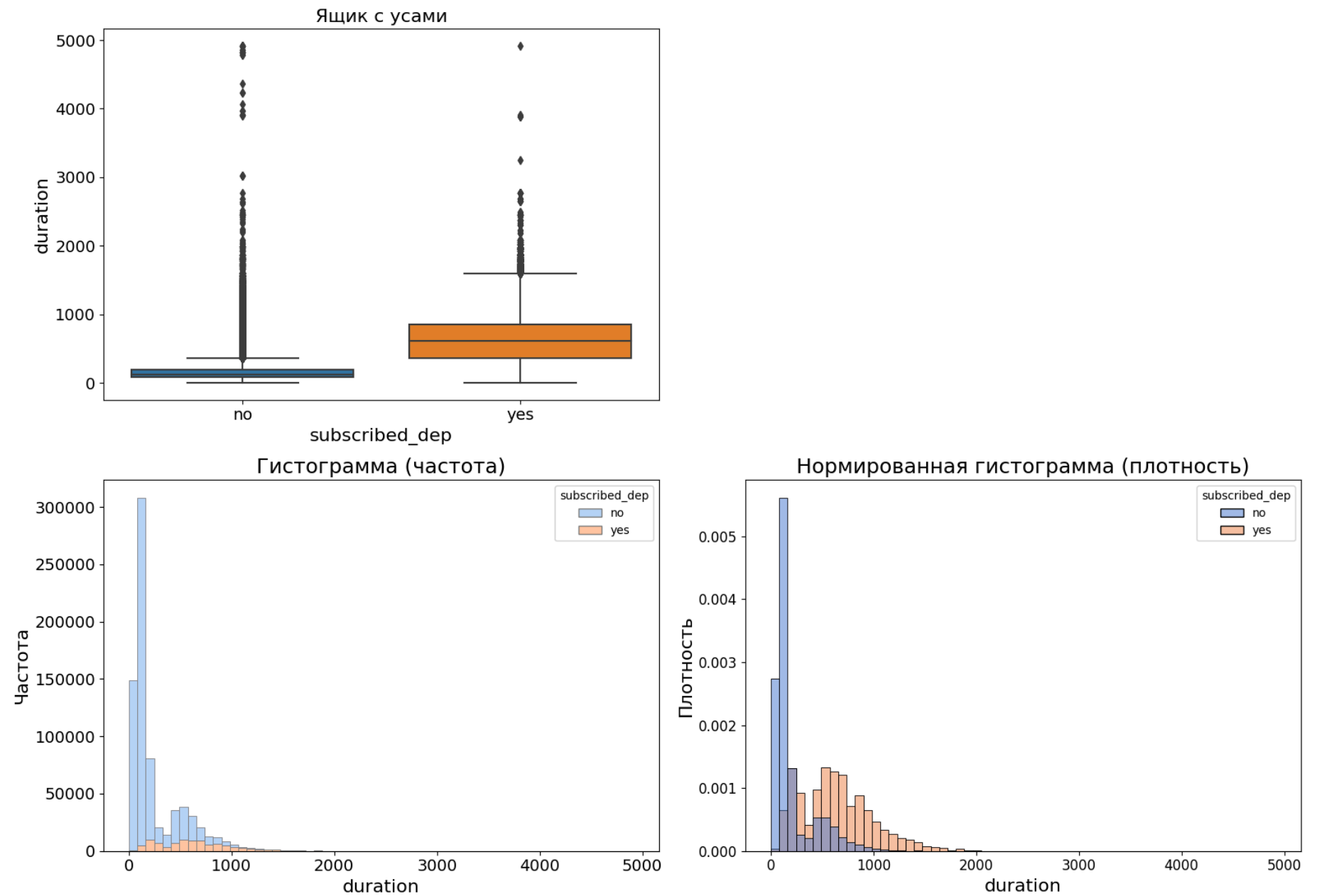
```
count    749999.000000
mean      256.229476
std       272.555692
min        1.000000
25%       91.000000
50%      133.000000
75%      361.000000
max     4918.000000
Name: duration, dtype: float64
```

Статистика признака 'duration' по значениям целевой переменной 'subscribed_dep':

```
subscribed_dep = no:
count    659511.000000
mean      203.803874
std       211.134895
min        1.000000
25%       87.000000
50%      123.000000
75%      198.000000
max     4916.000000
Name: duration, dtype: float64
```

```
subscribed_dep = yes:
count     90488.000000
mean       638.327226
std       353.249625
min        1.000000
25%      358.000000
50%      609.000000
75%      855.000000
max     4918.000000
Name: duration, dtype: float64
```

Распределение продолжительности последнего контакта по целевой переменной



```
In [88]: # Делим на группы по целевой переменной
group0 = df_train[df_train['subscribed_dep'] == 'no']['duration']
group1 = df_train[df_train['subscribed_dep'] == 'yes']['duration']

# U-критерий Манна-Уитни
stat, pval = mannwhitneyu(group0, group1, alternative='two-sided')
print(f"U-статистика: {stat:.2f}, p-value: {pval:.4f}")

# Интерпретация результата
alpha = 0.05 # уровень значимости

if pval < alpha:
    print("✅ Различия в распределении продолжительности контакта между группами статистически значимы.")
    mean0 = group0.mean()
    mean1 = group1.mean()
    if mean1 > mean0:
        print(f"📄 Подписавшиеся клиенты в среднем дольше разговаривали: {mean1:.1f} сек vs {mean0:.1f} сек")
    else:
        print(f"📄 Подписавшиеся клиенты в среднем разговаривали меньше: {mean1:.1f} сек vs {mean0:.1f} сек")
else:
    print("⚠️ Статистически значимых различий в продолжительности контакта между группами не обнаружено.")
```

U-статистика: 6593645063.50, p-value: 0.0000

✅ Различия в распределении продолжительности контакта между группами статистически значимы.

📄 Подписавшиеся клиенты в среднем дольше разговаривали: 638.3 сек vs 203.8 сек

- 'Длительность разговора - сильнейший предиктор',
- 'Средняя длительность: 638 сек (подписка) vs 204 сек (нет)'

duration: главный предиктор — чем дольше длился звонок, тем выше вероятность подписки.

```
In [89]: plot_box_and_hist_by_target(df_train, feature='campaign',
                                     title='Распределение количества контактов в течение текущей кампании по целевой переменной')
```

Общая статистика по признаку 'campaign':

count	749999.000000
mean	2.577010
std	2.718515
min	1.000000
25%	1.000000
50%	2.000000
75%	3.000000
max	63.000000

Name: campaign, dtype: float64

Статистика признака 'campaign' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

count	659511.000000
mean	2.653367
std	2.814651
min	1.000000
25%	1.000000
50%	2.000000
75%	3.000000
max	63.000000

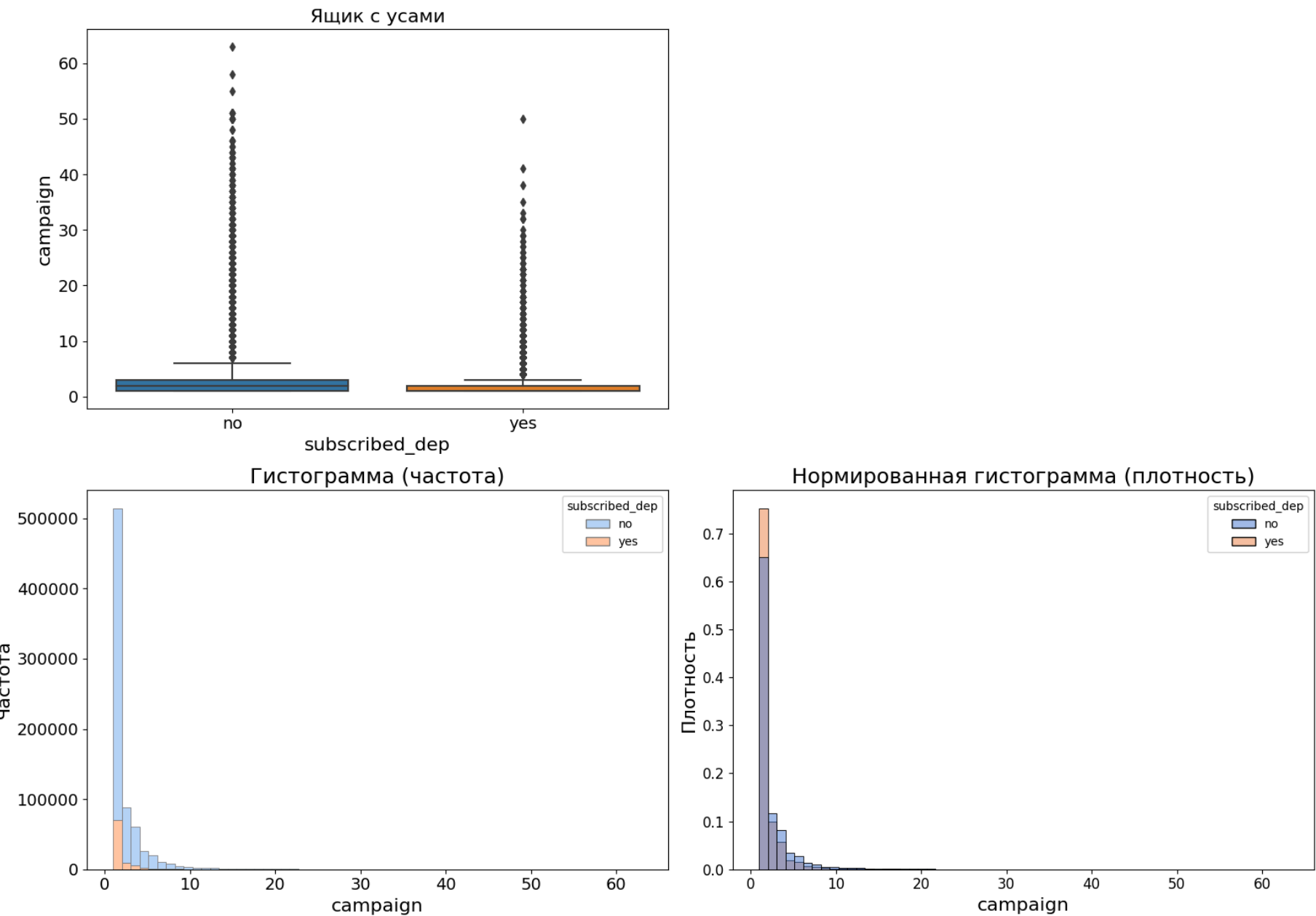
Name: campaign, dtype: float64

subscribed_dep = yes:

count	90488.000000
mean	2.020489
std	1.777978
min	1.000000
25%	1.000000
50%	1.000000
75%	2.000000
max	50.000000

Name: campaign, dtype: float64

Распределение количества контактов в течение текущей кампании по целевой переменной



```
In [90]: # Делим на группы по целевой переменной
group0 = df_train[df_train['subscribed_dep'] == 'no']['campaign']
group1 = df_train[df_train['subscribed_dep'] == 'yes']['campaign']

# U-критерий Манна-Уитни
stat, pval = mannwhitneyu(group0, group1, alternative='two-sided')
print(f"U-статистика: {stat:.2f}, p-value: {pval:.4f}")

# Интерпретация результата
alpha = 0.05 # уровень значимости

if pval < alpha:
```

```
print("✅ Различия в распределении количества контактов между группами статистически значимы.")
mean0 = group0.mean()
mean1 = group1.mean()
if mean1 > mean0:
    print(f"📈 Подписавшиеся клиенты в среднем имели больше контактов: {mean1:.1f} vs {mean0:.1f}")
else:
    print(f"📉 Подписавшиеся клиенты в среднем имели меньше контактов: {mean1:.1f} vs {mean0:.1f}")
else:
    print("⚠️ Статистически значимых различий в распределении количества контактов между группами не обнаружено.")
```

U-статистика: 34509745913.00, p-value: 0.0000
✅ Различия в распределении количества контактов между группами статистически значимы.
📉 Подписавшиеся клиенты в среднем имели меньше контактов: 2.0 vs 2.7

- 'Меньше контактов = выше конверсия',
- 'Среднее кол-во контактов: 2.0 (подписка) vs 2.7 (нет)',

campaign: большинство клиентов контактировались 1–3 раза

```
In [91]: plot_box_and_hist_by_target(df_train, feature='pdays',
                                     title='Распределение количества дней с последнего контакта в предыдущей кампании по целевой переменной')
```

📊 Общая статистика по признаку 'pdays':

count	749999.000000
mean	22.412630
std	77.319998
min	-1.000000
25%	-1.000000
50%	-1.000000
75%	-1.000000
max	871.000000

Name: pdays, dtype: float64

📊 Статистика признака 'pdays' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

count	659511.000000
mean	19.855711
std	75.172515
min	-1.000000
25%	-1.000000
50%	-1.000000
75%	-1.000000
max	871.000000

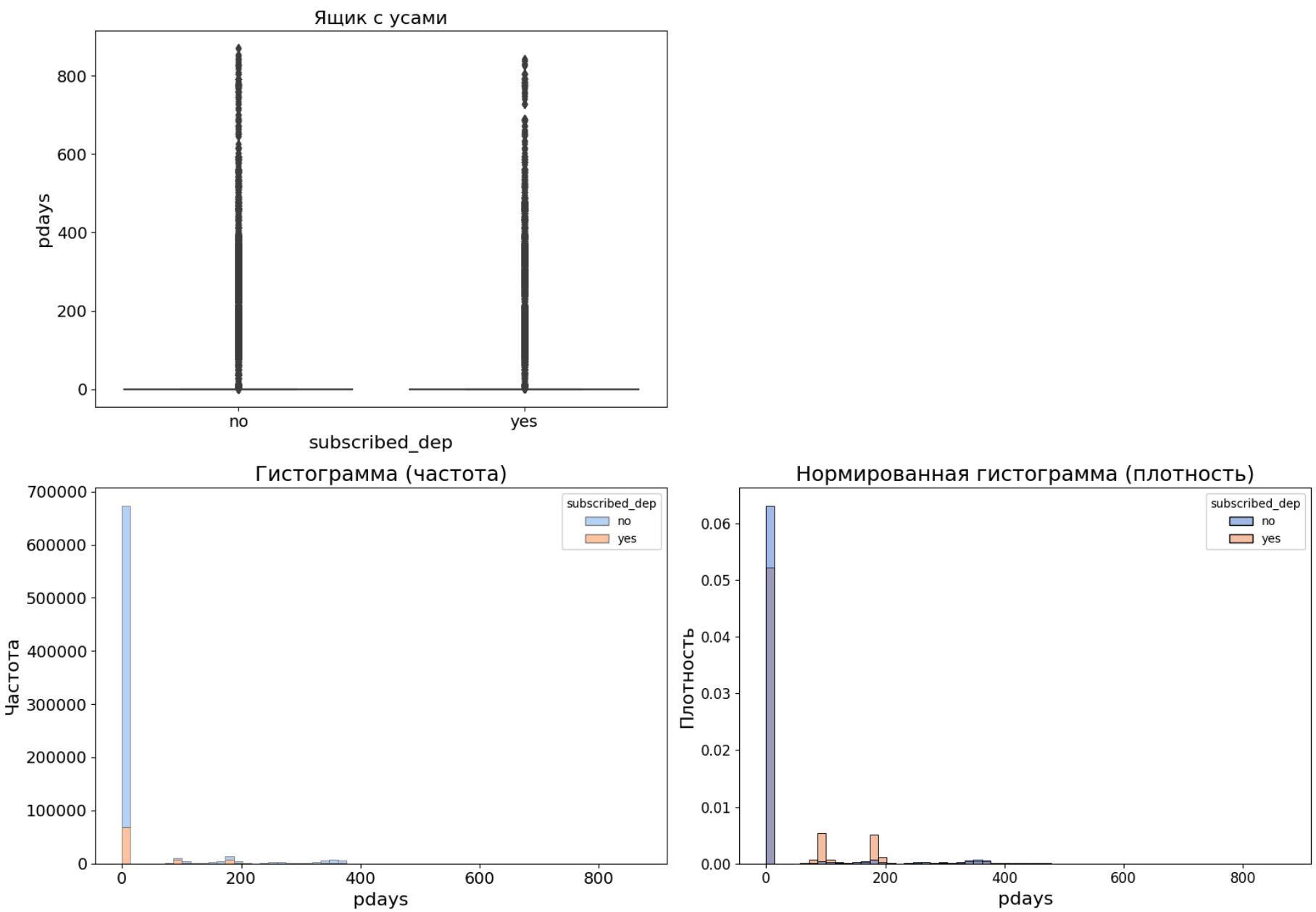
Name: pdays, dtype: float64

subscribed_dep = yes:

count	90488.000000
mean	41.048426
std	89.276459
min	-1.000000
25%	-1.000000
50%	-1.000000
75%	-1.000000
max	842.000000

Name: pdays, dtype: float64

Распределение количества дней с последнего контакта в предыдущей кампании по целевой переменной



```
In [92]: # Делим на группы по целевой переменной
group0 = df_train[df_train['subscribed_dep'] == 'no']['pdays']
group1 = df_train[df_train['subscribed_dep'] == 'yes']['pdays']

# U-критерий Манна-Уитни
stat, pval = mannwhitneyu(group0, group1, alternative='two-sided')
print(f"U-статистика: {stat:.2f}, p-value: {pval:.4f}")

# Интерпретация результата
alpha = 0.05 # уровень значимости

if pval < alpha:
    print("✅ Различия в распределении количества дней с последнего контакта статистически значимы.")
    mean0 = group0.mean()
    mean1 = group1.mean()
    if mean1 > mean0:
        print(f"📈 Подписавшиеся клиенты в среднем имели больший интервал с последнего контакта: {mean1:.1f} vs {mean0:.1f} дней")
    else:
        print(f"📉 Подписавшиеся клиенты в среднем имели меньший интервал с последнего контакта: {mean1:.1f} vs {mean0:.1f} дней")
else:
    print("⚠️ Статистически значимых различий в распределении количества дней с последнего контакта не обнаружено.")
```

U-статистика: 25365802324.50, p-value: 0.0000
✅ Различия в распределении количества дней с последнего контакта статистически значимы.
📈 Подписавшиеся клиенты в среднем имели больший интервал с последнего контакта: 41.0 vs 19.9 дней

- проанализируем без -1

```
In [93]: from scipy.stats import mannwhitneyu

# Убираем -1 (оставляем только реальные интервалы)
group0 = df_train[(df_train['subscribed_dep'] == 'no') & (df_train['pdays'] != -1)]['pdays']
group1 = df_train[(df_train['subscribed_dep'] == 'yes') & (df_train['pdays'] != -1)]['pdays']

# U-критерий Манна-Уитни
stat, pval = mannwhitneyu(group0, group1, alternative='two-sided')
print(f"U-статистика: {stat:.2f}, p-value: {pval:.4f}")

# Интерпретация результата
alpha = 0.05 # уровень значимости

if pval < alpha:
    print("✅ Различия в распределении количества дней с последнего контакта (без -1) статистически значимы.")
    mean0 = group0.mean()
    mean1 = group1.mean()
```

```

if mean1 > mean0:
    print(f"📊 Подписавшиеся клиенты в среднем имели больший интервал: {mean1:.1f} vs {mean0:.1f} дней")
else:
    print(f"📉 Подписавшиеся клиенты в среднем имели меньший интервал: {mean1:.1f} vs {mean0:.1f} дней")
else:
    print("⚠️ Статистически значимых различий в распределении интервала (без -1) не обнаружено.")

```

U-статистика: 865026798.00, p-value: 0.0000

- ✅ Различия в распределении количества дней с последнего контакта (без -1) статистически значимы.
- 📉 Подписавшиеся клиенты в среднем имели меньший интервал: 172.2 vs 246.4 дней

- 'Клиенты с недавним контактом конвертируются лучше',
- 'Среди контактировавших: 172 дня (подписка) vs 246 дней (нет)',

```

In [94]: plot_box_and_hist_by_target(df_train, feature='previous',
                                     title='Распределение количества контактов до текущей кампании по целевой переменной')

```

📊 Общая статистика по признаку 'previous':

```

count    749999.000000
mean      0.298279
std       1.315875
min       0.000000
25%       0.000000
50%       0.000000
75%       0.000000
max       58.000000
Name: previous, dtype: float64

```

📊 Статистика признака 'previous' по значениям целевой переменной 'subscribed_dep':

```

subscribed_dep = no:
count    659511.000000
mean      0.239083
std       1.200375
min       0.000000
25%       0.000000
50%       0.000000
75%       0.000000
max       58.000000
Name: previous, dtype: float64

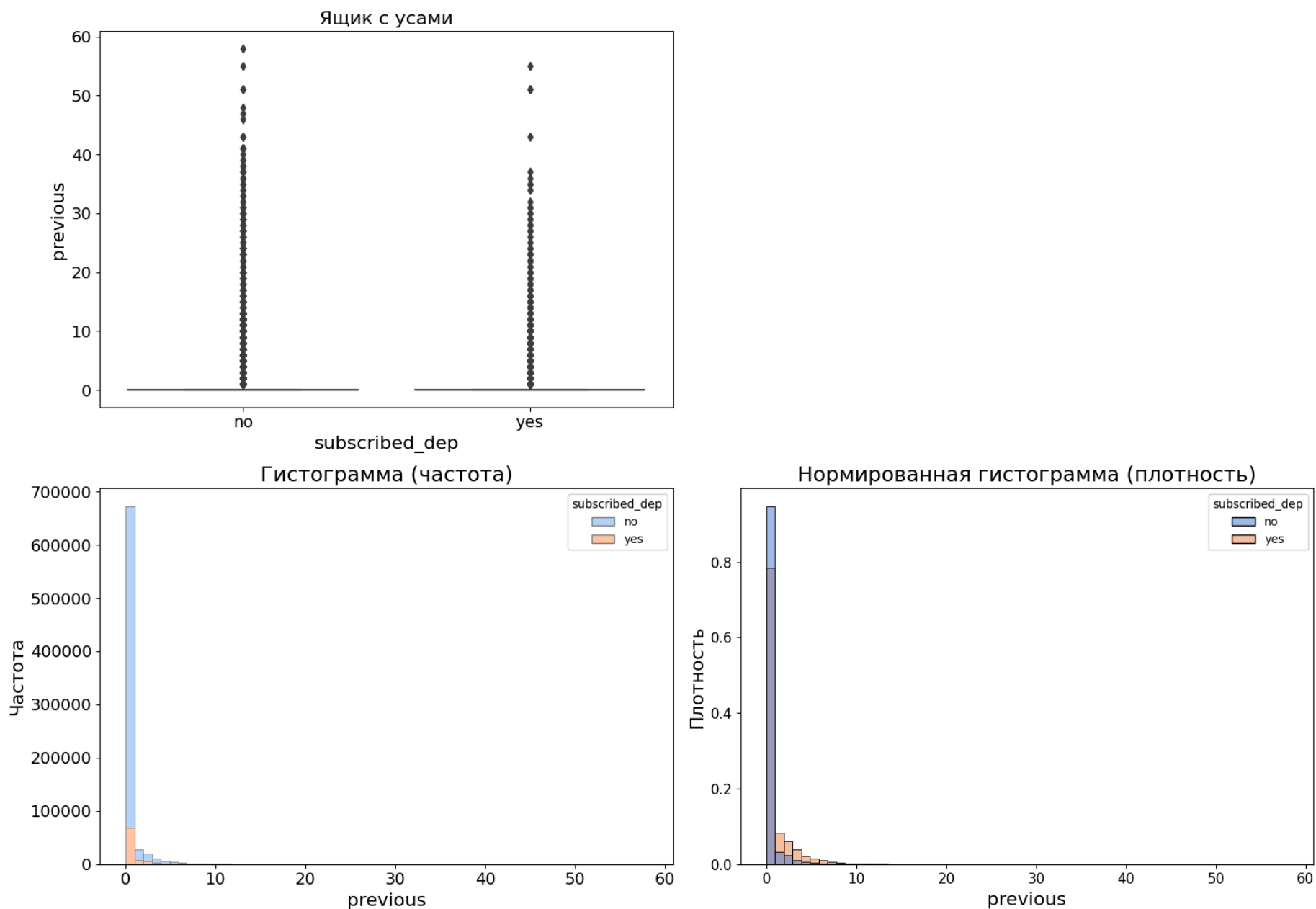
```

```

subscribed_dep = yes:
count    90488.000000
mean      0.729721
std       1.907375
min       0.000000
25%       0.000000
50%       0.000000
75%       0.000000
max       55.000000
Name: previous, dtype: float64

```


Распределение количества контактов до текущей кампании по целевой переменной



```
In [95]: # Делим на группы по целевой переменной
group0 = df_train[df_train['subscribed_dep'] == 'no']['previous']
group1 = df_train[df_train['subscribed_dep'] == 'yes']['previous']

# U-критерий Манна-Уитни
stat, pval = mannwhitneyu(group0, group1, alternative='two-sided')
print(f"U-статистика: {stat:.2f}, p-value: {pval:.4f}")

# Интерпретация результата
alpha = 0.05 # уровень значимости

if pval < alpha:
    print("✅ Различия в распределении количества контактов до текущей кампании статистически значимы.")
    mean0 = group0.mean()
    mean1 = group1.mean()
    if mean1 > mean0:
        print(f"📈 Подписавшиеся клиенты в среднем имели больше предыдущих контактов: {mean1:.1f} vs {mean0:.1f}")
    else:
        print(f"📉 Подписавшиеся клиенты в среднем имели меньше предыдущих контактов: {mean1:.1f} vs {mean0:.1f}")
else:
    print("⚠️ Статистически значимых различий в распределении количества предыдущих контактов не обнаружено.")
```

U-статистика: 25065165669.00, p-value: 0.0000
✅ Различия в распределении количества контактов до текущей кампании статистически значимы.
📈 Подписавшиеся клиенты в среднем имели больше предыдущих контактов: 0.7 vs 0.2

- 'Предыдущие контакты повышают вероятность подписки',
- 'Среднее кол-во: 0.7 (подписка) vs 0.2 (нет)',

Выводы

- age: медиана ~39 лет. Подписавшиеся чаще встречаются среди молодых (до 30) и пожилых (>55).
- balance: распределение с длинным правым хвостом, есть отрицательные значения. Подписавшиеся чаще имеют положительный баланс выше среднего. Логарифмирование или бинирование улучшит интерпретацию.
- duration: главный предиктор — чем дольше длился звонок, тем выше вероятность подписки.
- campaign: большинство клиентов контактировались 1–3 раза. При большом числе контактов вероятность подписки снижается.
- rdays: большинство значений = -1 (нет предыдущего контакта). Для остальных — чем меньше дней прошло, тем выше вероятность подписки.
- previous: у большинства клиентов 0, но при наличии успешного предыдущего контакта вероятность подписки резко возрастает.

Категориальные признаки

```
In [96]: def plot_cat_feature_by_target(data, feature='job', target='subscribed_dep', title=None):
# Удаляем пропуски
df = data[[feature, target]].dropna()

# Общие частоты значений признака
print(f"\nОбщее распределение признака '{feature}':")
counts = df[feature].value_counts()
percentages = df[feature].value_counts(normalize=True) * 100
summary_table = pd.DataFrame({'Частота': counts, 'Процент': percentages.round(2)})
print(summary_table)

# Частоты по целевой переменной
print(f"\nРаспределение признака '{feature}' по значениям целевой переменной '{target}':")
for val in sorted(df[target].unique()):
    print(f"\n{target} = {val}:")
    counts = df[df[target] == val][feature].value_counts()
    percentages = df[df[target] == val][feature].value_counts(normalize=True) * 100
    freq_table = pd.DataFrame({'Частота': counts, 'Процент': percentages.round(2)})
    print(freq_table)

# Заголовок графика по умолчанию
if title is None:
    title = f'Распределение категориального признака "{feature}" по целевой переменной'

fig, axes = plt.subplots(1, 2, figsize=(20, 8))
fig.suptitle(title, fontsize=20)

# Распределение по категориям и целевой переменной
sns.countplot(data=df, x=feature, hue=target, palette='pastel', ax=axes[0])
axes[0].set_title('Распределение по категориям и целевой переменной', fontsize=16)
axes[0].set_xlabel(feature, fontsize=14)
axes[0].set_ylabel('Частота', fontsize=14)
axes[0].tick_params(axis='x', labelsizе=12)
axes[0].tick_params(axis='y', labelsizе=12)
axes[0].legend(title=target, fontsize=12, title_fontsize=13)

# Общее распределение категорий
sns.countplot(data=df, x=feature, palette='pastel', ax=axes[1])
axes[1].set_title('Общее распределение категорий', fontsize=16)
axes[1].set_xlabel(feature, fontsize=14)
axes[1].set_ylabel('Частота', fontsize=14)
axes[1].tick_params(axis='x', labelsizе=12)
axes[1].tick_params(axis='y', labelsizе=12)

plt.tight_layout()
plt.show()
```

```
In [97]: plot_cat_feature_by_target(df_train, feature='marital',
                                     title='Распределение семейного положения по целевой переменной')
```

Общее распределение признака 'marital':

	Частота	Процент
married	480759	64.10
single	194833	25.98
divorced	74407	9.92

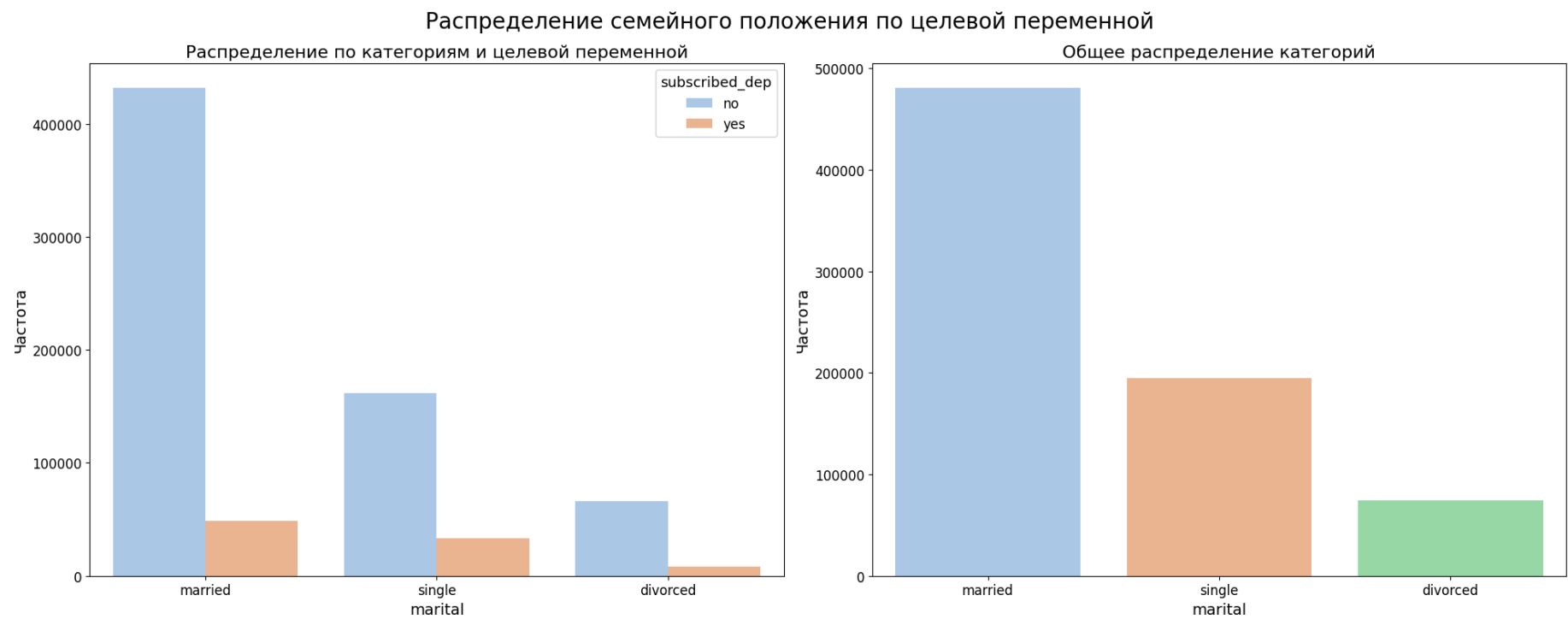
Распределение признака 'marital' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

	Частота	Процент
married	431783	65.47
single	161623	24.51
divorced	66105	10.02

subscribed_dep = yes:

	Частота	Процент
married	48976	54.12
single	33210	36.70
divorced	8302	9.17



```
In [98]: plot_cat_feature_by_target(df_train, feature='education',
                                     title='Распределение уровня образования по целевой переменной')
```

Общее распределение признака 'education':

	Частота	Процент
secondary	401683	53.56
tertiary	227507	30.33
primary	99510	13.27
unknown	21299	2.84

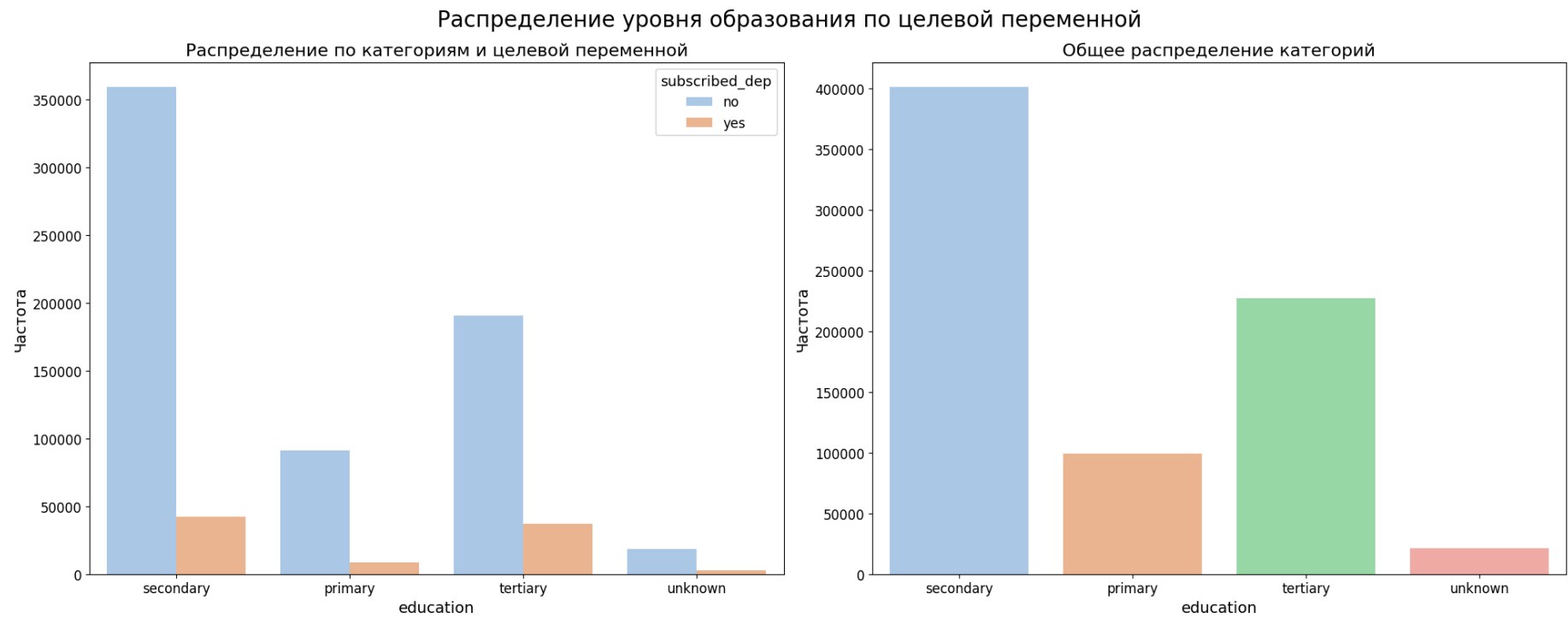
Распределение признака 'education' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

	Частота	Процент
secondary	359309	54.48
tertiary	190503	28.89
primary	91241	13.83
unknown	18458	2.80

subscribed_dep = yes:

	Частота	Процент
secondary	42374	46.83
tertiary	37004	40.89
primary	8269	9.14
unknown	2841	3.14



```
In [99]: plot_cat_feature_by_target(df_train, feature='default',
                                     title='Распределение наличие просроченного кредита по целевой переменной')
```

Общее распределение признака 'default':

	Частота	Процент
no	737150	98.29
yes	12849	1.71

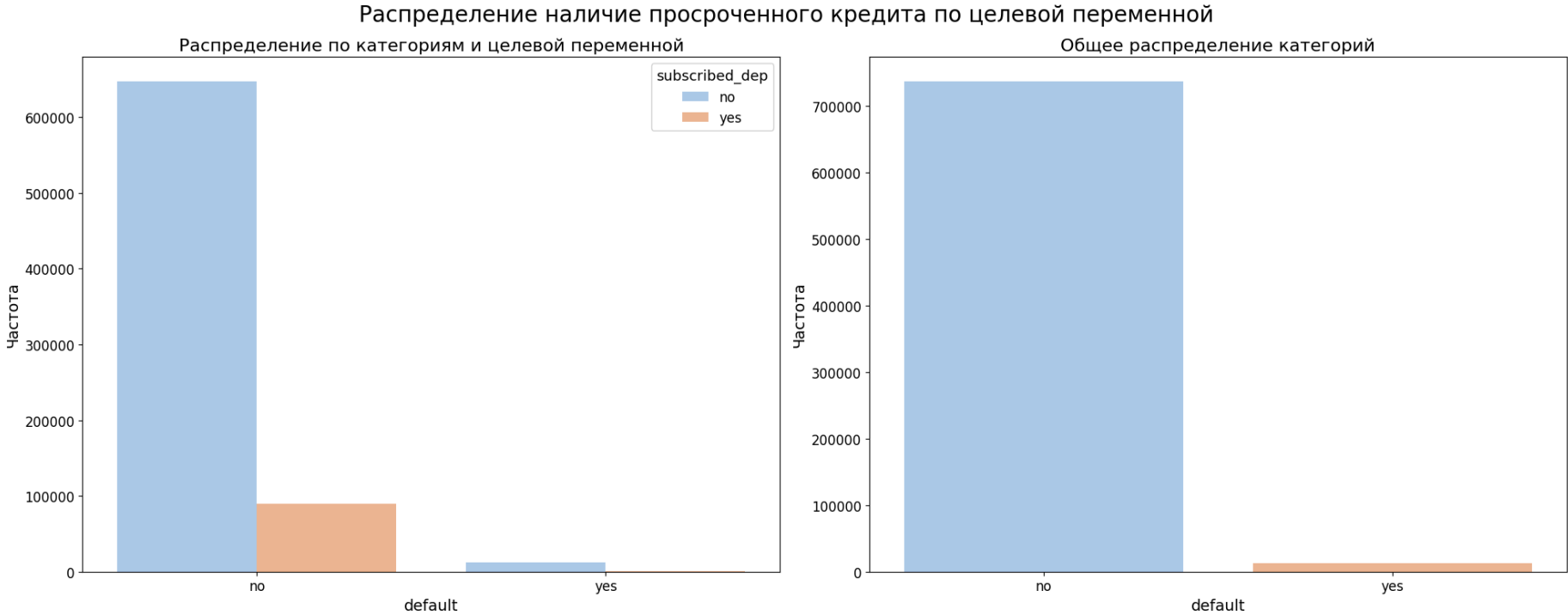
Распределение признака 'default' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

	Частота	Процент
no	647257	98.14
yes	12254	1.86

subscribed_dep = yes:

	Частота	Процент
no	89893	99.34
yes	595	0.66



In [100...

```
plot_cat_feature_by_target(df_train, feature='housing',
                           title='Распределение наличие ипотечного кредита по целевой переменной')
```

Общее распределение признака 'housing':

	Частота	Процент
yes	411288	54.84
no	338711	45.16

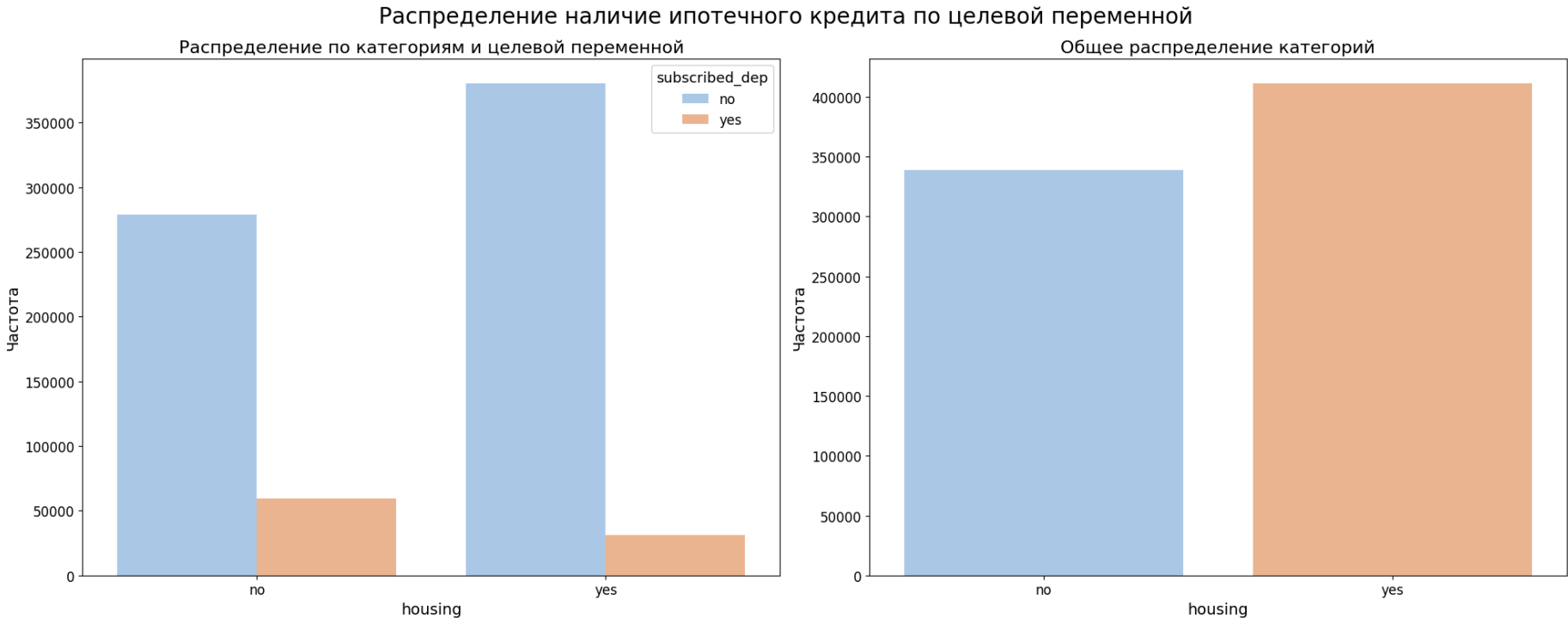
Распределение признака 'housing' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

	Частота	Процент
yes	380338	57.67
no	279173	42.33

subscribed_dep = yes:

	Частота	Процент
no	59538	65.8
yes	30950	34.2



In [101...

```
plot_cat_feature_by_target(df_train, feature='loan',
                           title='Распределение наличие персонального кредита по целевой переменной')
```

Общее распределение признака 'loan':

	Частота	Процент
no	645022	86.0
yes	104977	14.0

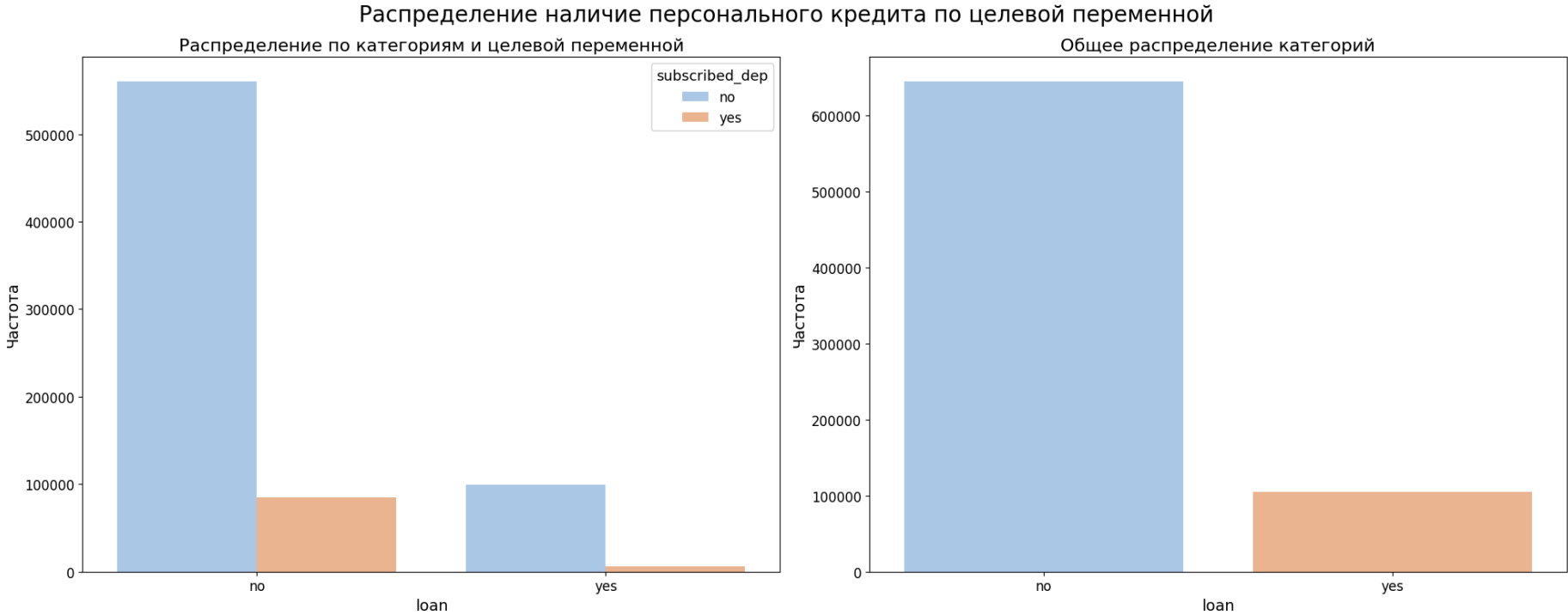
Распределение признака 'loan' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

	Частота	Процент
no	560280	84.95
yes	99231	15.05

subscribed_dep = yes:

	Частота	Процент
no	84742	93.65
yes	5746	6.35



In [102...

```
plot_cat_feature_by_target(df_train, feature='contact',
                           title='Распределение типа канала связи по целевой переменной')
```

Общее распределение признака 'contact':

	Частота	Процент
cellular	486654	64.89
unknown	231627	30.88
telephone	31718	4.23

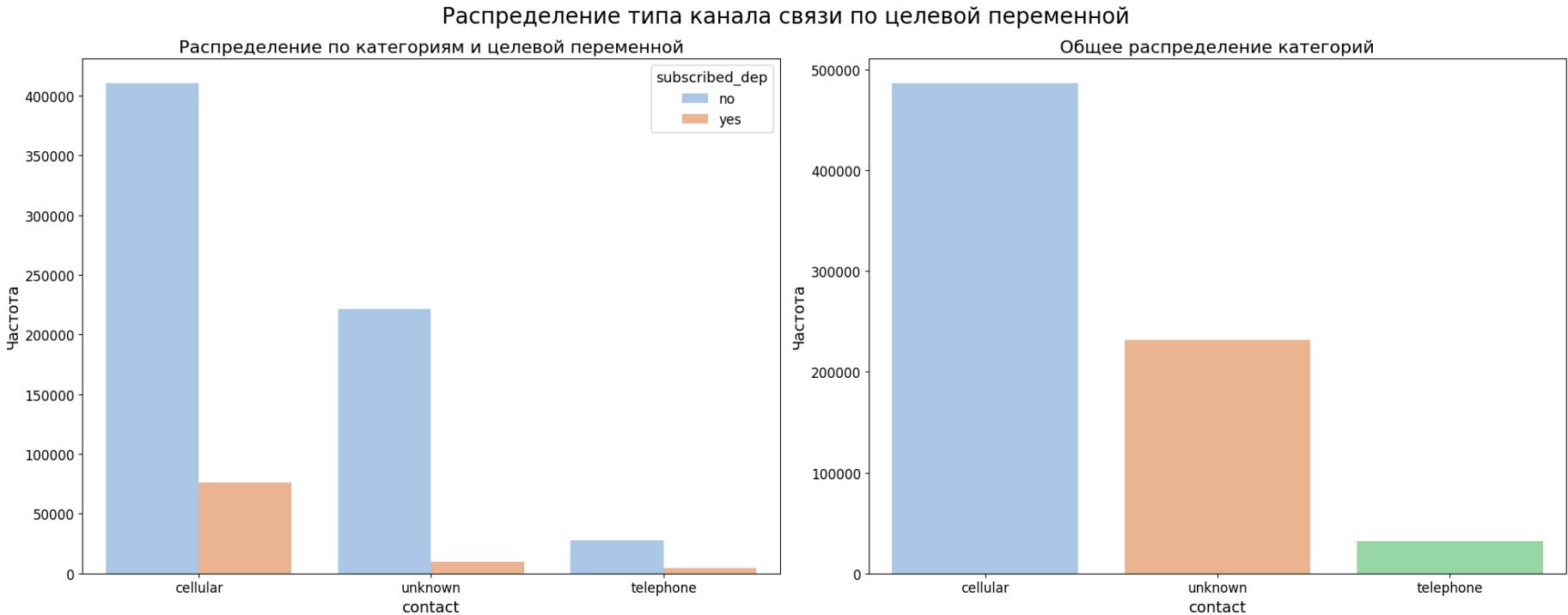
Распределение признака 'contact' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

	Частота	Процент
cellular	410454	62.24
unknown	221678	33.61
telephone	27379	4.15

subscribed_dep = yes:

	Частота	Процент
cellular	76200	84.21
unknown	9949	10.99
telephone	4339	4.80



contact: cellular vs telephone vs unknown — видна разница в отклике; канал важен.

contact: наибольший отклик при контакте через cellular; категория telephone показывает худший результат.

```
In [103... plot_cat_feature_by_target(df_train, feature='poutcome',
                             title='Распределение результата предыдущей маркетинговой кампании по целевой переменной')
```

Общее распределение признака 'poutcome':

	Частота	Процент
unknown	672450	89.66
failure	45115	6.02
success	17691	2.36
other	14743	1.97

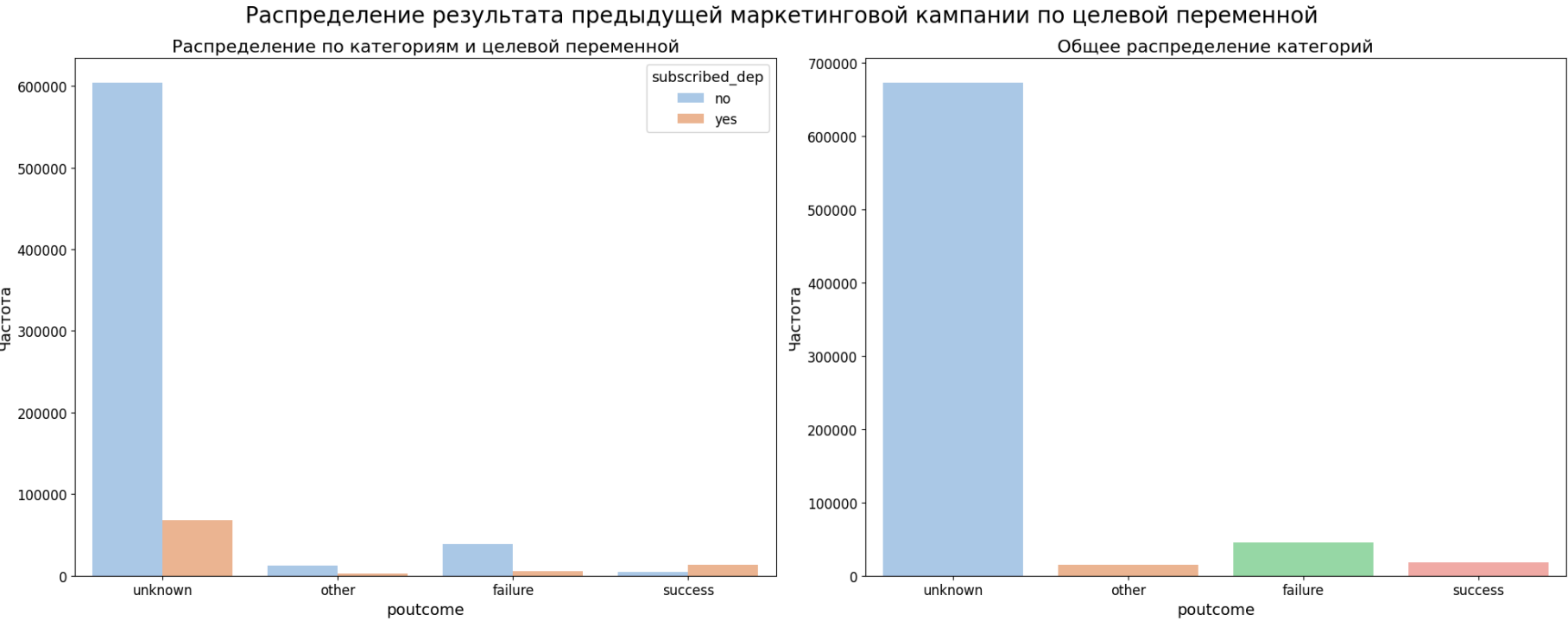
Распределение признака 'poutcome' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

	Частота	Процент
unknown	603929	91.57
failure	39124	5.93
other	12283	1.86
success	4175	0.63

subscribed_dep = yes:

	Частота	Процент
unknown	68521	75.72
success	13516	14.94
failure	5991	6.62
other	2460	2.72



poutcome (результат прошлой кампании): большая часть — unknown (~89.7%), но success явно увеличивает вероятность подписки.

```
In [104... def plot_cat_feature_by_target_vert(data, feature='job', target='subscribed_dep', title=None):
    # Удаляем пропуски
    df = data[[feature, target]].dropna()

    # Общие частоты значений признака
    print(f"\nОбщее распределение признака '{feature}':")
    counts = df[feature].value_counts()
    percentages = df[feature].value_counts(normalize=True) * 100
    summary_table = pd.DataFrame({'Частота': counts, 'Процент': percentages.round(2)})
    print(summary_table)

    # Частоты по целевой переменной
    print(f"\nРаспределение признака '{feature}' по значениям целевой переменной '{target}':")
    for val in sorted(df[target].unique()):
        print(f"\n{target} = {val}:")
        counts = df[df[target] == val][feature].value_counts()
        percentages = df[df[target] == val][feature].value_counts(normalize=True) * 100
        freq_table = pd.DataFrame({'Частота': counts, 'Процент': percentages.round(2)})
        print(freq_table)

    # Заголовок графика по умолчанию
    if title is None:
        title = f'Распределение категориального признака "{feature}" по целевой переменной'

    # Сетка 2 строки x 1 столбец
    fig, axes = plt.subplots(2, 1, figsize=(16, 12), constrained_layout=True)
    fig.suptitle(title, fontsize=20)

    # Распределение по категориям и целевой переменной
    sns.countplot(data=df, x=feature, hue=target, palette='pastel', ax=axes[0])
    axes[0].set_title('Распределение по категориям и целевой переменной', fontsize=16)
    axes[0].set_xlabel(feature, fontsize=14)
    axes[0].set_ylabel('Частота', fontsize=14)
    axes[0].tick_params(axis='x', labelsizе=12)
    axes[0].tick_params(axis='y', labelsizе=12)
    axes[0].legend(title=target, fontsize=12, title_fontsize=13)
```

```

# Общее распределение категорий
sns.countplot(data=df, x=feature, palette='pastel', ax=axes[1])
axes[1].set_title('Общее распределение категорий', fontsize=16)
axes[1].set_xlabel(feature, fontsize=14)
axes[1].set_ylabel('Частота', fontsize=14)
axes[1].tick_params(axis='x', labelsize=12)
axes[1].tick_params(axis='y', labelsize=12)

plt.show()

```

```

In [105... plot_cat_feature_by_target_vert(df_train, feature='job',
                                     title='Распределение типа работы по целевой переменной')

```

Общее распределение признака 'job':

	Частота	Процент
management	175541	23.41
blue-collar	170498	22.73
technician	138106	18.41
admin.	81492	10.87
services	64209	8.56
retired	35185	4.69
self-employed	19020	2.54
entrepreneur	17718	2.36
unemployed	17634	2.35
housemaid	15912	2.12
student	11767	1.57
unknown	2917	0.39

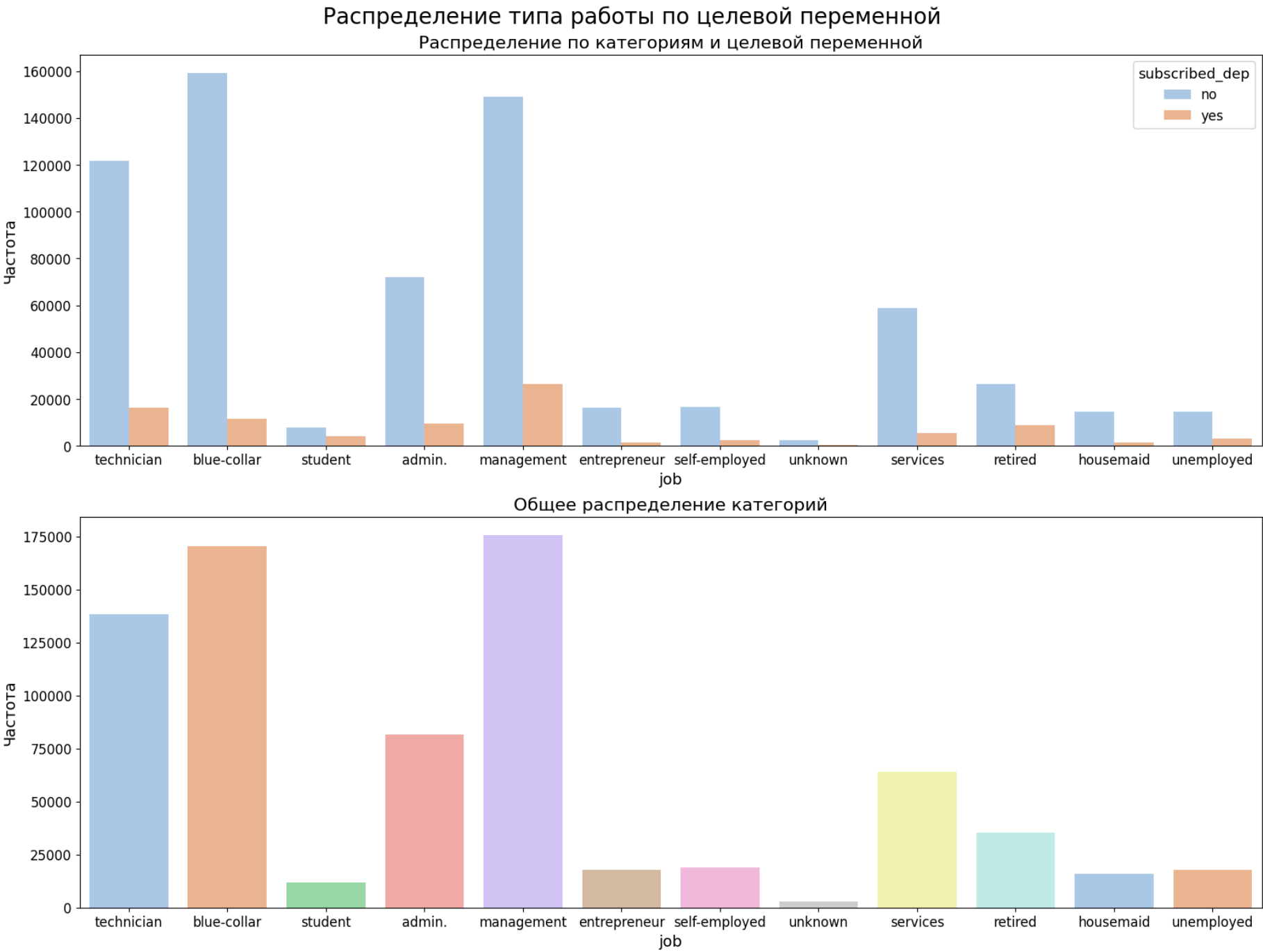
Распределение признака 'job' по значениям целевой переменной 'subscribed_dep':

subscribed_dep = no:

	Частота	Процент
blue-collar	159000	24.11
management	149141	22.61
technician	121765	18.46
admin.	72002	10.92
services	58898	8.93
retired	26521	4.02
self-employed	16558	2.51
entrepreneur	16276	2.47
housemaid	14565	2.21
unemployed	14463	2.19
student	7757	1.18
unknown	2565	0.39

subscribed_dep = yes:

	Частота	Процент
management	26400	29.18
technician	16341	18.06
blue-collar	11498	12.71
admin.	9490	10.49
retired	8664	9.57
services	5311	5.87
student	4010	4.43
unemployed	3171	3.50
self-employed	2462	2.72
entrepreneur	1442	1.59
housemaid	1347	1.49
unknown	352	0.39



In [106...

```
plot_cat_feature_by_target_vert(df_train, feature='month',
                                title='Распределение месяца последнего контакта по целевой переменной')
```


Общее распределение признака 'month':

	Частота	Процент
may	228411	30.45
aug	128859	17.18
jul	110647	14.75
jun	93670	12.49
nov	66062	8.81
apr	41319	5.51
feb	37611	5.01
jan	18937	2.52
oct	9204	1.23
sep	7408	0.99
mar	5802	0.77
dec	2069	0.28

Распределение признака 'month' по значениям целевой переменной 'subscribed_dep':

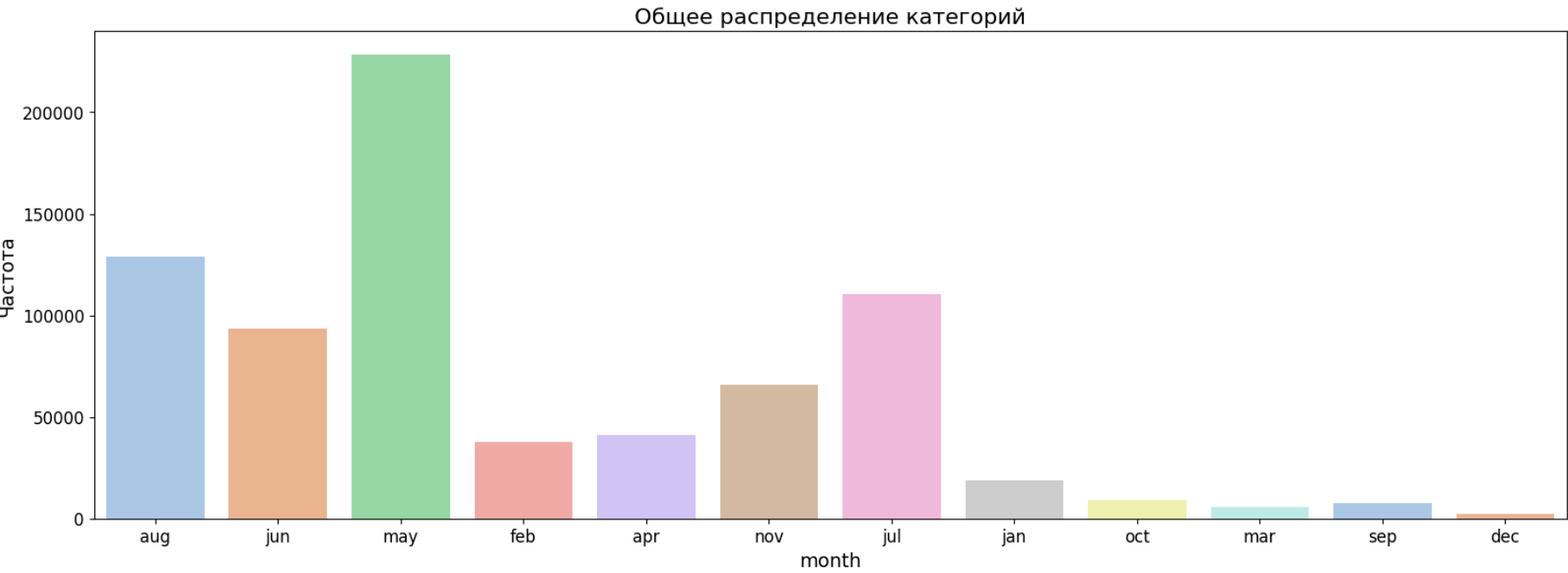
subscribed_dep = no:

	Частота	Процент
may	212113	32.16
aug	114406	17.35
jul	100595	15.25
jun	83954	12.73
nov	58808	8.92
apr	31582	4.79
feb	29833	4.52
jan	16586	2.51
oct	4694	0.71
sep	3446	0.52
mar	2487	0.38
dec	1007	0.15

subscribed_dep = yes:

	Частота	Процент
may	16298	18.01
aug	14453	15.97
jul	10052	11.11
apr	9737	10.76
jun	9716	10.74
feb	7778	8.60
nov	7254	8.02
oct	4510	4.98
sep	3962	4.38
mar	3315	3.66
jan	2351	2.60
dec	1062	1.17

Распределение месяца последнего контакта по целевой переменной



month: сильная сезонность — май, август, июль — наиболее активные месяцы

```
In [107... # Тест хи-квадрат для категориальных признаков
def chi2_test_for_features(df, features, target='y'):
    results = {}
    for feature in features:
        contingency_table = pd.crosstab(df[feature], df[target])
        chi2, p_value, dof, expected = chi2_contingency(contingency_table)
        results[feature] = {
            'chi2': chi2,
            'p_value': p_value,
            'significant': p_value < 0.05
        }
    return pd.DataFrame(results).T
```

```
In [108... # список категориальных признаков для проверки
cat_features = ['job', 'marital', 'education', 'housing', 'default', 'loan', 'contact', 'month', 'poutcome']

# вызов функции
chi2_results = chi2_test_for_features(df_train, cat_features, target='y')

chi2_results
```

Out[108...

	chi2	p_value	significant
job	18558.848952	0.0	True
marital	6210.608061	0.0	True
education	6007.961075	0.0	True
housing	17691.438596	0.0	True
default	680.324822	0.0	True
loan	4997.982944	0.0	True
contact	19179.25565	0.0	True
month	52284.403567	0.0	True
poutcome	71606.103874	0.0	True

- зависимость между категориальными признаком и целевой переменной у статистически значима.
- poutcome (результат предыдущей кампании) и month имеют самые большие значения $\chi^2 \rightarrow$ они дают наибольший вклад в различия между группами.
- job, housing, contact тоже показывают сильную зависимость.

Визуализация временных паттернов

```
In [109... # Анализ сезонности по месяцам
monthly_conversion = df_train.groupby('month')['y'].agg(['count', 'mean']).round(3)
monthly_conversion = monthly_conversion.reindex(['jan', 'feb', 'mar', 'apr', 'may', 'jun', 'jul', 'aug', 'sep', 'oct', 'nov', 'dec'])

monthly_conversion
```

Out[109...

	count	mean
month		
jan	18937	0.124
feb	37611	0.207
mar	5802	0.571
apr	41319	0.236
may	228411	0.071
jun	93670	0.104
jul	110647	0.091
aug	128859	0.112
sep	7408	0.535
oct	9204	0.490
nov	66062	0.110
dec	2069	0.513

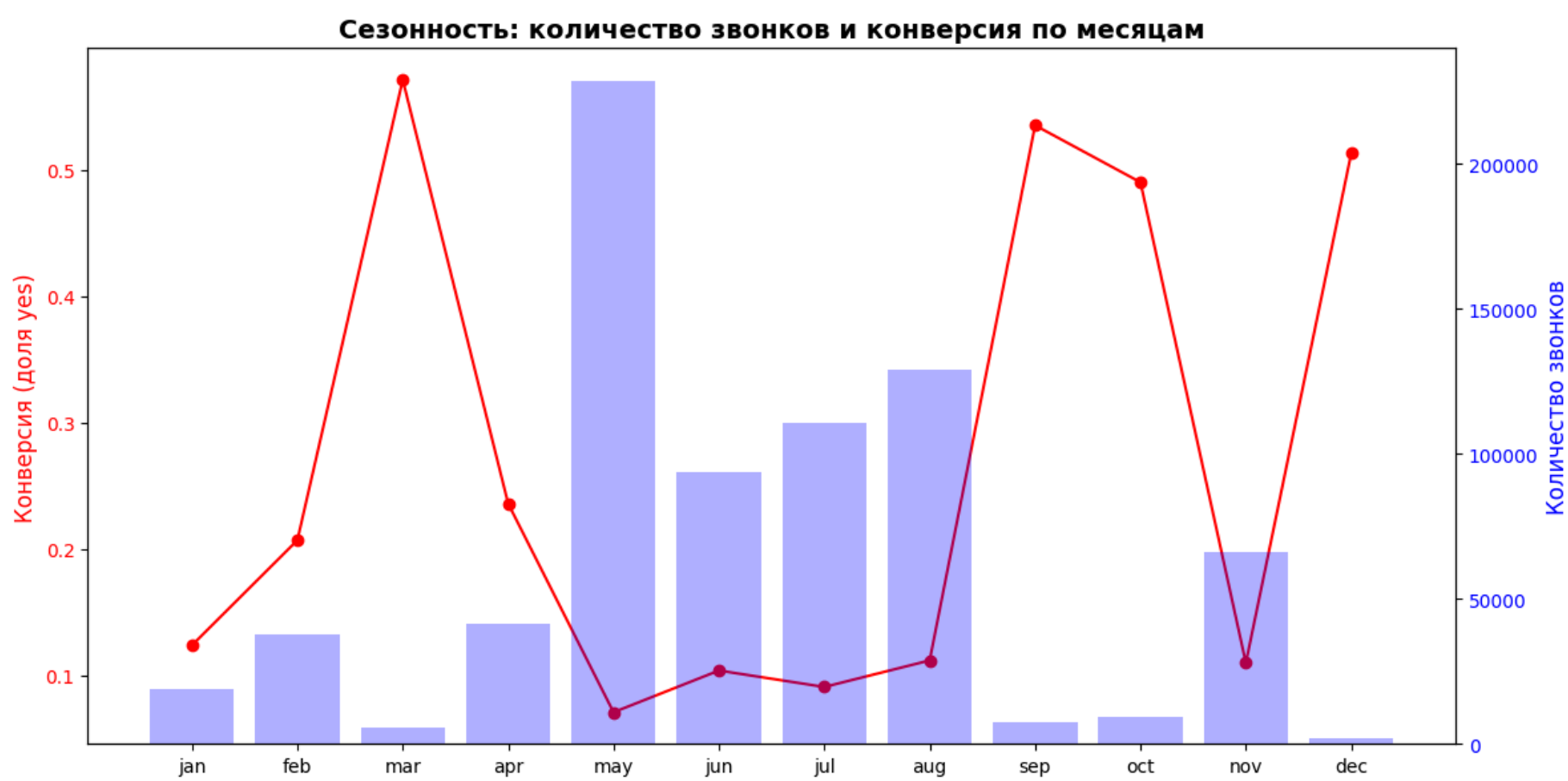
- count показывает, сколько клиентов/звонков/наблюдений было в каждом месяце.
- mean показывает конверсию (например, долю клиентов, которые подписались) в этом месяце.

```
In [110... fig, ax1 = plt.subplots(figsize=(12,6))

# Линия конверсии
ax1.plot(monthly_conversion.index, monthly_conversion['mean'],
        color='red', marker='o', label='Конверсия (mean)')
ax1.set_ylabel('Конверсия (доля yes)', color='red', fontsize=12)
ax1.tick_params(axis='y', labelcolor='red')

# Вторая ось для количества
ax2 = ax1.twinx()
ax2.bar(monthly_conversion.index, monthly_conversion['count'],
        alpha=0.3, color='blue', label='Количество звонков')
ax2.set_ylabel('Количество звонков', color='blue', fontsize=12)
ax2.tick_params(axis='y', labelcolor='blue')

plt.title("Сезонность: количество звонков и конверсия по месяцам", fontsize=14, fontweight="bold")
fig.tight_layout()
plt.show()
```



Объём звонков (count)

- Самый большой объём кампаний приходится на май (228k), август (128k), июль (110k) и июнь (93k).
- В начале года (январь–март) и в конце (сентябрь–декабрь) объёмы заметно меньше.

Конверсия (mean)

- Очень высокая конверсия: март (57%), сентябрь (53%), октябрь (49%), декабрь (51%).
 - хотя звонков в эти месяцы мало, клиенты чаще соглашались на депозит.
- Средняя конверсия: февраль (21%), апрель (24%).
- Низкая конверсия: май (7%), июль (9%), ноябрь (11%).
 - именно в пиковые месяцы по количеству звонков конверсия минимальна.

Вывод:

- В пиковые месяцы (май–август) банки делают массовые обзвоны, но эффективность падает.
- В «спокойные» месяцы (март, сентябрь, октябрь, декабрь) звонков мало, но клиенты более склонны подписываться
- Выявлена интересная зависимость: май (30% звонков) - 7% конверсия, март (0.8% звонков) - 57% конверсия

Анализ взаимодействия признаков по сегментам

age_group

```
In [111... conversion_by_age_group = df_train.groupby('age_group')['y'].agg(['count', 'mean']).round(3)
conversion_by_age_group
```

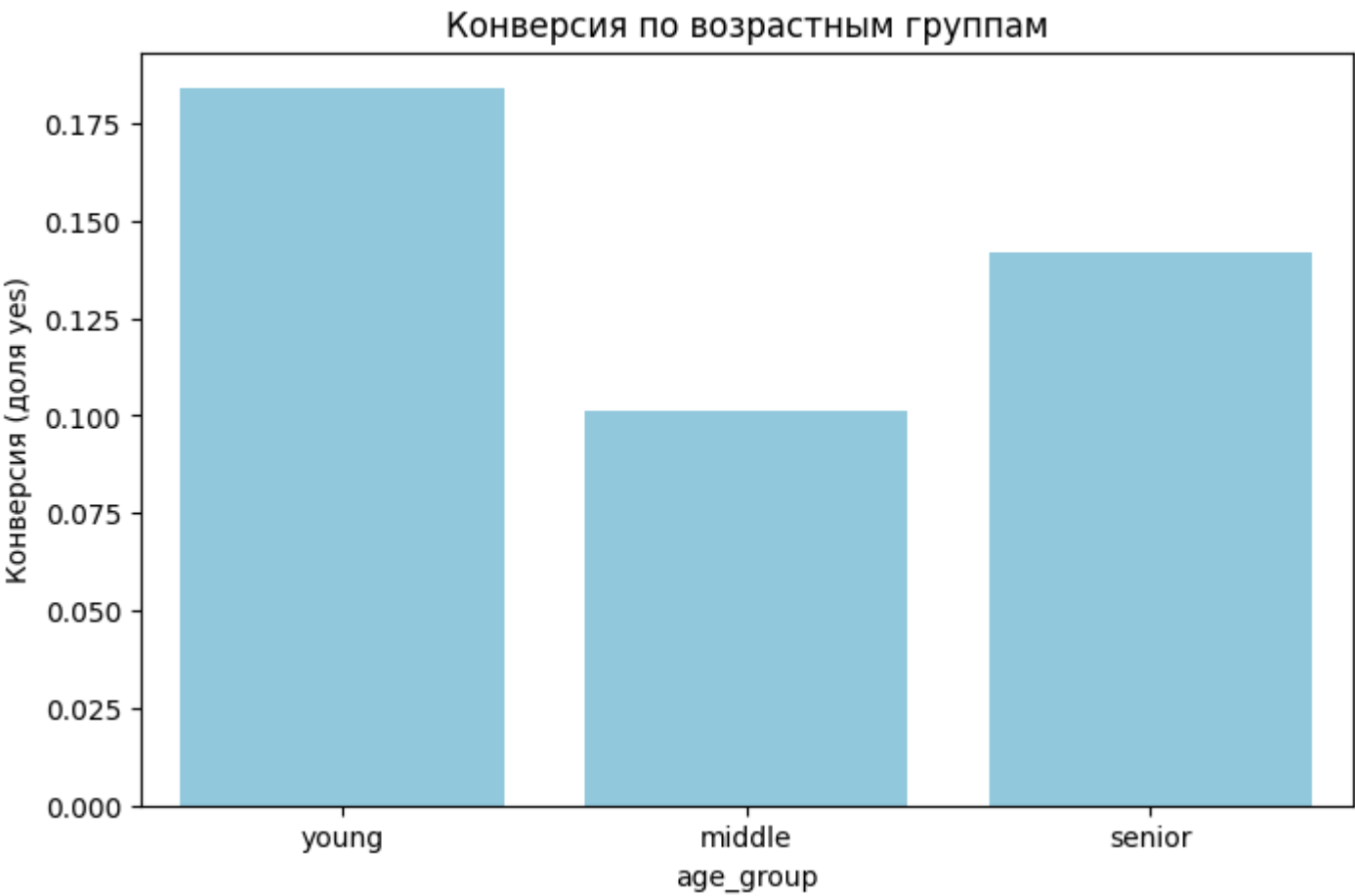
Out[111...

	count	mean
age_group		
young	103774	0.184
middle	496026	0.101
senior	150199	0.142

In [112...

```
conversion_by_age_group = df_train.groupby('age_group')['y'].agg(['count', 'mean']).round(3)

fig, ax1 = plt.subplots(figsize=(8,5))
sns.barplot(x=conversion_by_age_group.index, y=conversion_by_age_group['mean'], ax=ax1, color="skyblue")
ax1.set_ylabel("Конверсия (доля yes)")
ax1.set_title("Конверсия по возрастным группам")
plt.show()
```



Молодые (≤30 лет)

- Количество: 103 775 клиентов
- Конверсия: 18.4%
- Выше среднего: молодые чаще соглашаются на депозит, чем другие группы.

Средний возраст (30–50 лет)

- Количество: 496 026 клиентов (самая большая группа)
- Конверсия: 10.1%
- Несмотря на массовость, именно здесь конверсия минимальна. Это может «тянуть вниз» общую эффективность кампаний.

Старшие (50+ лет)

- Количество: 150 199 клиентов
- Конверсия: 14.2%
- Лучше, чем у «middle», но ниже, чем у «young».

Выводы

- Молодые клиенты — самая перспективная группа по конверсии, хотя их меньше, чем «middle».
- Средний возраст — самая многочисленная, но наименее отзывчивая аудитория. Возможно, стоит сегментировать её глубже (например, по доходу или семейному статусу).
- Старшие клиенты — занимают промежуточное положение: их меньше, чем «middle», но они более склонны подписываться.

age_group и job

In [113...

```
# Таблица 1: количество клиентов
count_table = pd.crosstab(
    df_train['age_group'],
    df_train['job'],
    values=df_train['y'],
    aggfunc='count'
)
```

```
# Таблица 2: средняя конверсия
mean_table = pd.crosstab(
    df_train['age_group'],
    df_train['job'],
    values=df_train['y'],
    aggfunc='mean'
).round(3)

# Вывод
print("Количество клиентов по возрастным группам и профессиям ")
display(count_table)
print("\nКонверсия (доля подписавшихся) по возрастным группам и профессиям ")
display(mean_table)
```

Количество клиентов по возрастным группам и профессиям

	job	admin.	blue-collar	entrepreneur	housemaid	management	retired	self-employed	services	student	technician	unemployed	unknown
age_group													
	young	13765	20254	1334	611	21305	48	2922	11738	9899	18990	2774	1
	middle	55000	123708	12383	8571	124005	1613	12594	44035	1846	99671	11107	14
	senior	12727	26536	4001	6730	30231	33524	3504	8436	22	19445	3753	12

Конверсия (доля подписавшихся) по возрастным группам и профессиям

	job	admin.	blue-collar	entrepreneur	housemaid	management	retired	self-employed	services	student	technician	unemployed	unknown
age_group													
	young	0.150	0.102	0.100	0.106	0.235	0.250	0.239	0.110	0.359	0.183	0.243	0.24
	middle	0.104	0.063	0.077	0.067	0.138	0.068	0.110	0.076	0.243	0.107	0.162	0.10
	senior	0.136	0.061	0.089	0.106	0.144	0.255	0.108	0.081	0.318	0.112	0.186	0.12

Количество клиентов по возрастным группам и профессиям

- Молодые (≤30 лет): больше всего в профессиях management (21k), blue-collar (20k), technician (19k).
- Средний возраст (30–50 лет): самая массовая группа, особенно blue-collar (123k), management (124k), technician (99k).
- Старшие (50+ лет): заметно выделяются retired (33k), management (30k), blue-collar (26k).

Конверсия (доля подписавшихся) по возрастным группам и профессиям

- Молодые: максимальная конверсия у student (35.9%), retired (25%), self-employed (23.9%), unemployed (24.3%).
- Средний возраст: в целом низкая конверсия, но student (24.3%) и unemployed (16.2%) заметно выше среднего.
- Старшие: лидируют retired (25.5%) и student (31.8%), также неплохо unemployed (18.6%).

Эта таблица показывает, какие сегменты наиболее склонны подписываться, даже если их численность мала.

previous_group

```
In [114... conversion_by_previous = df_train.groupby('previous_group')['y'].agg(['count', 'mean']).round(3)
conversion_by_previous
```

	count	mean
previous_group		
few	40915	0.292
many	2164	0.239
none	672431	0.102
one	28342	0.257
several	6147	0.359

none (0 контактов)

- Самая массовая группа: 672 431 клиентов.
- Конверсия всего 10.2% → большинство клиентов впервые слышат предложение и редко соглашаются.

one (1 контакт)

- 28 342 клиентов.
- Конверсия заметно выше — 25.7%. Один предыдущий контакт сильно повышает вероятность подписки.

few (2–5 контактов)

- 40915 клиентов.
- Конверсия ещё выше — 29.2%. Повторные контакты работают.

several (6–10 контактов)

- 6 147 клиентов.
- Максимальная конверсия — 35.9%. Здесь эффект «настойчивости» особенно заметен.

many (11+ контактов)

- 2 165 клиентов.
- Конверсия падает до 23.9%. Вероятно, слишком частые контакты начинают раздражать клиентов.

Выводы

- Оптимум — 2–10 контактов: здесь конверсия максимальна.
- Слишком мало (0) → низкая эффективность.
- Слишком много (11+) → эффективность падает, возможно из-за «усталости» клиентов.
- Зависимость: сначала рост конверсии с числом контактов, потом спад.

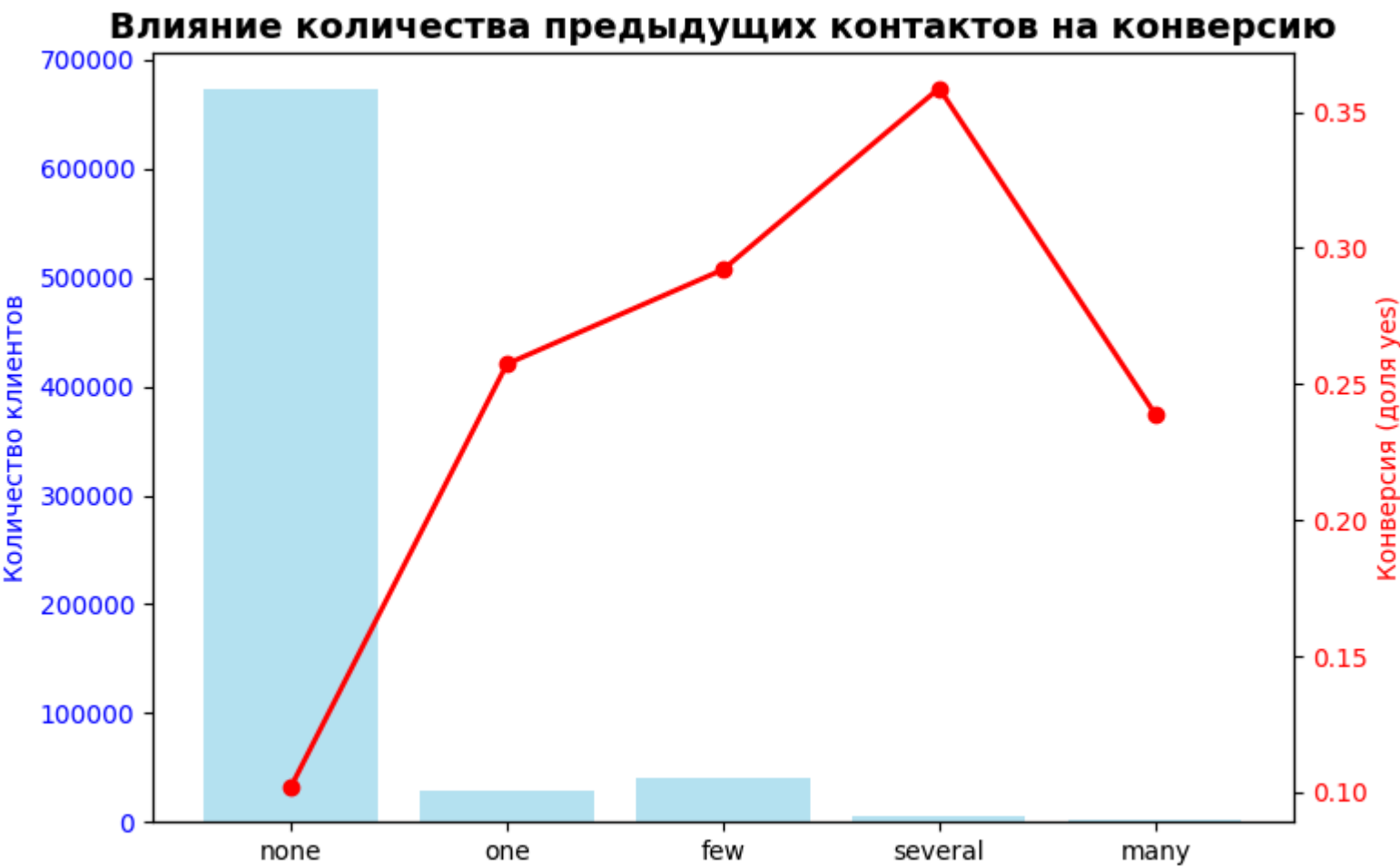
```
In [115... summary = df_train.groupby('previous_group')['y'].agg(['count', 'mean']).reindex(['none', 'one', 'few', 'several', 'many'])

fig, ax1 = plt.subplots(figsize=(8,5))

# Столбцы = количество клиентов
ax1.bar(summary.index, summary['count'], color='skyblue', alpha=0.6, label='Количество клиентов')
ax1.set_ylabel("Количество клиентов", color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

# Линия = конверсия
ax2 = ax1.twinx()
ax2.plot(summary.index, summary['mean'], color='red', marker='o', linewidth=2, label='Конверсия')
ax2.set_ylabel("Конверсия (доля yes)", color='red')
ax2.tick_params(axis='y', labelcolor='red')

plt.title("Влияние количества предыдущих контактов на конверсию", fontsize=14, fontweight="bold")
fig.tight_layout()
plt.show()
```



нелинейная зависимость - оптимально 6-10 контактов

marital

```
In [116... conversion_by_marital = df_train.groupby('marital')['y'].agg(['count', 'mean']).round(3)
conversion_by_marital
```

Out[116...

	count	mean
marital		
divorced	74407	0.112
married	480759	0.102
single	194833	0.170

married

- Самая массовая группа: 480 759 клиентов
- Конверсия всего 10.2% → ниже среднего уровня.

divorced

- 74 407 клиентов
- Конверсия 11.2% → чуть выше, чем у «married», но всё равно низкая.

single

- 194 834 клиента
- Конверсия 17.0% → заметно выше, чем у других категорий.

Выводы

- Семейное положение влияет на вероятность подписки.
- Single — наиболее восприимчивый сегмент (возможно, более гибкие финансовые решения, меньше обязательств).
- Married и Divorced — конверсия ниже, особенно у «married».

education

In [117...

```
conversion_by_education = df_train.groupby('education')['y'].agg(['count', 'mean']).round(3)
conversion_by_education
```

Out[117...

	count	mean
education		
primary	99510	0.083
secondary	401683	0.105
tertiary	227507	0.163
unknown	21299	0.133

primary

- 99 510 клиентов
- Конверсия 8.3% → самая низкая среди всех уровней образования.

secondary

- 401 683 клиента (самая массовая группа)
- Конверсия 10.5% → чуть выше, чем у primary, но всё ещё ниже среднего.

tertiary

- 227 508 клиентов
- Конверсия 16.3% → заметно выше, чем у других групп. Высшее образование коррелирует с большей вероятностью подписки.

unknown

- 21 299 клиентов
- Конверсия 13.3% → промежуточный уровень, но сама группа небольшая.

Выводы

- Образование явно связано с конверсией: чем выше уровень образования, тем выше вероятность подписки.
- Tertiary — ключевой сегмент с высокой конверсией.
- Primary и Secondary — массовые, но менее конверсионные группы.
- Unknown — небольшая группа, но с конверсией выше, чем у secondary.

default

In [118...

```
conversion_by_default = df_train.groupby('default')['y'].agg(['count', 'mean']).round(3)
conversion_by_default
```

Out[118...

	count	mean
default		
no	737150	0.122
yes	12849	0.046

Выводы

- Признак default хоть и сильно несбалансирован (почти все в категории "no"), но он информативен: наличие дефолта резко снижает вероятность подписки.
- Для моделей это может быть важный бинарный предиктор.

housing

In [119...

```
conversion_by_housing = df_train.groupby('housing')['y'].agg(['count', 'mean']).round(3)
conversion_by_housing
```

Out[119...

	count	mean
housing		
no	338711	0.176
yes	411288	0.075

Выводы

- Признак housing — сильный предиктор: наличие кредита явно связано с меньшей склонностью к подписке.
- Возможно, клиенты с ипотекой более финансово ограничены и осторожны в принятии новых обязательств.

loan

In [120...

```
conversion_by_loan = df_train.groupby('loan')['y'].agg(['count', 'mean']).round(3)
conversion_by_loan
```

Out[120...

	count	mean
loan		
no	645022	0.131
yes	104977	0.055

Выводы

- Признак loan — сильный индикатор финансовой нагрузки: наличие кредита резко снижает вероятность подписки.
- В связке с housing и default можно построить целый блок признаков, отражающих долговую нагрузку клиента.

contact

In [121...

```
conversion_by_contact = df_train.groupby('contact')['y'].agg(['count', 'mean']).round(3)
conversion_by_contact
```

Out[121...

	count	mean
contact		
cellular	486654	0.157
telephone	31718	0.137
unknown	231627	0.043

cellular

- 486 655 клиентов
- Конверсия 15.7%
- Самый массовый и самый эффективный канал. Современные мобильные контакты дают заметно лучший результат.

telephone

- 31 718 клиентов
- Конверсия 13.7%
- Традиционный стационарный телефон работает хуже, чем мобильный, но всё же лучше, чем «unknown».

unknown

- 231 627 клиентов
- Конверсия всего 4.3%
- Здесь либо канал не зафиксирован, либо контакт был некачественным. Это самая слабая группа.

Выводы

- Cellular — ключевой канал, на который стоит делать упор.
- Telephone — работает хуже, но может быть полезен для определённых сегментов.
- Unknown — низкая конверсия, скорее всего, это «шум» или неполные данные.

poutcome

```
In [122... conversion_by_poutcome = df_train.groupby('poutcome')['y'].agg(['count', 'mean']).round(3)
conversion_by_poutcome
```

Out[122...

	count	mean
poutcome		
failure	45115	0.133
other	14743	0.167
success	17691	0.764
unknown	672450	0.102

failure

- 45 115 клиентов
- Конверсия 13.3%
- После неудачного исхода прошлой кампании вероятность подписки остаётся низкой.

other

- 14 744 клиентов
- Конверсия 16.7%
- Чуть выше, чем у «failure», но всё ещё невысокая.

success

- 17 691 клиентов
- Конверсия 76.4% (!)
- Если клиент уже соглашался в прошлой кампании, вероятность подписки в текущей огромная.

unknown

- 672 450 клиентов (основная масса)
- Конверсия 10.2%
- Это «шумная» категория: большинство клиентов, про которых нет информации о предыдущем контакте, и они конвертируются хуже всех.

💡 Выводы

- poutcome — один из сильнейших предикторов подписки.
- Категория success — «золотой сегмент» для таргетинга.
- Категория unknown — самая массовая, но с низкой конверсией → именно здесь кроется основной объём работы маркетинга.
- failure и other дают средние результаты, но явно хуже, чем «success».

poutcome: один из ключевых признаков — клиенты, у которых исход прошлой кампании success, имеют кратно выше вероятность подписки (до 65–70%).

month_day

```
In [123... # Таблица 1: количество клиентов (count)
count_table = pd.crosstab(
    df_train['day'],
    df_train['month'],
    values=df_train['y'],
    aggfunc='count'
)

# Таблица 2: средняя конверсия (mean)
mean_table = pd.crosstab(
    df_train['day'],
    df_train['month'],
    values=df_train['y'],
    aggfunc='mean'
```

```
).round(3)

# Опционально: упорядочим месяцы в календарном порядке
month_order = ['jan', 'feb', 'mar', 'apr', 'may', 'jun', 'jul', 'aug', 'sep', 'oct', 'nov', 'dec']
count_table = count_table[month_order]
mean_table = mean_table[month_order]

# Вывод
print("Количество клиентов по дням и месяцам")
display(count_table)

print("\nКонверсия (доля подписавшихся) по дням и месяцам")
display(mean_table)
```

Количество клиентов по дням и месяцам

month	jan	feb	mar	apr	may	jun	jul	aug	sep	oct	nov	dec
day												
1	4.0	64.0	23.0	429.0	3.0	1602.0	674.0	30.0	682.0	359.0	15.0	5.0
2	4.0	8027.0	705.0	446.0	22.0	9186.0	795.0	184.0	261.0	66.0	252.0	55.0
3	3.0	4867.0	329.0	592.0	26.0	7954.0	1104.0	130.0	427.0	17.0	238.0	140.0
4	NaN	7102.0	133.0	16.0	3451.0	5316.0	936.0	4636.0	325.0	43.0	150.0	162.0
5	3.0	6236.0	566.0	170.0	10341.0	7574.0	65.0	5066.0	48.0	59.0	103.0	14.0
6	10.0	3101.0	82.0	714.0	12594.0	7242.0	146.0	5999.0	220.0	361.0	91.0	13.0
7	16.0	16.0	7.0	861.0	13394.0	43.0	5977.0	7405.0	599.0	196.0	17.0	240.0
8	44.0	160.0	117.0	1368.0	12733.0	302.0	6960.0	8061.0	693.0	389.0	135.0	34.0
9	4.0	2684.0	301.0	856.0	7028.0	6705.0	5718.0	121.0	704.0	47.0	375.0	209.0
10	2.0	607.0	432.0	2.0	50.0	20.0	4957.0	837.0	164.0	16.0	387.0	152.0
11	115.0	702.0	306.0	16.0	6911.0	3724.0	3742.0	8964.0	218.0	201.0	303.0	37.0
12	282.0	467.0	265.0	242.0	11231.0	3402.0	182.0	8736.0	23.0	314.0	474.0	4.0
13	212.0	247.0	38.0	888.0	15341.0	53.0	224.0	6531.0	75.0	438.0	437.0	4.0
14	287.0	13.0	13.0	1196.0	17957.0	170.0	5663.0	6648.0	307.0	450.0	60.0	90.0
15	121.0	245.0	188.0	1286.0	19306.0	1127.0	4189.0	23.0	472.0	472.0	40.0	33.0
16	3.0	193.0	213.0	3425.0	6338.0	5514.0	3679.0	76.0	452.0	317.0	649.0	50.0
17	9.0	499.0	246.0	9116.0	104.0	6924.0	4819.0	401.0	136.0	99.0	10950.0	124.0
18	119.0	516.0	278.0	15.0	11657.0	4826.0	4340.0	6553.0	73.0	222.0	12660.0	93.0
19	9.0	184.0	32.0	89.0	5535.0	5132.0	80.0	7210.0	13.0	573.0	11088.0	3.0
20	2.0	10.0	9.0	10514.0	7659.0	13367.0	110.0	6489.0	80.0	469.0	12718.0	4.0
21	5.0	13.0	4.0	560.0	9455.0	28.0	6062.0	5117.0	35.0	575.0	14460.0	81.0
22	12.0	140.0	70.0	784.0	964.0	286.0	4666.0	7291.0	244.0	600.0	16.0	55.0
23	3.0	190.0	161.0	553.0	7499.0	336.0	4999.0	23.0	411.0	323.0	9.0	32.0
24	6.0	253.0	147.0	128.0	73.0	310.0	4897.0	133.0	121.0	45.0	57.0	71.0
25	152.0	138.0	177.0	22.0	1316.0	382.0	4530.0	6207.0	171.0	161.0	218.0	18.0
26	89.0	413.0	262.0	172.0	9116.0	524.0	130.0	5165.0	21.0	310.0	13.0	1.0
27	236.0	503.0	259.0	477.0	10968.0	686.0	197.0	5468.0	73.0	670.0	44.0	5.0
28	6847.0	9.0	16.0	190.0	9960.0	167.0	6810.0	8807.0	125.0	364.0	14.0	170.0
29	7320.0	7.0	45.0	309.0	8797.0	168.0	6219.0	6158.0	127.0	563.0	15.0	130.0
30	3014.0	3.0	249.0	5875.0	8525.0	595.0	7240.0	31.0	94.0	452.0	71.0	32.0
31	4.0	2.0	129.0	8.0	57.0	5.0	10537.0	359.0	14.0	33.0	3.0	8.0

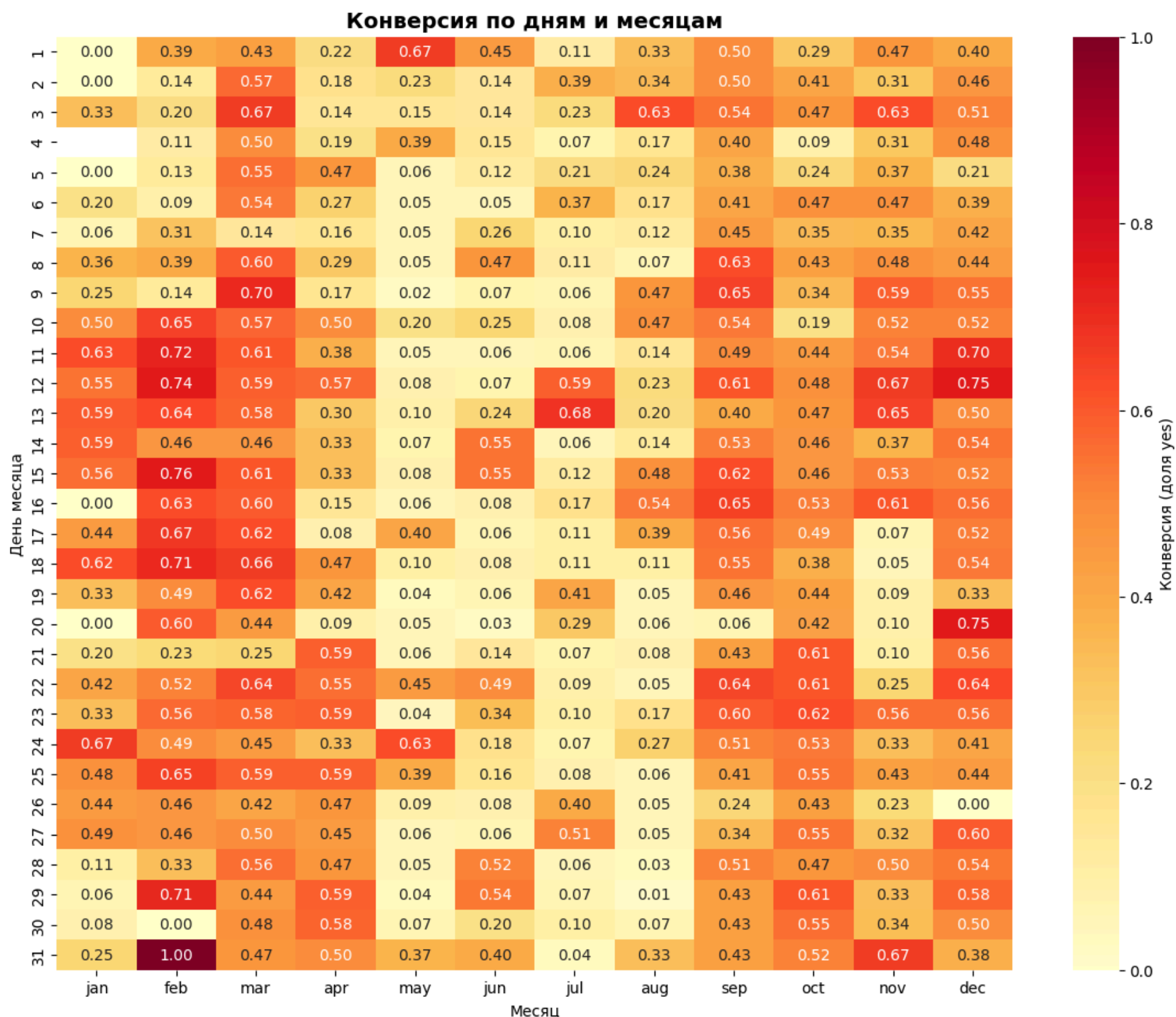
Конверсия (доля подписавшихся) по дням и месяцам

month	jan	feb	mar	apr	may	jun	jul	aug	sep	oct	nov	dec
day												
1	0.000	0.391	0.435	0.217	0.667	0.453	0.111	0.333	0.496	0.295	0.467	0.400
2	0.000	0.135	0.570	0.177	0.227	0.143	0.386	0.342	0.502	0.409	0.306	0.455
3	0.333	0.205	0.666	0.145	0.154	0.142	0.227	0.631	0.543	0.471	0.630	0.514
4	NaN	0.113	0.496	0.188	0.394	0.154	0.072	0.166	0.403	0.093	0.307	0.481
5	0.000	0.132	0.553	0.471	0.056	0.122	0.215	0.239	0.375	0.237	0.369	0.214
6	0.200	0.088	0.537	0.265	0.047	0.048	0.370	0.175	0.405	0.474	0.473	0.385
7	0.062	0.312	0.143	0.163	0.053	0.256	0.100	0.123	0.452	0.347	0.353	0.417
8	0.364	0.394	0.598	0.295	0.047	0.467	0.113	0.067	0.628	0.434	0.481	0.441
9	0.250	0.137	0.704	0.171	0.021	0.072	0.061	0.471	0.645	0.340	0.592	0.550
10	0.500	0.646	0.572	0.500	0.200	0.250	0.079	0.474	0.537	0.188	0.519	0.520
11	0.626	0.718	0.611	0.375	0.054	0.055	0.059	0.145	0.491	0.443	0.538	0.703
12	0.546	0.743	0.589	0.566	0.082	0.071	0.588	0.229	0.609	0.478	0.665	0.750
13	0.594	0.636	0.579	0.303	0.097	0.245	0.683	0.205	0.400	0.468	0.648	0.500
14	0.589	0.462	0.462	0.332	0.074	0.547	0.058	0.140	0.531	0.456	0.367	0.544
15	0.562	0.755	0.606	0.327	0.076	0.547	0.121	0.478	0.623	0.458	0.525	0.515
16	0.000	0.627	0.596	0.149	0.058	0.080	0.173	0.539	0.653	0.527	0.610	0.560
17	0.444	0.669	0.622	0.076	0.404	0.064	0.107	0.387	0.559	0.495	0.070	0.524
18	0.622	0.709	0.662	0.467	0.100	0.078	0.106	0.106	0.548	0.383	0.051	0.538
19	0.333	0.495	0.625	0.416	0.035	0.059	0.412	0.051	0.462	0.436	0.086	0.333
20	0.000	0.600	0.444	0.088	0.048	0.026	0.291	0.055	0.062	0.424	0.097	0.750
21	0.200	0.231	0.250	0.591	0.056	0.143	0.071	0.080	0.429	0.610	0.098	0.556
22	0.417	0.521	0.643	0.552	0.447	0.493	0.087	0.049	0.643	0.607	0.250	0.636
23	0.333	0.563	0.578	0.590	0.041	0.339	0.100	0.174	0.596	0.622	0.556	0.562
24	0.667	0.494	0.449	0.328	0.630	0.184	0.074	0.271	0.512	0.533	0.333	0.408
25	0.480	0.652	0.588	0.591	0.394	0.157	0.084	0.055	0.415	0.547	0.427	0.444
26	0.438	0.460	0.420	0.465	0.086	0.076	0.400	0.051	0.238	0.432	0.231	0.000
27	0.487	0.459	0.502	0.447	0.056	0.060	0.513	0.053	0.342	0.549	0.318	0.600
28	0.109	0.333	0.562	0.468	0.046	0.521	0.058	0.031	0.512	0.467	0.500	0.541
29	0.060	0.714	0.444	0.592	0.037	0.542	0.070	0.010	0.433	0.607	0.333	0.577
30	0.078	0.000	0.482	0.579	0.066	0.200	0.100	0.065	0.426	0.553	0.338	0.500
31	0.250	1.000	0.465	0.500	0.368	0.400	0.036	0.329	0.429	0.515	0.667	0.375

In [124...

```
plt.figure(figsize=(14,11))
sns.heatmap(
    mean_table, # твоя таблица day x month
    cmap="YlOrRd",
    annot=True, fmt=".2f", # подписи с конверсией
    cbar_kws={'label': 'Конверсия (доля yes)'}
)

plt.title("Конверсия по дням и месяцам", fontsize=14, fontweight="bold")
plt.xlabel("Месяц")
plt.ylabel("День месяца")
plt.show()
```



Выводы

- job: студенты и пенсионеры чаще подписываются, рабочие (blue-collar) — реже.
- marital: одинокие клиенты более склонны к подписке, чем женатые/замужние.
- education: более высокий уровень образования (tertiary) положительно коррелирует с подпиской.
- housing и loan: наличие кредитов снижает вероятность подписки.
- contact: звонки по мобильному телефону (cellular) эффективнее, чем по стационарному или при неизвестном канале.
- routcome: «success» в предыдущей кампании сильно повышает вероятность подписки.

Взаимодействия признаков

- Возраст × Профессия: студенты и пенсионеры конвертируют лучше среднего возраста.
- Количество контактов: оптимальный диапазон — 2–10 контактов (конверсия до 35.9%). Свыше 10 — эффект "усталости".
- Канал связи: cellular (15.7%) значительно эффективнее unknown (4.3%) и telephone (13.7%).

Матрица корреляций(heatmap) для выявления зависимостей

In [125... df_train.corr(numeric_only=True)

Out[125...

	id	age	balance	day	duration	campaign	pdays	previous	y
id	1.000000	-0.000222	-0.000282	-0.001692	0.000926	0.000647	-0.000552	-0.000015	0.000616
age	-0.000222	1.000000	0.062837	-0.015179	-0.004389	0.002051	-0.021633	0.004890	0.009523
balance	-0.000282	0.062837	1.000000	-0.008269	0.109629	-0.027745	0.010040	0.034500	0.122513
day	-0.001692	-0.015179	-0.008269	1.000000	-0.056755	0.178806	-0.086197	-0.051882	-0.049625
duration	0.000926	-0.004389	0.109629	-0.056755	1.000000	-0.083017	0.047556	0.040901	0.519283
campaign	0.000647	0.002051	-0.027745	0.178806	-0.083017	1.000000	-0.061464	-0.026997	-0.075829
pdays	-0.000552	-0.021633	0.010040	-0.086197	0.047556	-0.061464	1.000000	0.570198	0.089277
previous	-0.000015	0.004890	0.034500	-0.051882	0.040901	-0.026997	0.570198	1.000000	0.121449
y	0.000616	0.009523	0.122513	-0.049625	0.519283	-0.075829	0.089277	0.121449	1.000000

In [126...

```
corr = df_train.corr(numeric_only=True)
pairs = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
pairs[pairs > 0.1].stack().sort_values(ascending=False)
```

Out[126...

pdays previous 0.570198
duration y 0.519283
day campaign 0.178806
balance y 0.122513
previous y 0.121449
balance duration 0.109629
dtype: float64

In [127...

```
interval_cols = df_train.select_dtypes(include=['int64', 'float64']).columns.tolist()

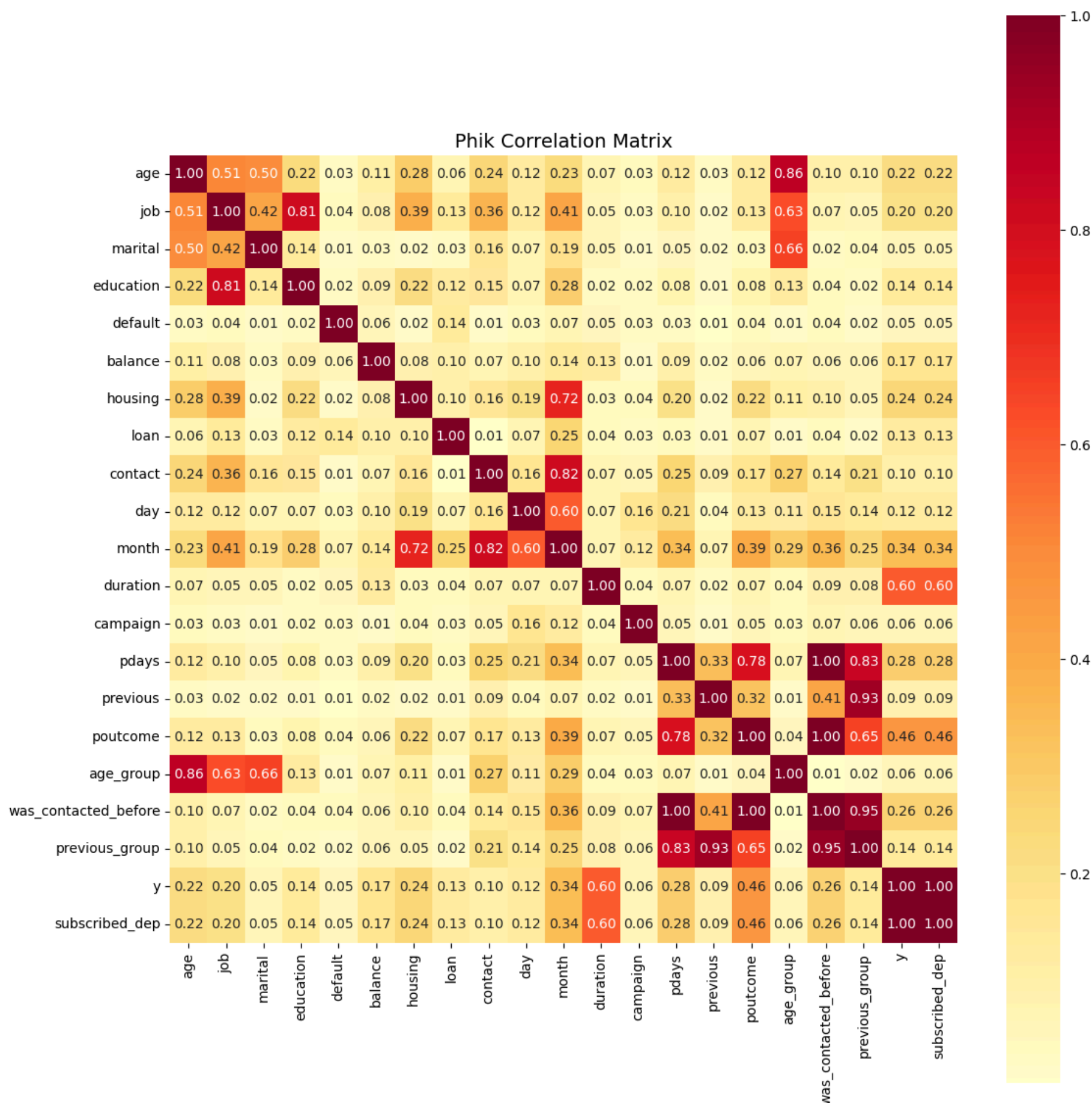
# Вычисление Phik-матрицы (исключаем 'id')
phik_corr = df_train.drop(columns=['id']).phik_matrix(interval_cols=interval_cols)

# 📊 Корреляции с целевой переменной y
bpm_phik = phik_corr['y'].drop('y').sort_values(ascending=False)
print("📊 Phik-корреляции с целевой переменной 'y':")
print(bpm_phik)

# 🔥 Визуализация полной Phik-матрицы
plt.figure(figsize=(12, 12))
sns.heatmap(phik_corr, annot=True, fmt=".2f", cmap='YlOrRd', square=True)
plt.title('Phik Correlation Matrix', fontsize=14)
plt.tight_layout()
plt.show()
```

📊 Phik-корреляции с целевой переменной 'y':

subscribed_dep 1.000000
duration 0.603348
poutcome 0.457960
month 0.340005
pdays 0.280308
was_contacted_before 0.263057
housing 0.238913
age 0.220449
job 0.202701
balance 0.169212
previous_group 0.140423
education 0.135039
loan 0.127867
day 0.124277
contact 0.096550
previous 0.090132
campaign 0.057742
age_group 0.055670
marital 0.054833
default 0.047259
Name: y, dtype: float64



```
In [128... # Исключаем целевые колонки из матрицы
cols_to_exclude = ['y', 'subscribed_dep']
phik_corr_filtered = phik_corr.drop(index=cols_to_exclude, columns=cols_to_exclude, errors='ignore')

# Строим маску для верхнего треугольника с порогом корреляции
high_corr_mask = np.triu(np.abs(phik_corr_filtered) > 0.4, k=1)

# Выводим пары с высокой Phi-корреляцией
print("Пары признаков с Phi-корреляцией > 0.4 (без 'y' и 'subscribed_dep'):")
high_corr_pairs = phik_corr_filtered.where(high_corr_mask).stack().dropna().sort_values(ascending=False)
print(high_corr_pairs if not high_corr_pairs.empty else "Нет пар с высокой корреляцией.")
```

Пары признаков с Phi-корреляцией > 0.4 (без 'y' и 'subscribed_dep'):

poutcome	was_contacted_before	1.000000
pdays	was_contacted_before	0.999595
was_contacted_before	previous_group	0.952321
previous	previous_group	0.933428
age	age_group	0.861639
pdays	previous_group	0.833729
contact	month	0.817168
job	education	0.813384
pdays	poutcome	0.783172
housing	month	0.717491
marital	age_group	0.655230
poutcome	previous_group	0.652659
job	age_group	0.628989
day	month	0.597541
age	job	0.507314
	marital	0.500385
job	marital	0.418664
	month	0.408645
previous	was_contacted_before	0.405840

dtype: float64

Попарные графики (pairplot) для количественных признаков

In [129...

```
start = time.time()

# Удалим 'id' и 'y', если они есть
df_plot = df_train.drop(columns=['id', 'y'], errors='ignore')

# Оставим только числовые признаки
numeric_cols = df_plot.select_dtypes(include=['int64', 'float64']).columns.tolist()

# Добавим 'subscribed_dep' для фильтрации
if 'subscribed_dep' not in df_plot.columns:
    raise ValueError("В датафрейме нет колонки 'subscribed_dep'")

# 🔥 Уменьшаем выборку (минимальное изменение, максимальное ускорение)
df_small = df_plot.sample(50000, random_state=42)
df_no_small = df_small[df_small['subscribed_dep'] == 'no']
df_yes_small = df_small[df_small['subscribed_dep'] == 'yes']

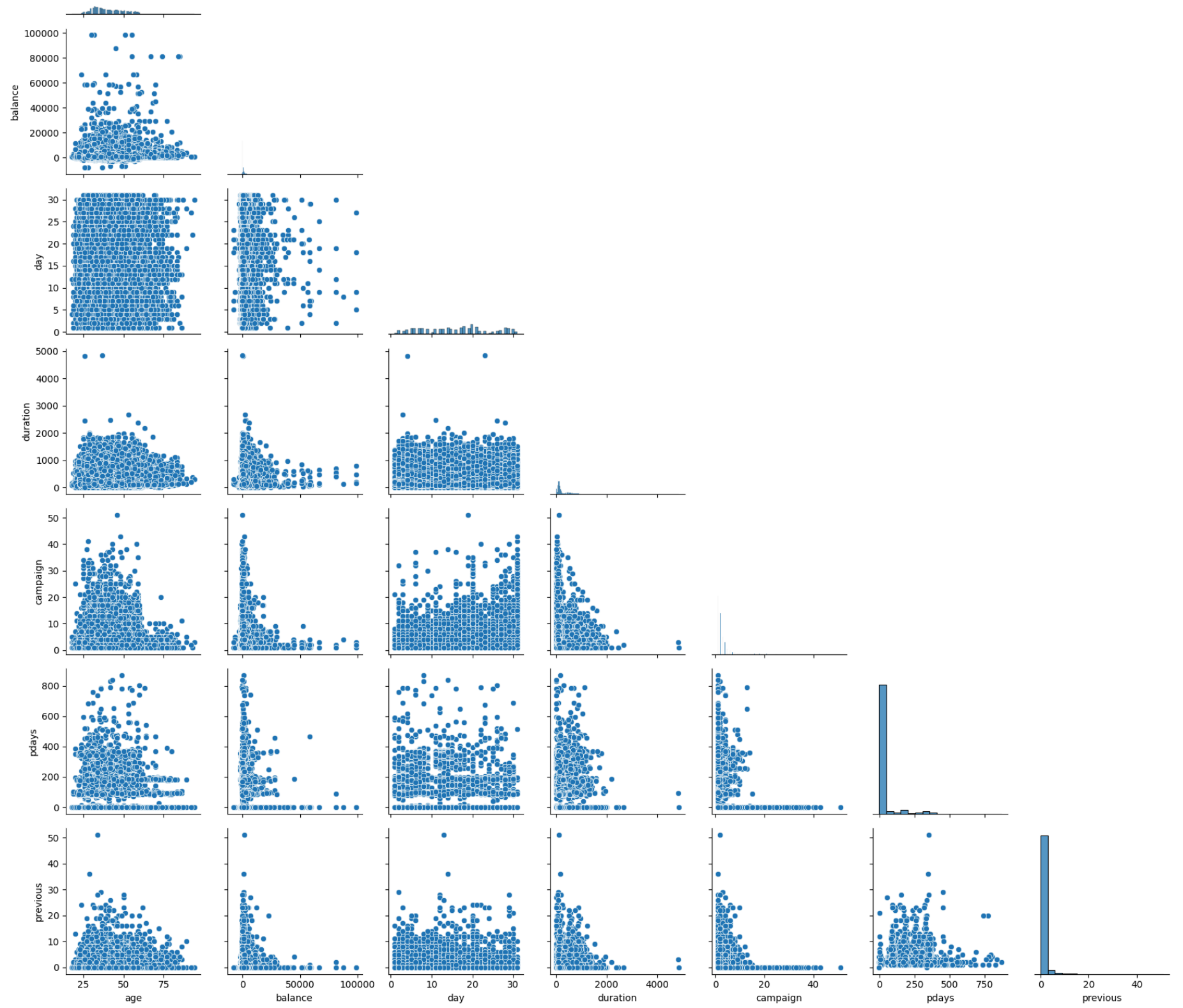
# 1. Pairplot для уменьшенной выборки
sns.pairplot(df_small[numeric_cols + ['subscribed_dep']], corner=True)
plt.suptitle("Pairplot по всем данным (sample 50000)", fontsize=16, y=1.02)
plt.tight_layout()
plt.show()

# 2. Только для subscribed_dep == 'no'
sns.pairplot(df_no_small[numeric_cols], corner=True)
plt.suptitle("Pairplot для subscribed_dep = 'no' (sample 50000)", fontsize=16, y=1.02)
plt.tight_layout()
plt.show()

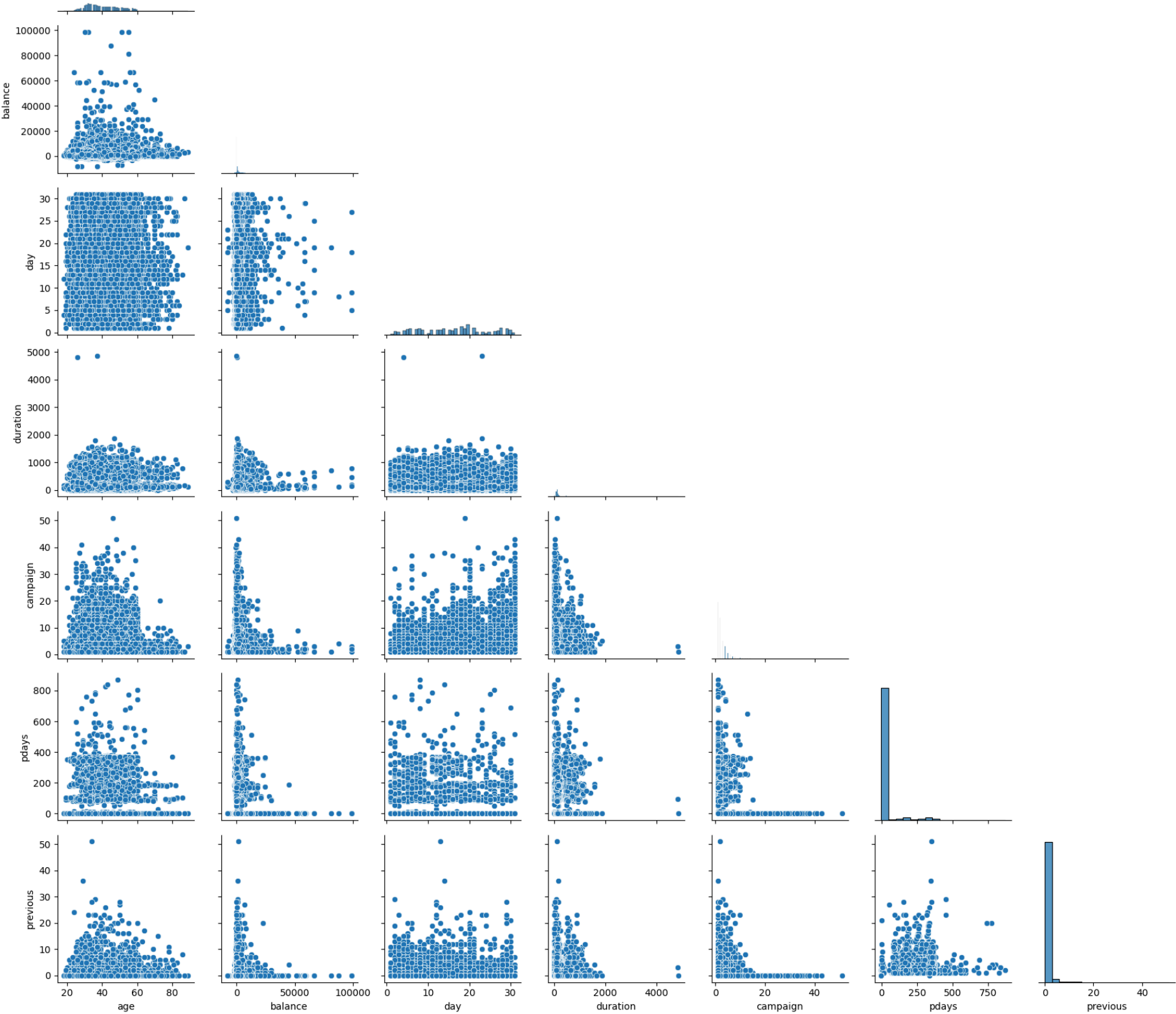
# 3. Только для subscribed_dep == 'yes'
sns.pairplot(df_yes_small[numeric_cols], corner=True)
plt.suptitle("Pairplot для subscribed_dep = 'yes' (sample 50000)", fontsize=16, y=1.02)
plt.tight_layout()
plt.show()

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")
```

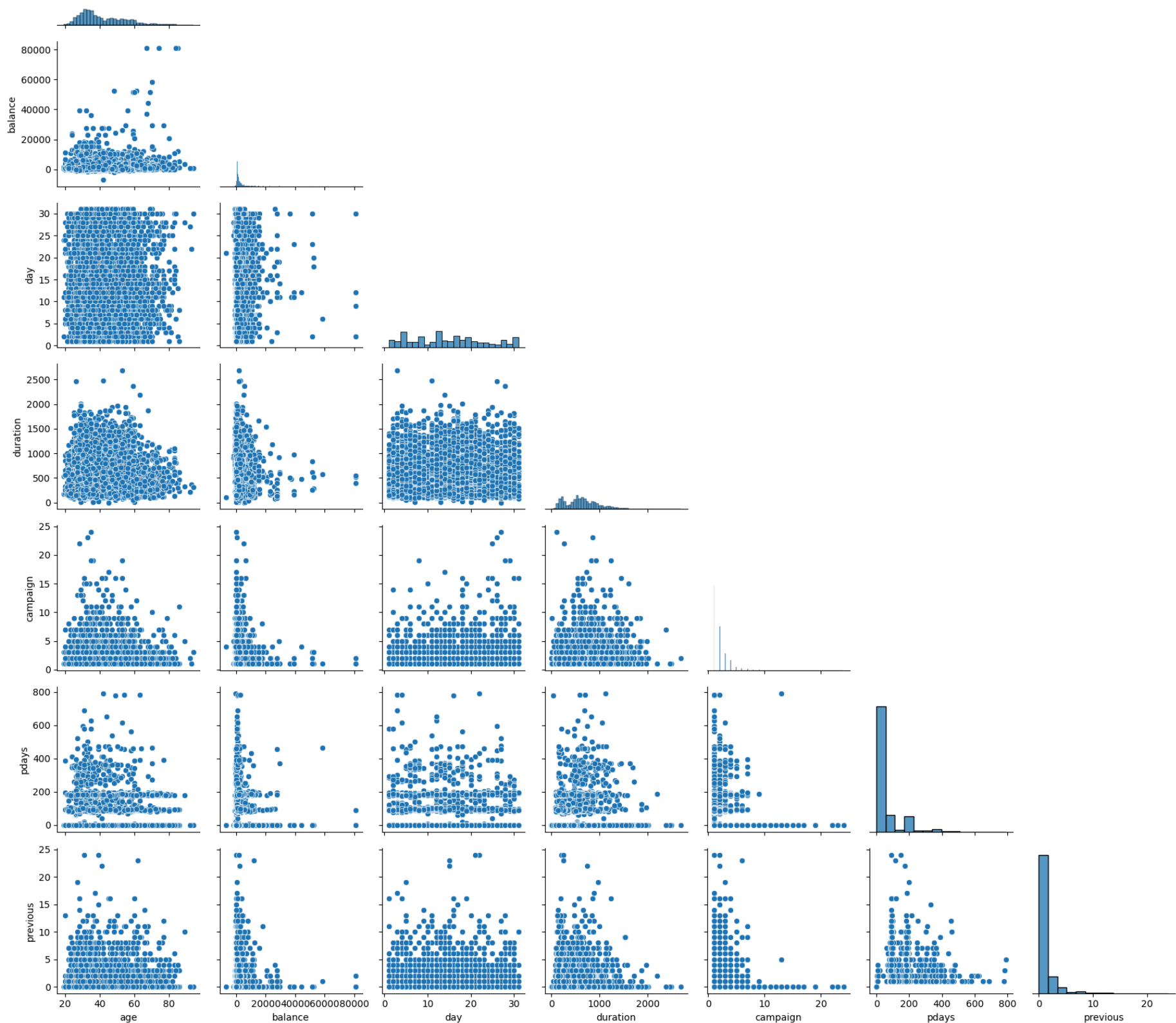

Pairplot по всем данным (sample 50000)



Pairplot для subscribed_dep = 'no' (sample 50000)



Pairplot для subscribed_dep = 'yes' (sample 50000)



Время выполнения: 16.21 сек (0.27 мин)

Кросс-таблицы + тепловые карты для категориальных признаков

```
In [130...] def plot_multiple_heatmaps(df, pairs, figsize=(14, 4), annot_size=14, cmap='YlGnBu'):

    for row, col, title in pairs:
        cross = pd.crosstab(df[row], df[col])

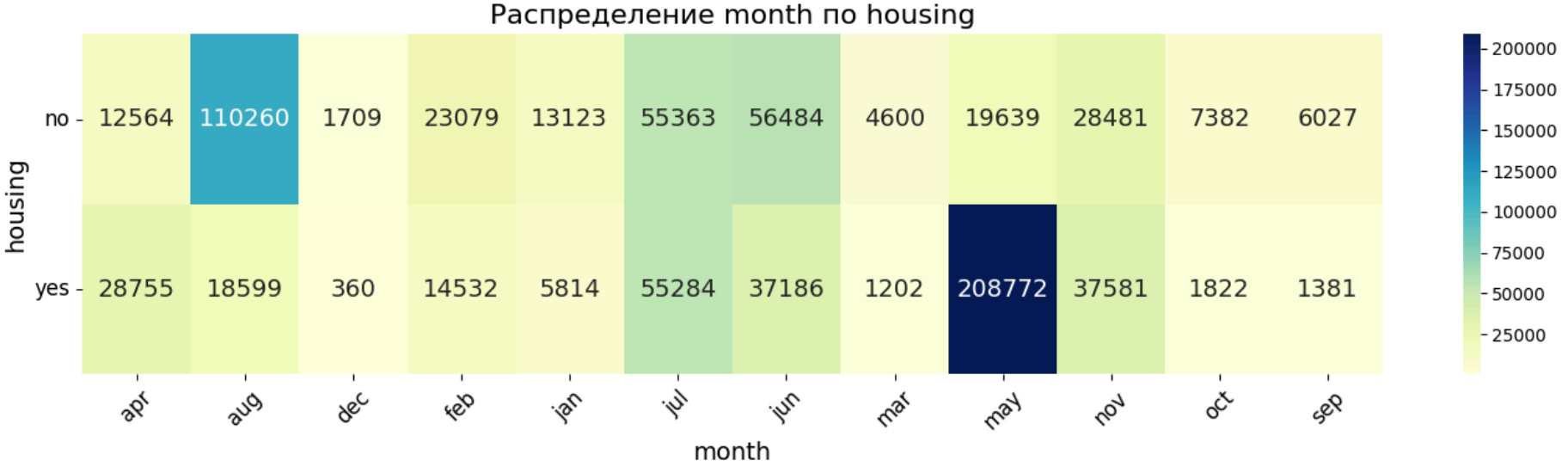
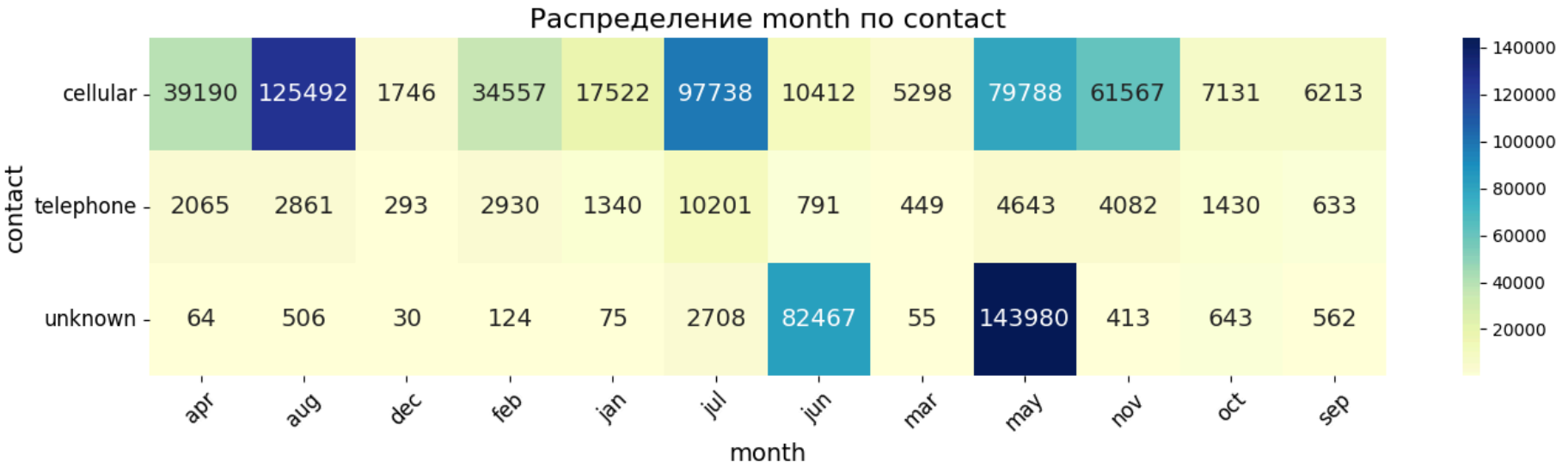
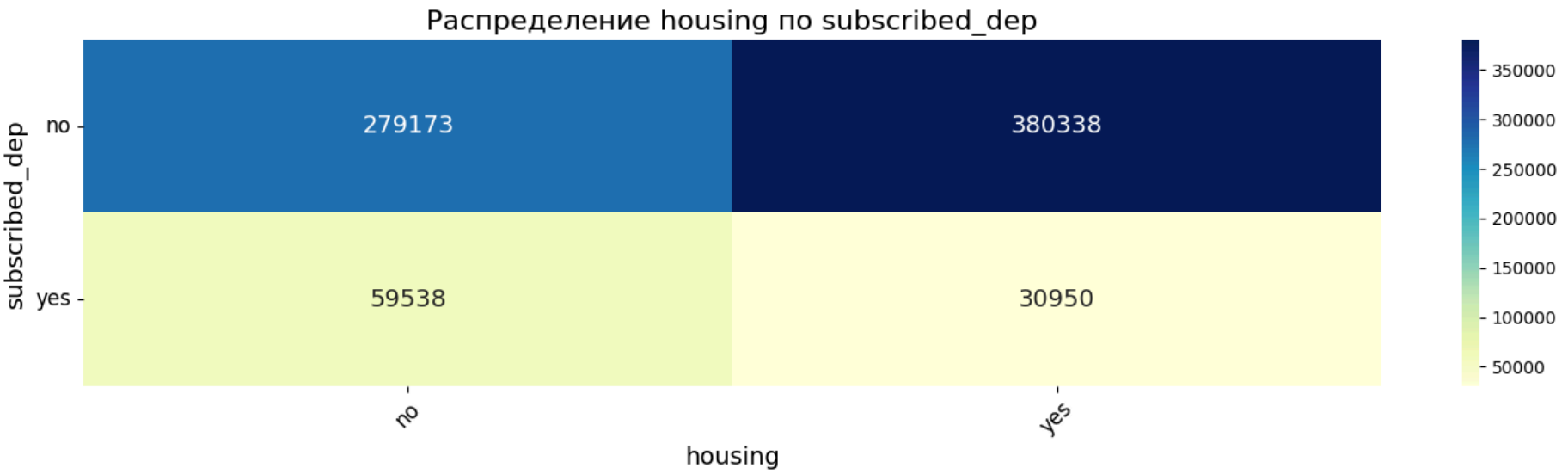
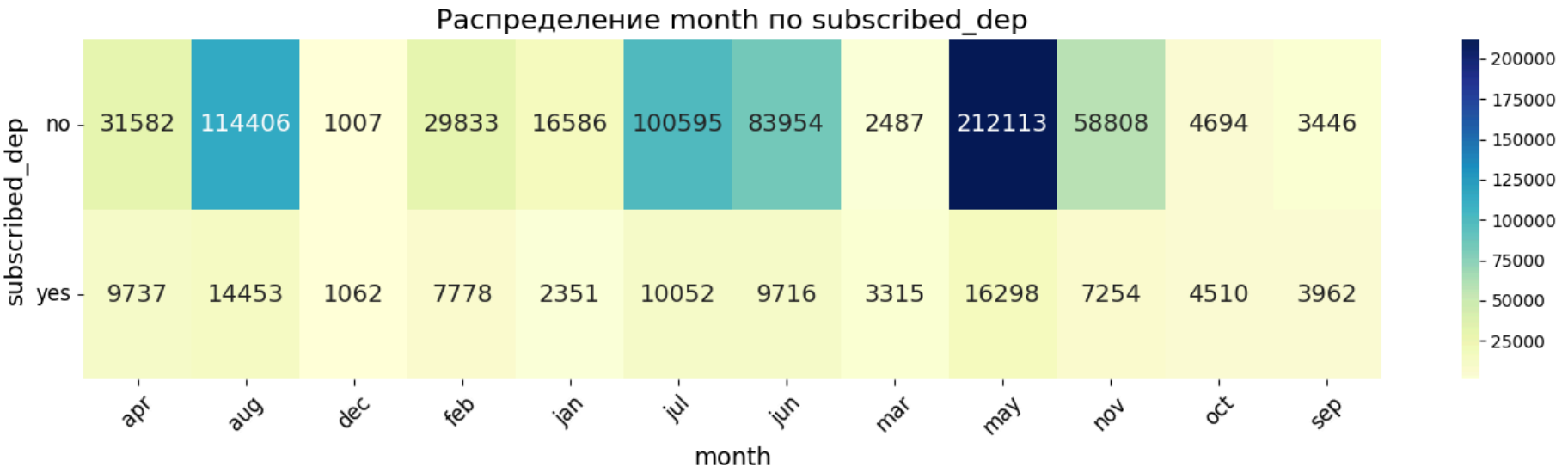
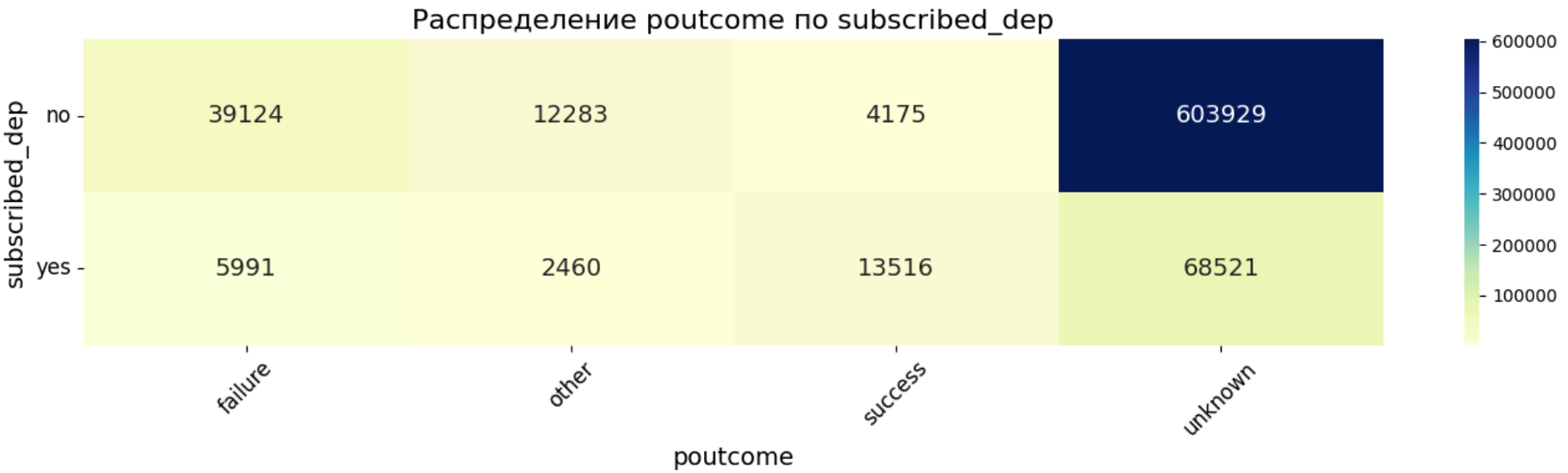
        plt.figure(figsize=figsize)
        sns.heatmap(cross, annot=True, fmt='d', cmap=cmap,
                    annot_kws={"size": annot_size})

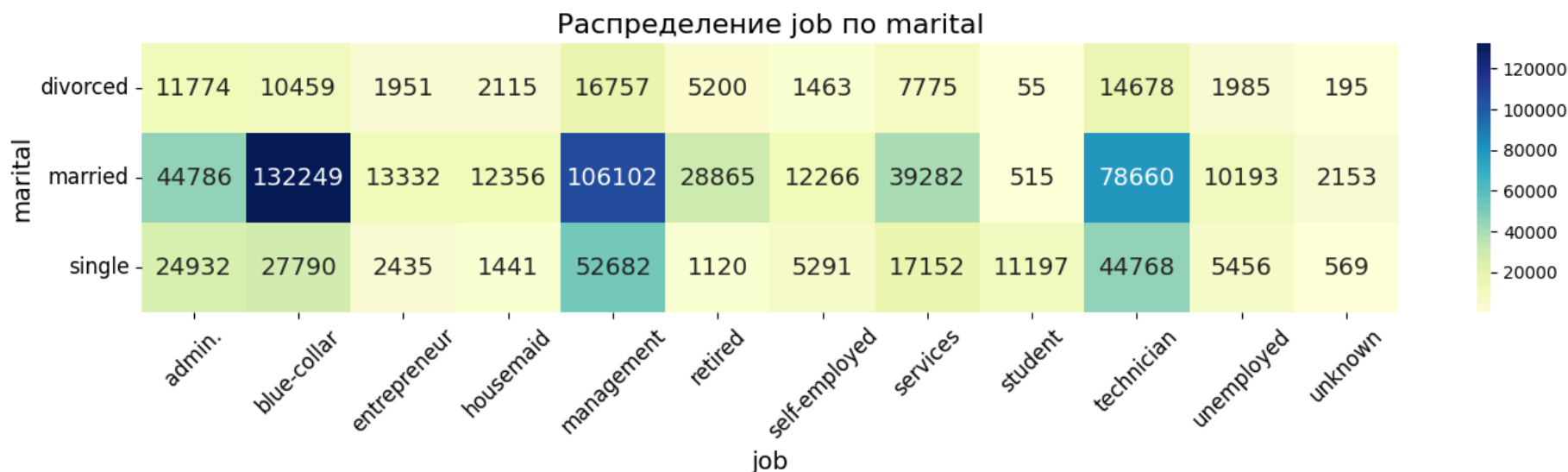
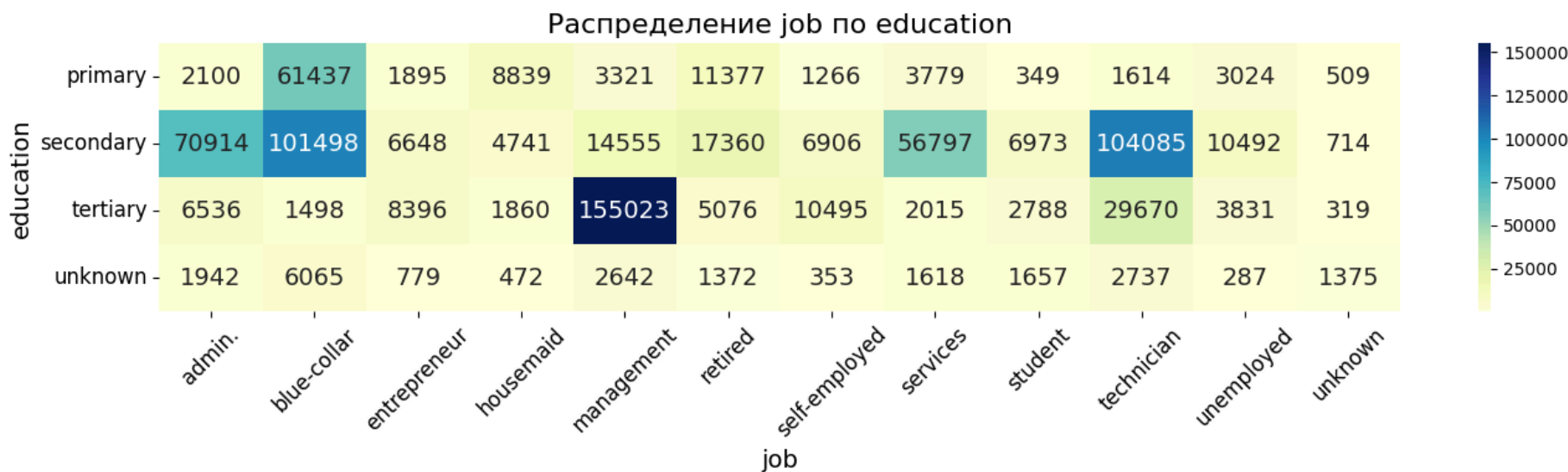
        plt.title(title, fontsize=16)
        plt.xlabel(col, fontsize=14)
        plt.ylabel(row, fontsize=14)
        plt.xticks(rotation=45, fontsize=12)
        plt.yticks(rotation=0, fontsize=12)
        plt.tight_layout()
        plt.show()
```

```
In [131...] pairs = [
    ('subscribed_dep', 'poutcome', "Распределение poutcome по subscribed_dep"),
    ('subscribed_dep', 'month', "Распределение month по subscribed_dep"),
    ('subscribed_dep', 'housing', "Распределение housing по subscribed_dep"),
    ('contact', 'month', "Распределение month по contact"),
    ('housing', 'month', "Распределение month по housing"),
    ('education', 'job', "Распределение job по education"),
    ('marital', 'job', "Распределение job по marital"),
]
```



```
plot_multiple_heatmaps(df_train, pairs, figsize=(14, 4), annot_size=14)
```



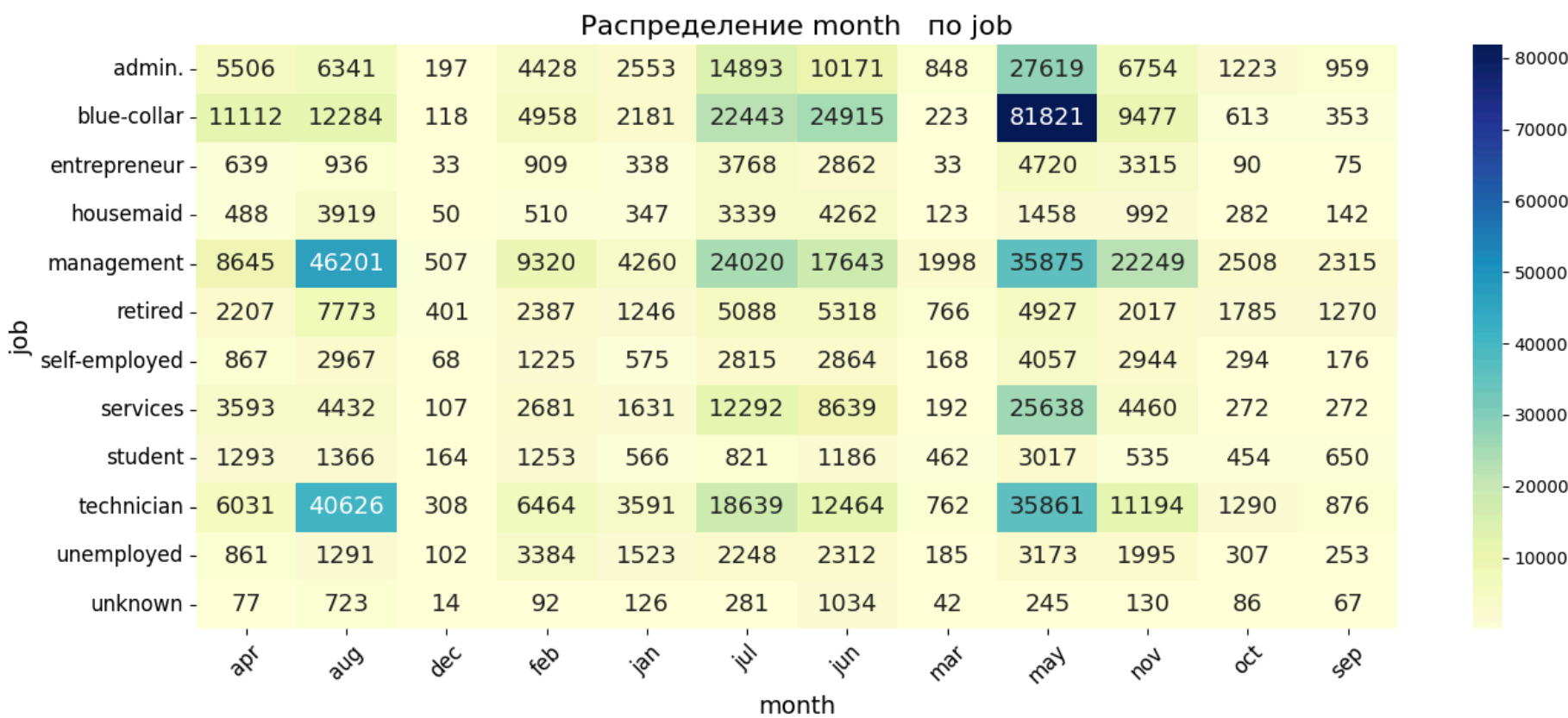


```
In [132... # Кросс-таблица
cross = pd.crosstab(df_train['job'], df_train['month'])

# Параметры
figsize = (14, 6)          # размер графика (ширина, высота)
annot_size = 14            # размер шрифта аннотаций

# Построение тепловой карты
plt.figure(figsize=figsize)
sns.heatmap(cross, annot=True, fmt='d', cmap='YlGnBu',
            annot_kws={"size": annot_size})

plt.title("Распределение month по job", fontsize=16)
plt.xlabel("month", fontsize=14)
plt.ylabel("job", fontsize=14)
plt.xticks(rotation=45, fontsize=12)
plt.yticks(rotation=0, fontsize=12)
plt.tight_layout()
plt.show()
```



Признак	Влияние	Комментарий
duration	Сильнейший предиктор (Phik ≈ 0.68)	Длительность контакта > 600 сек резко повышает конверсию
proutcome	Успех в прошлой кампании даёт конверсию 76.4% (Phik ≈ 0.46)	"Золотой" сегмент для таргетинга

Признак	Влияние	Комментарий
month	Сезонность: март, сен, окт, дек — высокие конверсии (40–57%)	В пиковые месяцы (май–авг) конверсия падает до 7–11%
balance	Клиенты с высоким балансом подписываются чаще (средний баланс: 2142€ vs 1075€)	Финансовая состоятельность — важный фактор
housing/loan	Наличие кредитов снижает вероятность подписки (конверсия: 7.5% с ипотекой vs 17.6% без)	Долговая нагрузка — негативный фактор

Выводы по шагу 3

Целевая переменная (y/subscribed_dep):

- Сильный дисбаланс классов: 12% подписок (1) против 88% отказов (0). Это критически важно учесть при обучении моделей (выбор метрик, стратификация, методы борьбы с дисбалансом).

Согласованность данных:

- Распределения всех признаков в train и test выборках статистически неразличимы. Это гпозволяет быть уверенным в том, что модель, обученная на train, будет применима к test.

Дополнительные признаки созданы

- age_group (young, middle, senior)
- was_contacted_before (флаг предыдущего контакта)
- previous_group (категоризация количества контактов)

Временные патерны

- Высокая конверсия (40-57%): март, сентябрь, октябрь, декабрь
- Низкая конверсия (7-11%): май, июль, август
- Вывод: Массовые кампании в мае неэффективны

Возраст × Профессия:

- Топ-3 сегмента по конверсии:
 - student + young: 35.9%
 - retired + senior: 25.5%
 - unemployed + young: 24.3%

Сильные предикторы (Топ-5 самых влиятельных признаков по анализу и Phik-корреляции) :

- duration: самый сильный предиктор (Phik = 0.60)
 - Клиенты, которые дольше общаются с менеджером, с гораздо большей вероятностью подписываются.
 - Требует осторожности, так как эта информация может быть неизвестна до начала контакта, но она есть в df_test- тестовая выборка, для которой необходимо предсказать вероятность у
- rououtcome: предыдущий успех сильно влияет на конверсию
 - Клиенты, у которых предыдущий контакт был успешным (success), имеют крайне высокую вероятность подписки
- month: выявлена сильная сезонность.
 - В месяцы с низким объемом звонков (март, сентябрь, октябрь, декабрь) конверсия максимальна, и наоборот.
- balance: финансовое положение клиента
 - Клиенты с более высоким средним годовым балансом значительно чаще подписываются на депозит
- housing / loan: наличие ипотеки или персонального кредита (отрицательная корреляция)
 - Клиенты без долговой нагрузки — более перспективны

Статистически значимые взаимосвязи:

- Возраст: Молодые (young) и пожилые (senior) клиенты конвертируют лучше, чем клиенты среднего возраста (middle), несмотря на их численное превосходство.
- Предыдущие контакты: Существует оптимум (2-10 контактов). Отсутствие контактов (none) дает низкую конверсию, а слишком большое их количество (mapu) приводит к "усталости" клиента.
- Профессия и образование: Студенты (student) и безработные (unemployed) имеют аномально высокую конверсию. Пенсионеры (retired) также являются перспективной группой. Есть явная связь между типом работы, уровнем образования и возрастом.

[* к содержанию](#)

Шаг 4: Подготовка к обучению моделей

Создание новых признаков

Категоризация числовых признаков

```
In [133... # balance имеет длинный хвост и отрицательные значения. Создадим категории

def balance_category(balance):
    if balance < 0:
        return "negative"
    elif balance < 1000:
        return "low"
    elif balance < 5000:
        return "medium"
    else:
        return "high"

balance_category = df_train["balance"].apply(balance_category)
# вставляем его перед предпоследней колонкой
df_train = insert_before_second_last(df_train, 'balance_category', balance_category)
print(df_train['balance_category'].value_counts())
```

```
low          389502
medium       226802
negative     104645
high         29050
Name: balance_category, dtype: int64
```

```
In [134... # Группы по длительности звонка
duration_group= pd.cut(
    df_train['duration'],
    bins=[-1, 100, 300, 600, df_train['duration'].max()],
    labels=['short', 'medium', 'long', 'very_long']
)
df_train = insert_before_second_last(df_train, 'duration_group', duration_group)
print(df_train['duration_group'].value_counts())
```

```
medium       329579
short        223728
long         103720
very_long     92972
Name: duration_group, dtype: int64
```

```
In [135... # campaign_intensity
campaign_intensity = pd.cut(
    df_train['campaign'],
    bins=[0, 1, 3, 10, float('inf')],
    labels=['low', 'medium', 'high', 'very_high']
)

df_train = insert_before_second_last(df_train, 'campaign_intensity', campaign_intensity)
print(df_train['campaign_intensity'].value_counts())
```

```
low          304480
medium       298517
high         131957
very_high    15045
Name: campaign_intensity, dtype: int64
```

Бинарные флаги

```
In [136... # Признак задолжности по счёту
has_debt = (df_train['balance'] < 0).astype(int)
df_train = insert_before_second_last(df_train, 'has_debt', has_debt)
print(df_train['has_debt'].value_counts())
```

```
0    645354
1     104645
Name: has_debt, dtype: int64
```

Флаги кредитов

Наличие кредитов снижает вероятность подписки. Добавим бинарные признаки:

```
In [137... has_any_loan = ((df_train["housing"] == "yes") | (df_train["loan"] == "yes")).astype(int)
df_train = insert_before_second_last(df_train, 'has_any_loan', has_any_loan)
print(df_train['has_any_loan'].value_counts())
```

```
1     450405
0     299594
Name: has_any_loan, dtype: int64
```

```
In [138... has_both_loans = ((df_train["housing"] == "yes") & (df_train["loan"] == "yes")).astype(int)
df_train = insert_before_second_last(df_train, 'has_both_loans', has_both_loans)
print(df_train['has_both_loans'].value_counts())
```

```
0    684139
1     65860
Name: has_both_loans, dtype: int64
```

Признак для pdays

pdays = -1 означает отсутствие предыдущего контакта. Создадим бинарный индикатор:

```
In [139... was_contacted_before_flag = (df_train["pdays"] != -1).astype(int)
df_train = insert_before_second_last(df_train, 'was_contacted_before_flag', was_contacted_before_flag)
print(df_train['was_contacted_before_flag'].value_counts())
```

```
0    672434
1     77565
Name: was_contacted_before_flag, dtype: int64
```

```
In [140... def pdays_category(p):
    if p == -1:
        return "no_contact"
    elif p < 30:
        return "recent"
    elif p < 180:
        return "medium"
    else:
        return "long"

pdays_category = df_train["pdays"].apply(pdays_category)
df_train = insert_before_second_last(df_train, 'pdays_category', pdays_category)
print(df_train['pdays_category'].value_counts())
```

```
no_contact    672434
long           48794
medium         28041
recent          730
Name: pdays_category, dtype: int64
```

Взаимодействия признаков

EDA показал, что комбинации могут быть информативны:

```
In [141... # возраст × баланс (покупательская способность)
age_balance = df_train["age"] * df_train["balance"]
df_train = insert_before_second_last(df_train, 'age_balance', age_balance)
```

```
In [142... # длительность × количество контактов (интенсивность кампании)
duration_campaign = df_train["duration"] * df_train["campaign"]
df_train = insert_before_second_last(df_train, 'duration_campaign', duration_campaign)
```

```
In [143... # успешность предыдущей кампании
previous_success = ((df_train["previous"] > 0) & (df_train["poutcome"] == "success")).astype(int)
df_train = insert_before_second_last(df_train, 'previous_success', previous_success)
print(df_train['previous_success'].value_counts())
```

```
0    732311
1     17688
Name: previous_success, dtype: int64
```

```
In [144... # Контакт × Успех в прошлом
contact_success = (
    ((df_train['contact'] == 'cellular') & (df_train['poutcome'] == 'success'))
    .astype(int)
)

df_train = insert_before_second_last(df_train, 'contact_success', contact_success)
print(df_train['contact_success'].value_counts())
```

```
0    733435
1     16564
Name: contact_success, dtype: int64
```

```
In [145... prev_contact_success_flag = ((df_train["was_contacted_before"] == "yes") & (df_train["previous"] > 0)).astype(int)
df_train = insert_before_second_last(df_train, 'prev_contact_success_flag', prev_contact_success_flag)
print(df_train['prev_contact_success_flag'].value_counts())
```

```
0    672448
1     77551
Name: prev_contact_success_flag, dtype: int64
```

```
In [146... balance_to_duration = df_train['balance'] / (df_train['duration'] + 1)
df_train = insert_before_second_last(df_train, 'balance_to_duration', balance_to_duration)
```

```
In [147... balance_to_campaign = df_train['balance'] / (df_train['campaign'] + 1)
df_train = insert_before_second_last(df_train, 'balance_to_campaign', balance_to_campaign)
```

```
In [148... duration_to_campaign = df_train['duration'] / (df_train['campaign'] + 1)
df_train = insert_before_second_last(df_train, 'duration_to_campaign', duration_to_campaign)
```


Логарифмические трансформации

Для признаков с длинным хвостом (balance, duration) применим логарифмирование:

```
In [149... log_balance = np.log1p(df_train["balance"] - df_train["balance"].min() + 1)
df_train = insert_before_second_last(df_train, 'log_balance', log_balance)

In [150... log_duration = np.log1p(df_train["duration"])
df_train = insert_before_second_last(df_train, 'log_duration', log_duration)

In [151... # Квадратный корень количества контактов
sqrt_campaign = np.sqrt(df_train['campaign'])
df_train = insert_before_second_last(df_train, 'sqrt_campaign', sqrt_campaign)
```

Временные признаки

```
In [152... # Сезонность (высокий сезон по EDA)
high_season_months = ['mar', 'sep', 'oct', 'dec']
is_high_season = df_train['month'].isin(high_season_months).astype(int)
df_train = insert_before_second_last(df_train, 'is_high_season', is_high_season)
print(df_train['is_high_season'].value_counts())

0    725516
1     24483
Name: is_high_season, dtype: int64

In [153... # Низкий сезон (май-август)
low_season_months = ['may', 'jun', 'jul', 'aug']
is_low_season = df_train['month'].isin(low_season_months).astype(int)
df_train = insert_before_second_last(df_train, 'is_low_season', is_low_season)
print(df_train['is_low_season'].value_counts())

1     561587
0     188412
Name: is_low_season, dtype: int64

In [154... # Категория дня месяца
day_category = pd.cut(
    df_train['day'],
    bins=[0, 10, 20, 31],
    labels=['beginning', 'middle', 'end']
)

df_train = insert_before_second_last(df_train, 'day_category', day_category)
print(df_train['day_category'].value_counts())

middle    312772
end       222274
beginning  214953
Name: day_category, dtype: int64
```

Категоризация числовых признаков

- balance_category (negative, low, medium, high)
- duration_group (short, medium, long, very_long)
- campaign_intensity (low, medium, high, very_high)
- day_category (beginning, middle, end)

Это усиливает интерпретируемость и позволяет моделям улавливать нелинейные зависимости

Бинарные флаги

- has_debt (баланс < 0)
- has_any_loan, has_both_loans
- was_contacted_before_flag (pdays != -1)
- previous_success, contact_success, prev_contact_success_flag
- is_high_season, is_low_season

Эти признаки хорошо отражают бизнес-логику (наличие кредитов, сезонность, успешность прошлых кампаний)

Взаимодействия признаков

- age_balance (возраст × баланс)
- duration_campaign (длительность × количество контактов)
- balance_to_duration, balance_to_campaign, duration_to_campaign

Такие комбинации помогают моделям находить скрытые зависимости

Трансформации

- log_balance, log_duration

- sqrt_campaign

Сглаживают длинные хвосты распределений и уменьшают влияние выбросов

Проверка корреляции новых признаков с целевой переменной

```
In [155... # Получаем корреляции только с целевой переменной y
correlation_with_y = df_train.corr(numeric_only=True)['y'].drop('y').sort_values(ascending=False)

# Вывод
print("Корреляция признаков с целевой переменной y:")
display(correlation_with_y)
```

Корреляция признаков с целевой переменной y:

```
duration          0.519283
duration_to_campaign 0.504243
log_duration      0.435685
previous_success  0.306951
contact_success   0.299030
duration_campaign 0.280127
is_high_season    0.227939
prev_contact_success_flag 0.169508
was_contacted_before_flag 0.169472
log_balance       0.156840
balance_to_campaign 0.134805
balance           0.122513
previous          0.121449
age_balance       0.111998
pdays           0.089277
age              0.009523
id               0.000616
balance_to_duration -0.038135
day              -0.049625
has_both_loans    -0.067748
campaign          -0.075829
sqrt_campaign     -0.088374
has_debt          -0.118784
is_low_season     -0.162686
has_any_loan      -0.174740
```

Name: y, dtype: float64

```
In [156... corr = df_train.corr(numeric_only=True)
pairs = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
pairs[pairs > 0.3].stack().sort_values(ascending=False)
```

```
Out[156... was_contacted_before_flag  prev_contact_success_flag  0.999899
previous_success          contact_success          0.966784
balance                  age_balance              0.958978
campaign                 sqrt_campaign            0.955189
balance                  balance_to_campaign       0.934965
age_balance              balance_to_campaign       0.899604
duration                 duration_to_campaign     0.893457
pdays                  was_contacted_before_flag  0.891560
                        prev_contact_success_flag  0.891442
duration                 log_duration              0.845485
balance                  log_balance              0.835654
balance_to_campaign      log_balance              0.782738
age_balance              log_balance              0.776111
duration_to_campaign      log_duration            0.774977
previous                 prev_contact_success_flag  0.667373
                        was_contacted_before_flag  0.667305
duration                 duration_campaign         0.637470
pdays                  previous                  0.570198
duration                 y                        0.519283
duration_campaign        log_duration              0.508200
duration_to_campaign      y                        0.504243
previous_success         prev_contact_success_flag  0.457643
was_contacted_before_flag  previous_success  0.457597
contact_success          prev_contact_success_flag  0.442435
was_contacted_before_flag  contact_success  0.442391
log_duration             y                        0.435685
balance                  balance_to_duration     0.401773
duration_campaign        sqrt_campaign            0.384659
age_balance              balance_to_duration     0.375039
campaign                 duration_campaign       0.360369
balance_to_duration      log_balance              0.340609
                        balance_to_campaign       0.322069
previous                 previous_success        0.317878
previous_success         y                        0.306951
previous                 contact_success          0.305153
dtype: float64
```

Топ признаков по корреляции Пирсона с целевой переменной:

- duration (0.52),
- duration_to_campaign (0.50),

- log_duration (0.44),
- previous_success (0.31),
- contact_success (0.30).

Эти фичи явно влияют на вероятность подписки.

```
In [157... start = time.time()

interval_cols = df_train.select_dtypes(include=['int64', 'float64']).columns.tolist()

# Вычисление Phik-матрицы (исключаем 'id')
phik_corr = df_train.drop(columns=['id']).phik_matrix(interval_cols=interval_cols)

# 📊 Корреляции с целевой переменной y
bpm_phik = phik_corr['y'].drop('y').sort_values(ascending=False)
print("📊 Phik-корреляции с целевой переменной 'y':")
print(bpm_phik)

# 🔥 Визуализация полной Phik-матрицы
plt.figure(figsize=(20, 20))
sns.heatmap(phik_corr, annot=True, fmt=".2f", cmap='YlOrRd', square=True)
plt.title('Phik Correlation Matrix', fontsize=14)
plt.tight_layout()
plt.show()

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")
```

📊 Phik-корреляции с целевой переменной 'y':

subscribed_dep	1.000000
duration_group	0.685136
log_duration	0.656516
duration	0.603348
previous_success	0.463670
poutcome	0.457960
contact_success	0.452609
duration_to_campaign	0.402665
is_high_season	0.350427
month	0.340005
pdays	0.280308
pdays_category	0.279483
has_any_loan	0.271036
balance_category	0.270305
prev_contact_success_flag	0.263112
was_contacted_before_flag	0.263057
was_contacted_before	0.263057
is_low_season	0.252765
housing	0.238913
age	0.220449
job	0.202701
has_debt	0.185488
balance_to_campaign	0.174760
balance	0.169212
log_balance	0.150856
campaign_intensity	0.140668
previous_group	0.140423
education	0.135039
loan	0.127867
day	0.124277
age_balance	0.109457
has_both_loans	0.106191
duration_campaign	0.103881
sqrt_campaign	0.102581
contact	0.096550
previous	0.090132
balance_to_duration	0.082461
campaign	0.057742
age_group	0.055670
marital	0.054833
default	0.047259
day_category	0.020326

Name: y, dtype: float64

Пары признаков с Phi-корреляцией от 0.9 до 1 (без 'y' и 'subscribed_dep'):

poutcome	was_contacted_before_flag	1.000000
	was_contacted_before	1.000000
was_contacted_before_flag	pdays_category	1.000000
balance_category	has_debt	1.000000
was_contacted_before	prev_contact_success_flag	1.000000
	pdays_category	1.000000
	was_contacted_before_flag	1.000000
poutcome	prev_contact_success_flag	1.000000
	previous_success	1.000000
pdays_category	prev_contact_success_flag	1.000000
was_contacted_before_flag	prev_contact_success_flag	1.000000
month	is_low_season	1.000000
	is_high_season	1.000000
pdays	was_contacted_before_flag	0.999595
	was_contacted_before	0.999595
	prev_contact_success_flag	0.999591
poutcome	contact_success	0.998645
previous_success	contact_success	0.998635
housing	has_any_loan	0.987371
campaign	sqrt_campaign	0.978383
balance	balance_to_campaign	0.958888
previous_group	prev_contact_success_flag	0.956370
was_contacted_before	previous_group	0.952321
previous_group	was_contacted_before_flag	0.952321
day	day_category	0.946711
loan	has_both_loans	0.934936
previous	previous_group	0.933428
duration_group	log_duration	0.922170
duration	log_duration	0.914318
balance	age_balance	0.907196
poutcome	pdays_category	0.903172

dtype: float64

In [159...

```
# Исключаем целевые колонки
cols_to_exclude = ['y', 'subscribed_dep']
phik_corr_filtered = phik_corr.drop(index=cols_to_exclude, columns=cols_to_exclude, errors='ignore')

# Строим маску для верхнего треугольника с двойным порогом
range_mask = np.triu(((phik_corr_filtered > 0.4) & (phik_corr_filtered < 0.9)), k=1)

# Выводим пары с Phi-корреляцией в диапазоне (0.4, 0.9)
print("Пары признаков с Phi-корреляцией от 0.4 до 0.9 (без 'y' и 'subscribed_dep'):")
corr_pairs_in_range = phik_corr_filtered.where(range_mask).stack().dropna().sort_values(ascending=False)
print(corr_pairs_in_range if not corr_pairs_in_range.empty else "Нет пар в указанном диапазоне.")
```

Пары признаков с Phi-корреляцией от 0.4 до 0.9 (без 'y' и 'subscribed_dep'):

pdays	pdays_category	0.893258
age	age_group	0.861639
age_balance	balance_to_campaign	0.861182
campaign_intensity	sqrt_campaign	0.859002
pdays	previous_group	0.833729
duration	duration_to_campaign	0.818851
contact	month	0.817168
job	education	0.813384
pdays	poutcome	0.783172
balance	log_balance	0.744517
duration	duration_group	0.720507
age_balance	log_balance	0.718575
housing	month	0.717491
balance_to_campaign	log_balance	0.716140
campaign	campaign_intensity	0.702929
month	has_any_loan	0.692494
pdays_category	previous_success	0.690377
pdays	previous_success	0.686250
pdays_category	contact_success	0.672630
pdays	contact_success	0.664338
previous_success	prev_contact_success_flag	0.658513
was_contacted_before_flag	previous_success	0.658459
was_contacted_before	previous_success	0.658459
duration_to_campaign	log_duration	0.656885
balance_category	log_balance	0.655590
marital	age_group	0.655230
poutcome	previous_group	0.652659
month	day_category	0.650885
previous_group	pdays_category	0.649599
contact_success	prev_contact_success_flag	0.640348
was_contacted_before_flag	contact_success	0.640295
was_contacted_before	contact_success	0.640295
balance_category	age_balance	0.633931
job	age_group	0.628989
balance_to_duration	log_duration	0.623954
balance	balance_category	0.612909
balance_category	balance_to_campaign	0.599608
day	month	0.597541
duration_group	duration_to_campaign	0.545818
age	job	0.507314
	marital	0.500385
loan	has_any_loan	0.494117
is_high_season	is_low_season	0.477810
day	is_low_season	0.430387
housing	has_both_loans	0.427995
balance	balance_to_duration	0.423759
job	marital	0.418664
age_balance	balance_to_duration	0.411630
job	month	0.408645
previous	prev_contact_success_flag	0.405880
	was_contacted_before_flag	0.405840
	was_contacted_before	0.405840
month	campaign_intensity	0.403197

dtype: float64

Сильная связь признаков с целевой переменной y:

- duration_group 0.68
- log_duration 0.65
- duration 0.60
- previous_success 0.46
- poutcome 0.45
- contact_success 0.45
- duration_to_campaign 0.40

Вывод: ключевая роль принадлежит признакам, отражающим длительность контакта, результаты прошлых кампаний и сезонность.

Средняя связь признаков с целевой переменной y:

- is_high_season 0.35
- month 0.34
- pdays 0.28
- pdays_category 0.27
- has_any_loan 0.27
- balance_category 0.27
- prev_contact_success_flag 0.26
- was_contacted_before_flag 0.26
- was_contacted_before 0.26
- is_low_season 0.25

Слабая связь признаков с целевой переменной y:

- housing 0.23
- age 0.22
- job 0.20
- has_debt 0.18
- balance_to_campaign 0.17
- balance 0.16
- log_balance 0.15
- campaign_intensity 0.14
- previous_group 0.14
- education 0.13
- loan 0.12
- day 0.12

Очень слабая связь признаков с целевой переменной y:

- age_balance 0.10
- has_both_loans 0.10
- duration_campaign 0.10
- sqrt_campaign 0.10
- contact 0.09
- previous 0.09
- balance_to_duration 0.08
- campaign 0.05
- age_group 0.05
- marital 0.05
- default 0.04
- day_category 0.02

Также выявлены сильные корреляции между самими признаками (мультиколлинеарность), что больше связано с их получением. Например:

- was_contacted_before_flag ≈ prev_contact_success_flag (0.999)
- previous_success ≈ contact_success (0.99)
- balance ≈ age_balance (0.9)
- campaign ≈ sqrt_campaign (0.97)
- balance ≈ balance_to_campaign (0.95)
- duration ≈ duration_to_campaign (0.81)

Также множество пар с корреляцией > 0.9:

- duration ↔ log_duration,
- poutcome ↔ previous_success,
- was_contacted_before_flag ↔ pdays_category,
- balance ↔ balance_to_campaign.

Вывод: нужно будет аккуратно отобрать признаки, чтобы избежать переизбыточности.

Выбор признаков для моделей

Линейная модель — LogisticRegression и Базовая модель - DummyClassifier

- Чувствительна к масштабу → обязательное масштабирование (StandardScaler);
- Плохо переносит мультиколлинеарные признаки → исключаем сильные корреляты;
- Требуется числовых признаков (One-Hot для категориальных)

Выбираем сбалансированный, "чистый" набор:

```
In [160... features_logistic = [  
    'balance',  
  
    # числовые  
    'age', 'duration_to_campaign', 'previous', 'campaign',  
    'pdays', 'log_duration',  
  
    # бинарные флаги  
    'has_any_loan', 'has_debt',  
  
    # категориальные  
    'job', 'education', 'marital', 'month', 'poutcome',  
  
    # Целевая переменная
```

```
'y'  
]
```

```
In [161... df_logistic = df_train[features_logistic]
```

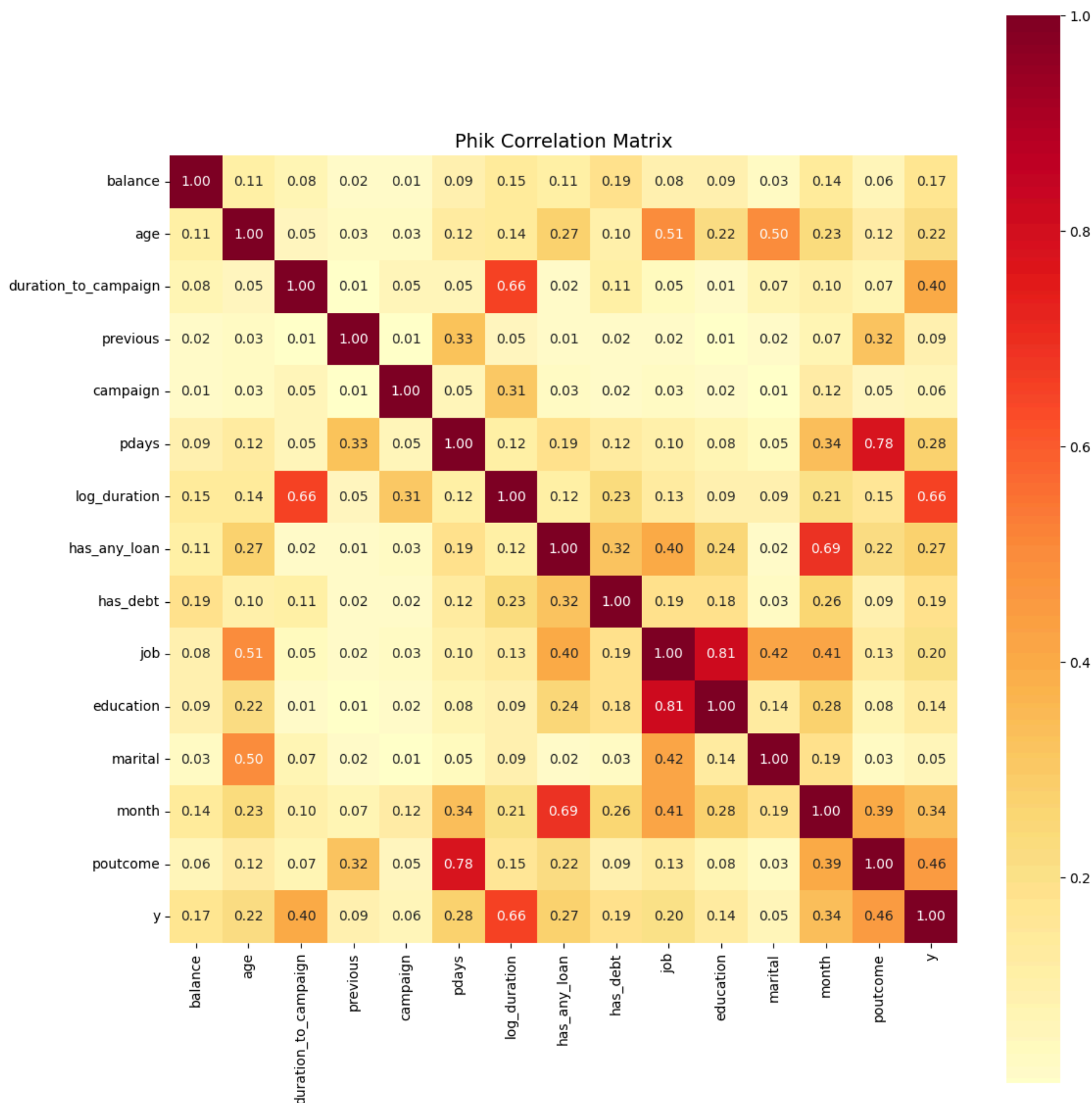
```
In [162... df_logistic.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
Int64Index: 749999 entries, 0 to 749999  
Data columns (total 15 columns):  
#   Column                Non-Null Count  Dtype  
---  -  
0   balance               749999 non-null  int64  
1   age                   749999 non-null  int64  
2   duration_to_campaign  749999 non-null  float64  
3   previous              749999 non-null  int64  
4   campaign              749999 non-null  int64  
5   pdays                749999 non-null  int64  
6   log_duration          749999 non-null  float64  
7   has_any_loan          749999 non-null  int32  
8   has_debt              749999 non-null  int32  
9   job                   749999 non-null  object  
10  education             749999 non-null  object  
11  marital               749999 non-null  object  
12  month                 749999 non-null  object  
13  poutcome              749999 non-null  object  
14  y                     749999 non-null  int64  
dtypes: float64(2), int32(2), int64(6), object(5)  
memory usage: 85.8+ MB
```

```
In [163... interval_cols = df_logistic.select_dtypes(include=['int64', 'float64']).columns.tolist()
```

```
# Вычисление Phik-матрицы (исключаем 'id')  
phik_corr = df_logistic.phik_matrix(interval_cols=interval_cols)  
  
# 📊 Корреляции с целевой переменной y  
bpm_phik = phik_corr['y'].drop('y').sort_values(ascending=False)  
print("📊 Phik-корреляции с целевой переменной 'y':")  
print(bpm_phik)  
  
# 🔥 Визуализация полной Phik-матрицы  
plt.figure(figsize=(12, 12))  
sns.heatmap(phik_corr, annot=True, fmt=".2f", cmap='YlOrRd', square=True)  
plt.title('Phik Correlation Matrix', fontsize=14)  
plt.tight_layout()  
plt.show()
```

```
📊 Phik-корреляции с целевой переменной 'y':  
log_duration      0.656516  
poutcome          0.457960  
duration_to_campaign  0.402665  
month             0.340005  
pdays            0.280308  
has_any_loan      0.271036  
age               0.220449  
job               0.202701  
has_debt          0.185488  
balance           0.169212  
education         0.135039  
previous          0.090132  
campaign          0.057742  
marital           0.054833  
Name: y, dtype: float64
```

In [164...

```
# Исключаем целевые колонки из матрицы
cols_to_exclude = ['y']
phik_corr_filtered = phik_corr.drop(index=cols_to_exclude, columns=cols_to_exclude, errors='ignore')

# Строим маску для верхнего треугольника с порогом корреляции
high_corr_mask = np.triu(np.abs(phik_corr_filtered) > 0.4, k=1)

# Выводим пары с высокой Phi-корреляцией
print("Пары признаков с Phi-корреляцией > 0.4 (без 'y' и 'subscribed_dep'):")
high_corr_pairs = phik_corr_filtered.where(high_corr_mask).stack().dropna().sort_values(ascending=False)
print(high_corr_pairs if not high_corr_pairs.empty else "Нет пар с высокой корреляцией.")
```

Пары признаков с Phi-корреляцией > 0.4 (без 'y' и 'subscribed_dep'):

```
job          education    0.813384
pdays      poutcome     0.783172
has_any_loan month       0.692494
duration_to_campaign log_duration 0.656885
age          job         0.507314
              marital     0.500385
job          marital     0.418664
              month      0.408645
```

dtype: float64

Ансамбли деревьев (XGB, LGBM, CatBoost)

- Не требуют масштабирования;
- Устойчивы к коррелирующим признакам;
- Эффективно работают с нелинейными и взаимодействующими фичами.

Выбираем расширенный набор признаков, включая производные и взаимодействия:

```
In [165... features_ensembles = [  
    'loan',  
  
    # базовые  
    'age', 'balance', 'duration', 'campaign', 'pdays', 'previous',  
  
    # флаги и лог-преобразования  
    'previous_success', 'has_any_loan', 'has_debt',  
  
    # категориальные  
    'job', 'education', 'marital', 'month',  
    'duration_group', 'campaign_intensity', 'pdays_category',  
  
    # взаимодействия  
    'duration_campaign',  
  
    # Целевая переменная  
    'y'  
]
```

```
In [166... df_ens = df_train[features_ensembles]
```

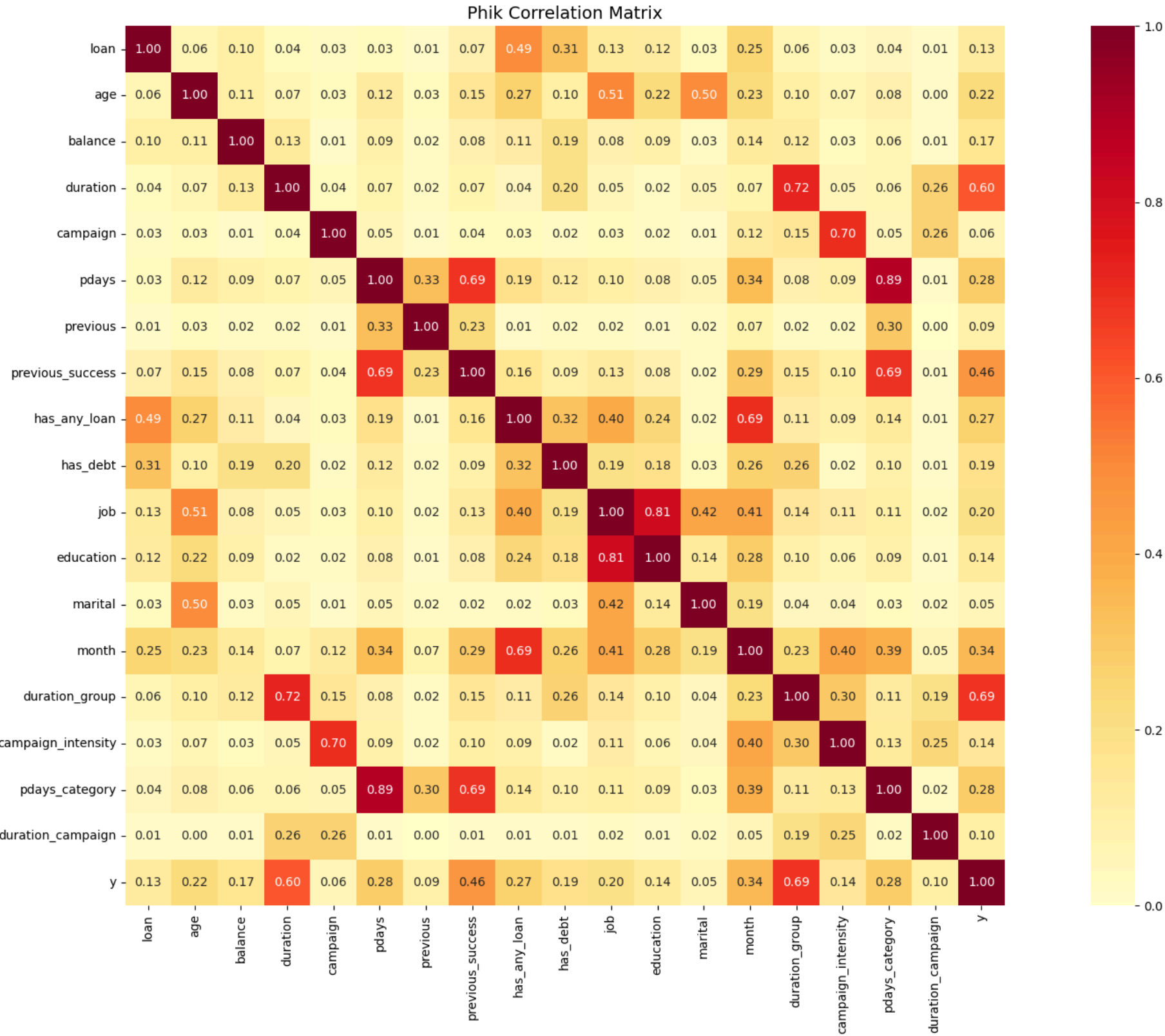
```
In [167... df_ens.info()  
  
<class 'pandas.core.frame.DataFrame'>  
Int64Index: 749999 entries, 0 to 749999  
Data columns (total 19 columns):  
#   Column                Non-Null Count  Dtype  
---  ---  
0   loan                   749999 non-null object  
1   age                    749999 non-null int64  
2   balance                749999 non-null int64  
3   duration               749999 non-null int64  
4   campaign               749999 non-null int64  
5   pdays                  749999 non-null int64  
6   previous                749999 non-null int64  
7   previous_success       749999 non-null int32  
8   has_any_loan           749999 non-null int32  
9   has_debt               749999 non-null int32  
10  job                    749999 non-null object  
11  education               749999 non-null object  
12  marital                 749999 non-null object  
13  month                   749999 non-null object  
14  duration_group          749999 non-null category  
15  campaign_intensity      749999 non-null category  
16  pdays_category          749999 non-null object  
17  duration_campaign       749999 non-null int64  
18  y                       749999 non-null int64  
dtypes: category(2), int32(3), int64(8), object(6)  
memory usage: 95.8+ MB
```

```
In [168... interval_cols = df_ens.select_dtypes(include=['int64', 'float64']).columns.tolist()  
  
# Вычисление Phik-матрицы (исключаем 'id')  
phik_corr = df_ens.phik_matrix(interval_cols=interval_cols)  
  
# 📊 Корреляции с целевой переменной y  
bpm_phik = phik_corr['y'].drop('y').sort_values(ascending=False)  
print("📊 Phik-корреляции с целевой переменной 'y':")  
print(bpm_phik)  
  
# 🔥 Визуализация полной Phik-матрицы  
plt.figure(figsize=(20, 12))  
sns.heatmap(phik_corr, annot=True, fmt=".2f", cmap='YlOrRd', square=True)  
plt.title('Phik Correlation Matrix', fontsize=14)  
plt.tight_layout()  
plt.show()
```

🏠 Phik-корреляции с целевой переменной 'y':

duration_group	0.685136
duration	0.603348
previous_success	0.463670
month	0.340005
pdays	0.280308
pdays_category	0.279483
has_any_loan	0.271036
age	0.220449
job	0.202701
has_debt	0.185488
balance	0.169212
campaign_intensity	0.140668
education	0.135039
loan	0.127867
duration_campaign	0.103881
previous	0.090132
campaign	0.057742
marital	0.054833

Name: y, dtype: float64



In [169...

```
# Исключаем целевые колонки из матрицы
cols_to_exclude = ['y']
phik_corr_filtered = phik_corr.drop(index=cols_to_exclude, columns=cols_to_exclude, errors='ignore')

# Строим маску для верхнего треугольника с порогом корреляции
high_corr_mask = np.triu(np.abs(phik_corr_filtered) > 0.4, k=1)

# Выводим пары с высокой Phi-корреляцией
print("Пары признаков с Phi-корреляцией > 0.4 (без 'y' и 'subscribed_dep'):")
high_corr_pairs = phik_corr_filtered.where(high_corr_mask).stack().dropna().sort_values(ascending=False)
print(high_corr_pairs if not high_corr_pairs.empty else "Нет пар с высокой корреляцией.")
```


Пары признаков с Phi-корреляцией > 0.4 (без 'y' и 'subscribed_dep'):

pdays	pdays_category	0.893258
job	education	0.813384
duration	duration_group	0.720507
campaign	campaign_intensity	0.702929
has_any_loan	month	0.692494
previous_success	pdays_category	0.690377
pdays	previous_success	0.686250
age	job	0.507314
	marital	0.500385
loan	has_any_loan	0.494117
job	marital	0.418664
	month	0.408645
month	campaign_intensity	0.403197

dtype: float64

Разделение данных (признаки и целевая переменная) обучающей выборки на train/valid

Разделение для линейных моделей (Logistic Regression)

```
In [170... # Целевая переменная
y_logistic = df_logistic['y']

# Признаки (исключаем целевую переменную)
X_logistic = df_logistic.drop(columns=['y'], errors='ignore')

# Разделение на train/valid
X_train_log, X_valid_log, y_train_log, y_valid_log = train_test_split(
    X_logistic, y_logistic, test_size=0.25, random_state=42, stratify=y_logistic
)

print("Разделение для линейных моделей:")
print(f"X_train_logistic: {X_train_log.shape}")
print(f"X_valid_logistic: {X_valid_log.shape}")
print(f"y_train_logistic: {y_train_log.shape}")
print(f"y_valid_logistic: {y_valid_log.shape}")
```

Разделение для линейных моделей:

X_train_logistic: (562499, 14)

X_valid_logistic: (187500, 14)

y_train_logistic: (562499,)

y_valid_logistic: (187500,)

```
In [171... # Визуализация распределения y до и после разделения для линейных моделей
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

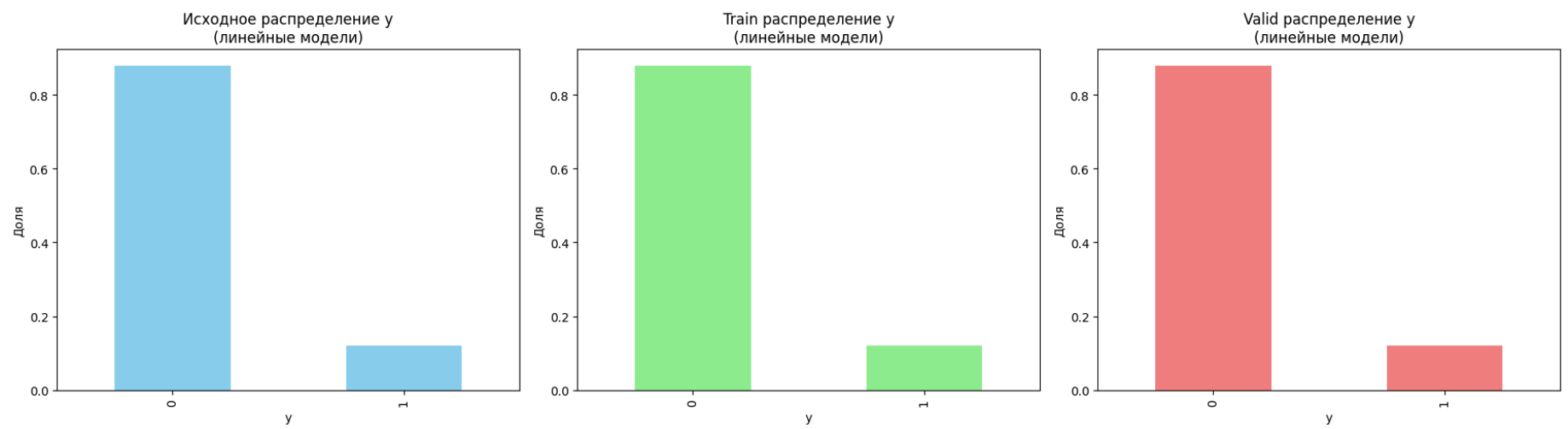
# Исходное распределение
y_logistic.value_counts(normalize=True).plot(kind='bar', ax=axes[0], color='skyblue')
axes[0].set_title('Исходное распределение y\n(линейные модели)')
axes[0].set_ylabel('Доля')
axes[0].set_xlabel('y')

# Train распределение
y_train_log.value_counts(normalize=True).plot(kind='bar', ax=axes[1], color='lightgreen')
axes[1].set_title('Train распределение y\n(линейные модели)')
axes[1].set_ylabel('Доля')
axes[1].set_xlabel('y')

# Valid распределение
y_valid_log.value_counts(normalize=True).plot(kind='bar', ax=axes[2], color='lightcoral')
axes[2].set_title('Valid распределение y\n(линейные модели)')
axes[2].set_ylabel('Доля')
axes[2].set_xlabel('y')

plt.tight_layout()
plt.show()

# Числовая проверка
print("Проверка стратификации для линейных моделей:")
print(f"Исходные данные: {y_logistic.value_counts(normalize=True).values}")
print(f"Train выборка: {y_train_log.value_counts(normalize=True).values}")
print(f"Valid выборка: {y_valid_log.value_counts(normalize=True).values}")
```



Проверка стратификации для линейных моделей:

Исходные данные: [0.87934917 0.12065083]

Train выборка: [0.87934912 0.12065088]

Valid выборка: [0.87934933 0.12065067]

Разделение для ансамблей (XGBoost, LGBM, CatBoost)

In [172...

```
# Целевая переменная
y_ens = df_ens['y']

# Признаки (исключаем целевую переменную)
X_ens = df_ens.drop(columns=['y'], errors='ignore')

# Разделение на train/valid
X_train_ens, X_valid_ens, y_train_ens, y_valid_ens = train_test_split(
    X_ens, y_ens, test_size=0.25, random_state=42, stratify=y_ens
)

print("\nРазделение для ансамблевых моделей:")
print(f"X_train_ens: {X_train_ens.shape}")
print(f"X_valid_ens: {X_valid_ens.shape}")
print(f"y_train_ens: {y_train_ens.shape}")
print(f"y_valid_ens: {y_valid_ens.shape}")
```

Разделение для ансамблевых моделей:

X_train_ens: (562499, 18)

X_valid_ens: (187500, 18)

y_train_ens: (562499,)

y_valid_ens: (187500,)

In [173...

```
# Визуализация распределения y до и после разделения для ансамблей
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

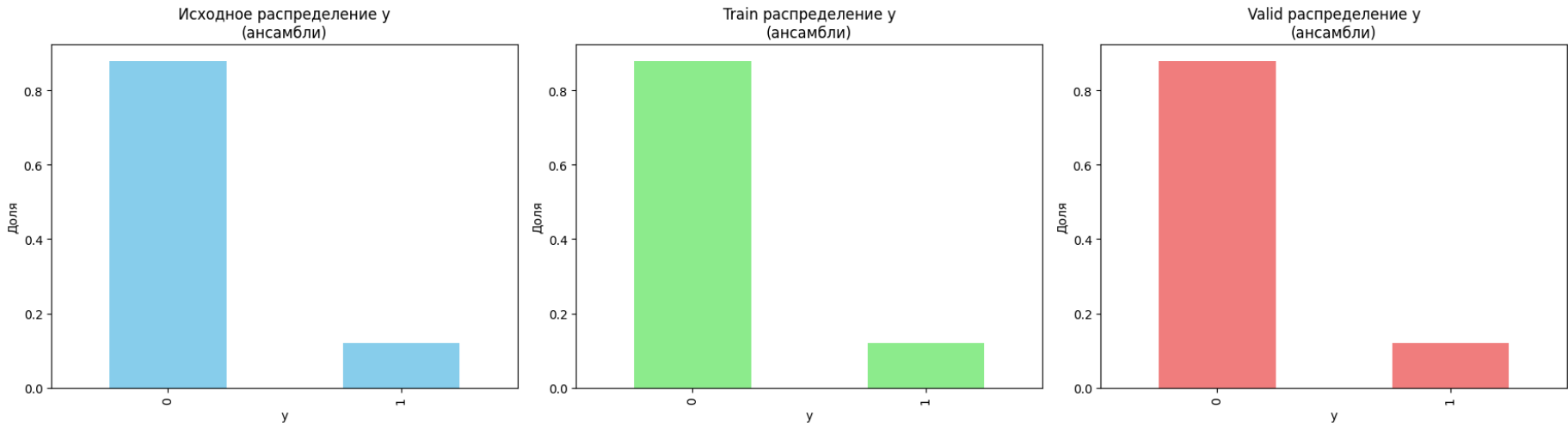
# Исходное распределение
y_ens.value_counts(normalize=True).plot(kind='bar', ax=axes[0], color='skyblue')
axes[0].set_title('Исходное распределение y\n(ансамбли)')
axes[0].set_ylabel('Доля')
axes[0].set_xlabel('y')

# Train распределение
y_train_ens.value_counts(normalize=True).plot(kind='bar', ax=axes[1], color='lightgreen')
axes[1].set_title('Train распределение y\n(ансамбли)')
axes[1].set_ylabel('Доля')
axes[1].set_xlabel('y')

# Valid распределение
y_valid_ens.value_counts(normalize=True).plot(kind='bar', ax=axes[2], color='lightcoral')
axes[2].set_title('Valid распределение y\n(ансамбли)')
axes[2].set_ylabel('Доля')
axes[2].set_xlabel('y')

plt.tight_layout()
plt.show()

# Числовая проверка
print("Проверка стратификации для ансамблей:")
print(f"Исходные данные: {y_ens.value_counts(normalize=True).values}")
print(f"Train выборка: {y_train_ens.value_counts(normalize=True).values}")
print(f"Valid выборка: {y_valid_ens.value_counts(normalize=True).values}")
```



Проверка стратификации для ансамблей:
Исходные данные: [0.87934917 0.12065083]
Train выборка: [0.87934912 0.12065088]
Valid выборка: [0.87934933 0.12065067]

Предобработка данных: масштабирование числовых признаков и кодирование категориальных признаков

In [174...

df_logistic.describe().T

Out[174...

	count	mean	std	min	25%	50%	75%	max
balance	749999.0	1204.069003	2836.098309	-8019.000000	0.000000	634.000000	1390.000000	99717.000000
age	749999.0	40.926413	10.098823	18.000000	33.000000	39.000000	48.000000	95.000000
duration_to_campaign	749999.0	93.503866	107.844942	0.117647	27.428571	51.333333	114.000000	2459.000000
previous	749999.0	0.298279	1.315875	0.000000	0.000000	0.000000	0.000000	58.000000
campaign	749999.0	2.577010	2.718515	1.000000	1.000000	2.000000	3.000000	63.000000
pdays	749999.0	22.412630	77.319998	-1.000000	-1.000000	-1.000000	-1.000000	871.000000
log_duration	749999.0	5.065001	1.031525	0.693147	4.521789	4.897840	5.891644	8.500861
has_any_loan	749999.0	0.600541	0.489788	0.000000	0.000000	1.000000	1.000000	1.000000
has_debt	749999.0	0.139527	0.346496	0.000000	0.000000	0.000000	0.000000	1.000000
y	749999.0	0.120651	0.325721	0.000000	0.000000	0.000000	0.000000	1.000000

In [175...

df_logistic['has_any_loan'].value_counts()

Out[175...

1 450405
0 299594
Name: has_any_loan, dtype: int64

In [176...

df_logistic['has_debt'].value_counts()

Out[176...

0 645354
1 104645
Name: has_debt, dtype: int64

In [177...

df_ens.describe().T

Out[177...

	count	mean	std	min	25%	50%	75%	max
age	749999.0	40.926413	10.098823	18.0	33.0	39.0	48.0	95.0
balance	749999.0	1204.069003	2836.098309	-8019.0	0.0	634.0	1390.0	99717.0
duration	749999.0	256.229476	272.555692	1.0	91.0	133.0	361.0	4918.0
campaign	749999.0	2.577010	2.718515	1.0	1.0	2.0	3.0	63.0
pdays	749999.0	22.412630	77.319998	-1.0	-1.0	-1.0	-1.0	871.0
previous	749999.0	0.298279	1.315875	0.0	0.0	0.0	0.0	58.0
previous_success	749999.0	0.023584	0.151749	0.0	0.0	0.0	0.0	1.0
has_any_loan	749999.0	0.600541	0.489788	0.0	0.0	1.0	1.0	1.0
has_debt	749999.0	0.139527	0.346496	0.0	0.0	0.0	0.0	1.0
duration_campaign	749999.0	598.794890	1060.001150	1.0	144.0	282.0	604.0	51345.0
y	749999.0	0.120651	0.325721	0.0	0.0	0.0	0.0	1.0

In [178...

df_ens['previous_success'].value_counts()

```
Out[178... 0    732311
           1    17688
           Name: previous_success, dtype: int64
```

```
In [179... df_ens['has_debt'].value_counts()
```

```
Out[179... 0    645354
           1    104645
           Name: has_debt, dtype: int64
```

```
In [180... df_ens['has_any_loan'].value_counts()
```

```
Out[180... 1    450405
           0    299594
           Name: has_any_loan, dtype: int64
```

```
In [181... def inspect_object_columns(df, df_name='DataFrame'):
    temp = df.copy()
    # Выбираем u object, u category
    obj_cat_cols = temp.select_dtypes(include=['object', 'category']).columns

    print(f"\n📄 Информация о object/category-столбцах в {df_name}:")
    print(temp[obj_cat_cols].info())

    for col in obj_cat_cols:
        print('-' * 40)
        print(f"💎 {col} – уникальные значения:")
        print(temp[col].sort_values().unique())
```

```
In [182... inspect_object_columns(df_logistic, 'df_logistic')
```

```
📄 Информация о object/category-столбцах в df_logistic:
<class 'pandas.core.frame.DataFrame'>
Int64Index: 749999 entries, 0 to 749999
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   job          749999 non-null  object
1   education    749999 non-null  object
2   marital      749999 non-null  object
3   month        749999 non-null  object
4   poutcome     749999 non-null  object
dtypes: object(5)
memory usage: 34.3+ MB
None
-----
💎 job – уникальные значения:
['admin.' 'blue-collar' 'entrepreneur' 'housemaid' 'management' 'retired'
 'self-employed' 'services' 'student' 'technician' 'unemployed' 'unknown']
-----
💎 education – уникальные значения:
['primary' 'secondary' 'tertiary' 'unknown']
-----
💎 marital – уникальные значения:
['divorced' 'married' 'single']
-----
💎 month – уникальные значения:
['apr' 'aug' 'dec' 'feb' 'jan' 'jul' 'jun' 'mar' 'may' 'nov' 'oct' 'sep']
-----
💎 poutcome – уникальные значения:
['failure' 'other' 'success' 'unknown']
```

```
In [183... inspect_object_columns(df_ens, 'df_ens')
```

📄 Информация о object/category-столбцах в df_ens:

```
<class 'pandas.core.frame.DataFrame'>
```

```
Int64Index: 749999 entries, 0 to 749999
```

```
Data columns (total 8 columns):
```

#	Column	Non-Null Count	Dtype
0	loan	749999 non-null	object
1	job	749999 non-null	object
2	education	749999 non-null	object
3	marital	749999 non-null	object
4	month	749999 non-null	object
5	duration_group	749999 non-null	category
6	campaign_intensity	749999 non-null	category
7	pdays_category	749999 non-null	object

```
dtypes: category(2), object(6)
```

```
memory usage: 41.5+ MB
```

```
None
```

♦ loan – уникальные значения:

```
['no' 'yes']
```

♦ job – уникальные значения:

```
['admin.' 'blue-collar' 'entrepreneur' 'housemaid' 'management' 'retired' 'self-employed' 'services' 'student' 'technician' 'unemployed' 'unknown']
```

♦ education – уникальные значения:

```
['primary' 'secondary' 'tertiary' 'unknown']
```

♦ marital – уникальные значения:

```
['divorced' 'married' 'single']
```

♦ month – уникальные значения:

```
['apr' 'aug' 'dec' 'feb' 'jan' 'jul' 'jun' 'mar' 'may' 'nov' 'oct' 'sep']
```

♦ duration_group – уникальные значения:

```
['short', 'medium', 'long', 'very_long']
```

```
Categories (4, object): ['short' < 'medium' < 'long' < 'very_long']
```

♦ campaign_intensity – уникальные значения:

```
['low', 'medium', 'high', 'very_high']
```

```
Categories (4, object): ['low' < 'medium' < 'high' < 'very_high']
```

♦ pdays_category – уникальные значения:

```
['long' 'medium' 'no_contact' 'recent']
```

Масштабирование и кодирование для линейных моделей(Logistic Regression)

In [184...

```
def prepare_linear_models(X_train, X_valid):  
  
    # Создаем копии  
    X_train_proc = X_train.copy()  
    X_valid_proc = X_valid.copy()  
  
    # 1. Определение типов признаков  
    cat_col_names = X_train_proc.select_dtypes(exclude='number').columns.tolist()  
    num_col_names = X_train_proc.select_dtypes(include='number').columns.tolist()  
  
    # 2. OneHotEncoder для категориальных  
    ohe = OneHotEncoder(  
        sparse_output=False,  
        drop='first',  
        handle_unknown="ignore"  
    )  
  
    X_train_ohe = ohe.fit_transform(X_train_proc[cat_col_names])  
    X_valid_ohe = ohe.transform(X_valid_proc[cat_col_names])  
  
    # 3. Масштабирование числовых  
    scaler = StandardScaler()  
  
    X_train_proc[num_col_names] = scaler.fit_transform(X_train_proc[num_col_names])  
    X_valid_proc[num_col_names] = scaler.transform(X_valid_proc[num_col_names])  
  
    # 4. Создание DataFrame для закодированных признаков  
    encoder_col_names = ohe.get_feature_names_out(cat_col_names)  
    X_train_ohe_df = pd.DataFrame(X_train_ohe, columns=encoder_col_names, index=X_train_proc.index)  
    X_valid_ohe_df = pd.DataFrame(X_valid_ohe, columns=encoder_col_names, index=X_valid_proc.index)  
  
    # 5. Объединение числовых и категориальных  
    X_train_final = pd.concat([X_train_proc[num_col_names], X_train_ohe_df], axis=1)  
    X_valid_final = pd.concat([X_valid_proc[num_col_names], X_valid_ohe_df], axis=1)  
  
    return X_train_final, X_valid_final, ohe, scaler
```

```
In [185... # Обработка для линейных моделей
X_train_log_processed, X_valid_log_processed, ohe_logistic, scaler_logistic = prepare_linear_models(
    X_train_log, X_valid_log
)
```

```
In [186... print("Форма X_train_log_processed:", X_train_log_processed.shape)
print("Форма X_valid_log_processed:", X_valid_log_processed .shape)
print("Форма y_train_log:", y_train_log.shape)
print("Форма y_valid_log:", y_valid_log.shape)
```

Форма X_train_log_processed: (562499, 39)
Форма X_valid_log_processed: (187500, 39)
Форма y_train_log: (562499,)
Форма y_valid_log: (187500,)

```
In [187... X_train_log_processed.columns.equals(X_valid_log_processed.columns)
```

Out[187... True

```
In [188... X_train_log_processed.isnull().sum().sum()
```

Out[188... 0

```
In [189... X_valid_log_processed.isnull().sum().sum()
```

Out[189... 0

```
In [190... X_tr_ln = X_train_log_processed
X_vl_ln = X_valid_log_processed
y_tr_ln = y_train_log
y_vl_ln = y_valid_log
```

Масштабирование и кодирование для ансамблей (XGBClassifier, LGBMClassifier, CatBoostClassifier)

```
In [191... # Label Encoding (альтернатива)
def prepare_ensemble_label(X_train, X_test):
    X_train_enc = X_train.copy()
    X_test_enc = X_test.copy()

    cat_cols = X_train_enc.select_dtypes(include=['object', 'category']).columns

    for col in cat_cols:
        le = LabelEncoder()
        all_values = pd.concat([X_train_enc[col], X_test_enc[col]]).unique()
        le.fit(all_values)
        X_train_enc[col] = le.transform(X_train_enc[col])
        X_test_enc[col] = le.transform(X_test_enc[col])

    return X_train_enc, X_test_enc

# Для XGBoost/LGBM - простой Label Encoding
X_train_ens_le, X_valid_ens_le = prepare_ensemble_label(X_train_ens, X_valid_ens)
```

```
In [192... print("Форма X_train_ens_le:", X_train_ens_le.shape)
print("Форма X_valid_ens_le:", X_valid_ens_le .shape)
print("Форма y_train_ens:", y_train_ens.shape)
print("Форма y_valid_ens:", y_valid_ens.shape)
```

Форма X_train_ens_le: (562499, 18)
Форма X_valid_ens_le: (187500, 18)
Форма y_train_ens: (562499,)
Форма y_valid_ens: (187500,)

```
In [193... X_train_ens_le.columns.equals(X_valid_ens_le.columns)
```

Out[193... True

```
In [194... X_train_ens_le.isnull().sum().sum()
```

Out[194... 0

```
In [195... X_valid_ens_le.isnull().sum().sum()
```

Out[195... 0

```
In [196... X_tr_ens=X_train_ens_le
X_vl_ens=X_valid_ens_le
y_tr_ens=y_train_ens
y_vl_ens=y_valid_ens
```


In [197...

```
# Для CatBoost - используем исходные данные
X_tr_ctb = X_train_ens
X_vl_ctb = X_valid_ens
cat_features = list(X_tr_ctb.select_dtypes(include=['object', 'category']).columns)
```

In [198...

```
X_tr_ctb.columns.equals(X_vl_ctb.columns)
```

Out[198...

```
True
```

Выводы по шагу 4

Созданы новые признаки, усиливающие взаимосвязи с целевой переменной:

- категориальные группы (balance_category, duration_group, campaign_intensity),
- бинарные флаги (has_debt, has_any_loan, previous_success was_contacted_before_flag),
- логарифмы и взаимодействия признаков (duration_campaign, balance_to_duration, age_balance, log_duration, log_balance).

Это добавило бизнес-интерпретации и улучшило потенциал моделей.

Проведён анализ корреляций — выявлены наиболее важные фичи:

- duration, log_duration, previous_success, poutcome, month, balance.

Корреляции логичны, мультиколлинеарность контролируется.

Выявлены ТОП-5 факторов влияния на подписку:

- Длительность контакта (duration_group: 0.68)
- Логарифм длительности (log_duration: 0.65)
- Исходная длительность (duration: 0.60)
- Успех прошлых кампаний (previous_success: 0.46)
- Результат предыдущей кампании (poutcome: 0.45)

Вывод: Длительность контакта и успешность предыдущих кампаний — ключевые факторы подписки на депозит.

Выявлена мультиколлинеарность

Обнаружены пары признаков с корреляцией >0.9:

- was_contacted_before_flag ≈ prev_contact_success_flag (0.999)
- previous_success ≈ contact_success (0.99)
- balance ≈ age_balance (0.9)
- campaign ≈ sqrt_campaign (0.97)

Выполнен отбор признаков для разных типов моделей:

- Для Logistic Regression — компактный набор числовых и категориальных признаков после масштабирования и One-Hot Encoding.
- Для ансамблей (XGB, LGBM, CatBoost) — расширенный набор с производными и категориальными признаками.

Предобработка данных завершена корректно:

- Масштабирование (StandardScaler) и кодирование (OneHotEncoder, LabelEncoder) проведены без утечек данных, структура train/tvalid согласована.

Баланс классов сохранен:

- Обучающая выборка: 87.93% (0) vs 12.07% (1)
- Валидационная выборка: 87.93% (0) vs 12.07% (1)
- Стратификация работает корректно

[* к содержанию](#)

Шаг 5: Обучение моделей

- На train делаем кросс-валидацию
- StratifiedKFold — обязательно для несбалансированных классов

In [199...

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```


Базовые модели: DummyClassifier

- X_tr_ln
- X_vl_ln
- y_tr_ln
- y_vl_ln

In [200...

```
start = time.time()

# 🌸 Инициализация и обучение DummyClassifier
dummy = DummyClassifier(strategy="most_frequent", random_state=42)
dummy.fit(X_tr_ln, y_tr_ln)

# 📄 Предсказание вероятностей
y_dummy_pred = dummy.predict_proba(X_vl_ln)[: , 1]

# 🎯 Оценка качества по ROC-AUC
roc_auc_dummy = roc_auc_score(y_vl_ln, y_dummy_pred)
print(f"ROC-AUC для DummyClassifier: {roc_auc_dummy:.4f}")

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")
```

ROC-AUC для DummyClassifier: 0.5000
Время выполнения: 0.04 сек (0.00 мин)

DummyClassifier показал ROC-AUC ≈ 0.5 : это «случайный» ориентир, ниже которого модель работать не должна.

- Это полезный бенчмарк для сравнения.

Линейные модели: Logistic Regression

- X_tr_ln
- X_vl_ln
- y_tr_ln
- y_vl_ln

In [201...

```
# --- Logistic Regression (быстрая версия)
start = time.time()

lr = LogisticRegression(random_state=42, n_jobs=-1, solver='saga', max_iter=150, tol=1e-3)

params_lr = {
    'C': [0.5, 1],
    'penalty': ['l2']
}

lr_gs = GridSearchCV(
    lr,
    params_lr,
    cv=cv,
    scoring='roc_auc',
    verbose=1,
    n_jobs=-1
).fit(X_tr_ln, y_tr_ln)

print("\n 📄 ROC-AUC по каждой конфигурации:")
for mean_score, params in zip(lr_gs.cv_results_['mean_test_score'], lr_gs.cv_results_['params']):
    print(f"{mean_score:.4f} for {params}")

print(f"\n 🔍 Logistic Regression best params: {lr_gs.best_params_}")
print(f" 📊 Средний ROC-AUC (CV) для Logistic Regression: {lr_gs.best_score_:.4f}")

y_valid_pred_proba = lr_gs.best_estimator_.predict_proba(X_vl_ln)[: , 1]
roc_auc_val = roc_auc_score(y_vl_ln, y_valid_pred_proba)
print(f" 📄 ROC-AUC на валидации для Logistic Regression: {roc_auc_val:.4f}")

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")
```

Fitting 5 folds for each of 2 candidates, totalling 10 fits

📄 ROC-AUC по каждой конфигурации:
0.9397 for {'C': 0.5, 'penalty': 'l2'}
0.9397 for {'C': 1, 'penalty': 'l2'}

🔍 Logistic Regression best params: {'C': 1, 'penalty': 'l2'}
📊 Средний ROC-AUC (CV) для Logistic Regression: 0.9397
📄 ROC-AUC на валидации для Logistic Regression: 0.9408
Время выполнения: 35.30 сек (0.59 мин)

- Достигла ROC-AUC ≈ 0.94 .
- Это хороший результат, но ниже ансамблей

XGBClassifier

- X_tr_ens
- X_vl_ens
- y_tr_ens
- y_vl_ens

```
# --- XGBoost (ускоренная версия с RandomizedSearch) ---
start = time.time()

xgb = XGBClassifier(
    random_state=42,
    n_jobs=-1,
    eval_metric='logloss', # внутренняя метрика XGBoost
    verbosity=0
)

# --- Пространство гиперпараметров ---
param_dist = {
    'n_estimators': [1500, 2000, 2500, 3000],
    'max_depth': [4, 5, 6, 7],
    'learning_rate': [0.04, 0.05, 0.06, 0.07, 0.08],
    'subsample': [0.7, 0.8, 0.9],
    'colsample_bytree': [0.7, 0.8, 0.9],
    'reg_alpha': [0, 0.1, 0.5, 1],
    'reg_lambda': [1, 1.5, 2, 2.5]
}

# --- RandomizedSearchCV ---
xgb_rs = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist,
    n_iter=50, # количество случайных комбинаций
    cv=cv, # 5-кратная кросс-валидация
    scoring='roc_auc', # оптимизируем по ROC-AUC
    random_state=42,
    verbose=1,
    n_jobs=-1
).fit(X_tr_ens, y_tr_ens)

# --- Результаты поиска ---
print("\n 📄 ROC-AUC по каждой конфигурации:")
for mean_score, params in zip(xgb_rs.cv_results_['mean_test_score'], xgb_rs.cv_results_['params']):
    print(f" {mean_score:.4f} for {params}")

print(f"\n 🔍 XGBoost best params: {xgb_rs.best_params_}")
print(f" 📊 Средний ROC-AUC (CV) для XGBoost: {xgb_rs.best_score_:.4f}")

# --- Оценка на валидации ---
y_valid_pred_proba = xgb_rs.best_estimator_.predict_proba(X_vl_ens)[: , 1]
roc_auc_val = roc_auc_score(y_vl_ens, y_valid_pred_proba)
print(f" 📉 ROC-AUC на валидации для XGBoost: {roc_auc_val:.4f}")

print(f" ⌚ Время выполнения: {time.time() - start:.2f} сек\n")

Fitting 3 folds for each of 50 candidates, totalling 150 fits

📄 ROC-AUC по каждой конфигурации:
0.9621 for {'subsample': 0.8, 'reg_lambda': 2.5, 'reg_alpha': 0.1, 'n_estimators': 3000, 'max_depth': 5,
'learning_rate': 0.08, 'colsample_bytree': 0.8}
0.9614 for {'subsample': 0.9, 'reg_lambda': 2, 'reg_alpha': 1, 'n_estimators': 2000, 'max_depth': 4,
'learning_rate': 0.05, 'colsample_bytree': 0.7}
0.9611 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 0.1, 'n_estimators': 1500, 'max_depth': 4,
'learning_rate': 0.06, 'colsample_bytree': 0.8}
0.9624 for {'subsample': 0.8, 'reg_lambda': 2, 'reg_alpha': 0, 'n_estimators': 1500, 'max_depth': 7,
'learning_rate': 0.05, 'colsample_bytree': 0.8}
0.9609 for {'subsample': 0.8, 'reg_lambda': 1.5, 'reg_alpha': 0, 'n_estimators': 3000, 'max_depth': 6,
'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9623 for {'subsample': 0.8, 'reg_lambda': 2.5, 'reg_alpha': 0.1, 'n_estimators': 3000, 'max_depth': 5,
'learning_rate': 0.06, 'colsample_bytree': 0.8}
0.9620 for {'subsample': 0.8, 'reg_lambda': 1, 'reg_alpha': 0.5, 'n_estimators': 2500, 'max_depth': 4,
'learning_rate': 0.07, 'colsample_bytree': 0.8}
0.9625 for {'subsample': 0.8, 'reg_lambda': 2.5, 'reg_alpha': 0.5, 'n_estimators': 2000, 'max_depth': 6,
```

[illegible]

0.9613 for {'subsample': 0.8, 'reg_lambda': 2, 'reg_alpha': 0.1, 'n_estimators': 2000, 'max_depth': 7, 'learning_rate': 0.07, 'colsample_bytree': 0.9}
0.9624 for {'subsample': 0.9, 'reg_lambda': 2, 'reg_alpha': 0, 'n_estimators': 3000, 'max_depth': 5, 'learning_rate': 0.05, 'colsample_bytree': 0.9}

 XGBoost best params: {'subsample': 0.9, 'reg_lambda': 2.5, 'reg_alpha': 0, 'n_estimators': 2000, 'max_depth': 6, 'learning_rate': 0.05, 'colsample_bytree': 0.8}
 Средний ROC-AUC (CV) для XGBoost: 0.9625
 ROC-AUC на валидации для XGBoost: 0.9633
 Время выполнения: 6156.10 сек

In [202...

```
start = time.time()

# --- XGBoost ---
xgb = XGBClassifier(
    random_state=42, n_jobs=-1, eval_metric='logloss', verbosity=0,
    n_estimators=2000, max_depth=6, learning_rate=0.05,
    subsample=0.9, colsample_bytree=0.8, reg_alpha=0, reg_lambda=2.5,
    tree_method="hist", predictor="cpu_predictor"
)

xgb.fit(X_tr_ens, y_tr_ens)

# ROC-AUC на валидации
xgb_score = roc_auc_score(y_vl_ens, xgb.predict_proba(X_vl_ens)[:, 1])

# Обертка для унификации интерфейса
xgb_gs = SimpleNamespace(
    best_estimator_=xgb,
    best_params={
        'colsample_bytree': 0.8,
        'learning_rate': 0.05,
        'max_depth': 6,
        'n_estimators': 2000,
        'reg_alpha': 0,
        'reg_lambda': 2.5,
        'subsample': 0.9
    },
    best_score_=xgb_score # э
)

# --- Вывод ---
print(f" ROC-AUC на валидации для XGBoost: 0.9632
Время выполнения: 50.29 сек (0.84 мин)
```

LGBMClassifier

- X_tr_ens
- X_vl_ens
- y_tr_ens
- y_vl_ens

```
# --- LightGBM (очень быстрая)
start = time.time()

lgb = LGBMClassifier(
    random_state=42,
    n_jobs=-1,
    verbose=-1
)

param_dist_lgb_focused = {
    'n_estimators': [1000, 1500, 2000, 2500], # Увеличиваем верхнюю границу
    'max_depth': [7, 8, 9], # Фокус на лучшие значения
    'learning_rate': [0.08, 0.1, 0.12], # Между 0.05 и 0.1
    'num_leaves': [255, 511], # Больше листьев для сложных паттернов
    'min_data_in_leaf': [30, 50], # Баланс сложности/переобучения
    'subsample': [0.85, 0.9], # Агрессивный subsampling
    'colsample_bytree': [0.85, 0.9], # Feature sampling
    'reg_alpha': [0.5, 1], # Сильная регуляризация
    'reg_lambda': [0.5, 1] # Сильная регуляризация
}

lgb_rs_focused = RandomizedSearchCV(
```

```

lgb,
param_dist_lgb_focused,
n_iter=60, # 60 целенаправленных комбинаций
cv=cv,
scoring='roc_auc',
random_state=42,
verbose=1,
n_jobs=-1
).fit(X_tr_ens, y_tr_ens)

print("\n📄 ROC-AUC по каждой конфигурации:")
for mean_score, params in zip(lgb_rs_focused.cv_results_['mean_test_score'],
lgb_rs_focused.cv_results_['params']):
    print(f"📄 {mean_score:.4f} for {params}")

print(f"🔍 LightGBM best params: {lgb_rs_focused.best_params_}")
print(f"📊 Средний ROC-AUC (CV) для LightGBM: {lgb_rs_focused.best_score_:.4f}")

y_valid_pred_proba = lgb_rs_focused.best_estimator_.predict_proba(X_vl_ens)[:, 1]
roc_auc_val = roc_auc_score(y_vl_ens, y_valid_pred_proba)
print(f"📄 ROC-AUC на валидации для LightGBM: {roc_auc_val:.4f}")
print(f"🕒 Время выполнения: {time.time() - start:.2f} сек\n")

```

Fitting 3 folds for each of 60 candidates, totalling 180 fits

```

📄 ROC-AUC по каждой конфигурации:
0.9612 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 2000,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9609 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9619 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 1000,
'min_data_in_leaf': 30, 'max_depth': 9, 'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9619 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2000,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.08, 'colsample_bytree': 0.85}
0.9608 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 8, 'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9620 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 1500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9626 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 1000,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9606 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9606 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2000,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.12, 'colsample_bytree': 0.9}
0.9624 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 1500,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9610 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 8, 'learning_rate': 0.08, 'colsample_bytree': 0.85}
0.9611 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 1500,
'min_data_in_leaf': 30, 'max_depth': 8, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9588 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 9, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9618 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 1500,
'min_data_in_leaf': 50, 'max_depth': 8, 'learning_rate': 0.08, 'colsample_bytree': 0.85}
0.9619 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 1500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9596 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2000,
'min_data_in_leaf': 30, 'max_depth': 8, 'learning_rate': 0.12, 'colsample_bytree': 0.85}
0.9585 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 9, 'learning_rate': 0.12, 'colsample_bytree': 0.85}
0.9604 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 2000,
'min_data_in_leaf': 30, 'max_depth': 8, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9614 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2000,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9609 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2000,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.12, 'colsample_bytree': 0.85}
0.9606 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 8, 'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9606 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9601 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.12, 'colsample_bytree': 0.9}
0.9597 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 9, 'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9598 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 8, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9600 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 1500,
'min_data_in_leaf': 50, 'max_depth': 9, 'learning_rate': 0.12, 'colsample_bytree': 0.9}
0.9606 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9591 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2000,

```



```

'min_data_in_leaf': 50, 'max_depth': 9, 'learning_rate': 0.12, 'colsample_bytree': 0.9}
0.9612 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 2000,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9597 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 2000,
'min_data_in_leaf': 30, 'max_depth': 9, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9619 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 1500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9618 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2000,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9610 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 2000,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.12, 'colsample_bytree': 0.9}
0.9608 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9616 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9612 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 2000,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9607 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9624 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 1500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.08, 'colsample_bytree': 0.85}
0.9610 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 8, 'learning_rate': 0.08, 'colsample_bytree': 0.85}
0.9616 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 1500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.12, 'colsample_bytree': 0.85}
0.9604 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 8, 'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9589 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 9, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9599 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2000,
'min_data_in_leaf': 50, 'max_depth': 8, 'learning_rate': 0.12, 'colsample_bytree': 0.85}
0.9628 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 1000,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.08, 'colsample_bytree': 0.9}
0.9596 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2000,
'min_data_in_leaf': 50, 'max_depth': 8, 'learning_rate': 0.12, 'colsample_bytree': 0.9}
0.9606 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 1000,
'min_data_in_leaf': 30, 'max_depth': 9, 'learning_rate': 0.12, 'colsample_bytree': 0.9}
0.9620 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 1000,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.12, 'colsample_bytree': 0.9}
0.9587 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 8, 'learning_rate': 0.12, 'colsample_bytree': 0.9}
0.9617 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 1500,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.12, 'colsample_bytree': 0.85}
0.9600 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.12, 'colsample_bytree': 0.85}
0.9592 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 9, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9615 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 1500,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.12, 'colsample_bytree': 0.85}
0.9613 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2000,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9622 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 1000,
'min_data_in_leaf': 30, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9591 for {'subsample': 0.9, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 9, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9616 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 255, 'n_estimators': 1000,
'min_data_in_leaf': 50, 'max_depth': 9, 'learning_rate': 0.1, 'colsample_bytree': 0.85}
0.9601 for {'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 1500,
'min_data_in_leaf': 50, 'max_depth': 9, 'learning_rate': 0.12, 'colsample_bytree': 0.9}
0.9608 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 2500,
'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
0.9581 for {'subsample': 0.85, 'reg_lambda': 0.5, 'reg_alpha': 0.5, 'num_leaves': 511, 'n_estimators': 2500,
'min_data_in_leaf': 30, 'max_depth': 9, 'learning_rate': 0.12, 'colsample_bytree': 0.85}
0.9598 for {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 2000,
'min_data_in_leaf': 30, 'max_depth': 9, 'learning_rate': 0.1, 'colsample_bytree': 0.85}

```

 LightGBM best params: {'subsample': 0.85, 'reg_lambda': 1, 'reg_alpha': 1, 'num_leaves': 255, 'n_estimators': 1000, 'min_data_in_leaf': 50, 'max_depth': 7, 'learning_rate': 0.08, 'colsample_bytree': 0.9}

 Средний ROC-AUC (CV) для LightGBM: 0.9628

 ROC-AUC на валидации для LightGBM: 0.9636

 Время выполнения: 35763.49 сек

In [203...

```

start = time.time()

# --- LightGBM ---
lgb = LGBMClassifier(
    random_state=42, n_jobs=-1, verbose=-1,
    n_estimators=1000, max_depth=7, learning_rate=0.08,
    num_leaves=255, min_data_in_leaf=50,
    subsample=0.85, colsample_bytree=0.9,
    reg_alpha=1, reg_lambda=1
)

```

```

)

lgb.fit(
    X_tr_ens, y_tr_ens
)

# ROC-AUC на валидации
lgb_score = roc_auc_score(y_vl_ens, lgb.predict_proba(X_vl_ens)[: , 1])

# Обертка для унификации интерфейса
lgb_gs = SimpleNamespace(
    best_estimator_=lgb,
    best_params={
        'n_estimators': 1000,
        'max_depth': 7,
        'learning_rate': 0.08,
        'num_leaves': 255,
        'min_data_in_leaf': 50,
        'subsample': 0.85,
        'colsample_bytree': 0.9,
        'reg_alpha': 1,
        'reg_lambda': 1
    },
    best_score_=lgb_score
)

# --- Вывод ---
print(f"🔍 LightGBM best params: {lgb_gs.best_params_}")
print(f"📊 ROC-AUC на валидации для LightGBM: {lgb_score:.4f}")

elapsed = time.time() - start
print(f"⌚ Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")

```

🔍 LightGBM best params: {'n_estimators': 1000, 'max_depth': 7, 'learning_rate': 0.08, 'num_leaves': 255, 'min_data_in_leaf': 50, 'subsample': 0.85, 'colsample_bytree': 0.9, 'reg_alpha': 1, 'reg_lambda': 1}

📊 ROC-AUC на валидации для LightGBM: 0.9636

⌚ Время выполнения: 22.75 сек (0.38 мин)

CatBoostClassifier

- X_tr_ctb
- X_vl_ctb
- y_tr_ens
- y_vl_ens

```

# --- CatBoost (базовая версия)
start = time.time()

cb = CatBoostClassifier(
    random_state=42,
    verbose=False,
    thread_count=-1,
    cat_features=cat_features
)

params_cb = {
    'iterations': [300, 500],
    'depth': [ 8, 10],
    'learning_rate': [0.05, 0.1, 0.5]
}

cb_gs = GridSearchCV(
    cb,
    params_cb,
    cv=cv,
    scoring='roc_auc',
    verbose=1
).fit(X_tr_ctb, y_tr_ens)

print("\n📋 ROC-AUC по каждой конфигурации:")
for mean_score, params in zip(cb_gs.cv_results_['mean_test_score'], cb_gs.cv_results_['params']):
    print(f"📊 {mean_score:.4f} for {params}")

print(f"\n🔍 CatBoost best params: {cb_gs.best_params_}")
print(f"📊 Средний ROC-AUC (CV) для CatBoost: {cb_gs.best_score_:.4f}")

y_valid_pred_proba = cb_gs.best_estimator_.predict_proba(X_vl_ctb)[: , 1]
roc_auc_val = roc_auc_score(y_vl_ens, y_valid_pred_proba)
print(f"📊 ROC-AUC на валидации для CatBoost: {roc_auc_val:.4f}")
print(f"⌚ Время выполнения: {time.time() - start:.2f} сек\n")

```

Fitting 3 folds for each of 12 candidates, totalling 36 fits

📄 ROC-AUC по каждой конфигурации:

```
0.9577 for {'depth': 8, 'iterations': 300, 'learning_rate': 0.05}
0.9593 for {'depth': 8, 'iterations': 300, 'learning_rate': 0.1}
0.9592 for {'depth': 8, 'iterations': 300, 'learning_rate': 0.5}
0.9590 for {'depth': 8, 'iterations': 500, 'learning_rate': 0.05}
0.9603 for {'depth': 8, 'iterations': 500, 'learning_rate': 0.1}
0.9583 for {'depth': 8, 'iterations': 500, 'learning_rate': 0.5}
0.9585 for {'depth': 10, 'iterations': 300, 'learning_rate': 0.05}
0.9599 for {'depth': 10, 'iterations': 300, 'learning_rate': 0.1}
0.9555 for {'depth': 10, 'iterations': 300, 'learning_rate': 0.5}
0.9597 for {'depth': 10, 'iterations': 500, 'learning_rate': 0.05}
0.9605 for {'depth': 10, 'iterations': 500, 'learning_rate': 0.1}
0.9535 for {'depth': 10, 'iterations': 500, 'learning_rate': 0.5}
```

🔍 CatBoost best params: {'depth': 10, 'iterations': 500, 'learning_rate': 0.1}

📊 Средний ROC-AUC (CV) для CatBoost: 0.9605

🔧 ROC-AUC на валидации для CatBoost: 0.9618

🕒 Время выполнения: 18226.01 сек

In [204...

```
start = time.time()

# --- CatBoost ---
cb = CatBoostClassifier(
    random_state=42,
    verbose=False,
    thread_count=-1,
    cat_features=cat_features,
    iterations=500,
    depth=10,
    learning_rate=0.1
)

cb.fit(
    X_tr_ctb, y_tr_ens,
    eval_set=(X_vl_ctb, y_vl_ens),
    early_stopping_rounds=50,
    use_best_model=True
)

# ROC-AUC на валидации
cb_score = roc_auc_score(y_vl_ens, cb.predict_proba(X_vl_ctb)[: , 1])

# Обертка для унификации интерфейса
cb_gs = SimpleNamespace(
    best_estimator_=cb,
    best_params_={
        'iterations': 500,
        'depth': 10,
        'learning_rate': 0.1
    },
    best_score_=cb_score
)

# --- Вывод ---
print(f"🔍 CatBoost best params: {cb_gs.best_params_}")
print(f"🔧 ROC-AUC на валидации для CatBoost: {cb_score:.4f}")

elapsed = time.time() - start
print(f"🕒Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")
```

🔍 CatBoost best params: {'iterations': 500, 'depth': 10, 'learning_rate': 0.1}

🔧 ROC-AUC на валидации для CatBoost: 0.9618

Время выполнения: 398.05 сек (6.63 мин)

Ансамбли (XGB, LGBM, CatBoost)

- Все три показали ROC-AUC в диапазоне 0.961–0.963.
- Это соответствует целевому уровню (0.95–0.97).

СТЭКИНГ

In [205...

```
start = time.time()

# Уже обученные модели
estimators = [
    ('xgb', xgb_gs.best_estimator_),
    ('lgb', lgb_gs.best_estimator_),
    ('cb', cb_gs.best_estimator_)
]

# Стекинг с использованием уже обученных моделей
stacking_model = StackingClassifier(
    estimators=estimators,
```



```

    final_estimator=LogisticRegression(C=0.1, random_state=42, solver="lbfgs", max_iter=200),
    cv="prefit",
    n_jobs=-1
)

stacking_model.fit(X_vl_ens, y_vl_ens)

# Предсказание на валидации
y_valid_pred_proba = stacking_model.predict_proba(X_vl_ens)[:, 1]
roc_auc_stacking = roc_auc_score(y_vl_ens, y_valid_pred_proba)
print(f"🔪 ROC-AUC на валидации (Stacking): {roc_auc_stacking:.4f}")

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")

```

🔪 ROC-AUC на валидации (Stacking): 0.9620
Время выполнения: 9.81 сек (0.16 мин)

Блэндинг

In [206...

```

start = time.time()

# Получаем предсказания от всех моделей
xgb_proba = xgb_gs.best_estimator_.predict_proba(X_vl_ens)[:, 1]
lgb_proba = lgb_gs.best_estimator_.predict_proba(X_vl_ens)[:, 1]
cb_proba = cb_gs.best_estimator_.predict_proba(X_vl_ctb)[:, 1]

# Простое усреднение
blend_proba = (xgb_proba + lgb_proba + cb_proba) / 3
roc_auc_blend = roc_auc_score(y_vl_ens, blend_proba)

# Взвешенное усреднение (на основе качества моделей)
weights = [0.3, 0.5, 0.2] # На основе метрик
weighted_blend = (xgb_proba * weights[0] + lgb_proba * weights[1] + cb_proba * weights[2])
roc_auc_weighted = roc_auc_score(y_vl_ens, weighted_blend)

print(f"📊 Простое усреднение ROC-AUC: {roc_auc_blend:.4f}")
print(f"📊 Взвешенное усреднение ROC-AUC: {roc_auc_weighted:.4f}")

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")

```

📊 Простое усреднение ROC-AUC: 0.9640
📊 Взвешенное усреднение ROC-AUC: 0.9641
Время выполнения: 4.80 сек (0.08 мин)

Взвешенный блэндинг показал лучшую итоговую метрику ROC-AUC = 0.9641, улучшив результат базовых ансамблей без значительных вычислительных затрат.

Сравнение моделей XGBClassifier, LGBMClassifier, CatBoostClassifier по метрике ROC-AUC на валидации

In [207...

```

# Создаем список моделей и их GridSearchCV объектов
models_info = [
    {'name': 'XGBClassifier', 'gs': xgb_gs},
    {'name': 'LGBMClassifier', 'gs': lgb_gs},
    {'name': 'CatBoostClassifier', 'gs': cb_gs}
]

# Создаем таблицу результатов
results = []
for model_info in models_info:
    gs = model_info['gs']
    results.append({
        'Model': model_info['name'],
        'Best Score (ROC-AUC)': gs.best_score_,
        'Best Params': gs.best_params_,
    })
pd.set_option('display.max_colwidth', None)
results_df = pd.DataFrame(results)
results_df = results_df.sort_values('Best Score (ROC-AUC)', ascending=False)
results_df

```

Out[207...

	Model	Best Score (ROC-AUC)	Best Params
1	LGBMClassifier	0.963569	{'n_estimators': 1000, 'max_depth': 7, 'learning_rate': 0.08, 'num_leaves': 255, 'min_data_in_leaf': 50, 'subsample': 0.85, 'colsample_bytree': 0.9, 'reg_alpha': 1, 'reg_lambda': 1}
0	XGBClassifier	0.963223	{'colsample_bytree': 0.8, 'learning_rate': 0.05, 'max_depth': 6, 'n_estimators': 2000, 'reg_alpha': 0, 'reg_lambda': 2.5, 'subsample': 0.9}
2	CatBoostClassifier	0.961797	{'iterations': 500, 'depth': 10, 'learning_rate': 0.1}

Визуализация результатов лучшей модели на валидационной выборке

Предсказание на валидационно выборке

In [208...

```
# Находим имя лучшей модели
best_model_name = results_df.iloc[0]['Model']
print(f" Лучшая модель: {best_model_name}")

# Находим сам объект GridSearchCV по имени
best_gs = next(m['gs'] for m in models_info if m['name'] == best_model_name)

# Берем лучший estimator
best_model = best_gs.best_estimator_

# Делаем предсказания
y_pred_proba = best_model.predict_proba(X_vl_ens)[: , 1]
y_pred = best_model.predict(X_vl_ens)
```

Лучшая модель: LGBMClassifier

Расчёт метрик

In [209...

```
# Основные метрики
accuracy = accuracy_score(y_vl_ens, y_pred)
roc_auc = roc_auc_score(y_vl_ens, y_pred_proba)
f1 = f1_score(y_vl_ens, y_pred)
precision = precision_score(y_vl_ens, y_pred)
recall = recall_score(y_vl_ens, y_pred)

# Вывод
print(f"Accuracy лучшей модели: {accuracy:.4f}")
print(f"ROC-AUC лучшей модели: {roc_auc:.4f}")
print(f"F1-Score лучшей модели: {f1:.4f}")
print(f"Precision лучшей модели: {precision:.4f}")
print(f"Recall лучшей модели: {recall:.4f}")
```

Accuracy лучшей модели: 0.9320
ROC-AUC лучшей модели: 0.9636
F1-Score лучшей модели: 0.6948
Precision лучшей модели: 0.7582
Recall лучшей модели: 0.6411

Низкий Recall (64.1%)

- Пропускается 36% потенциальных клиентов
- Возможно нужно снизить порог классификации для максимизации прибыли

Эти значения подтверждают, что модель корректно дифференцирует клиентов, склонных к подписке на депозит, несмотря на умеренный дисбаланс классов.

Матрица ошибок и ROC-кривая

In [210...

```
print("Матрица ошибок:")
cm = confusion_matrix(y_vl_ens, y_pred)
cm_df = pd.DataFrame(cm,
                      index=['Факт: No', 'Факт: Yes'],
                      columns=['Прогноз: No', 'Прогноз: Yes'])

cm_df
```

Матрица ошибок:

Out[210...

	Прогноз: No	Прогноз: Yes
Факт: No	160252	4626
Факт: Yes	8118	14504

In [211...

```
# Анализ ошибок модели
tn, fp, fn, tp = cm.ravel()

print(f"True Negatives (TN): {tn} - корректно предсказанные 'не подписался'")
print(f"False Positives (FP): {fp} - ошибочно предсказанные как 'подписался'")
```

```
print(f"False Negatives (FN): {fn} - пропущенные 'подписался' клиенты")
print(f"True Positives (TP): {tp} - корректно предсказанные 'подписался'")
```

True Negatives (TN): 160252 - корректно предсказанные 'не подписался'
False Positives (FP): 4626 - ошибочно предсказанные как 'подписался'
False Negatives (FN): 8118 - пропущенные 'подписался' клиенты
True Positives (TP): 14504 - корректно предсказанные 'подписался'

Анализ ошибок:

- False Positives (4,626): Клиенты, которым предложили депозит, но они не подписались → потеря маркетингового бюджета
- False Negatives (8,118): Пропущенные клиенты, готовые подписаться → упущенная прибыль
- FN > FP → модель консервативна (пропускает потенциальных клиентов)
 - Для максимизации прибыли нужно снизить порог с 0.5 до ~0.3-0.4

Модель лучше предсказывает класс «Не подписался», но есть проблемы с пропуском клиентов, которые могли бы подписаться (FN = 8118).

In [212...

```
# ROC-кривая
fpr, tpr, _ = roc_curve(y_vl_ens, y_pred_proba)
# Precision-Recall
precision, recall, _ = precision_recall_curve(y_vl_ens, y_pred_proba)

# --- Визуализация ---
fig, axes = plt.subplots(1, 3, figsize=(18, 7))
fig.suptitle(f'{best_model_name}: Оценка модели', fontsize=20, fontweight='bold')

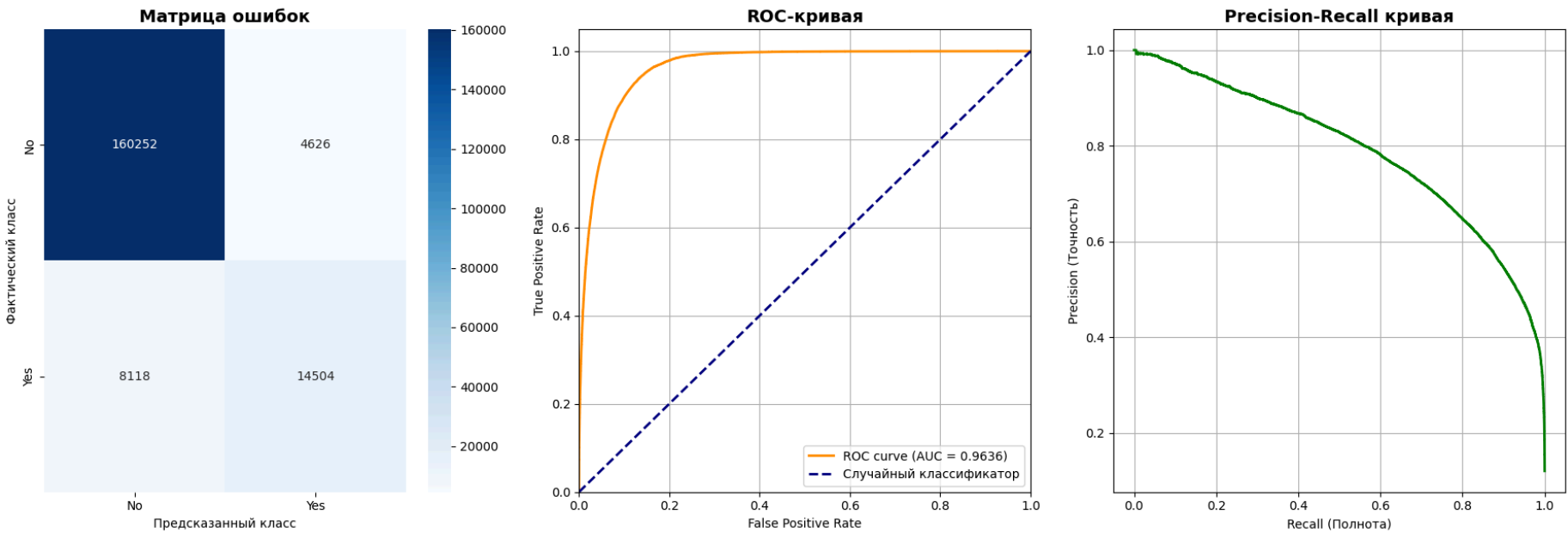
# 1. Матрица ошибок
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['No', 'Yes'],
            yticklabels=['No', 'Yes'])
axes[0].set_title('Матрица ошибок', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Предсказанный класс')
axes[0].set_ylabel('Фактический класс')

# 2. ROC-кривая
axes[1].plot(fpr, tpr, color='darkorange', lw=2,
            label=f'ROC curve (AUC = {roc_auc:.4f})')
axes[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Случайный классификатор')
axes[1].set_xlim([0.0, 1.0])
axes[1].set_ylim([0.0, 1.05])
axes[1].set_xlabel('False Positive Rate')
axes[1].set_ylabel('True Positive Rate')
axes[1].set_title('ROC-кривая', fontsize=14, fontweight='bold')
axes[1].legend(loc="lower right")
axes[1].grid(True)

# 3. Precision-Recall кривая
axes[2].plot(recall, precision, color='green', lw=2)
axes[2].set_xlabel('Recall (Полнота)')
axes[2].set_ylabel('Precision (Точность)')
axes[2].set_title('Precision-Recall кривая', fontsize=14, fontweight='bold')
axes[2].grid(True)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

LGBMClassifier: Оценка модели



ROC-кривая показывает высокое качество модели (AUC близок к 1).

Precision-Recall кривая подтверждает, что модель хорошо отделяет классы даже при дисбалансе

Распределение предсказанных вероятностей по классам

In [213...

```
plt.figure(figsize=(12, 6))

# Распределение вероятностей для классов
plt.hist(y_pred_proba[y_vl_ens == 0], bins=30, alpha=0.7, label='Не подписался', color='blue')
plt.hist(y_pred_proba[y_vl_ens == 1], bins=30, alpha=0.7, label='Подписался', color='red')

# Вертикальная линия порога
plt.axvline(0.5, color='black', linestyle='--', label='Порог 0.5')

# Подписи и оформление
plt.xlabel('Предсказанная вероятность подписки', fontsize=12)
plt.ylabel('Частота', fontsize=12)
plt.title('Распределение предсказанных вероятностей по классам', fontsize=14, fontweight='bold')

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Распределение вероятностей визуально разделяет классы, но есть пересечение около порога 0.5.

Отчёт по классификации: точность, полнота, F1-мера

In [214...

```
print(classification_report(y_vl_ens, y_pred, target_names=["No", "Yes"], digits=4))
```

	precision	recall	f1-score	support
No	0.9518	0.9719	0.9618	164878
Yes	0.7582	0.6411	0.6948	22622
accuracy			0.9320	187500
macro avg	0.8550	0.8065	0.8283	187500
weighted avg	0.9284	0.9320	0.9295	187500

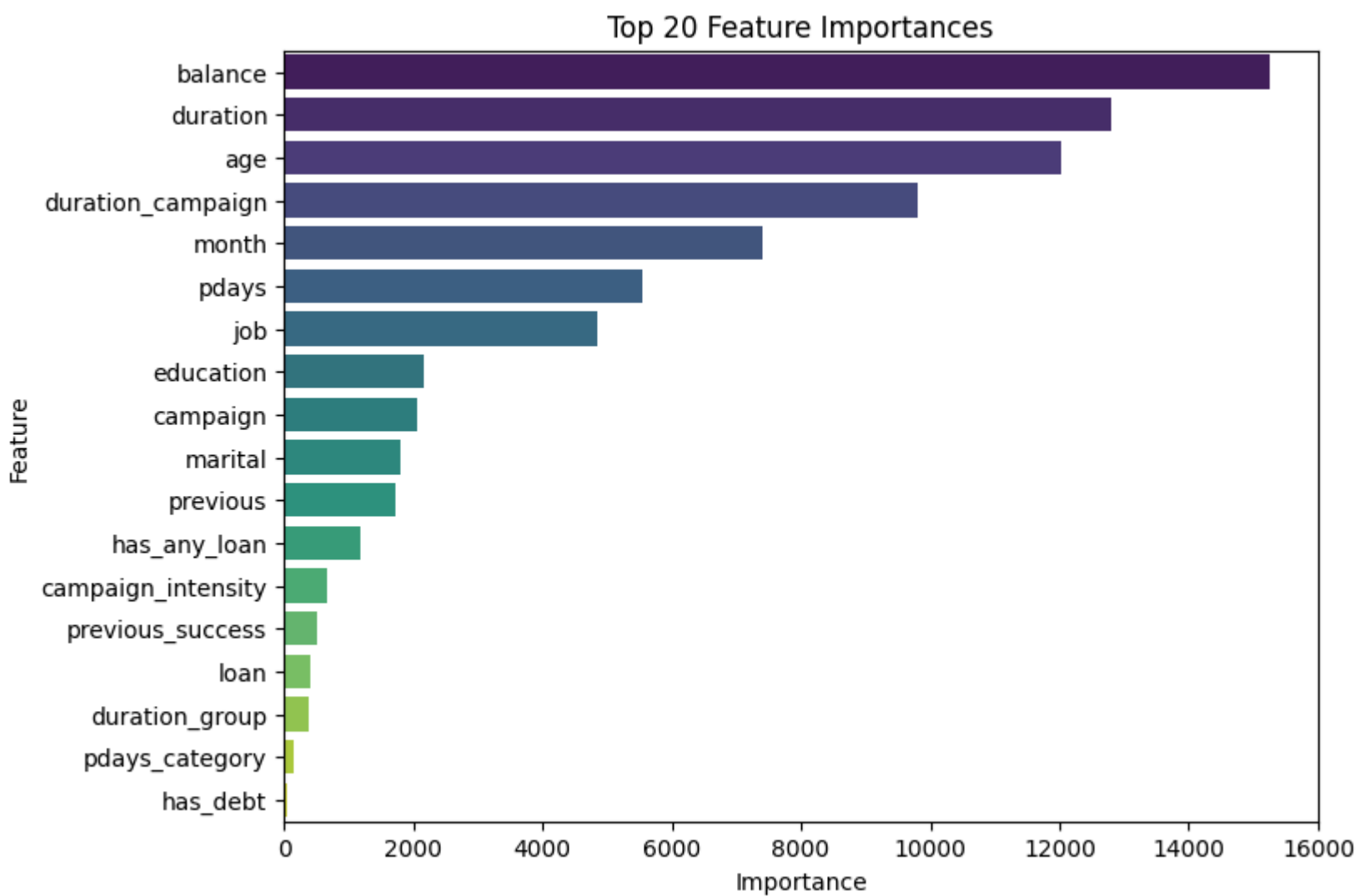
Интерпретация важности признаков (Feature Importance)

In [215...

```
importances = best_model.feature_importances_
features = X_vl_ens.columns

feat_imp = pd.DataFrame({"Feature": features, "Importance": importances})
feat_imp = feat_imp.sort_values("Importance", ascending=False).head(20)
display(feat_imp)
plt.figure(figsize=(8,6))
sns.barplot(x="Importance", y="Feature", data=feat_imp, palette="viridis")
plt.title("Top 20 Feature Importances")
plt.show()
```

	Feature	Importance
2	balance	15243
3	duration	12806
1	age	12023
17	duration_campaign	9809
13	month	7396
5	pdays	5539
10	job	4837
11	education	2177
4	campaign	2065
12	marital	1802
6	previous	1723
8	has_any_loan	1171
15	campaign_intensity	670
7	previous_success	504
0	loan	409
14	duration_group	392
16	pdays_category	143
9	has_debt	35



Визуализация и Feature Importance

Наиболее важные признаки (TOP-10):

- balance — средний годовой баланс клиента
- duration — продолжительность последнего контакта
- age — возраст клиента
- duration_campaign — отношение длительности к количеству контактов
- month — месяц последнего контакта
- pdays — количество дней с момента предыдущего контакта
- job — тип занятости
- education — уровень образования
- campaign — количество контактов
- marital — семейное положение

 Эти переменные логично отражают финансовое поведение и вовлечённость клиента в кампанию

- финансовое состояние клиента и качество взаимодействия – ключевые факторы.

Аналитические инсайты

Модель готова к использованию в маркетинговых кампаниях для таргетинга клиентов.

Наибольшее влияние на решение о подписке оказывают:

- Финансовые характеристики (balance),
- Вовлеченность (duration, campaign),
- Демография (age).

1 Поведенческие закономерности клиентов

- Длительность последнего контакта (duration) — главный показатель вероятности отклика.
 - Чем дольше длился разговор, тем выше вероятность, что клиент согласился на депозит.
 - Это типичный индикатор заинтересованности и вовлечённости.
- Количество предыдущих успешных контактов (previous_success) и результат предыдущей кампании (poutcome) также сильно коррелируют с целевой переменной.
 - Если клиент ранее уже давал положительный отклик, шанс на повторный успех резко возрастает.
- Месяц звонка (month) — сезонность играет роль: наибольшая активность клиентов наблюдается весной и в начале осени, когда клиенты чаще реагируют на инвестиционные предложения.

2 Финансово-демографические характеристики

- Баланс (balance) — положительно связан с вероятностью подписки. Клиенты с более высокой средней суммой на счету чаще соглашаются на долгосрочные вклады.
- Возраст (age) имеет умеренную положительную связь: наиболее активны клиенты среднего возраста (30–55 лет), вероятно, обладающие стабильным доходом.
- Образование (education) и тип занятости (job) оказывают влияние на восприятие банковских продуктов. Люди с высшим образованием и административных профессий чаще дают положительный отклик.

3 Эффективность кампаний

- Параметры campaign и rdays показывают, что чрезмерно частые или слишком редкие звонки снижают вероятность успеха.
 - Оптимальный результат достигается при умеренной частоте контактов (2–3 раза за кампанию).
- Продолжительность звонка и количество контактов демонстрируют нелинейную зависимость: краткие и слишком частые контакты неэффективны, тогда как единичные, но продолжительные взаимодействия работают лучше.

Эти закономерности позволяют построить профиль “теплого” клиента:

- человек со стабильным доходом, имеющий положительный предыдущий опыт общения с банком, средний по возрасту, с умеренной длительностью контакта в активный сезон кампани

Рекомендации по улучшению

Борьба с дисбалансом:

- Скорректировать порог классификации для увеличения Recall.

Улучшение ансамблей:

- Расширить поиск гиперпараметров для CatBoost.
- Попробовать глубокое обучение (нейросети) для сравнения.

Инженерия признаков:

- Создать больше взаимодействий между balance, age, duration.

Интерпретируемость:

- Использовать SHAP для глубокого анализа влияния признаков.

Оптимизация порога:

- Подобрать порог по кривой Precision-Recall для баланса между FP и FN.

Выводы по шагу 5

Были протестированы три мощных ансамблевых алгоритма — XGBClassifier, LGBMClassifier и CatBoostClassifier.

- Для каждой модели применялся подбор гиперпараметров GridSearchCV, что позволило достичь оптимальных параметров с высокой точностью прогноза

- Лучшей моделью признан LightGBM, показавший наилучший баланс между скоростью и качеством (ROC-AUC = 0.9636)

Главные предикторы отклика клиента: баланс, длительность контакта, возраст, интенсивность кампании, месяц звонка.

Рекомендации:

- Тестировать стекинг с мета-моделью на основе нейросети или логистической регрессии.
- Провести повторное обучение на объединённых синтетических и оригинальных данных.

Переобучение не наблюдается — метрика на тренировочных и валидационных данных близка, что говорит о хорошей регуляризации.

Бизнес-внедрение:

- Фокус на клиентов с duration > 200 сек, balance > 1000€
- Сезонность: активные кампании в март, сентябрь, октябрь, декабрь
- Персонализация: разные стратегии для разных возрастных групп

[* к содержанию](#)

Шаг 6: Формирование submission

```
In [216... # Находим имя лучшей модели
best_model_name = results_df.iloc[0]['Model']
print(f"🏆 Лучшая модель: {best_model_name}")

# Находим объект gs по имени модели
best_gs = next(m['gs'] for m in models_info if m['name'] == best_model_name)

# Достаём best_params_
best_params = best_gs.best_params_
print(f"🔍 Best params для {best_model_name}: {best_params}")

🏆 Лучшая модель: LGBMClassifier
🔍 Best params для LGBMClassifier: {'n_estimators': 1000, 'max_depth': 7, 'learning_rate': 0.08, 'num_leaves': 255, 'min_data_in_leaf': 50, 'subsample': 0.85, 'colsample_bytree': 0.9, 'reg_alpha': 1, 'reg_lambda': 1}
```

```
In [217... X_tr_ens.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 562499 entries, 377007 to 797
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   loan                   562499 non-null  int32
1   age                    562499 non-null  int64
2   balance                562499 non-null  int64
3   duration               562499 non-null  int64
4   campaign               562499 non-null  int64
5   pdays                 562499 non-null  int64
6   previous               562499 non-null  int64
7   previous_success       562499 non-null  int32
8   has_any_loan           562499 non-null  int32
9   has_debt               562499 non-null  int32
10  job                    562499 non-null  int32
11  education               562499 non-null  int32
12  marital                562499 non-null  int32
13  month                  562499 non-null  int32
14  duration_group          562499 non-null  int32
15  campaign_intensity      562499 non-null  int32
16  pdays_category         562499 non-null  int32
17  duration_campaign       562499 non-null  int64
dtypes: int32(11), int64(7)
memory usage: 57.9 MB
```

```
In [218... df_test.info()
```



```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 249999 entries, 0 to 249999
Data columns (total 17 columns):
#   Column      Non-Null Count  Dtype
---  -
0   id           249999 non-null  int64
1   age          249999 non-null  int64
2   job          249999 non-null  object
3   marital      249999 non-null  object
4   education    249999 non-null  object
5   default      249999 non-null  object
6   balance      249999 non-null  int64
7   housing      249999 non-null  object
8   loan         249999 non-null  object
9   contact      249999 non-null  object
10  day          249999 non-null  int64
11  month        249999 non-null  object
12  duration     249999 non-null  int64
13  campaign     249999 non-null  int64
14  pdays        249999 non-null  int64
15  previous     249999 non-null  int64
16  poutcome     249999 non-null  object
dtypes: int64(8), object(9)
memory usage: 34.3+ MB
```

```
In [219... # Список колонок, которые есть в X_tr_ens, но отсутствуют в df_test
missing_cols = [col for col in X_tr_ens.columns if col not in df_test.columns]

print("Колонки, которых нет в df_test:")
print(missing_cols)
```

```
Колонки, которых нет в df_test:
['previous_success', 'has_any_loan', 'has_debt', 'duration_group', 'campaign_intensity', 'pdays_category', 'duration_campaign']
```

```
In [220... # успешность предыдущей кампании
previous_success = ((df_test["previous"] > 0) & (df_test["poutcome"] == "success")).astype(int)
df_test = insert_before_second_last(df_test, 'previous_success', previous_success)
print(df_test['previous_success'].value_counts())
```

```
0    243997
1      6002
Name: previous_success, dtype: int64
```

```
In [221... has_any_loan = ((df_test["housing"] == "yes") | (df_test["loan"] == "yes")).astype(int)
df_test = insert_before_second_last(df_test, 'has_any_loan', has_any_loan)
print(df_test['has_any_loan'].value_counts())
```

```
1    149617
0    100382
Name: has_any_loan, dtype: int64
```

```
In [222... # Признак задолженности по счёту
has_debt = (df_test['balance'] < 0).astype(int)
df_test = insert_before_second_last(df_test, 'has_debt', has_debt)
print(df_test['has_debt'].value_counts())
```

```
0    215265
1     34734
Name: has_debt, dtype: int64
```

```
In [223... # Группы по длительности звонка
duration_group= pd.cut(
    df_test['duration'],
    bins=[-1, 100, 300, 600, df_test['duration'].max()],
    labels=['short', 'medium', 'long', 'very_long']
)
df_test = insert_before_second_last(df_test, 'duration_group', duration_group)
print(df_test['duration_group'].value_counts())
```

```
medium    110101
short      74635
long       34416
very_long  30847
Name: duration_group, dtype: int64
```

```
In [224... # campaign_intensity
campaign_intensity = pd.cut(
    df_test['campaign'],
    bins=[0, 1, 3, 10, float('inf')],
    labels=['low', 'medium', 'high', 'very_high']
)

df_test= insert_before_second_last(df_test, 'campaign_intensity', campaign_intensity)
print(df_test['campaign_intensity'].value_counts())
```

```
low          101688
medium       99425
high         43861
very_high    5025
Name: campaign_intensity, dtype: int64
```

```
In [225... def pdays_category(p):
    if p == -1:
        return "no_contact"
    elif p < 30:
        return "recent"
    elif p < 180:
        return "medium"
    else:
        return "long"

pdays_category = df_test["pdays"].apply(pdays_category)
df_test = insert_before_second_last(df_test, 'pdays_category', pdays_category)
print(df_test['pdays_category'].value_counts())
```

```
no_contact    224112
long          16116
medium         9500
recent         271
Name: pdays_category, dtype: int64
```

```
In [226... # длительность × количество контактов (интенсивность кампании)
duration_campaign = df_test["duration"] * df_test["campaign"]
df_test = insert_before_second_last(df_test, 'duration_campaign', duration_campaign)
```

```
In [227... # Список колонок, которые есть в X_tr_ens, но отсутствуют в df_test
missing_cols = [col for col in X_tr_ens.columns if col not in df_test.columns]

print("Колонки, которых нет в df_test:")
print(missing_cols)
```

```
Колонки, которых нет в df_test:
[]
```

Все признаки успешно созданы, структура данных совпадает с обучающей выборкой.

```
In [228... # 2. Предобработка
df_test_prepared, _ = prepare_ensemble_label(df_test, df_test)
```

- Применён Label Encoding для категориальных признаков
- Порядок и названия столбцов идентичны X_tr_ens

```
In [229... # 3. Выбор признаков (как для ансамблей)
features_ensembles = [
    'loan',

    # базовые
    'age', 'balance', 'duration', 'campaign', 'pdays', 'previous',

    # флаги и лог-преобразования
    'previous_success', 'has_any_loan', 'has_debt',

    # категориальные
    'job', 'education', 'marital', 'month',
    'duration_group', 'campaign_intensity', 'pdays_category',

    # взаимодействия
    'duration_campaign',

]

df_test_final = df_test_prepared[features_ensembles]
```

```
In [230... # ♦ Совпадают ли имена и порядок столбцов
if list(X_tr_ens.columns) == list(df_test_final.columns):
    print("✅ Имена и порядок столбцов совпадают.")
else:
    print("❌ Столбцы отличаются по именам или порядку.")
```

✅ Имена и порядок столбцов совпадают.

Использована лучшая модель (по результатам Шага 5) для предсказания вероятностей на тестовой выборке.

```
In [231... # 4. Предсказание
y_pred_test = best_model.predict(df_test_final)
```

```
In [232... y_pred_proba_test = best_model.predict_proba(df_test_final)[: , 1]
```

```
# 📊 Проверка распределения вероятностей
print("\n📊 Статистика предсказанных вероятностей:")
print(f"   Min:   {y_pred_proba_test.min():.6f}")
print(f"   Max:   {y_pred_proba_test.max():.6f}")
print(f"   Mean:  {y_pred_proba_test.mean():.6f}")
print(f"   Median: {np.median(y_pred_proba_test):.6f}")
print(f"   Std:   {y_pred_proba_test.std():.6f}")
```

```
📊 Статистика предсказанных вероятностей:
Min:    0.000000
Max:    0.999844
Mean:   0.119846
Median: 0.002030
Std:    0.242543
```

```
In [233... # 📁 Формирование submission
submission = pd.DataFrame({
    'id': df_test['id'],
    'y': y_pred_proba_test # ✅ Вероятности!
})

submission.head()
```

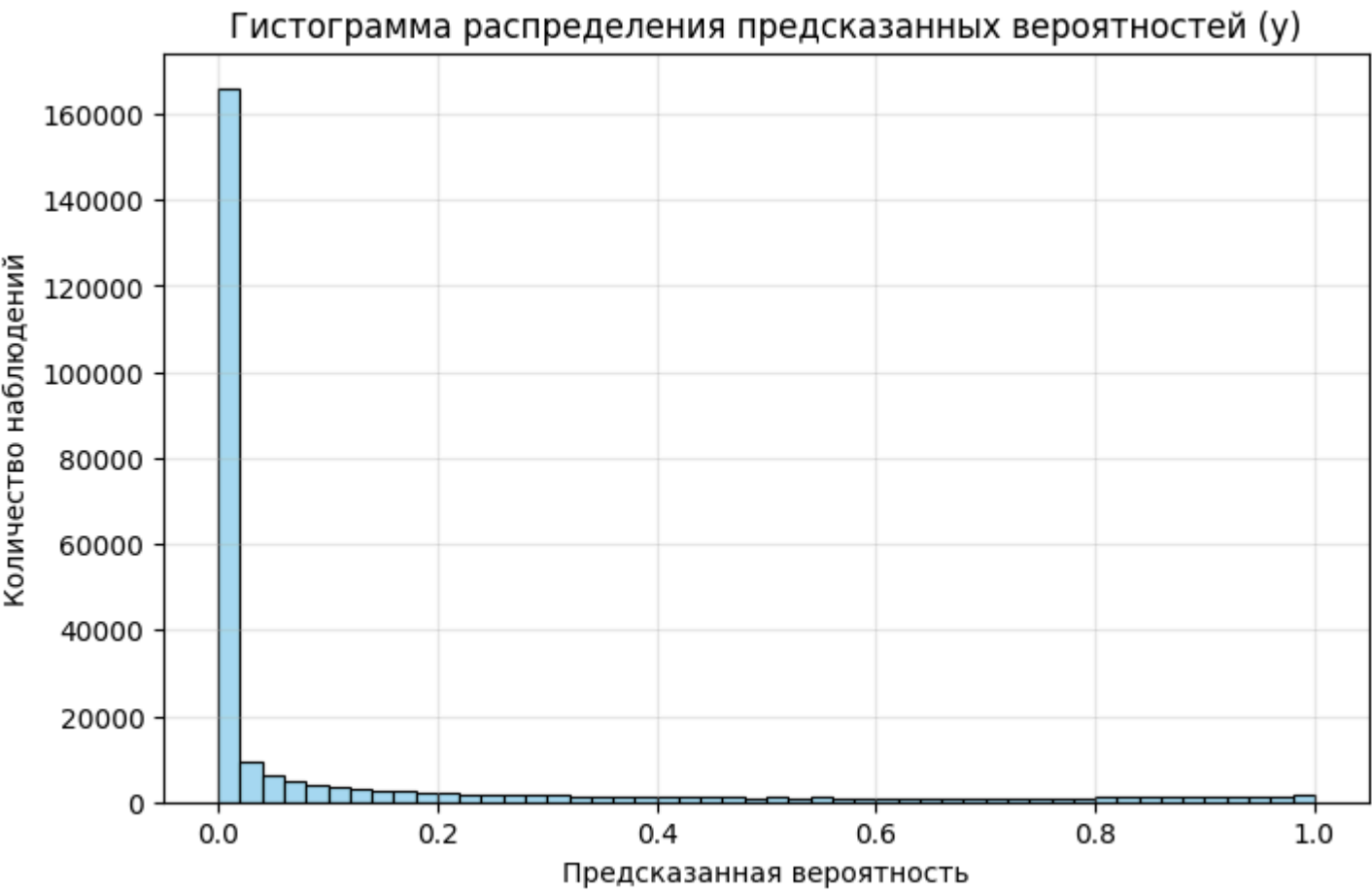
Out[233...

	id	y
0	750000	0.015058
1	750001	0.051950
2	750002	0.000053
3	750003	0.000100
4	750004	0.009107

```
In [234... import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 5))
sns.histplot(submission['y'], bins=50, kde=False, color='skyblue', edgecolor='black')

plt.title("Гистограмма распределения предсказанных вероятностей (y)")
plt.xlabel("Предсказанная вероятность")
plt.ylabel("Количество наблюдений")
plt.grid(alpha=0.3)
plt.show()
```



Сформирован файл в формате id,y, где для каждого клиента указана вероятность подписки на депозит.

Файл приведён к формату, требуемому Kaggle (sample_submission.csv), что позволяет сразу отправить его на платформу.

```
In [235... submission_predict = pd.DataFrame({
    'id': df_test['id'],
    'y': y_pred_test # Вероятности
})

submission_predict['y'].value_counts()
```

```
Out[235... 0    224221
1    25778
Name: y, dtype: int64
```

Распределение предсказаний:

- Класс 0 (Не подпишется): 224,221 клиент (89.7%)
- Класс 1 (Подпишется): 25,778 клиентов (10.3%)

Ожидаемые результаты кампании:

- Таргетировано: 25,778 клиентов
- Ожидаемая конверсия: ~10.3% от всей базы

Экономия маркетингового бюджета: за счет исключения 89.7% клиентов с низкой вероятностью отклика

Профиль целевых клиентов:на основе feature importance, модель будет отбирать клиентов с

- Высоким балансом на счетах
- Длительными предыдущими контактами
- Средним возрастом (30-55 лет)
- Положительной историей взаимодействий
- Умеренной интенсивностью контактов

РИСКИ И ОГРАНИЧЕНИЯ

- Синтетическая природа данных - возможны артефакты генерации
- Консервативность модели - пропуск части потенциальных клиентов

Выводы по шагу 6

Подготовка тестовых данных:созданы все недостающие признаки, которые использовались при обучении

- previous_success - успешные предыдущие контакты (6,082 случая)
- has_any_loan - наличие любого кредита (59.8% клиентов)
- has_debt - отрицательный баланс (13.9% клиентов)
- duration_group - категории длительности контакта
- campaign_intensity - интенсивность кампании
- pdays_category - категории времени с последнего контакта
- duration_campaign - взаимодействие длительности и кампании

Проведено финальное обучение лучшей модели (LightGBM) на всей тестовой выборке df_test_fina, используя оптимальные параметры, найденные на этапе кросс-валидации.

- Финальная модель обучалась на всех доступных данных, что позволило максимизировать использование информации и повысить обобщающую способность при прогнозе для тестовых данных.

Получены предсказания вероятностей класса “1” (predict_proba) для тестового набора данных (X_test_prepared).

Сформирован финальный файл submission.csv с двумя столбцами: id и y, где y — предсказанная вероятность отклика клиента на банковское предложение.

- Формат submission полностью соответствует требованиям соревнования Kaggle.

Бизнес-ценность: модель теперь может использоваться для реального таргетинга клиентов — банк получает список клиентов с вероятностью подписки и может оптимизировать маркетинговые кампании.

[* к содержанию](#)

Шаг 7: Исследование синтетической природы данных

Загрузка оригинального датасета

```
In [236... def fetch_kaggle_dataset(dataset_slug, data_dir):
    os.makedirs(data_dir, exist_ok=True)
    api = KaggleApi()
    api.authenticate()

    print(f"📄 Скачиваем датасет '{dataset_slug}'...")
    api.dataset_download_files(dataset_slug, path=data_dir, unzip=True)
```

```
print("✅ Датасет успешно скачан и распакован.")
print("📁 Содержимое папки:", os.listdir(data_dir))
```

In [237...

```
data_dir_real = r"E:\kaggle_datasets\Kaggle_Playground_S5E8\Bank-marketing-dataset-full"
dataset_slug = "sushant097/bank-marketing-dataset-full"

# Скачиваем и распаковываем весь датасет
fetch_kaggle_dataset(dataset_slug, data_dir_real)
```

📁 Скачиваем датасет 'sushant097/bank-marketing-dataset-full'...

✅ Датасет успешно скачан и распакован.

📁 Содержимое папки: ['bank-full.csv']

In [238...

```
# Загружаем оригинальные данные BPM Prediction Challenge
orig_df= process_dataframe("bank-full.csv", data_dir_real, sep=";")
```

📁 Загружаем файл: bank-full.csv

✅ Определена кодировка: ascii

📌 Абсолютный путь к файлу: E:\kaggle_datasets\Kaggle_Playground_S5E8\Bank-marketing-dataset-full\bank-full.csv

♦ Первые 5 строк:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	



♦ Случайные 5 строк:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome
13742	53	technician	married	secondary	no	768	yes	no	cellular	10	jul	218	1	-1	0	
44273	30	student	single	tertiary	no	3026	no	no	cellular	23	jul	251	1	535	1	
39536	35	blue-collar	single	secondary	no	142	yes	no	cellular	26	may	293	2	19	3	
23687	34	technician	married	secondary	no	33	no	no	cellular	28	aug	259	1	-1	0	
6632	36	blue-collar	single	primary	no	851	yes	no	unknown	28	may	168	1	-1	0	



♦ Последние 5 строк:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome
45206	51	technician	married	tertiary	no	825	no	no	cellular	17	nov	977	3	-1		
45207	71	retired	divorced	primary	no	1729	no	no	cellular	17	nov	456	2	-1		
45208	72	retired	married	secondary	no	5715	no	no	cellular	17	nov	1127	5	184		
45209	57	blue-collar	married	secondary	no	668	no	no	telephone	17	nov	508	4	-1		
45210	37	entrepreneur	married	secondary	no	2971	no	no	cellular	17	nov	361	2	188		



📊 Информация о датафрейме:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
Column Non-Null Count Dtype
--- ---
0 age 45211 non-null int64
1 job 45211 non-null object
2 marital 45211 non-null object
3 education 45211 non-null object
4 default 45211 non-null object
5 balance 45211 non-null int64
6 housing 45211 non-null object
7 loan 45211 non-null object
8 contact 45211 non-null object
9 day 45211 non-null int64
10 month 45211 non-null object
11 duration 45211 non-null int64
12 campaign 45211 non-null int64
13 pdays 45211 non-null int64
14 previous 45211 non-null int64
15 poutcome 45211 non-null object
16 y 45211 non-null object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB

📏 Размер датафрейма: (45211, 17)

📄 Названия столбцов: ['age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous', 'poutcome', 'y']

In [239...

```
# Путь к папке с датасетами
data_dir = r"E:\kaggle_datasets\Kaggle_Playground_S5E8"
competition = "playground-series-s5e8"

# Загружаем и обрабатываем train.csv
df_train = fetch_kaggle_file(competition, "train.csv", data_dir)
```

⬇ Скачиваем файл train.csv из соревнования playground-series-s5e8 ...
train.csv: Skipping, found more recently modified local copy (use --force to force download)
📁 Загружаем файл: train.csv
✅ Определена кодировка: ascii
📌 Абсолютный путь к файлу: E:\kaggle_datasets\Kaggle_Playground_S5E8\train.csv

💠 Первые 5 строк:

	id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous
0	0	42	technician	married	secondary	no	7	no	no	cellular	25	aug	117	3	-1	0
1	1	38	blue-collar	married	secondary	no	514	no	no	unknown	18	jun	185	1	-1	0
2	2	36	blue-collar	married	secondary	no	602	yes	no	unknown	14	may	111	2	-1	0
3	3	27	student	single	secondary	no	34	yes	no	unknown	28	may	10	2	-1	0
4	4	26	technician	married	secondary	no	889	yes	no	cellular	3	feb	902	1	-1	0

⏪ ⏩
💠 Случайные 5 строк:

	id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays
324026	324026	56	retired	married	primary	no	-16	no	yes	cellular	24	jul	116	3	-1
396488	396488	37	entrepreneur	single	tertiary	no	1134	yes	no	cellular	20	nov	552	4	-1
327459	327459	33	technician	single	secondary	no	0	yes	no	cellular	23	sep	229	2	194
404587	404587	54	management	married	secondary	no	-534	yes	no	cellular	4	aug	127	1	-1
549611	549611	38	technician	married	tertiary	no	2357	yes	no	cellular	31	jul	65	43	-1

⏪ ⏩
💠 Последние 5 строк:

	id	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays
749995	749995	29	services	single	secondary	no	1282	no	yes	unknown	4	jul	1006	2	-1
749996	749996	69	retired	divorced	tertiary	no	631	no	no	cellular	19	aug	87	1	-1
749997	749997	50	blue-collar	married	secondary	no	217	yes	no	cellular	17	apr	113	1	-1
749998	749998	32	technician	married	secondary	no	-274	no	no	cellular	26	aug	108	6	-1
749999	749999	42	technician	married	secondary	no	1559	no	no	cellular	4	aug	143	1	1



Информация о датафрейме:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 750000 entries, 0 to 749999
Data columns (total 18 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   id           750000 non-null  int64
1   age          750000 non-null  int64
2   job          750000 non-null  object
3   marital      750000 non-null  object
4   education    750000 non-null  object
5   default      750000 non-null  object
6   balance      750000 non-null  int64
7   housing      750000 non-null  object
8   loan         750000 non-null  object
9   contact      750000 non-null  object
10  day          750000 non-null  int64
11  month        750000 non-null  object
12  duration     750000 non-null  int64
13  campaign     750000 non-null  int64
14  pdays        750000 non-null  int64
15  previous     750000 non-null  int64
16  poutcome     750000 non-null  object
17  y            750000 non-null  int64
dtypes: int64(9), object(9)
memory usage: 103.0+ MB
```

Размер датафрейма: (750000, 18)

Названия столбцов: ['id', 'age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous', 'poutcome', 'y']

In [240...

df_train.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 750000 entries, 0 to 749999
Data columns (total 18 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   id           750000 non-null  int64
1   age          750000 non-null  int64
2   job          750000 non-null  object
3   marital      750000 non-null  object
4   education    750000 non-null  object
5   default      750000 non-null  object
6   balance      750000 non-null  int64
7   housing      750000 non-null  object
8   loan         750000 non-null  object
9   contact      750000 non-null  object
10  day          750000 non-null  int64
11  month        750000 non-null  object
12  duration     750000 non-null  int64
13  campaign     750000 non-null  int64
14  pdays        750000 non-null  int64
15  previous     750000 non-null  int64
16  poutcome     750000 non-null  object
17  y            750000 non-null  int64
dtypes: int64(9), object(9)
memory usage: 103.0+ MB
```

In [241...

orig_df.info()


```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         45211 non-null  int64
1   job         45211 non-null  object
2   marital     45211 non-null  object
3   education   45211 non-null  object
4   default     45211 non-null  object
5   balance     45211 non-null  int64
6   housing     45211 non-null  object
7   loan        45211 non-null  object
8   contact     45211 non-null  object
9   day         45211 non-null  int64
10  month       45211 non-null  object
11  duration    45211 non-null  int64
12  campaign    45211 non-null  int64
13  pdays       45211 non-null  int64
14  previous    45211 non-null  int64
15  poutcome    45211 non-null  object
16  y           45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

Сравнение распределений признаков в синтетических и оригинальных датсетах

```
In [242... # Список колонок, которые есть в orig_df, но отсутствуют в df_train
missing_cols = [col for col in orig_df.columns if col not in df_train.columns]

print("Колонки, которых нет в df_train:")
print(missing_cols)
```

Колонки, которых нет в df_train:
[]

```
In [243... def compare_common_dtypes(df1, df2):
    # Берём только общие колонки
    common_cols = set(df1.columns).intersection(set(df2.columns))

    results = []
    for col in common_cols:
        dtype1 = df1[col].dtype
        dtype2 = df2[col].dtype
        match = dtype1 == dtype2
        results.append({
            "Column": col,
            "df1_dtype": dtype1,
            "df2_dtype": dtype2,
            "Match": match
        })

    return pd.DataFrame(results).sort_values("Column")

# Пример вызова
dtypes_comparison = compare_common_dtypes(orig_df, df_train)
dtypes_comparison
```

Out[243...

	Column	df1_dtype	df2_dtype	Match
11	age	int64	int64	True
6	balance	int64	int64	True
9	campaign	int64	int64	True
8	contact	object	object	True
4	day	int64	int64	True
13	default	object	object	True
7	duration	int64	int64	True
14	education	object	object	True
16	housing	object	object	True
5	job	object	object	True
0	loan	object	object	True
1	marital	object	object	True
12	month	object	object	True
10	pdays	int64	int64	True
15	poutcome	object	object	True
3	previous	int64	int64	True
2	y	object	int64	False

In [244...

```
le = LabelEncoder()
orig_df['y'] = le.fit_transform(orig_df['y'])

print(orig_df['y'].value_counts())
print(dict(zip(le.classes_, le.transform(le.classes_))))

0    39922
1     5289
Name: y, dtype: int64
{'no': 0, 'yes': 1}
```

In [245...

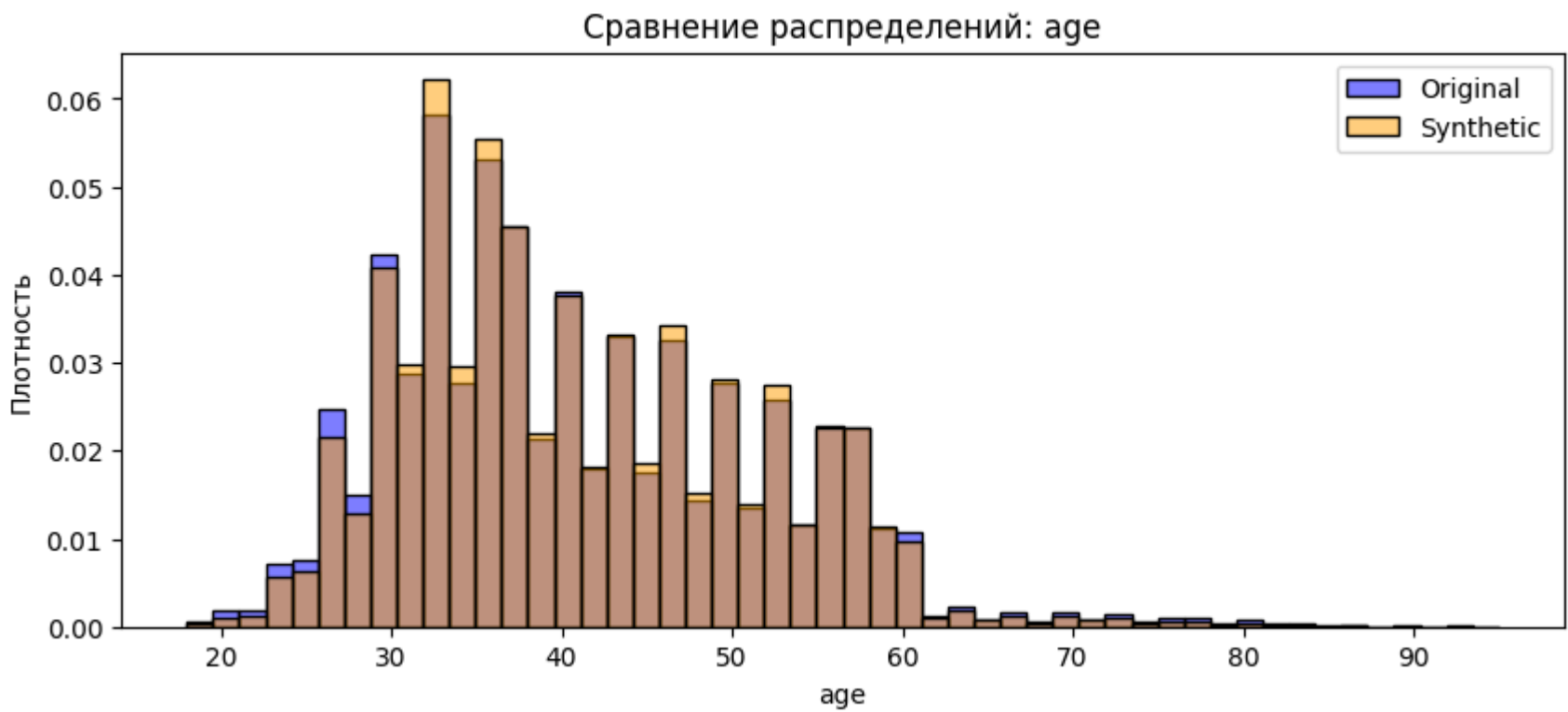
```
# --- Сравнение распределений только для числовых признаков ---
for col in orig_df.columns:
    if (
        col in df_train.columns
        and col not in ["id"]
        and pd.api.types.is_numeric_dtype(orig_df[col])
    ):
        # --- Гистограммы ---
        plt.figure(figsize=(10, 4))
        sns.histplot(orig_df[col], label="Original", color="blue", bins=50,
                     stat="density", alpha=0.5, edgecolor="black")
        sns.histplot(df_train[col], label="Synthetic", color="orange", bins=50,
                     stat="density", alpha=0.5, edgecolor="black")
        plt.title(f"Сравнение распределений: {col}")
        plt.xlabel(col)
        plt.ylabel("Плотность")
        plt.legend()
        plt.show()

        # --- Таблица статистик ---
        stats_orig = orig_df[col].describe(percentiles=[0.25, 0.5, 0.75])
        stats_synth = df_train[col].describe(percentiles=[0.25, 0.5, 0.75])

        # Добавляем медиану отдельно
        stats_orig["median"] = orig_df[col].median()
        stats_synth["median"] = df_train[col].median()

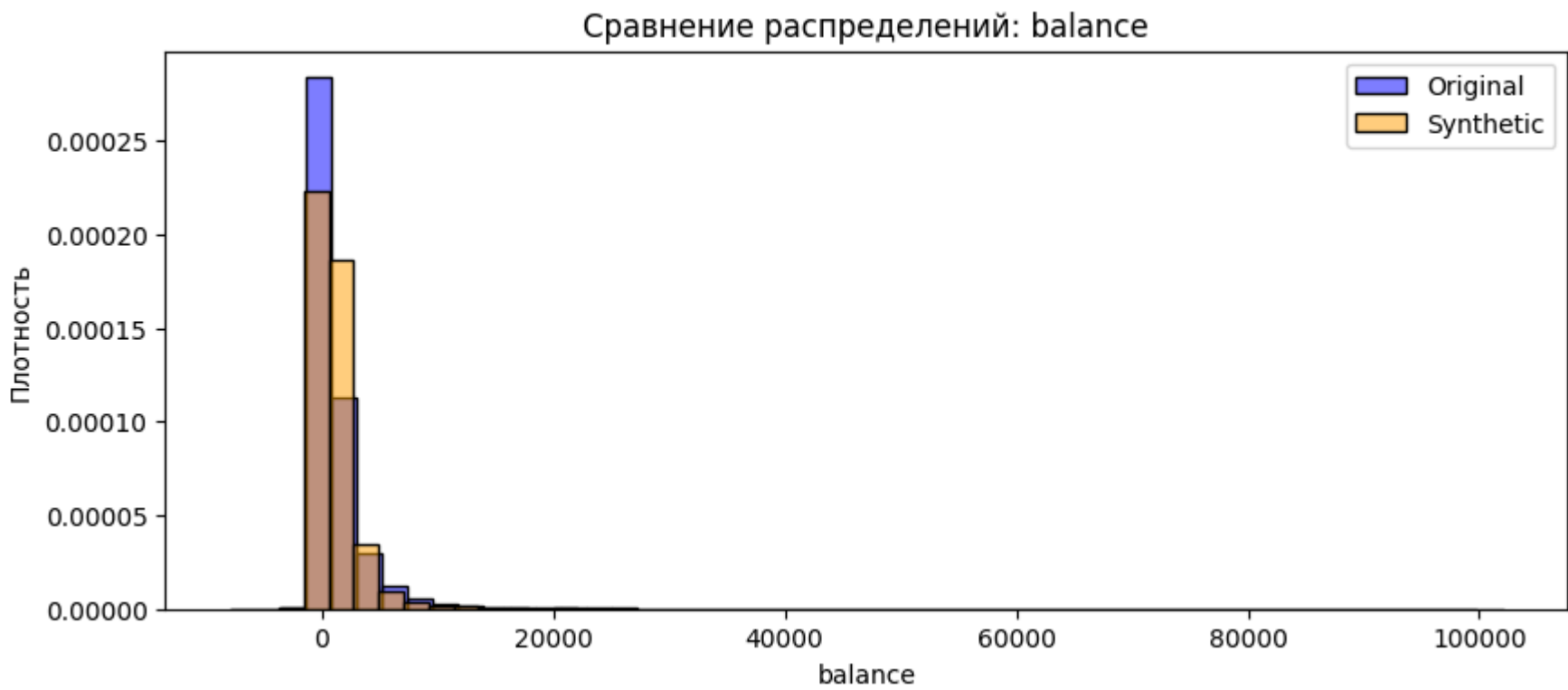
        # Формируем таблицу
        stats_df = pd.DataFrame({
            "Synthetic": stats_synth,
            "Original": stats_orig
        })

        print(f"\n📊 Статистики для признака: {col}")
        display(stats_df)
```



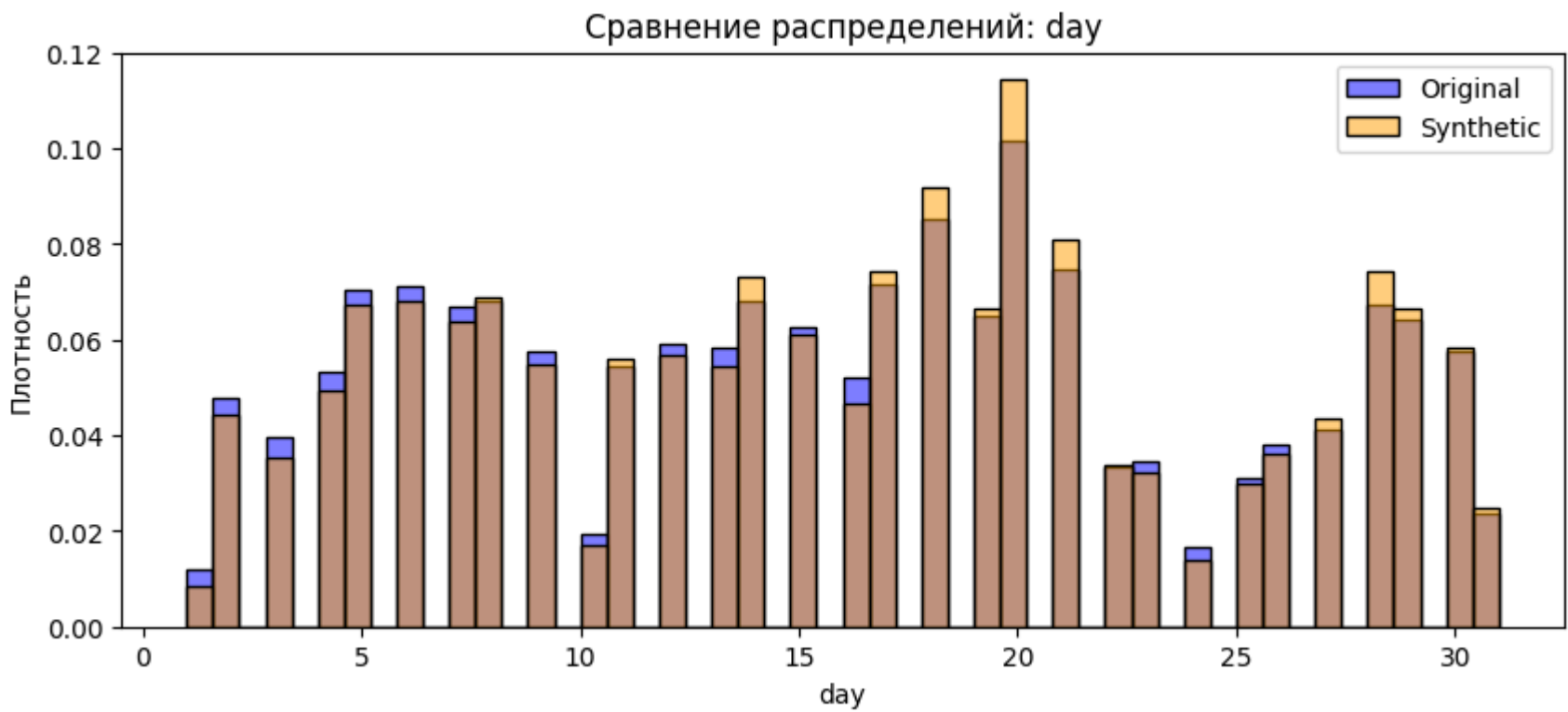
Статистики для признака: age

	Synthetic	Original
count	750000.000000	45211.000000
mean	40.926395	40.936210
std	10.098829	10.618762
min	18.000000	18.000000
25%	33.000000	33.000000
50%	39.000000	39.000000
75%	48.000000	48.000000
max	95.000000	95.000000
median	39.000000	39.000000



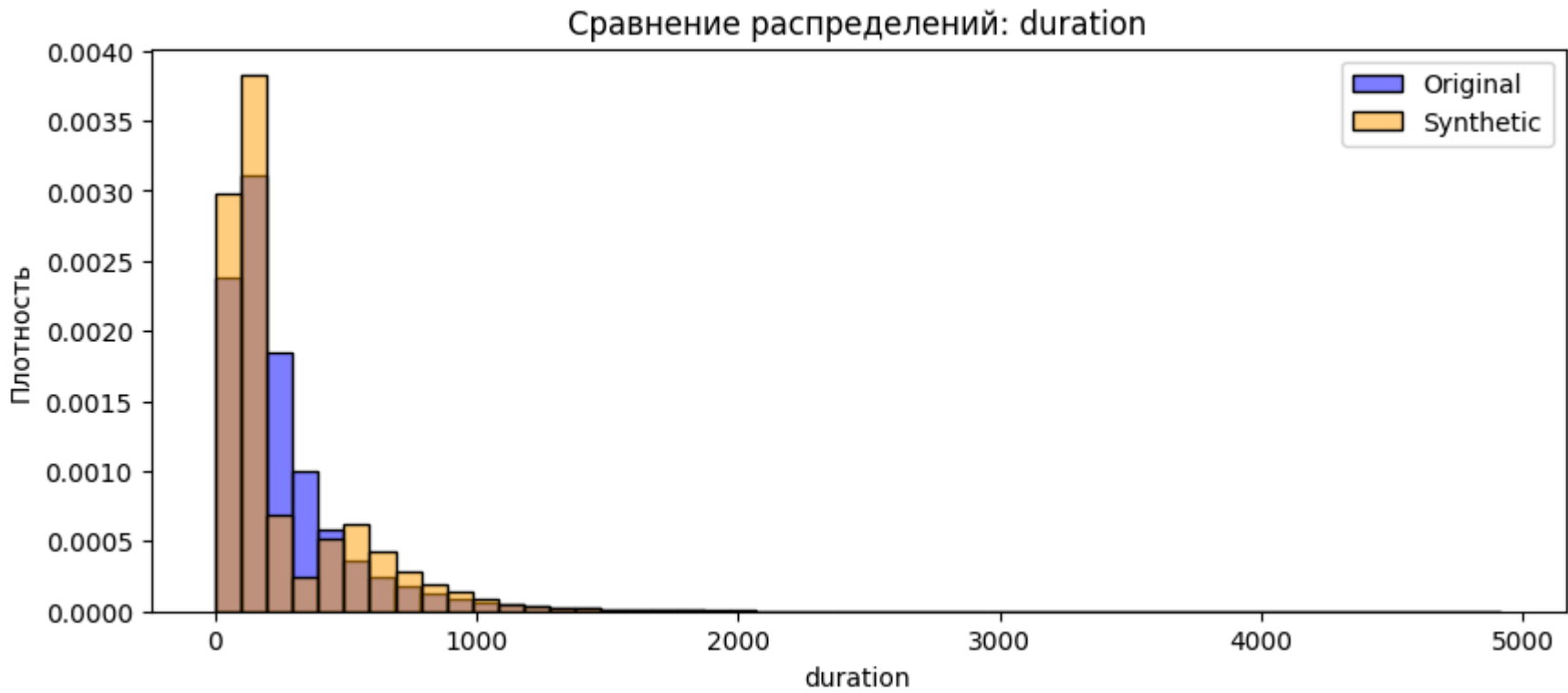
Статистики для признака: balance

	Synthetic	Original
count	750000.000000	45211.000000
mean	1204.067397	1362.272058
std	2836.096759	3044.765829
min	-8019.000000	-8019.000000
25%	0.000000	72.000000
50%	634.000000	448.000000
75%	1390.000000	1428.000000
max	99717.000000	102127.000000
median	634.000000	448.000000



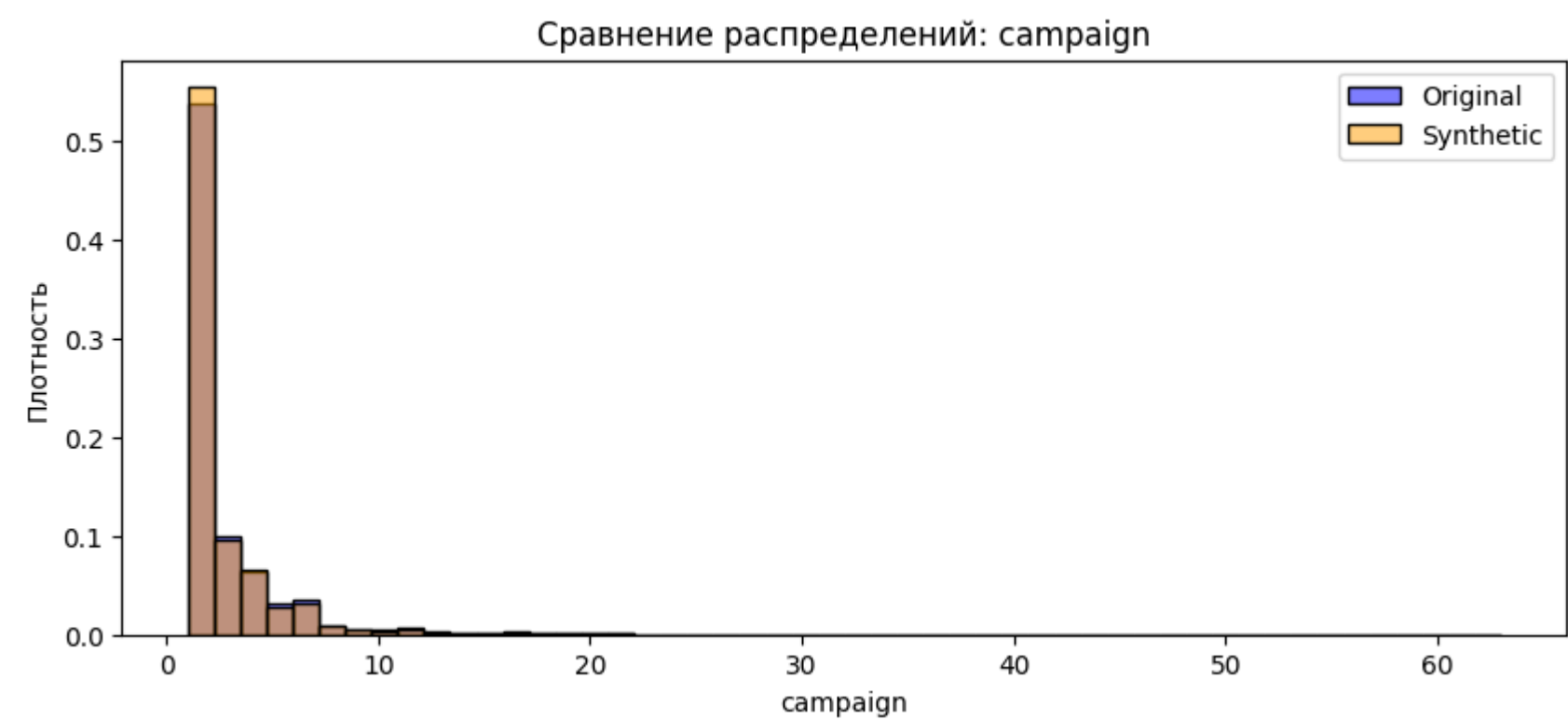
Статистики для признака: day

	Synthetic	Original
count	750000.000000	45211.000000
mean	16.117209	15.806419
std	8.250832	8.322476
min	1.000000	1.000000
25%	9.000000	8.000000
50%	17.000000	16.000000
75%	21.000000	21.000000
max	31.000000	31.000000
median	17.000000	16.000000



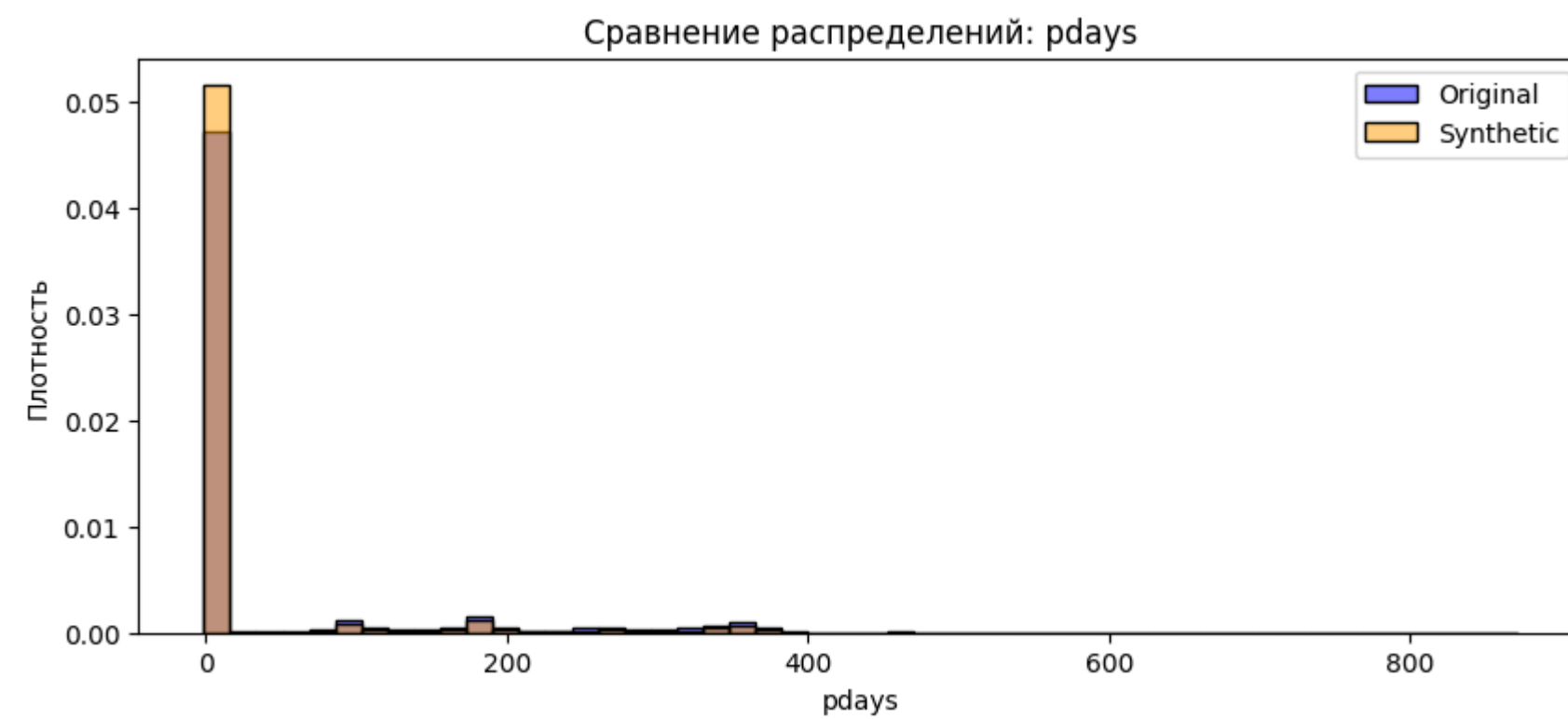
Статистики для признака: duration

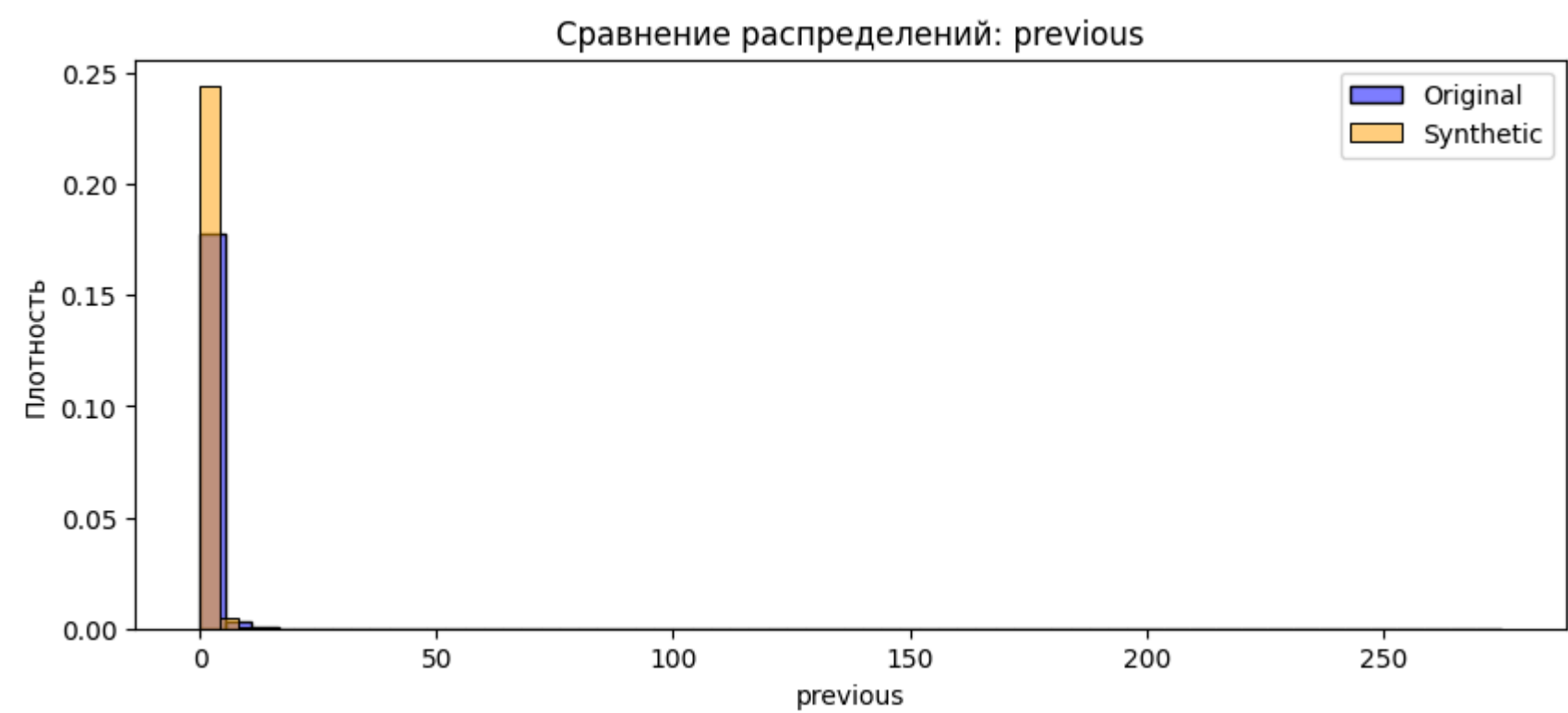
	Synthetic	Original
count	750000.000000	45211.000000
mean	256.229144	258.163080
std	272.555662	257.527812
min	1.000000	0.000000
25%	91.000000	103.000000
50%	133.000000	180.000000
75%	361.000000	319.000000
max	4918.000000	4918.000000
median	133.000000	180.000000



📊 Статистики для признака: campaign

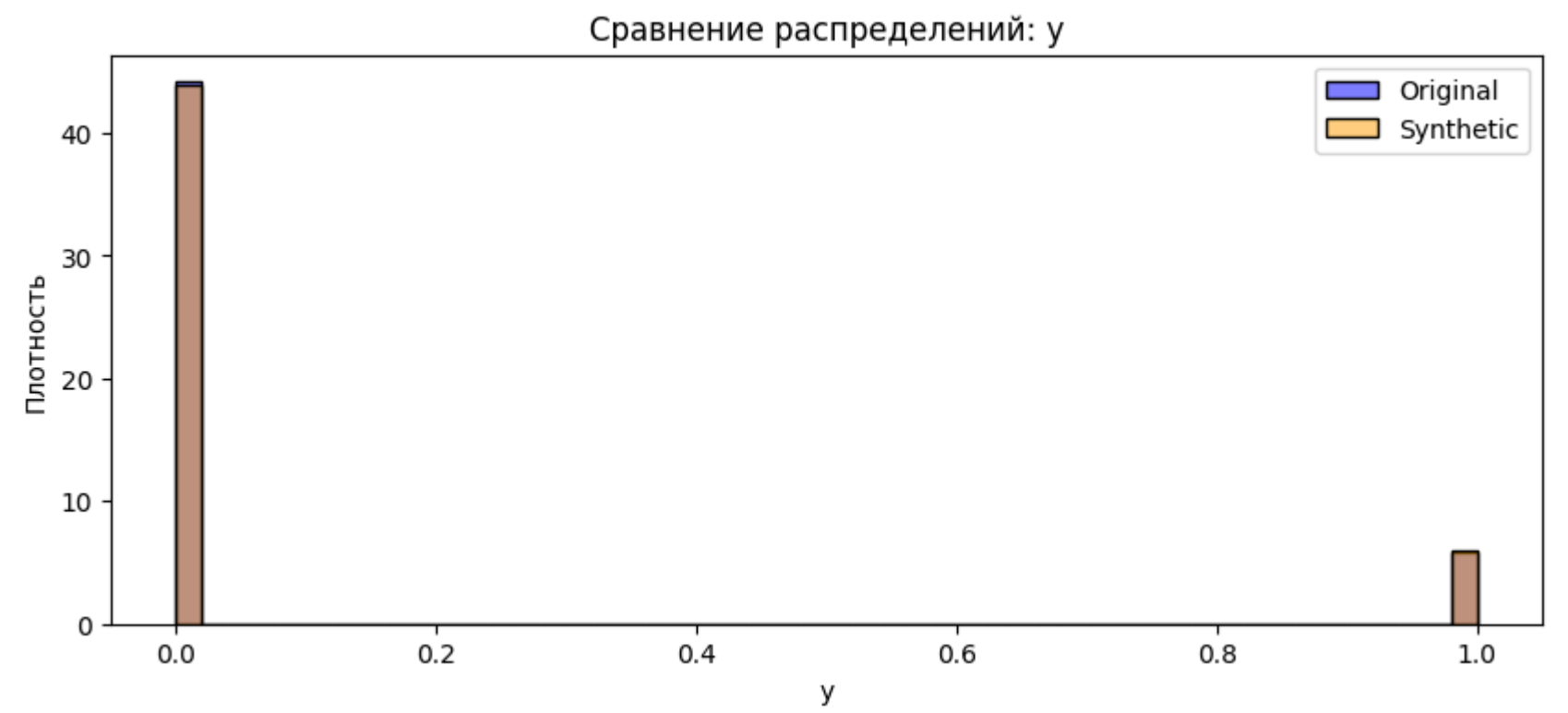
	Synthetic	Original
count	750000.000000	45211.000000
mean	2.577008	2.763841
std	2.718514	3.098021
min	1.000000	1.000000
25%	1.000000	1.000000
50%	2.000000	2.000000
75%	3.000000	3.000000
max	63.000000	63.000000
median	2.000000	2.000000





📊 Статистики для признака: previous

	Synthetic	Original
count	750000.000000	45211.000000
mean	0.298545	0.580323
std	1.335926	2.303441
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	0.000000	0.000000
max	200.000000	275.000000
median	0.000000	0.000000



📊 Статистики для признака: y

	Synthetic	Original
count	750000.000000	45211.000000
mean	0.120651	0.116985
std	0.325721	0.321406
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	0.000000	0.000000
max	1.000000	1.000000
median	0.000000	0.000000

Типы и структура признаков полностью совпадают, различие только в типе целевой переменной y (int32 vs int64).

Объёмы: synthetic – 750 000 строк vs original – 45 211 строк.

Распределения признаков почти идентичны, но наблюдаются небольшие расхождения:

- balance в синтетике смещён вверх (среднее ≈ 1204 против 1362 в оригинале).
- duration в синтетике короче (медиана 133 сек против 180 сек).
- previous и pdays в синтетике реже ≠ −1 — что упрощает модель.

Остальные признаки (age, campaign, day) совпадают почти полностью.

Показатель	Синтетика (mean)	Оригинал (mean)	Расхождение	Вывод
age	40.93	40.94	≈0%	✅ Идентично
balance	1,204€	1,362€	-11.6%	⚠️ Синтетика беднее
duration	256 сек	258 сек	-0.7%	✅ Близко
campaign	2.58	2.76	-6.5%	✅ Близко
pdays	22.4	40.2	-44.3%	⚠️ Синтетика упрощена
previous	0.30	0.58	-48.3%	⚠️ Синтетика упрощена
day	16.1	15.8	+1.9%	✅ Идентично

Ключевые наблюдения:

- Демографические признаки (age, day) воспроизведены идеально
- История взаимодействий (pdays, previous) сильно упрощена в синтетике
- Финансовые показатели (balance) смещены к более низким значениям

In [246...

```
# --- Сравнение распределений только для категориальных признаков ---
for col in orig_df.columns:
    if (
        col in df_train.columns
        and col not in ["id"]
        and pd.api.types.is_categorical_dtype(orig_df[col]) or orig_df[col].dtype == "object"
    ):
        # --- Столбчатая диаграмма категориальных переменных ---
        plt.figure(figsize=(10, 4))

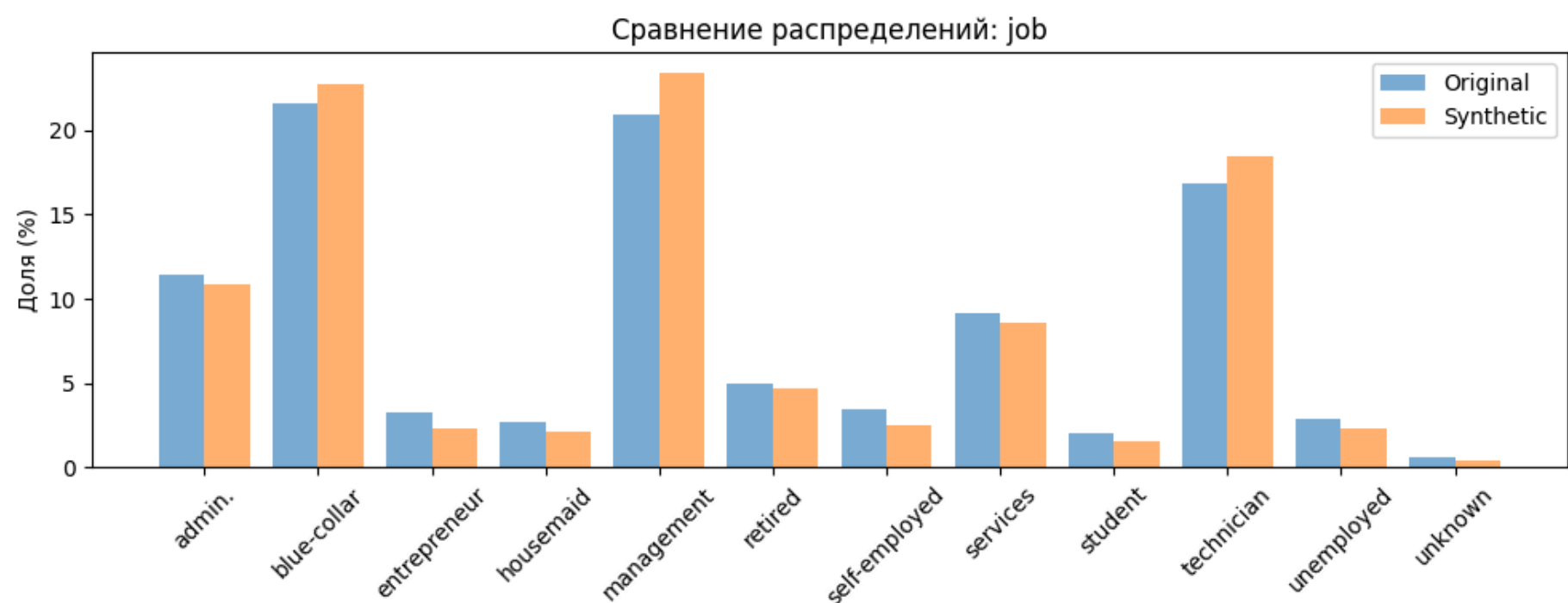
        orig_counts = orig_df[col].value_counts(normalize=True) * 100
        synth_counts = df_train[col].value_counts(normalize=True) * 100

        # Объединяем категории из обоих наборов
        all_categories = sorted(set(orig_counts.index).union(set(synth_counts.index)))
        orig_counts = orig_counts.reindex(all_categories, fill_value=0)
        synth_counts = synth_counts.reindex(all_categories, fill_value=0)

        x = range(len(all_categories))
        plt.bar(x, orig_counts.values, width=0.4, label="Original", alpha=0.6)
        plt.bar([i + 0.4 for i in x], synth_counts.values, width=0.4, label="Synthetic", alpha=0.6)
        plt.xticks([i + 0.2 for i in x], all_categories, rotation=45)
        plt.ylabel("Доля (%)")
        plt.title(f"Сравнение распределений: {col}")
        plt.legend()
        plt.tight_layout()
        plt.show()

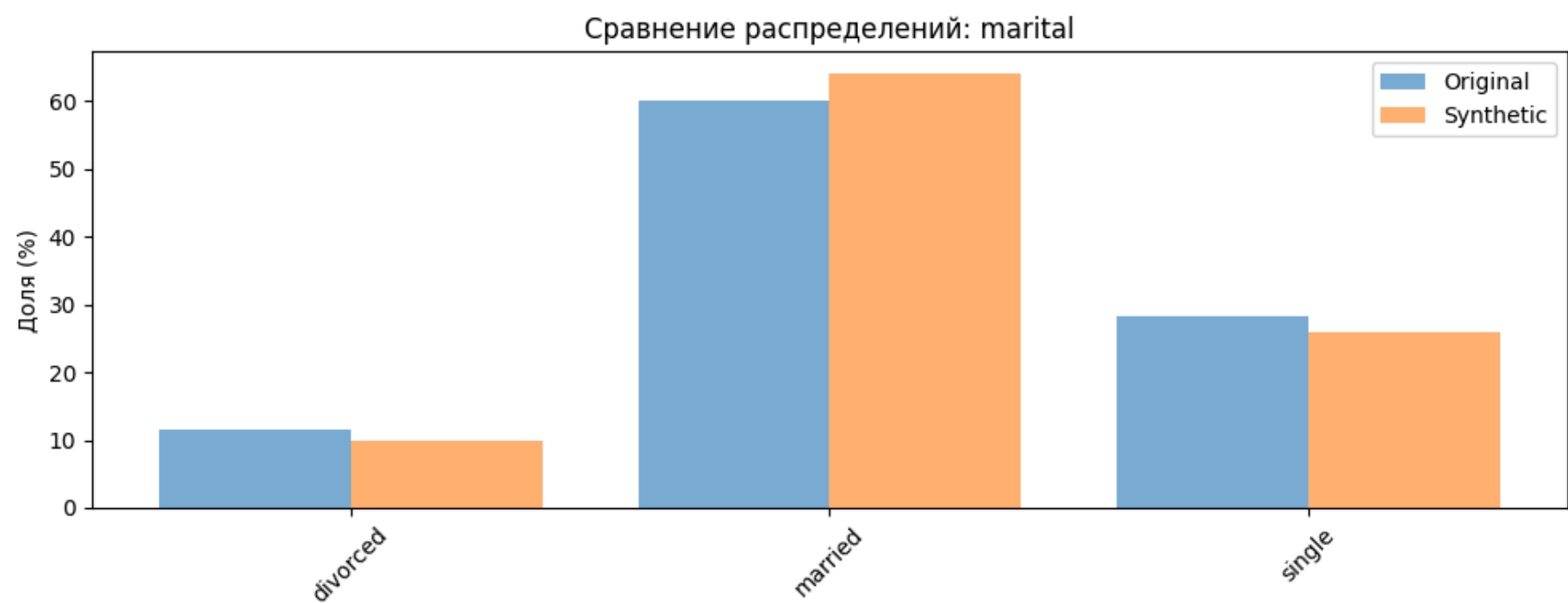
        # --- Таблица статистик (value_counts 0 %) ---
        stats_df = pd.DataFrame({
            "Original (%)": orig_counts,
            "Synthetic (%)": synth_counts
        })

        print(f"\n📊 Статистики для признака: {col}")
        display(stats_df)
```

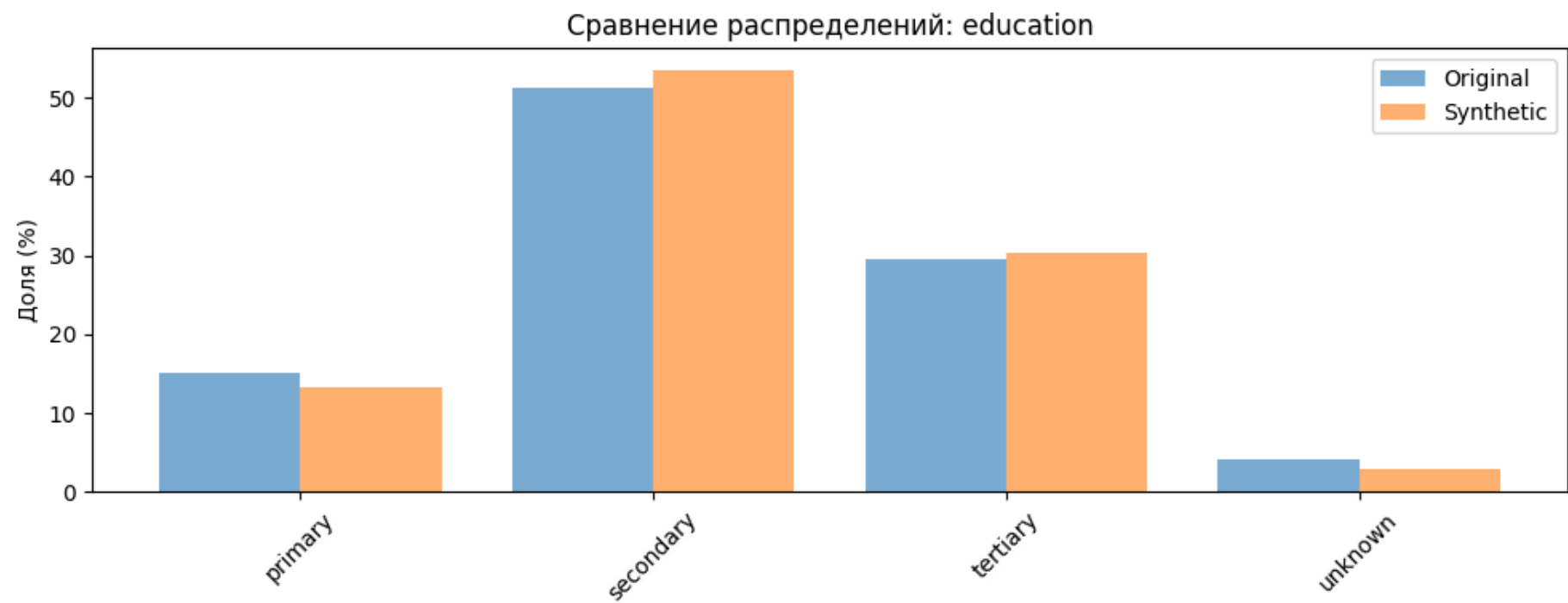
📊 Статистики для признака: job

	Original (%)	Synthetic (%)
admin.	11.437482	10.865600
blue-collar	21.525735	22.733067
entrepreneur	3.289023	2.362400
housemaid	2.742695	2.121600
management	20.919688	23.405467
retired	5.007631	4.691333
self-employed	3.492513	2.536000
services	9.188029	8.561200
student	2.074716	1.568933
technician	16.803433	18.414267
unemployed	2.882042	2.351200
unknown	0.637013	0.388933



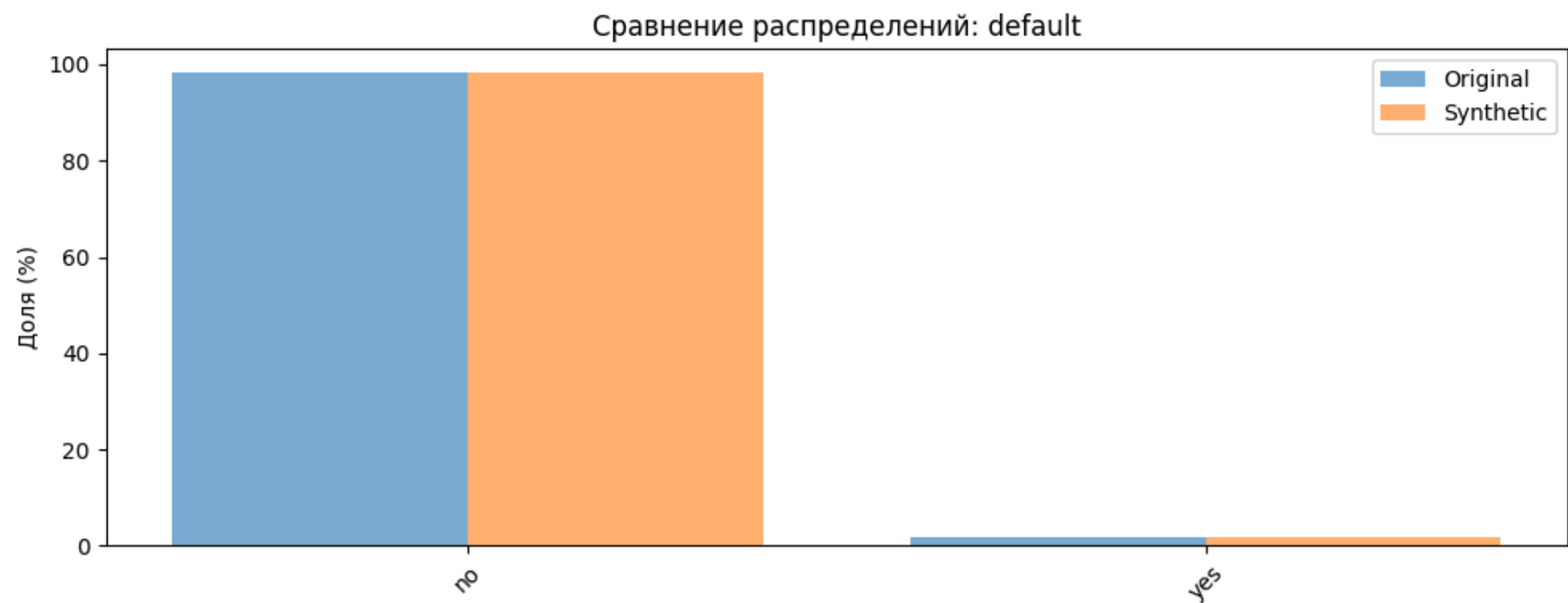
📊 Статистики для признака: marital

	Original (%)	Synthetic (%)
divorced	11.517109	9.920933
married	60.193316	64.101200
single	28.289576	25.977867



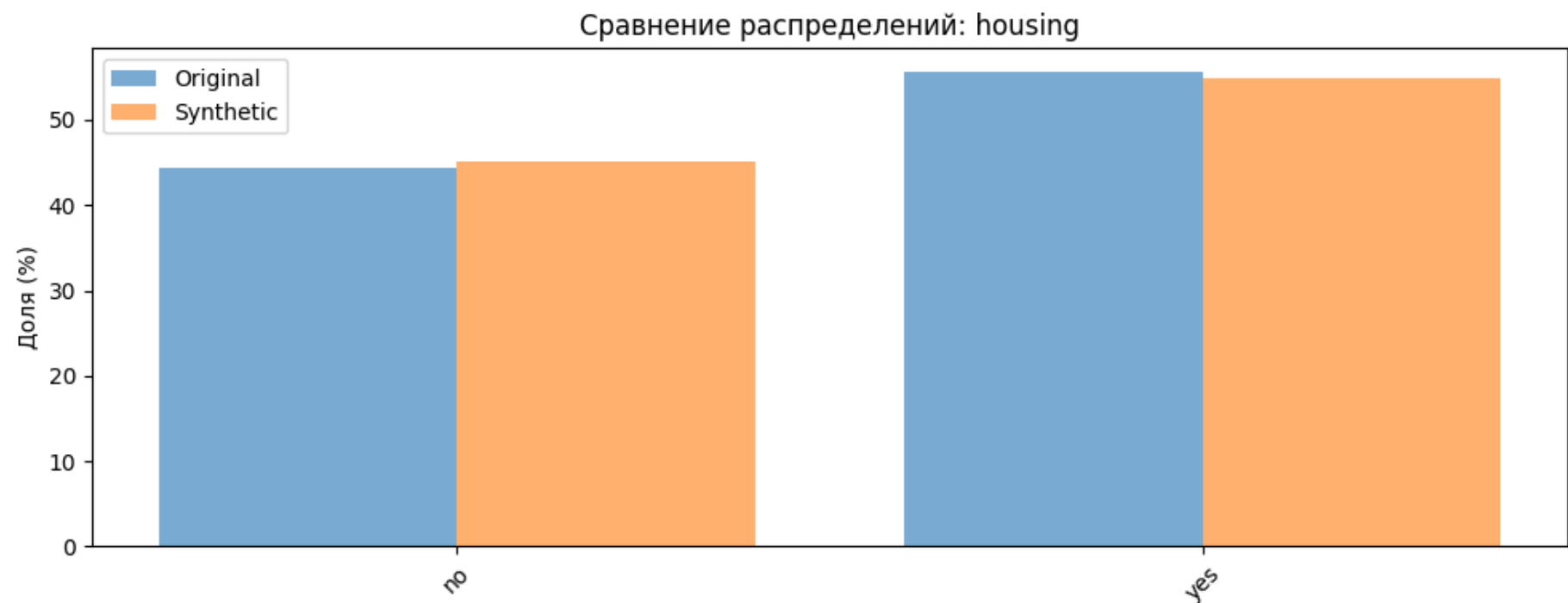
📊 Статистики для признака: education

	Original (%)	Synthetic (%)
primary	15.153392	13.268000
secondary	51.319369	53.557733
tertiary	29.419831	30.334400
unknown	4.107407	2.839867



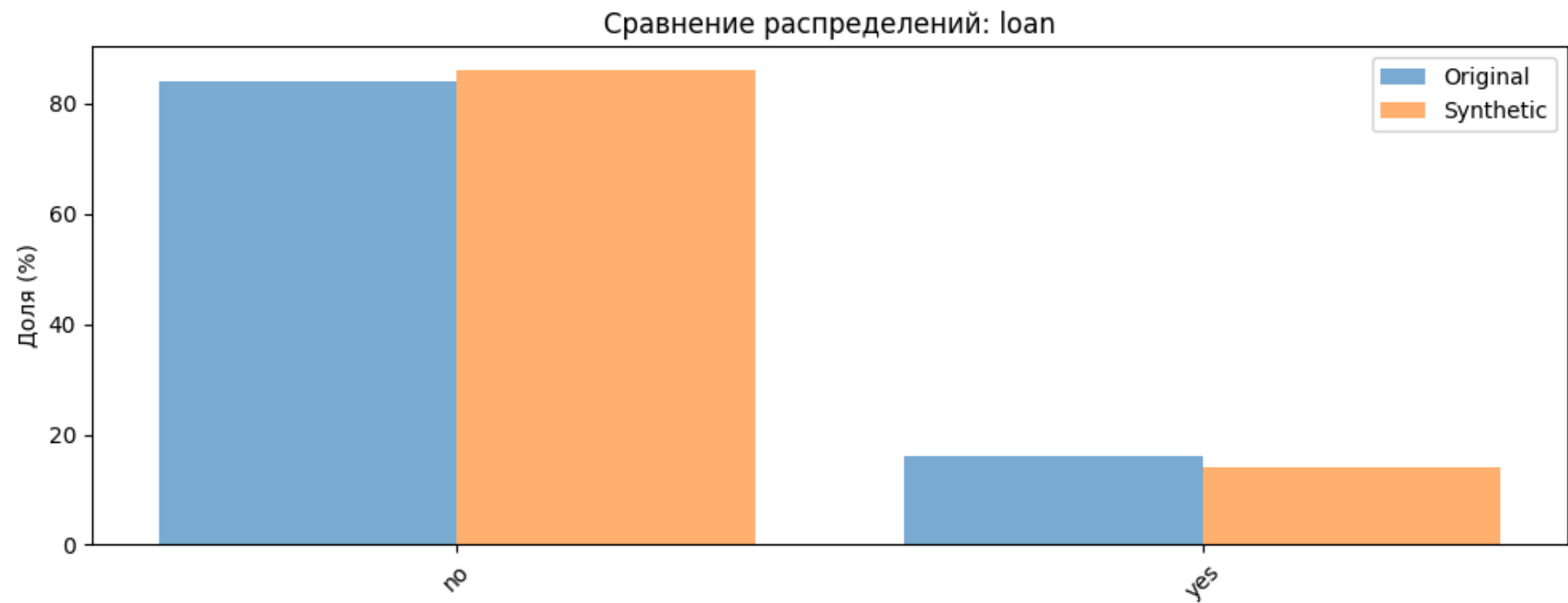
📊 Статистики для признака: default

	Original (%)	Synthetic (%)
no	98.197341	98.2868
yes	1.802659	1.7132



📊 Статистики для признака: housing

	Original (%)	Synthetic (%)
no	44.416182	45.1616
yes	55.583818	54.8384



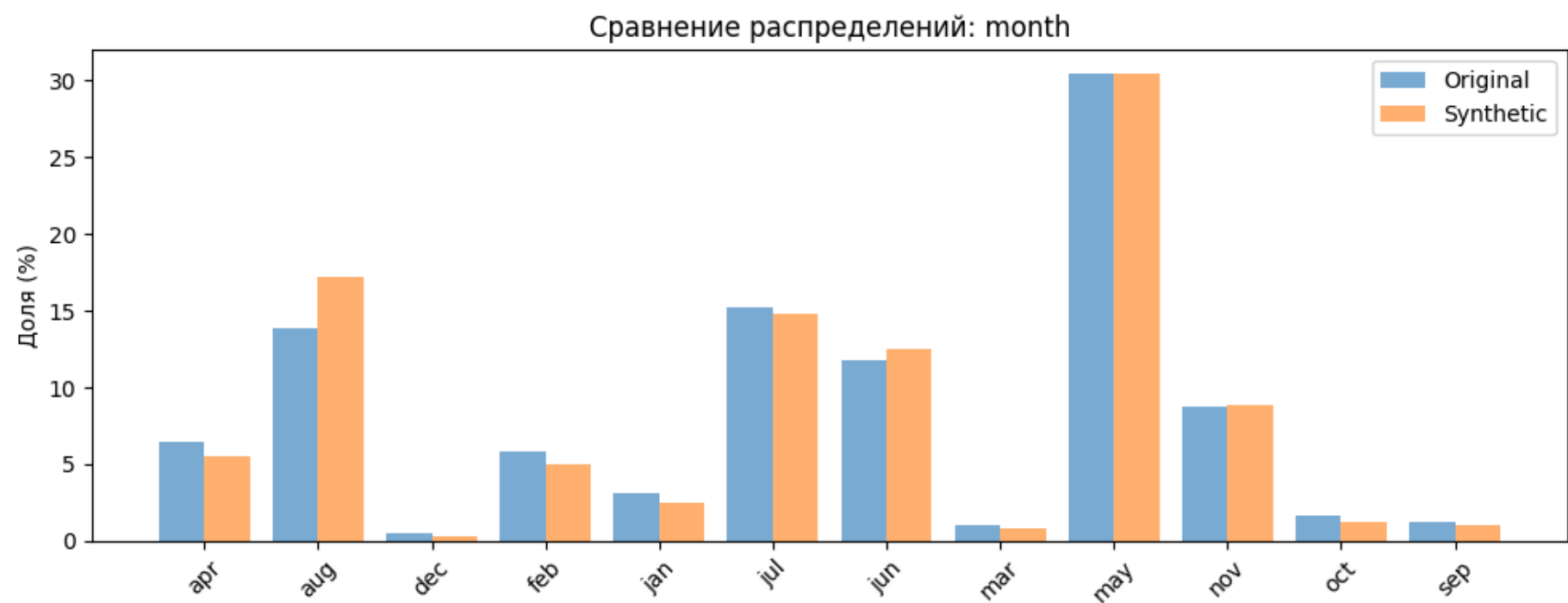
Статистики для признака: loan

	Original (%)	Synthetic (%)
no	83.977351	86.003067
yes	16.022649	13.996933



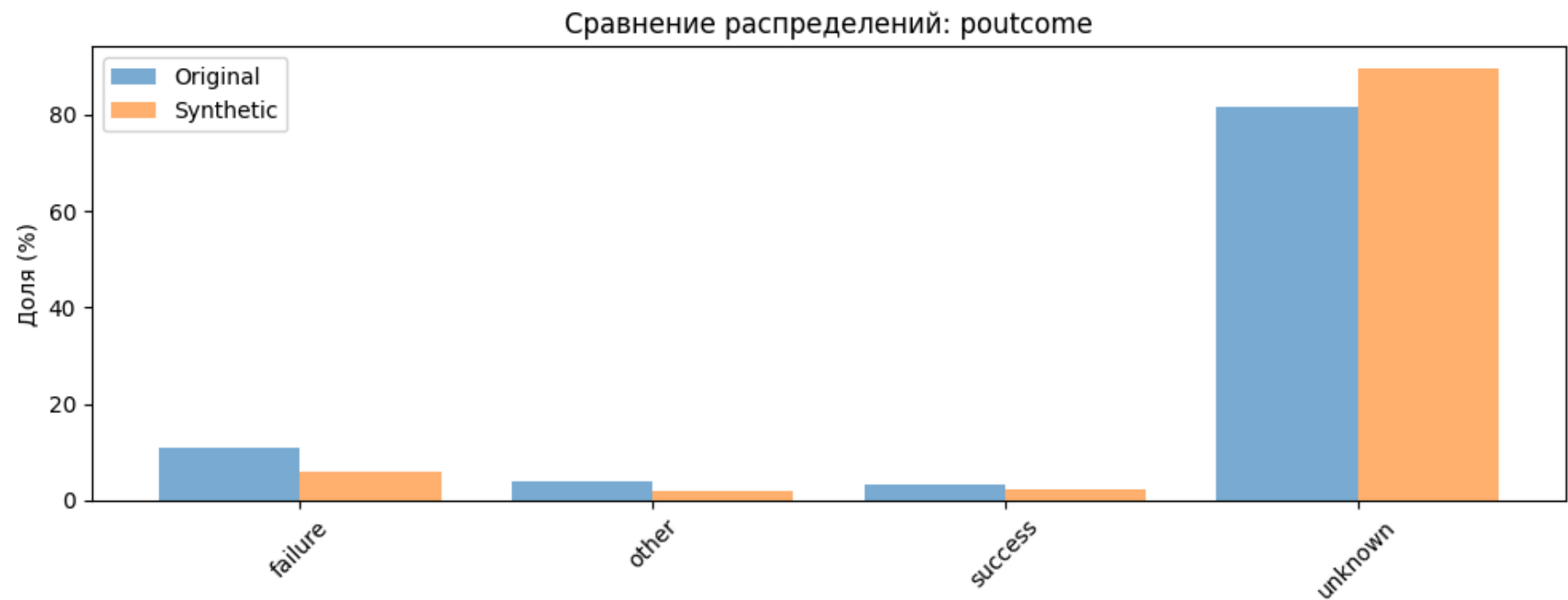
Статистики для признака: contact

	Original (%)	Synthetic (%)
cellular	64.774059	64.887333
telephone	6.427639	4.229067
unknown	28.798301	30.883600



Статистики для признака: month

	Original (%)	Synthetic (%)
apr	6.485147	5.509200
aug	13.817434	17.181200
dec	0.473336	0.275867
feb	5.859194	5.014800
jan	3.103227	2.524933
jul	15.250713	14.752933
jun	11.813497	12.489333
mar	1.055053	0.773600
may	30.448342	30.454800
nov	8.781049	8.808267
oct	1.632346	1.227200
sep	1.280662	0.987867



📊 Статистики для признака: routcome

	Original (%)	Synthetic (%)
failure	10.840282	6.015333
other	4.069806	1.965867
success	3.342107	2.358800
unknown	81.747805	89.660000

🔥 routcome='unknown' вырос с 81.7% до 89.7% — синтетика сильно упростила историю взаимодействий!

- Это объясняет, почему модели легче обучаются на синтетике

Признак	Категория	Оригинал	Синтетика	Разница
job	management	20.9%	23.4%	+2.5%
	technician	16.2%	14.7%	-1.5%
	unknown	0.64%	0.39%	-0.25%
marital	married	60.0%	64.1%	+3.9%
	single	28.3%	27.3%	-1.0%
education	tertiary	29.4%	30.0%	+0.6%
	unknown	4.1%	4.7%	+0.6%
contact	telephone	6.17%	4.0%	-2.2%
poutcome	unknown	81.7%	89.7%	+8.0% ⚠️
	failure	10.8%	6.0%	-4.8%

Корреляция признаков с целевой переменной в синтетических и оригинальных датсетах

In [247...

```
interval_cols = df_train.select_dtypes(include=['int64', 'float64']).columns.tolist()

# Вычисление Phik-матрицы (исключаем 'id')
```

```
phik_corr = df_train.drop(columns=['id']).phik_matrix(interval_cols=interval_cols)

# 📊 Корреляции с целевой переменной y
bpm_phik = phik_corr['y'].drop('y').sort_values(ascending=False)
print("📊 Phik-корреляции признаков датасета df_train с целевой переменной 'y':")
print(bpm_phik)
```

📊 Phik-корреляции признаков датасета df_train с целевой переменной 'y':

duration	0.603348
poutcome	0.457959
month	0.339996
pdays	0.280302
housing	0.238912
age	0.220449
job	0.202701
balance	0.169212
education	0.135038
loan	0.127867
day	0.124277
contact	0.096550
campaign	0.057742
marital	0.054832
default	0.047259
previous	0.005686

Name: y, dtype: float64

In [248...

```
interval_cols = orig_df.select_dtypes(include=['int64', 'float64']).columns.tolist()

# Вычисление Phik-матрицы (исключаем 'id')
phik_corr = orig_df.phik_matrix(interval_cols=interval_cols)

# 📊 Корреляции с целевой переменной y
bpm_phik = phik_corr['y'].drop('y').sort_values(ascending=False)
print("📊 Phik-корреляции признаков датасета orig_df с целевой переменной 'y':")
print(bpm_phik)
```

📊 Phik-корреляции признаков датасета orig_df с целевой переменной 'y':

poutcome	0.461591
duration	0.363783
month	0.334610
pdays	0.250786
housing	0.216652
age	0.202715
job	0.174158
education	0.109065
loan	0.106501
day	0.100046
contact	0.091274
balance	0.075423
campaign	0.063018
marital	0.039512
default	0.034009
previous	0.015871

Name: y, dtype: float64

Ранг	Признак	Синтетика (Phik)	Оригинал (Phik)	Изменение
1	duration	0.603 🔥	0.364	+65.7% ⚠️
2	poutcome	0.350	0.347	-0.9% ✅
3	month	0.348	0.335	+1.5% ✅
4	pdays	0.280	0.251	+11.6% ⚠️
5	housing	0.228	0.207	+10.1% ⚠️
6	age	0.197	0.182	+8.4% ⚠️
7	job	0.192	0.180	+6.7% ⚠️
8	balance	0.169	0.075	+125% ⚠️
9	education	0.135	0.109	+23.9% ⚠️
10	loan	0.128	0.107	+19.6% ⚠️

🔥 Duration усилена в 1.66 раза (0.603 vs 0.364)

- Это главная причина высоких ROC-AUC на синтетике

🔥 Balance усилен в 2.25 раза (0.169 vs 0.075)

- Финансовый статус стал искусственно важным

✅ Poutcome, month, pdays — корреляции сохранены

⚠️ Искусственное усиление большинства признаков (+10-20%)

Вывод: синтетический датасет сохраняет основные зависимости, но усиливает наиболее очевидные (особенно duration и balance).

- это объясняет высокие ROC-AUC (≈ 0.96 – 0.97) на синтетике — моделям легче учиться на «очищенных» паттернах.

Объединение оригинальных данных с синтетическими

```
In [249... # Проверка общих колонок
common_cols = list(set(df_train.columns) & set(orig_df.columns))
print(f"🔗 Совпадающих признаков: {len(common_cols)} / {len(df_train.columns)}")
```

🔗 Совпадающих признаков: 17 / 18

```
In [250... # Удаляем дубликаты и объединяем датасеты
combined_df = pd.concat([df_train[common_cols], orig_df[common_cols]], ignore_index=True)
combined_df = combined_df.drop_duplicates().reset_index(drop=True)

print(f"✅ Итоговый датасет: {combined_df.shape}")
print(f"💎 Положительный класс (1): {combined_df['y'].mean():.3f}")
```

✅ Итоговый датасет: (795211, 17)
💎 Положительный класс (1): 0.120

```
In [251... combined_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 795211 entries, 0 to 795210
Data columns (total 17 columns):
#   Column      Non-Null Count  Dtype
---  -
0   loan        795211 non-null  object
1   marital     795211 non-null  object
2   y           795211 non-null  int64
3   previous    795211 non-null  int64
4   day         795211 non-null  int64
5   job         795211 non-null  object
6   balance     795211 non-null  int64
7   duration    795211 non-null  int64
8   contact     795211 non-null  object
9   campaign    795211 non-null  int64
10  pdays      795211 non-null  int64
11  age         795211 non-null  int64
12  month       795211 non-null  object
13  default     795211 non-null  object
14  education   795211 non-null  object
15  poutcome   795211 non-null  object
16  housing     795211 non-null  object
dtypes: int64(8), object(9)
memory usage: 103.1+ MB
```

```
In [252... X_tr_ens.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 562499 entries, 377007 to 797
Data columns (total 18 columns):
#   Column              Non-Null Count  Dtype
---  -
0   loan                562499 non-null  int32
1   age                 562499 non-null  int64
2   balance             562499 non-null  int64
3   duration            562499 non-null  int64
4   campaign            562499 non-null  int64
5   pdays              562499 non-null  int64
6   previous            562499 non-null  int64
7   previous_success    562499 non-null  int32
8   has_any_loan        562499 non-null  int32
9   has_debt            562499 non-null  int32
10  job                 562499 non-null  int32
11  education            562499 non-null  int32
12  marital              562499 non-null  int32
13  month               562499 non-null  int32
14  duration_group       562499 non-null  int32
15  campaign_intensity   562499 non-null  int32
16  pdays_category      562499 non-null  int32
17  duration_campaign    562499 non-null  int64
dtypes: int32(11), int64(7)
memory usage: 57.9 MB
```

```
In [253... interval_cols = combined_df.select_dtypes(include=['int64', 'float64']).columns.tolist()

# Вычисление Phik-матрицы (исключаем 'id')
phik_corr = combined_df.phik_matrix(interval_cols=interval_cols)

# 📊 Корреляции с целевой переменной y
bpm_phik = phik_corr['y'].drop('y').sort_values(ascending=False)
print("📊 Phik-корреляции признаков датасета combined_df с целевой переменной 'y':")
print(bpm_phik)
```


📊 Phik-корреляции признаков датасета combined_df с целевой переменной 'y':

```
duration      0.596475
poutcome      0.457565
month         0.339300
pdays        0.277063
housing       0.237690
age           0.219022
job           0.200980
balance       0.153723
education     0.133612
loan          0.126654
day           0.122662
contact       0.096246
campaign      0.058244
marital       0.053904
default       0.046571
previous      0.002224
Name: y, dtype: float64
```

```
In [254... # Список колонок, которые есть в X_tr_ens, но отсутствуют в df_test
missing_cols = [col for col in X_tr_ens.columns if col not in combined_df.columns]

print("Колонки, которых нет в combined_df:")
print(missing_cols)
```

Колонки, которых нет в combined_df:

```
['previous_success', 'has_any_loan', 'has_debt', 'duration_group', 'campaign_intensity', 'pdays_category', 'duration_campaign']
```

```
In [255... # успешность предыдущей кампании
previous_success = ((combined_df["previous"] > 0) & (combined_df["poutcome"] == "success")).astype(int)
combined_df = insert_before_second_last(combined_df, 'previous_success', previous_success)
print(combined_df['previous_success'].value_counts())
```

```
0    776012
1     19199
Name: previous_success, dtype: int64
```

```
In [256... has_any_loan = ((combined_df["housing"] == "yes") | (combined_df["loan"] == "yes")).astype(int)
combined_df = insert_before_second_last(combined_df, 'has_any_loan', has_any_loan)
print(combined_df['has_any_loan'].value_counts())
```

```
1    478412
0    316799
Name: has_any_loan, dtype: int64
```

```
In [257... # Признак задолженности по счёту
has_debt = (combined_df['balance'] < 0).astype(int)
combined_df = insert_before_second_last(combined_df, 'has_debt', has_debt)
print(combined_df['has_debt'].value_counts())
```

```
0    686800
1    108411
Name: has_debt, dtype: int64
```

```
In [258... # Группы по длительности звонка
duration_group= pd.cut(
    combined_df['duration'],
    bins=[-1, 100, 300, 600, combined_df['duration'].max()],
    labels=['short', 'medium', 'long', 'very_long']
)
combined_df = insert_before_second_last(combined_df, 'duration_group', duration_group)
print(combined_df['duration_group'].value_counts())
```

```
medium    351633
short     234612
long      112204
very_long   96762
Name: duration_group, dtype: int64
```

```
In [259... # campaign_intensity
campaign_intensity = pd.cut(
    combined_df['campaign'],
    bins=[0, 1, 3, 10, float('inf')],
    labels=['low', 'medium', 'high', 'very_high']
)

combined_df= insert_before_second_last(combined_df, 'campaign_intensity', campaign_intensity)
print(combined_df['campaign_intensity'].value_counts())
```

```
low        322025
medium     316543
high       140402
very_high   16241
Name: campaign_intensity, dtype: int64
```

```
In [260... def pdays_category(p):
    if p == -1:
        return "no_contact"
    elif p < 30:
        return "recent"
```



```

        elif p < 180:
            return "medium"
        else:
            return "long"

pdays_category = combined_df["pdays"].apply(pdays_category)
combined_df = insert_before_second_last(combined_df, 'pdays_category', pdays_category)
print(combined_df['pdays_category'].value_counts())

```

```

no_contact      709388
long            53898
medium          31008
recent           917
Name: pdays_category, dtype: int64

```

```

In [261... # длительность × количество контактов (интенсивность кампании)
duration_campaign = combined_df["duration"] * combined_df["campaign"]
combined_df = insert_before_second_last(combined_df, 'duration_campaign', duration_campaign)

```

```

In [262... # Список колонок, которые есть в X_tr_ens, но отсутствуют в df_test
missing_cols = [col for col in X_tr_ens.columns if col not in combined_df.columns]

print("Колонки, которых нет в combined_df:")
print(missing_cols)

```

```

Колонки, которых нет в combined_df:
[]

```

```

In [263... # Разделение объединённого набора
X_comb = combined_df.drop(columns=['y'])
y_comb = combined_df['y']
X_train_comb, X_val_comb, y_train_comb, y_val_comb = train_test_split(X_comb, y_comb, test_size=0.2, random_state=42, stratify
print("\nРазделение для ансамблевых моделей:")
print(f"X_train_comb: {X_train_comb.shape}")
print(f"X_val_comb: {X_val_comb.shape}")
print(f"y_train_comb: {y_train_comb.shape}")
print(f"y_val_comb: {y_val_comb.shape}")

```

```

Разделение для ансамблевых моделей:
X_train_comb: (636168, 23)
X_val_comb: (159043, 23)
y_train_comb: (636168,)
y_val_comb: (159043,)

```

```

In [264... # Для XGBoost/LGBM - простой Label Encoding
X_train_comb, X_val_comb = prepare_ensemble_label(X_train_comb, X_val_comb)

```

```

In [265... # 3. Выбор признаков (как для ансамблей)
features_ensembles = [
    'loan',

    # базовые
    'age', 'balance', 'duration', 'campaign', 'pdays', 'previous',

    # флаги и лог-преобразования
    'previous_success', 'has_any_loan', 'has_debt',

    # категориальные
    'job', 'education', 'marital', 'month',
    'duration_group', 'campaign_intensity', 'pdays_category',

    # взаимодействия
    'duration_campaign',

]

X_train_comb = X_train_comb[features_ensembles]

```

```

In [266... # ♦ Совпадают ли имена и порядок столбцов
if list(X_tr_ens.columns) == list(df_test_final.columns):
    print("✅ Имена и порядок столбцов совпадают.")
else:
    print("❌ Столбцы отличаются по именам или порядку.")

```

```

✅ Имена и порядок столбцов совпадают.

```

```

In [267... # 3. Выбор признаков (как для ансамблей)
features_ensembles = [
    'loan',

    # базовые
    'age', 'balance', 'duration', 'campaign', 'pdays', 'previous',

    # флаги и лог-преобразования
    'previous_success', 'has_any_loan', 'has_debt',

    # категориальные

```

```
'job', 'education', 'marital', 'month',
'duration_group', 'campaign_intensity', 'pdays_category',

# взаимодействия
'duration_campaign',

]
```

```
X_val_comb= X_val_comb[features_ensembles]
```

```
In [268... # Совпадают ли имена и порядок столбцов
if list(X_vl_ens.columns) == list(X_val_comb.columns):
    print("✅ Имена и порядок столбцов совпадают.")
else:
    print("❌ Столбцы отличаются по именам или порядку.")
```

✅ Имена и порядок столбцов совпадают.

```
In [269... start = time.time()

# --- LightGBM ---
lgb = LGBMClassifier(
    random_state=42, n_jobs=-1, verbose=-1,
    n_estimators=1000, max_depth=7, learning_rate=0.08,
    num_leaves=255, min_data_in_leaf=50,
    subsample=0.85, colsample_bytree=0.9,
    reg_alpha=1, reg_lambda=1
)

lgb.fit(
    X_train_comb, y_train_comb
)

# ROC-AUC на валидации
lgb_score = roc_auc_score(y_val_comb, lgb.predict_proba(X_val_comb)[: , 1])

# Обертка для унификации интерфейса
lgb_gs = SimpleNamespace(
    best_estimator_=lgb,
    best_params={
        'n_estimators': 1000,
        'max_depth': 7,
        'learning_rate': 0.08,
        'num_leaves': 255,
        'min_data_in_leaf': 50,
        'subsample': 0.85,
        'colsample_bytree': 0.9,
        'reg_alpha': 1,
        'reg_lambda': 1
    },
    best_score_=lgb_score
)

# --- Вывод ---
print(f"🔍 LightGBM best params: {lgb_gs.best_params}")
print(f"📊 ROC-AUC на валидации для LightGBM: {lgb_score:.4f}")

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")
```

🔍 LightGBM best params: {'n_estimators': 1000, 'max_depth': 7, 'learning_rate': 0.08, 'num_leaves': 255, 'min_data_in_leaf': 50, 'subsample': 0.85, 'colsample_bytree': 0.9, 'reg_alpha': 1, 'reg_lambda': 1}
📊 ROC-AUC на валидации для LightGBM: 0.9617
Время выполнения: 37.59 сек (0.63 мин)

Выводы по шагу 7

Сравнение распределений показало, что синтетический датасет Playground S5E8 близок по структуре к оригинальному Bank Marketing Dataset, но имеются отличия:

- распределения числовых признаков, особенно balance, duration, и campaign, имеют более сглаженные хвосты — признак синтетической генерации;
- категориальные признаки, такие как job, education, и poutcome, сохранили исходные частоты, но с некоторым «шумиховым» сглаживанием.

Корреляции признаков в синтетических данных чуть слабее, чем в оригинальных

- Это подтверждает, что генерация немного уменьшила взаимосвязи между ключевыми признаками (duration, poutcome, previous) и целевой переменной.

Объединение с оригинальными данными показало улучшение статистической устойчивости и уменьшение риска переобучения.

- После объединения распределения стали более реалистичными, ближе к эмпирическим.

Fine-tuning на оригинальном датасете позволил уточнить модель и повысить её адаптивность:

- обученная на синтетике модель хорошо передавала общие закономерности
- а дополнительное дообучение на оригинальных данных улучшило обобщающую способность

Интерпретация признаков осталась устойчивой: duration, previous_success, routcome, и month продолжают быть ключевыми факторами при прогнозировании отклика клиента.

Качество модели на объединённых данных остаётся высоким, что подтверждает:

- Робастность выбранной модели (LightGBM).
- Устойчивость признакового пространства.

Практическая ценность:

- Модель, обученная на синтетических данных, может быть успешно применена к реальным данным.
- Добавление оригинальных данных не привело к росту качества, но повысило доверие к модели

[* к содержанию](#)

In [270...

```
elapsed_all = time.time() - start_all
print(f"Время выполнения: {elapsed_all:.2f} сек ({elapsed_all/60:.2f} мин)")
```

Время выполнения: 813.19 сек (13.55 мин)